Introduction to Data Analytics for Business and Economics

Using Python for Data Preprocessing, Visualization, and Modeling

Matthias Bogaert

2026

Table of contents

Preface **1**
Who this book is for . . . 1
What this book covers . . . 1
How to read it . . . 1
A note on generative AI . . . 1
License . . . 2

1 Introduction **3**
1.1 Why data analytics now? . . . 3
1.2 What is data analytics? . . . 5
1.2.1 The four types of analytics . . . 6
1.2.2 AI, machine learning, and data analytics . . . 6
1.2.3 Machine learning vs. statistics . . . 9
1.2.4 Big data and generative AI . . . 9
1.2.5 What makes a good analytical model . . . 11
1.2.6 Data . . . 11
1.2.7 Supervised and unsupervised learning . . . 12
1.3 The data analytics process: CRISP-DM . . . 14
1.4 Where data analytics shows up . . . 16
1.5 Why Python . . . 17
1.6 How this book is organized . . . 17

2 Python for Data Analytics **19**
2.1 Variables . . . 20
2.1.1 Numeric Variables . . . 20
2.1.2 Boolean Variables . . . 22
2.1.3 String Variables . . . 22
2.1.4 None Variables . . . 24
2.2 Collections . . . 25
2.2.1 Lists . . . 25
2.2.2 Dictionaries . . . 27
2.2.3 Tuples . . . 28
2.3 Conditionals . . . 30
2.3.1 If-Else . . . 30
2.3.2 For . . . 32
2.3.3 While . . . 32
2.3.4 Try-Except . . . 34
2.4 Reading and Writing Files . . . 34
2.4.1 Reading . . . 34
2.4.2 Writing Files . . . 37
2.5 Functions . . . 38

2.5.1 Definition . . . 38
2.5.2 Parameters . . . 38
2.5.3 Return . . . 39
2.5.4 Flexible Functions . . . 40
2.6 Packages . . . 41
2.6.1 Usage . . . 41
2.7 NumPy . . . 42
2.7.1 Definition . . . 42
2.7.2 Multi-dimensional array . . . 42
2.7.3 Array Manipulation . . . 46
2.8 Pandas . . . 47
2.8.1 Definition . . . 48
2.8.2 DataFrame Creation . . . 48
2.8.3 DataFrame Exploration . . . 49
2.8.4 DataFrame Manipulation . . . 53

3 Data Understanding 65
3.1 The three levels of data understanding . . . 65
3.2 Table level: how your tables relate . . . 65
3.2.1 Entity-relationship diagrams . . . 65
3.2.2 Confirming a relationship in code . . . 66
3.2.3 Table descriptions . . . 67
3.3 Data level: getting types and samples right . . . 68
3.3.1 Data types . . . 68
3.3.2 Sampling . . . 69
3.4 Variable level: a first statistical read . . . 70
3.4.1 Working with dates . . . 70
3.4.2 Describing individual variables . . . 70
3.4.3 Saving your progress . . . 72
3.5 Where does the data come from . . . 72
3.6 Data regulation: privacy and intellectual property rights . . . 73
3.6.1 GDPR . . . 73
3.7 Where we go from here . . . 74

4 Data Exploration 75
4.1 Why visualize . . . 75
4.1.1 Matplotlib basics . . . 77
4.2 The core chart types . . . 78
4.2.1 Comparing groups: bar charts . . . 78
4.2.2 Showing a trend: line charts . . . 81
4.2.3 Showing relationships: scatter plots . . . 83
4.2.4 Showing a distribution: histograms and box plots . . . 87
4.3 Data quality: multicollinearity . . . 93
4.4 Data quality: missing values . . . 96
4.4.1 A churn model with missing values . . . 97
4.4.2 Treating missing values . . . 98
4.4.3 Doing this properly: fitting only on training data . . . 99
4.5 Data quality: outliers . . . 101
4.5.1 Detecting outliers . . . 101
4.5.2 Treating outliers . . . 108
4.6 Saving your progress . . . 109
4.7 Where we go from here . . . 109

5 Data Preparation 111

5.1 Value transformations . . . 112
5.1.1 Continuous categorization (binning) . . . 112
5.2 Value representations . . . 115
5.2.1 One-hot encoding . . . 115
5.2.2 Weight of Evidence . . . 117
5.3 Feature engineering . . . 120
5.3.1 Non-linear relationships . . . 121
5.4 Standardization and normalization . . . 125
5.5 Variable selection . . . 127
5.5.1 Filter methods . . . 128
5.6 Ordinal encoding . . . 129
5.7 Putting it all together . . . 131
5.8 Where we go from here . . . 132

6 Model Evaluation 133
6.1 The experimental setup . . . 133
6.1.1 The holdout set: train and test . . . 134
6.1.2 The holdout set: train, validation, and test . . . 135
6.1.3 Cross-validation . . . 137
6.1.4 Hyperparameter tuning with K-fold CV . . . 139
6.1.5 Nested cross-validation . . . 140
6.1.6 Leave-one-out cross-validation . . . 142
6.1.7 The bootstrap . . . 143
6.1.8 Comparing the resampling strategies . . . 143
6.2 Data leakage . . . 144
6.3 The bias–variance trade-off . . . 145
6.4 Performance metrics . . . 148
6.4.1 Regression metrics . . . 148
6.4.2 Classification metrics . . . 150
6.5 Where we go from here . . . 162

7 Modeling 165
7.1 Linear Regression . . . 165
7.1.1 When the linear assumption fails . . . 166
7.1.2 Fitting the line: sklearn versus statsmodels . . . 167
7.1.3 A worked example: predicting Airbnb demand . . . 168
7.1.4 Where linear regression breaks down . . . 172
7.2 Selecting Variables and Taming Complexity . . . 172
7.2.1 Wrapper methods: sequential feature selection . . . 172
7.2.2 Embedded methods: regularization . . . 173
7.2.3 Importance-based selection: SelectFromModel . . . 177
7.2.4 Comparing the approaches . . . 178
7.3 Logistic Regression . . . 178
7.3.1 Why not just use linear regression? . . . 178
7.3.2 Interpreting coefficients: log-odds and odds ratios . . . 180
7.3.3 A worked example: predicting churn . . . 180
7.3.4 Beyond two classes . . . 183
7.3.5 Where logistic regression breaks down . . . 184
7.4 Where we go from here . . . 184

Appendices 185

A Installation Guide 185

A.1 Getting the course files . . . 185
A.2 Create a virtual environment . . . 185
A.2.1 VS Code . . . 186
A.2.2 Command line . . . 186
A.3 Installing required packages . . . 186
A.4 Selecting the right kernel . . . 186
A.5 Running Python . . . 187

References **189**

Preface

This book stems from years of experience teaching introductory and advanced data analytics courses across different institutions. There is no shortage of excellent books in this space: theoretical handbooks like *An Introduction to Statistical Learning* build the statistical foundations rigorously, and applied books like *Hands-On Machine Learning* cover the modeling toolkit in depth. What none of them focus on is what business and economics students actually need most: the tools and methods to turn a messy, real-world business problem into a proper data analytics project. That is what this book tries to do differently. There is no lengthy mathematical derivation here, but plenty of hands-on code, and the practical tips and tricks that make the difference between a model that works on a slide and one that survives contact with a real business problem.

Who this book is for

This book offers a gentle introduction to Python, so no prior programming experience is required to start. That said, a basic introductory programming or Python course would benefit the reader, since the programming covered here is aimed squarely at what you need for data analytics, not the general fundamentals of programming as such. A basic statistics course is similarly useful, since not every statistical concept is re-derived from first principles. In practice, this makes the book a good fit for undergraduate or Bachelor-level teaching once students have already had some exposure to programming and statistics, even a light one. It is also the primary teaching material for the Data Analytics course at the University of Namur (UNamur), though it is written to be useful to any reader with the right background, regardless of institution.

What this book covers

The book walks through a full data analytics pipeline, from raw, messy data to a working, evaluated predictive model: Python fundamentals, data understanding, data exploration, data preparation, model evaluation, and modeling. Each chapter pairs a concept with runnable code, so the code is never a black box you're asked to accept on faith.

How to read it

Chapters build on each other and are meant to be read in order. Each one comes with a companion Jupyter notebook so you can run the code yourself, not just read it. These notebooks aren't published on this website; they live in the `notebooks_student/` folder of the book's GitHub repository, linked from the sidebar on every page of this site. If you want to run the code locally, the Installation Guide appendix walks through setting up everything you need.

A note on generative AI

Parts of this book's text, code examples, and figures were drafted, polished, or generated with the help of a generative AI assistant, always under the direct guidance of, and with final review by, the author. Every

explanation, dataset, line of code, and figure has been checked for accuracy before publication. Chapter 1 discusses this shift in writing and coding practice in a bit more detail.

License

This book is licensed under the CC BY-NC-SA 4.0 license.



Figure 1: Creative Commons License

Chapter 1

Introduction

Analytics has taken on a pivotal role in modern society, and we rely on it on a daily basis often without realizing it. The omnipresence of and reliance on analytics has increased sharply over the past few years, with the introduction of large language models and generative AI into everyday products. This book itself could even serve as a small example of that shift: like most writing and coding today, parts of it were co-produced and polished with the help of a generative AI assistant working with a human in the loop. The boom in LLMs and generative AI has understandably pulled artificial intelligence into the public conversation and even the kitchen table.

But generative AI is only the most visible recent chapter of a much longer history of analytics. Long before ChatGPT, Gemini and Claude, data analytical models were already quietly shaping the everyday decisions businesses make about their customers and products. When scanning your loyalty card (or mobile app nowadays) at the supermarket, a model decides which promotional folder lands in your inbox next. When scrolling through your social media feed, a ranking algorithm determines which posts are most likely to keep you engaged. On streaming services a recommender system has quietly reshuffled its entire catalogue around what it predicts you'll want to watch or listen to next. If you apply for a loan or an insurance policy, a risk model estimates how likely you are to default or file a claim, and a bank or insurer uses that estimate to decide whether to approve your application and at what price. However, you might assume that the realm of analytics is limited to business-to-consumer applications. It's not! In industrial settings, analytics is used to optimize production processes, reduce energy consumption, and improve product quality. On the factory floor, sensor data feeds predictive maintenance models that monitor when a machine is likely to fail before it actually does, such that preventive actions can be taken. These models also help to decide how many spare parts to keep in stock for (unexpected) breakdowns. On a broader supply chain level, demand forecasting models also assist on how much raw material to order, how to design the manufacturing site layout, how much inventory to hold, and how many to reorder weeks in advance.

This chapter introduces what data analytics actually is, why it has become central to business decision-making, and how the rest of this book is organized around the process of turning raw data into a working analytical model.

1.1 Why data analytics now?

Three things happened at roughly the same time: the amount of data organizations collect exploded, computing power to process it became cheap, and the algorithms to make sense of it matured. Mortenson et al. (2015) call this convergence the **sixth period** in the century-long evolution of operational research and analytics (running from the mid-2000s to today). Given the prevalence of online data sources, businesses had continuous access to data from customers, competitors, and the general public via social networks. Even their own machines and products were being connected and continuously monitored, in the "internet of things". That data quickly outgrew traditional databases and business intelligence architecture, giving rise to an entire

new ecosystem of technologies such as Hadoop, NoSQL and NewSQL databases, and cloud computing. All these systems were built specifically to store, process, and analyze data at scale. The result is that analytical models now sit behind decisions that used to rely purely on human judgment and increasingly outperform it.

A **seventh period** arguably began with the arrival of generative AI, though this is our own extension, not part of Mortenson et al. (2015)'s original 2015 framework, which predates the release of ChatGPT by several years. The shift from models that *describe and predict* to models that *generate and respond* (a distinction we return to later in this chapter), and, with agentic AI, even *act on their own*, marks as sharp a break with the analytics period as the analytics period itself marked with what came before it.

A few examples make this concrete. In late December 2019, an AI system monitoring news reports and airline ticketing data flagged unusual pneumonia activity in Wuhan, China, days before public health authorities issued their own warnings. A nice illustration of how analytics can act as an early detection system for a signal humans hadn't yet noticed. That signal, of course, would go on to become COVID-19, and everything else from there is history. Around the same period, revelations about Cambridge Analytica showed how predictive models built on Facebook data (e.g., likes, comments, status updates, groups, etc.) could infer personality traits and how these personality traits could be used to target voters individually. Progress has only accelerated since: modern large language models now routinely match or exceed professional-level performance on standardized exams in law and medicine, and in 2025 both OpenAI and Google DeepMind's reasoning models achieved a gold-medal-level score at the International Mathematical Olympiad, solving 5 of the 6 contest problems under official conditions. OpenAI's result was graded by former IMO medalists; DeepMind's performance was independently verified by official IMO coordinators.[1] In medicine specifically, deep learning models have been shown to match or outperform radiologists at detecting breast cancer in screening mammograms.

These capabilities cut both ways. As analytical models are given more influence over decisions that affect people's lives (e.g., credit, hiring, medical diagnosis, criminal justice) regulators have started drawing hard lines around how they can be used. One notable example of careless use of AI systems was the COMPAS (Correctional Offender Management Profiling for Alternative Sanctions) algorithm, used across the U.S. to assess a defendant's risk of recidivism for general and violent crime. An investigation found the tool was biased against Black defendants, who were flagged as high-risk more often despite not reoffending at a higher rate. What makes the case genuinely instructive, though, is not just *that* the model was biased, but *why*: Lagioia et al. (2022) show that COMPAS was, overall, roughly equally accurate for Black and White defendants, yet it still made one particular kind of mistake, wrongly flagging someone as high-risk who never went on to reoffend, far more often for Black defendants. Trying to remove that specific problem can, in turn, make a different imbalance worse elsewhere, because the two groups didn't reoffend at the same rate to begin with. In other words, "being fair" isn't a single, clear-cut target a data scientist can just optimize for: different, equally reasonable ideas of what counts as fair can pull in different directions, so choosing between them ends up being as much a value judgment as a technical one.

The EU has responded to cases like this in stages. GDPR (European Parliament and Council of the European Union 2016), which became enforceable in 2018, was a first step. It doesn't target AI specifically, but it sets out several key principles for processing personal data (among them fairness, purpose limitation, and data minimisation) together with a right to meaningful information about automated decisions made about you. We'll come back to these principles in more detail later in this book, once we start working with real personal data ourselves. The EU AI Act (European Parliament and Council of the European Union 2024) builds on that foundation with a comprehensive, risk-based framework specifically for AI systems: it classifies applications into risk tiers, from "unacceptable" (e.g., real-time biometric scoring for social credit purposes) down to "minimal" (e.g., spam filters), with correspondingly stricter requirements for higher risks (e.g., high-stakes criminal-justice risk scoring like COMPAS).

In sum, data analytics is no longer a specialized department that runs in the back-office. It shapes what products get built, what prices get charged, who gets a loan, and increasingly, what rules govern all of the above, which is exactly why it is crucial to business and economics.

[1] "AI Reasoning Models Achieve Gold-Medal Performance at the International Mathematical Olympiad", IntuitionLabs, accessed 2026.

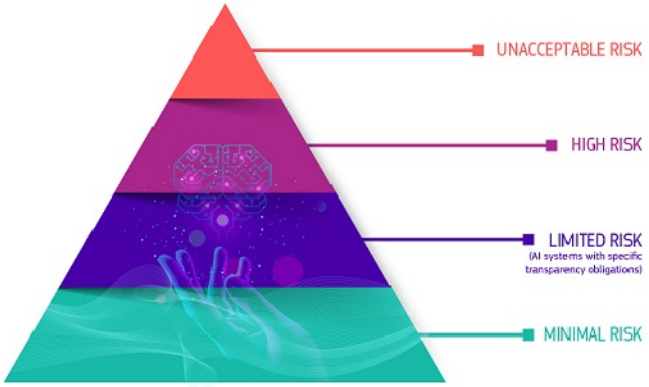


Figure 1.1: EU AI Act risk pyramid: unacceptable, high, limited, and minimal risk tiers. Source: European Commission.

1.2 What is data analytics?

In essence, **data analytics is about transforming raw data into actionable insights for business**. It does so by applying descriptive, diagnostic, predictive, and prescriptive models to heterogeneous, and often large, data sources, so that organizations can make quicker, better, and more intelligent decisions that create business value.

Business analytics sits at the intersection of several disciplines: operations research, information systems, and machine learning all contribute methods. However, domain knowledge and understanding of the actual business problem determine whether those methods produce anything useful for business (Mortenson et al. 2015; Hindle et al. 2020). That combination is exactly what makes it a natural fit for business and economics students: your training gives you the technical grounding, and your business and economic mindset gives you the domain framing to point that technique at a problem worth solving.

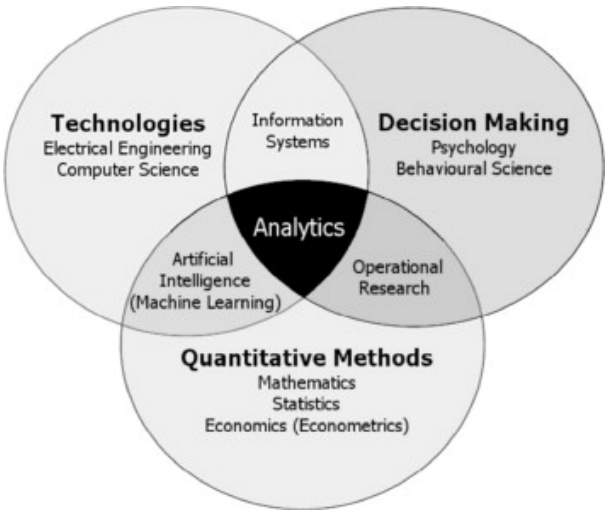


Figure 1.2: Business analytics. Source: Figure 1 in Mortenson et al. (2015), licensed under CC BY 3.0.

A long list of near-synonyms for this field has been used throughout the years (mainly depending on the hype of that moment): artificial intelligence (AI), big data, machine learning, data mining, data science, deep learning, and in the old days, knowledge discovery, pattern recognition. In practice, these terms overlap heavily and are often used interchangeably, though later in this chapter we'll draw out a few real distinctions (particularly between AI, machine learning and statistics) that matter once you start building models yourself.

1.2.1 The four types of analytics

Data analytics is usually broken down into four types, distinguished by the kind of question they answer:

- **Descriptive analytics** describes historical trends or *what* happened. Reporting dashboards, business intelligence tools, and services like Google Analytics are almost entirely descriptive.
- **Diagnostic analytics** explains *why* something happened, digging into the causes behind a trend that descriptive analytics only shows you.
- **Predictive analytics** estimates what *will* happen in the future, based on patterns in historical data.
- **Prescriptive analytics** recommends the best next *action* or decision to take, given a predicted outcome.

Let's consider a simple example to make the distinction more concrete. Imagine tracking a product's yearly sales. Descriptive analytics plots the historical sales curve. Diagnostic analytics explains why sales dropped two years ago (a competitor launched a similar, cheaper product). Predictive analytics forecasts that, on the current trajectory, sales will keep declining next year. Prescriptive analytics recommends a concrete intervention (e.g., a price adjustment or a product redesign) to reverse that trend.

A slightly more elaborate version of the same idea: a B2C e-commerce platform wants to optimize its pricing strategy. Descriptive analytics tells it how prices and sales have historically moved together. Diagnostic analytics uncovers why prices had an impact on sales in the past (e.g., perhaps a past price cut drove a spike in demand). Predictive analytics forecasts how a proposed price change would affect future sales. Prescriptive analytics goes one step further and recommends the actual pricing strategy (e.g., dynamic pricing, product bundling) that maximizes profit given that forecast.

These four types are also hierarchical in complexity and organizational maturity: most organizations start with descriptive analytics, move into predictive analytics as their data and modeling capabilities mature, and only later exploit prescriptive analytics: the analytics equivalent of asking not just "what will happen," but "what should I do about it," using optimization, simulation, and decision modeling techniques (Delen and Ram 2018). In practice the boundaries blur: an organization can be firmly in descriptive-analytics territory for most of its reporting while already piloting predictive models in one specific area.

1.2.2 AI, machine learning, and data analytics

These three terms are related but not identical:

- **Artificial intelligence (AI)** is the broad field of computer science dedicated to solving problems that would otherwise require human intelligence.
- **Machine learning** is a subfield of AI: the study of algorithms that build a mathematical model from sample (training) data in order to make predictions or decisions, without being explicitly programmed for the task. Much like a person improves at an instrument through repeated practice, a machine learning model improves its predictions as it's given more training data.
- **Data analytics** uses big data and mathematical models (often, but not exclusively, machine learning models) to explain or predict a business outcome and support (and preferably improve) decision-making.

Within that space, **predictive analytics** deserves a closer look, since much of this book is about building predictive models. It has been defined as "the process by which a model is created or chosen to try to best predict the probability of an outcome" (Geisser 1993) and, more generally, as "the process of developing a mathematical model that generates an accurate prediction" (Kuhn and Johnson 2013). In recent years predictive analytics, data mining, and machine learning have been used almost interchangeably, but if you want a precise distinction: predictive analytics is about predicting the *future*, while machine learning is about predicting *unseen data* more generally (which is often, but not always, in the future). Under that reading, predictive analytics is a subfield of machine learning, even though the underlying algorithms are frequently identical; only the setup differs.

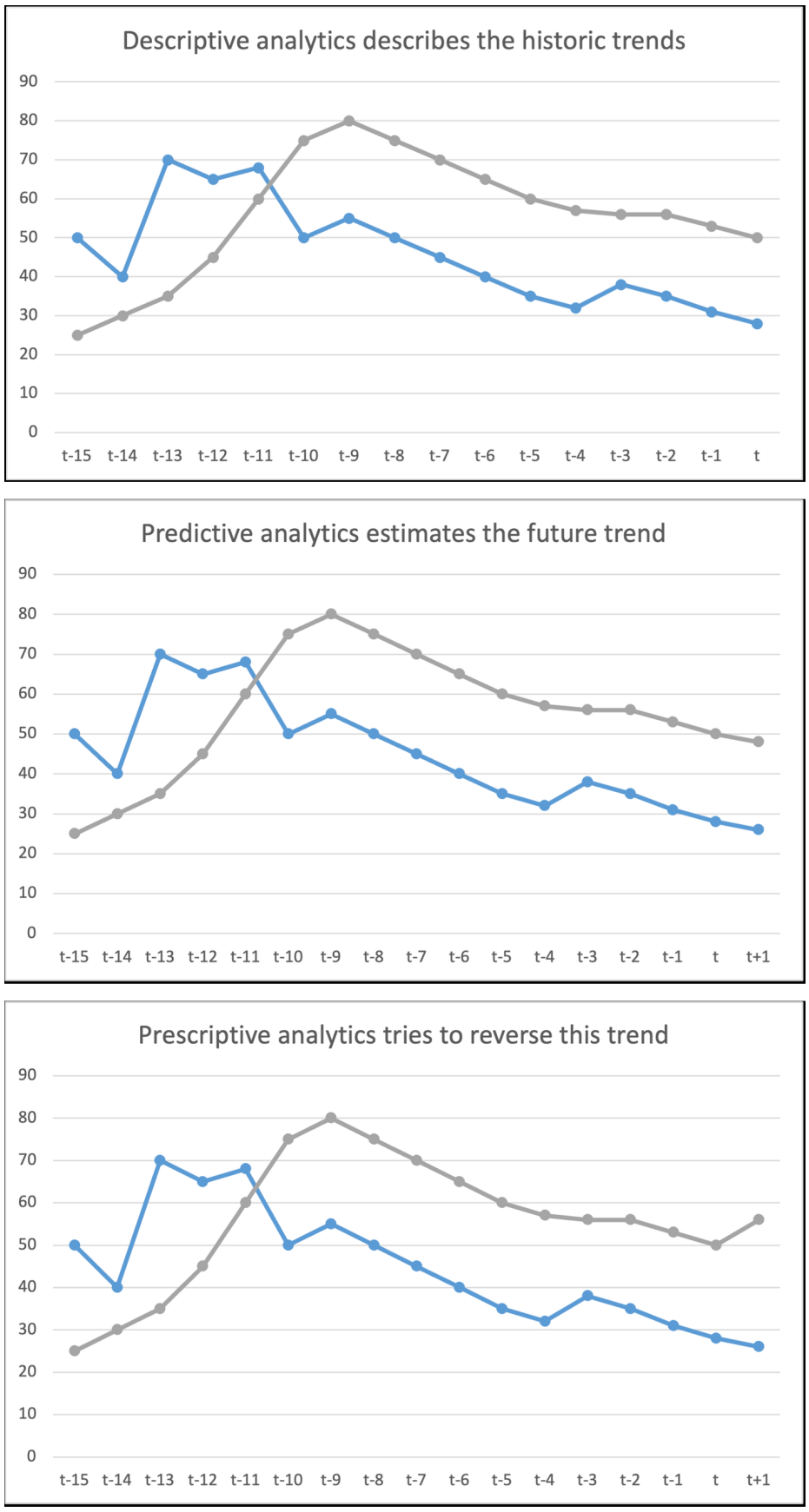


Figure 1.3: Sales trend of two products across three analytics stages. Top: descriptive analytics plots the historical trend through the present (t). Middle: predictive analytics extends that trend one period into the future (t+1), forecasting a continued decline for both products. Bottom: prescriptive analytics shows the outcome of an prescriptive action: one product's trend (grey line) is reversed at t+1, turning the forecasted decline back upward.

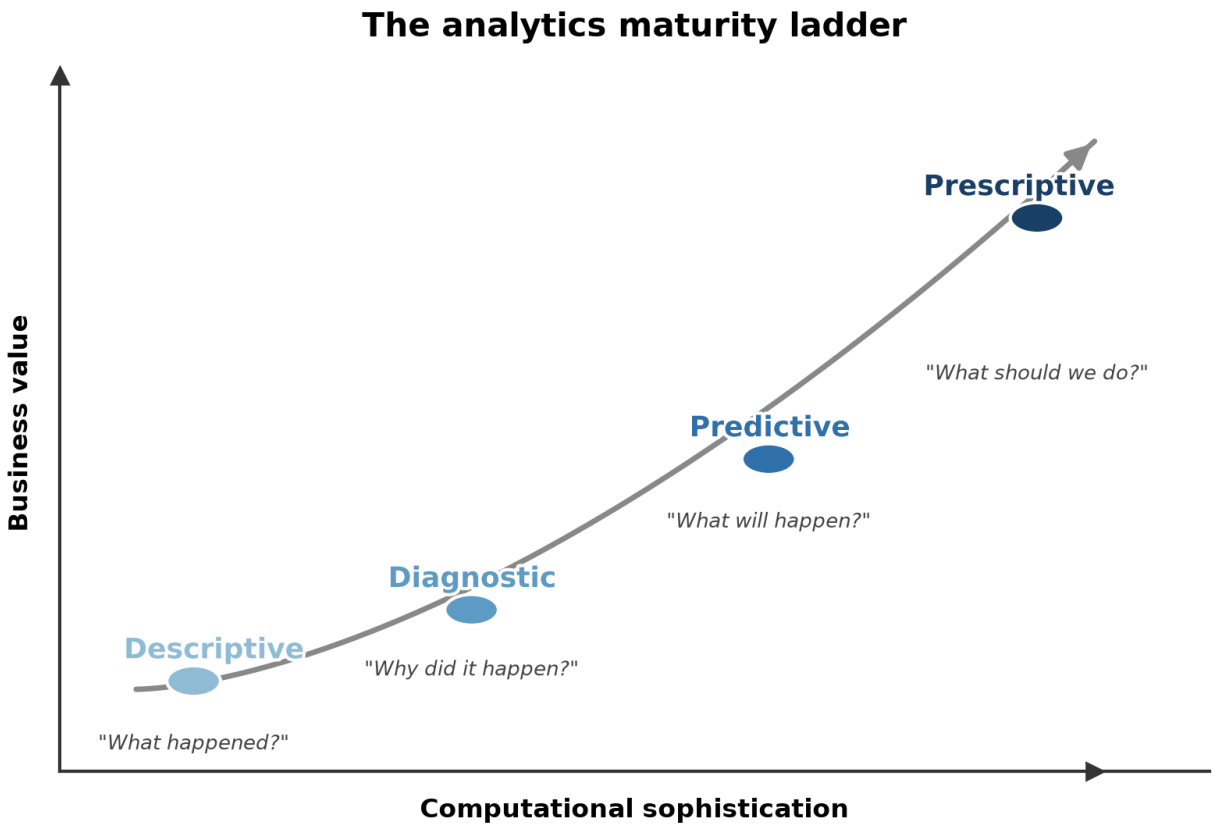


Figure 1.4: The analytics maturity ladder: descriptive, diagnostic, predictive, and prescriptive analytics plotted against computational sophistication and business value, each paired with the question it answers.

1.2.3 Machine learning vs. statistics

It's tempting to think that you don't need to know statistics to do machine learning. That's not quite right, since machine learning builds on statistical foundations. The difference between them matters for how you'll approach modeling problems throughout this book.

- **Machine learning** is interested in maximizing prediction accuracy (or, equivalently, minimizing error) on unseen data. It is generally *not* interested in explaining *why* something happens. Although interpretable machine learning is an active research area (De Bock et al. 2024), the objective function being optimized is still accuracy.
- **Statistics**, by contrast, is often more concerned with the *explanatory power* of a model: how do the independent variables influence the dependent variable, and by how much? This goal is *causal analysis* rather than pure prediction.

This distinction between prediction and causal analysis has practical consequences for how you build and evaluate a model. Take customer churn as a running example: a bank might build a model to *predict* which customers are about to leave, or a model to *explain* what actually *causes* customers to leave in the first place. Same data, sometimes even the same techniques, but the goal changes what you should worry about, along at least three dimensions:

1. **Omitted variables.** If you're *explaining* churn, leaving out a variable that both drives churn and correlates with a variable already in your model will distort your other coefficients. For example, you might conclude that high monthly charges cause churn, when really both are driven by something you failed to include, like contract type. If you're *predicting* churn, that same omission only matters if adding the missing variable would have made your predictions more accurate. A distorted coefficient is irrelevant if you were never trying to isolate one variable's true effect to begin with.
2. **Accuracy.** A *causal* churn model can have fairly low explanatory power and still be useful. Even if it only accounts for a small share of the variation in churn, you can still say with confidence that a specific driver (e.g., a recent price change) has a genuine effect, especially with a large enough sample. A *predictive* churn model doesn't get that amount of slack: if it can't accurately flag which customers are actually about to leave, it's useless for prioritizing retention offers, no matter how large the training data was.
3. **Multicollinearity.** Suppose tenure and number of products owned are highly correlated across your customers. For a *causal* model, that's a real problem since it becomes hard to say how much of the churn effect is really due to tenure vs product count, because the two are so intertwined. For a *predictive* model, that ambiguity barely matters: as long as the correlated variables jointly improve how well you flag at-risk customers, you don't need to cleanly separate out each one's individual contribution. We'll return to multicollinearity once we dive into data preprocessing later in this book.

1.2.4 Big data and generative AI

Two developments are worth calling out explicitly, since they shape almost everything else in this field.

Big data is usually characterized by four Vs: **volume** (the total amount of data), **velocity** (how fast the data is generated and needs to be processed), **variety** (how many different forms the data takes, like structured, text, image, or network data), and **veracity** (how uncertain or noisy the data is, especially when it comes from many different, unchecked sources). In this book, we propose adding a fifth V: **value**. We see this as one of the core Vs rather than an optional add-on: data that doesn't create value for a business, in any of its processes, simply isn't worth collecting, storing, or analyzing in the first place. Extracting that value is arguably the point of this entire book. Most routine business activity now generates data that can be stored, visualized, and analyzed, and as storage and compute keep getting cheaper, previously uneconomical analyses now become viable and create value.

Generative AI has pushed analytics from *describing and predicting* into *responding and creating*. Generative models produce text, images, audio, or video from a user-supplied prompt, after being trained on very large amounts of data. Whereas these models can respond by supplying the prompt without any training data, under the hood they're still built on a predictive learning task: a large language model is, at its core,

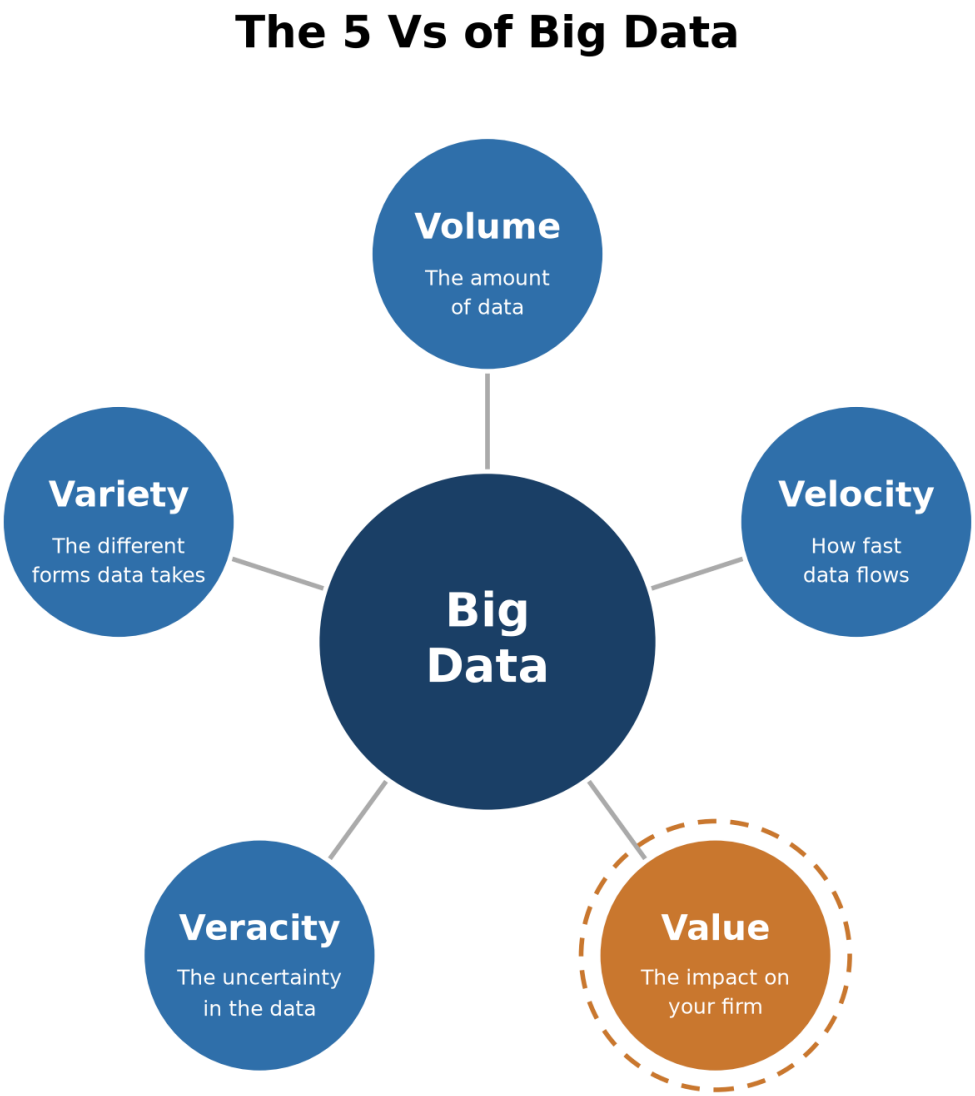


Figure 1.5: The 5 Vs of big data (volume, velocity, variety, veracity, and value) and their impact on your firm.

repeatedly predicting "given this context, what's the most likely next word?"; an image diffusion model is repeatedly predicting "given this noisy image, what does a slightly less noisy version look like?" In this book, you should use LLMs as a *learning aid*. For example, as a way to generate extra practice exercises once you already understand the underlying concept. Don't use them as a substitute for understanding the material yourself, since a tool that merely reproduces patterns it has seen before is a poor foundation on its own.

1.2.5 What makes a good analytical model

Not every model that achieves good accuracy on a test set is actually a *good* model for the business. Five requirements tend to come up again and again (Baesens 2014):

- **Relevance.** Does the model actually solve the business problem it was built for? A telecom's data science team might build a churn model that flags customers at-risk six months before they leave only to discover the retention budget can only fund targeting customers in their final 30 days. The model is accurate but irrelevant, because nobody agreed on the decision it actually needed to support before anyone started building it.
- **Performance.** Does the model generalize well to new, unseen cases and not just the data it was trained on? A credit-scoring model might reach 95% accuracy on the applicants it was trained on, and then barely beat a coin flip on next month's applicants. This concept is called *overfitting*: the model memorized its training data instead of learning a pattern that actually holds up. What "good performance" means varies by problem type (discrimination power for classification, cluster homogeneity for clustering). We'll cover the relevant metrics in the chapter on model evaluation.
- **Interpretability and justifiability.** Interpretability means you can understand the patterns the model has captured; justifiability means those patterns are consistent with prior business knowledge and intuition (Martens et al. 2011). Amazon famously scrapped an internal AI recruiting tool after discovering it had taught itself to downgrade résumés containing words like "women's" (e.g., "women's chess club captain"). This is a pattern that helped predict who'd historically been hired, but one no HR department could ever justify using for obvious reasons.[2] High-performing models (e.g., neural networks) are often black boxes; simpler, transparent models trade some performance for interpretability, and how much that trade-off matters depends on the setting. For example, hiring and credit decisions usually demand interpretable models, while a spam filter can happily stay a black box. However, thorough feature engineering and careful model evaluation can often produce interpretable models that are almost as accurate as black-box alternatives, so the trade-off isn't always inevitable.
- **Operational efficiency.** How much effort does it take to collect the data, prepare it, evaluate the model, put it into production and afterward, to monitor it, backtest it, and re-estimate it as conditions change? For example, a fraud-detection model that takes three days to score a transaction is worthless no matter how accurate it is, since the decision to approve or block a payment has to happen in milliseconds.
- **Regulatory compliance.** Especially for models using personal or sensitive data, does the model comply with data protection and AI regulation, like GDPR, the EU AI Act, or sector-specific rules? For example, a lending model that never uses race as an input can still be found discriminatory if it leans heavily on a variable like zip code that correlates strongly with it. In sum, following the letter of the law isn't enough if the outcome still discriminates in practice.

1.2.6 Data

1.2.6.1 Terminology and structure

Before working with data hands-on in the next chapters, it helps to fix some vocabulary about a data set, terms you'll see used more or less interchangeably throughout this book and elsewhere:

- **Independent variables**: also called features, predictors, inputs, attributes, or descriptors. These are the variables you use to predict something.

[2] Dastin, J. (2018). "Amazon scraps secret AI recruiting tool that showed bias against women." Reuters, October 10, 2018.

- **Dependent variable**: also called the target, outcome, output, response, or (in classification) the label or class. This is the thing you're trying to predict.
- **Data point**: a single observation, instance, or record.
- **Modeling**: also called model building, or training.
- **Prediction**: also called estimation, or inference.

Data itself comes in a few standard shapes:

- **Cross-sectional data** captures many units at a single point in time: one row per observation, useful for estimating a model that applies at "this moment."
- **Time series data** tracks a single variable over time at regular intervals: rainfall, stock prices, sales, energy prices.
- **Panel data** (also called longitudinal data) combines both: repeated observations of the same units (households, products, firms) across repeated time periods.

Regardless of which of these shapes your data takes, how you *arrange* it also matters. **Tidy data** (Wickham 2014) is the standard way to structure a dataset so that later analysis (descriptive statistics, visualizations, further processing) is easy: each variable forms a column, each observation forms a row, and each type of observational unit forms its own table. A "messy" table with one column per store and one row per year, versus a "tidy" table with one row per store-year combination, contains the same information, but only one of them is easy to filter, aggregate, and model. We'll come back to tidying data concretely once we start working with `pandas`.

Be aware that not all data is tabular. Text, images, audio, and networks are increasingly common data sources, and they require dedicated modeling approaches, though generative AI has made working with some of them (especially text and images) far more approachable than it used to be. This book focuses on tabular data, since it remains the form business and economics students encounter most, but the same tidying and preparation mindset carries over once you move to other data types. And no dataset, tabular or not, arrives clean: missing values, outliers, and messy relationships between variables are the norm, which is exactly why data preparation and preprocessing gets its own chapter later in this book.

1.2.6.2 Data samples are always biased

Every data sample you work with is biased in some way, and it's worth internalizing this before you trust a model's output. The classic illustration in this case is **survivorship bias**. In WWII engineers were studying where returning bomber aircrafts had been hit in order to know where to add armor. The natural instinct is to reinforce the areas with the most bullet holes. However, those planes *made it back*! The planes that didn't survive were hit somewhere else entirely, and that data was never collected. The correct conclusion was the opposite of the intuitive one: reinforce the spots with *no* damage on the survivors, since those are the hits that bring a plane down.

The business analogy is straightforward. If you build a model of "what makes a store successful" using only data from your current top-performing stores, you'll miss out on the factors that made other locations fail and your conclusions will be biased toward the survivors. Because any sample is collected at a particular point in time, under a particular strategy and economic climate, it's worth asking not just "is this sample biased," but "biased in what way, and does that bias matter for the decision I'm about to make?" For example, a model trained on pre-COVID-19 retail data might be biased toward in-store shopping behavior, and a model trained on 2020-2021 data might be biased toward online shopping behavior. If you use either of those models to make decisions about the post-pandemic retail landscape, you'll likely make mistakes.

1.2.7 Supervised and unsupervised learning

Machine learning methods are split into two broad families, depending on whether you have a dependent variable to predict.

Supervised learning aims to approximate reality with a function: given inputs, infer a reliable pattern that maps them onto a known output. This inferred function is designed in a way that generalizes to new, unseen

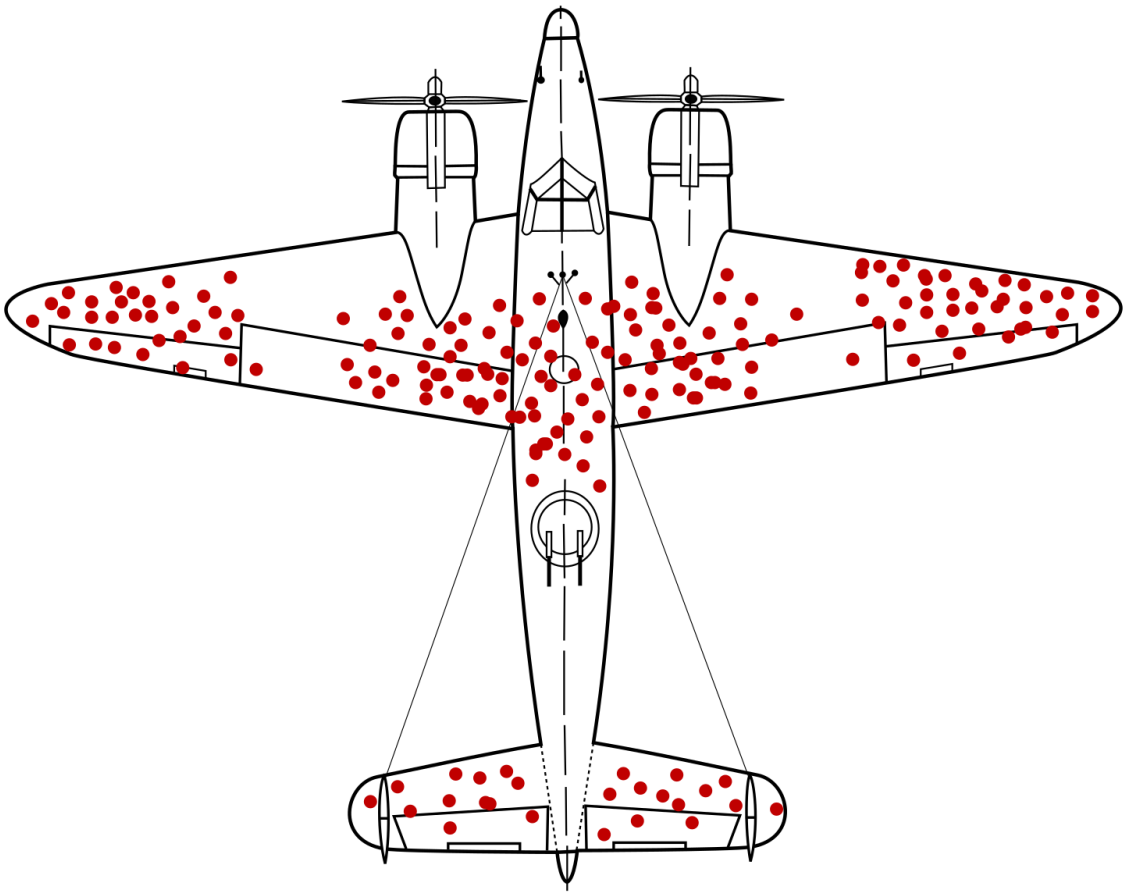


Figure 1.6: WWII bomber survivorship bias: hypothetical damage pattern on a returning aircraft. The areas *without* red dots are the ones that actually bring a plane down, since planes hit there never made it back. Source: Martin Grandjean, McGeddon, and the US Air Force, licensed under CC BY-SA 4.0.

cases. This requires a response variable, and splits further by the type of that response:

- **Classification**, for a categorical or discrete response (binary, multiclass, multilabel, or ordinal).
- **Regression**, for a continuous or numerical response (predicting either an absolute value or a change (delta) in value).

Supervised models are also commonly grouped by how they represent the underlying relationship:

- **Parametric models** first assume a functional form for the relationship (e.g., linear), then estimate a fixed number of parameters for that form. This is much easier to estimate, but performs poorly if the assumed functional form doesn't match reality.
- **Nonparametric models** make no such assumption and let the data determine the shape of the relationship. This is far more flexible and can be more accurate, but typically needs substantially more data. The danger is that it can end up fitting noise rather than signal ("too accurate" on the training data, at the expense of generalization on the test data).

Unsupervised learning has no dependent variable, only independent variables, and is correspondingly more subjective to evaluate. Its goals include:

- **Clustering**: finding homogeneous subgroups in the data.
- **Dimensionality reduction**: compressing many variables down to two or three for visualization.
- **Association rule mining**: finding "if antecedent, then consequent" rules that describe co-occurring patterns (e.g., market basket analysis).
- **Anomaly detection**: finding outliers or anomalies that don't fit the rest of the data.

1.3 The data analytics process: CRISP-DM

Producing a reliable, robust analytical model isn't something you improvise: practitioners follow a fixed framework that lets a project be carefully defined, organized, and structured. The two best-known frameworks are KDD or Knowledge Discovery in Databases (Fayyad et al. 1996) and, more widely used today, **CRISP-DM** or the Cross-Industry Standard Process for Data Mining (Chapman et al. 2000).

CRISP-DM organizes an analytics project into six phases:

1. **Business understanding**: determine the business objectives and translate them into data mining goals.
2. **Data understanding**: describe and explore the available data, and identify data quality issues.
3. **Data preparation**: select, clean, construct, aggregate, and integrate data into a single basetable containing every variable you'll model with.
4. **Modeling**: set up a proper test design, select an appropriate modeling technique, and build and interpret the model.
5. **Evaluation**: evaluate the model using appropriate performance metrics.
6. **Deployment**: put the model into production.

CRISP-DM is considered the gold standard process because it's well documented, widely adopted, and more clearly organized than competing processes. Two things are worth knowing before you start your own first project. First, data understanding and preparation are consistently the most time-consuming steps: often cited as taking up to 80% of total project time! Second, the process is *iterative*, not linear. You will regularly find yourself going back a step (or several) once something you learn in modeling changes what you thought you understood about the data.

This same six-phase structure maps directly onto how the rest of this book is organized: the coming chapters on data understanding, data preparation, model evaluation, and modeling each correspond to one phase of CRISP-DM, applied in a business setting.

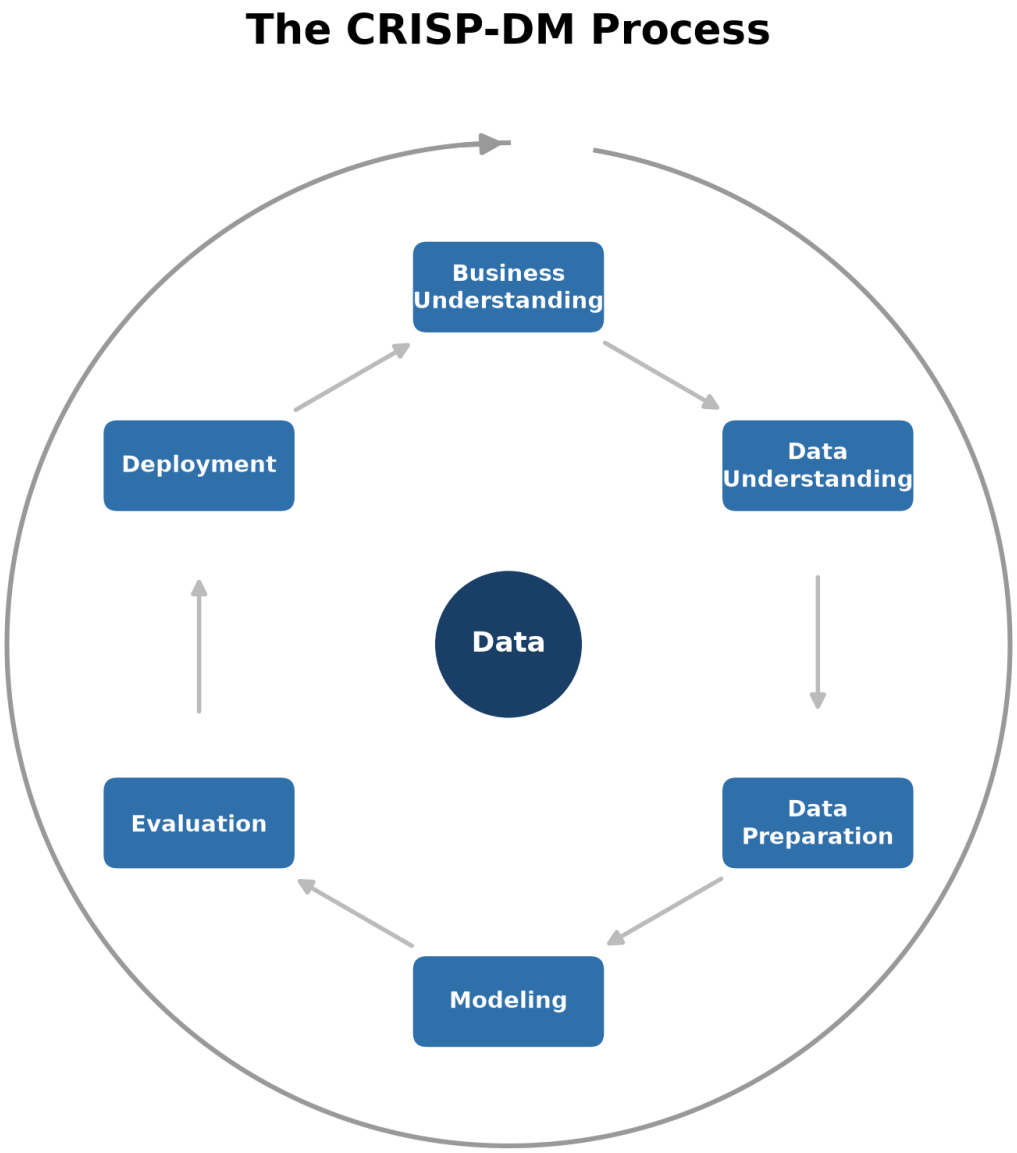


Figure 1.7: The CRISP-DM process: business understanding, data understanding, data preparation, modeling, evaluation, and deployment, cycling back around a central data store.

1.4 Where data analytics shows up

Data analytics is genuinely everywhere in modern business, not just in the usual tech companies, and well beyond the handful of examples already mentioned in this chapter's opening. A streaming platform's next-series recommendations, the dynamically adjusted fare prices on a ride-hailing app, a spam filter quietly sorting your inbox, an automatic reorder triggered the moment a warehouse's stock dips below a threshold, a warning signal when the condition of an industrial asset deteriorates: all of these are analytics applications running in the background of ordinary business operations.

More systematically, applications span nearly every business function:

Marketing	Finance	Manufacturing & Operations	Supply Chain & Logistics	Public Sector	Digital & Other
Response and uplift modeling	Credit scoring	Predictive maintenance	Demand forecasting	Tax fraud detection	Web and social media analytics
Customer churn	Fraud detection	Condition monitoring	Route and network optimization	Social benefit fraud detection	A/B and multivariate testing
Customer segmentation	Market risk modeling	Spare-parts inventory optimization	Warehouse and inventory management	Money laundering detection	Misinformation and rumor detection
Market basket analysis	Operational risk modeling	Personnel and shift scheduling		Security and terrorism risk detection	Hospitality and tourism analytics
Recommender systems		Quality control and defect detection			HR analytics
					Healthcare analytics
					Learning analytics

Two applications are worth walking through in a bit more detail, since they recur throughout this book.

Customer churn prediction. Analytical customer relationship management (CRM) uses modeling techniques on customer characteristics and behavior to support cost-effective, targeted marketing for acquisition modeling, up-sell and cross-sell modeling, customer lifetime value estimation, and churn modeling. Churn, defined as the loss of a customer, matters because acquiring a new customer typically costs five to six times more than retaining an existing one (Bhattacharya 1998), and established customers tend to be more profitable and more likely to refer others (Van den Poel and Larivière 2004). Churn prediction models assign each customer an individual propensity to churn, so the company can proactively target the highest-risk customers with a retention offer *before* they leave, rather than reactively after the fact. Useful features typically include demographic variables, relationship variables (tenure, number of products), and RFM variables (Recency, Frequency, and Monetary value) of past purchases. Because comprehensibility lets business experts sanity-check a churn model and understand what's actually driving churn, logistic regression is often treated as the gold standard here: easy to interpret, reasonably accurate, and well understood in both research and practice.

Credit risk modeling. Credit scoring is concerned with assessing financial risk and informing lending decisions: a credit score is a model-based estimate of the probability that a borrower will default and fail to meet their financial obligations (Lessmann et al. 2015). Consumer loans and mortgages often exceed corporate lending in total volume at the country level, which is exactly why credit scoring has been called "one of the most successful applications of statistics and OR in industry" (Crook et al. 2007). It's also one of the most mature and heavily benchmarked applications of predictive analytics in business. Despite a steady stream of

more complex algorithms, logistic regression has remained a surprisingly hard baseline to beat in practice (Lessmann et al. 2015). There are two main types of credit scoring models: application scorecards and behavioral scorecards. *Application scorecards* score new credit applications for creditworthiness, typically built from two snapshots; application data and credit bureau data. Both snapshots are taken at loan origination and matched against whether the applicant actually defaulted 12 or 18 months later. *Behavioral scorecards* go further, scoring the existing customer portfolio using both application data and behavioral trends (account balance movement, delinquency history, credit limit changes), and tend to outperform application scorecards since they have more (and more current) information to work with.

1.5 Why Python

The data, AI, analytics, and machine learning tooling landscape is enormous, ranging from off-the-shelf business-intelligence (BI) tools optimized for point-and-click visual analysis on one end, to full programming languages on the other. As a business student, you'll mostly work with the programming-language end of that spectrum, since it's what gives you the flexibility to go beyond what an off-the-shelf BI tool can do.

Among programming languages, Python and R have long been the two dominant choices for data science, but Python has clearly emerged as the winner in recent years when measured by job postings, community size, and package ecosystem maturity. Python wasn't originally designed as a data science language, but it became the standard one for several reasons:

- its syntax is logical and comparatively easy to learn;
- it's genuinely general-purpose, useful for everything from short scripts to full enterprise applications;
- its ecosystem includes a handful of foundational packages this book relies on directly (`numpy`, `pandas`, `matplotlib`, `scikit-learn`, among others);
- and it has an enormous community across Stack Overflow, GitHub, and beyond, so help is rarely far away.

One more note before you start writing Python yourself: large language models (LLMs) are useful coding *assistants* for debugging, for explaining unfamiliar code, for generating extra practice exercises, but they are not a substitute for understanding the fundamentals. An LLM doesn't reason about your code from first principles; it produces output based on patterns it has seen before, and that output is often subtly wrong. The proper order is to learn the basics and write the code yourself first, and use an LLM to accelerate that process second and not the other way around.

1.6 How this book is organized

The chapters that follow mirror the CRISP-DM process introduced above, applied consistently to a single running business case so techniques build on each other instead of feeling like disconnected exercises:

- **Python for Data Analytics**: the Python fundamentals used throughout the rest of the book.
- **Data Understanding**: manipulating and making sense of raw datasets.
- **Data Exploration**: visualizing data to find patterns, relationships, and quality issues. In CRISP-DM terms, this is part of the data understanding phase, but we separate it out because it's a distinct skill set that deserves its own chapter.
- **Data Preparation**: cleaning and transforming data so it's ready for modeling.
- **Model Evaluation**: how to set up a fair experiment and measure whether a model is any good, *before* worrying about which model to use.
- **Modeling**: building and interpreting linear and logistic regression models.

If you're setting up Python and an IDE for the first time, see the installation guide in the appendix.

Chapter 2

Python for Data Analytics

This chapter reviews the fundamental building blocks of Python that the rest of the book relies on: variables, collections, control flow, reading and writing files, functions, modules, NumPy arrays, and finally `pandas`, the core data manipulation library within this book. If you already know Python well, treat this chapter as a quick reference; if you're new to the language, work through every code cell yourself before moving on to the running business case in the next chapter.

> **Practice with generative AI**
>
> If you want to exercise your Python skills even more, consider asking a Large Language Model (LLM) to create additional practice exercises tailored to the concepts in this chapter. For example, you could ask it to show you how to select specific rows from a pandas DataFrame or filter a NumPy array based on a condition, then implement and test them yourself. This is a useful way to learn a new programming language and a proper way of using generative AI in your learning experience.

Throughout, we use small business-flavored examples (e.g., customers and newspaper subscriptions data) rather than simulated toy examples. In that way, the syntax remains close to the context you'll actually use it in.

> **Using Google Colab**
>
> If you're running these examples in Google Colab rather than locally, you first need to mount your Google Drive and point Python at the folder containing your notebook, since Colab's default working directory isn't the same as where your notebook is saved.

```
# Code to access drive
from google.colab import drive
drive.mount('/content/drive')
```

```
# Set working directory to proper location
import os
os.chdir("/content/drive/MyDrive/Colab Notebooks/Data Analytics/2026/notebooks_student")
```

> If you're working locally with VS Code (see the installation guide in the appendix), your working directory is already the folder containing your notebook, so you can skip both cells above.

2.1 Variables

A variable is simply a name that points to a value in memory. Python figures out a variable's type automatically from the value you assign to it: you never have to declare it up front.

2.1.1 Numeric Variables

Python has two built-in numeric types you'll use constantly: `int` for whole numbers and `float` for decimal numbers.

2.1.1.1 Definition

A numeric variable's type is inferred automatically from how you write the value: no decimal point makes it an `int`, while a decimal point makes it a `float`.

```
# assign integer variable
x = 3
print(type(x))
```

```
<class 'int'>
```

```
# assign float variable
x = 3.0
print(type(x))
```

```
<class 'float'>
```

2.1.1.2 Arithmetic operators

The standard arithmetic operators work as you'd expect, including `%` for the remainder (modulus), `//` for floor division, and `**` for exponentiation.

```
# arithmetic operators
x = 5.
y = 3.
print("addition: ", x + y)
print("subtraction: ", x - y)
print("multiplication: ", x * y)
print("division: ", x / y)
print("modulus: ", x % y)
print("floor division: ", x // y)
print("exponent: ", x ** y)
```

```
addition:  8.0
subtraction:  2.0
multiplication:  15.0
division:  1.6666666666666667
modulus:  2.0
floor division:  1.0
exponent:  125.0
```

Division behaves the same whether the operands are `int` or `float`: since Python 3, `/` always returns a `float`, so you no longer need to manually cast one of the operands.

```
# integer division (no issues from python 3.X onwards)
x = 5
y = 3
print(x / y)
```

```
1.6666666666666667
```

```
# float division (no need to type cast from python 3.X onwards)
x = 5.
y = 3.
print(x / y)
```

```
1.6666666666666667
```

2.1.1.3 Type conversion

You can convert explicitly between numeric types using `float()` and `int()`.

```
# convert integer to float
x = 3
y = float(x)
print(y)
print(type(y))
```

```
3.0
<class 'float'>
```

```
# convert float to integer
x = 3.
y = int(x)
print(y)
print(type(y))
```

```
3
<class 'int'>
```

2.1.1.4 Comparison operators

Comparison operators return a `bool` (`True`/`False`), and Python compares across numeric types transparently: an `int` and a `float` with the same value are considered equal.

```
# comparison operators
x = 3
y = 5
print("greater than: ", x > y)
print("greater than or equal to: ", x >= y)
print("less than: ", x < y)
print("less than or equal to: ", x <= y)
print("equal to: ", x == y)
print("not equal to: ", x != y)
```

```
greater than:  False
greater than or equal to:  False
less than:  True
less than or equal to:  True
equal to:  False
not equal to:  True
```

```
x = 3
y = 3.
x == y
```

```
True
```

2.1.2 Boolean Variables

2.1.2.1 Definition

A `bool` variable holds one of exactly two values: `True` or `False`. They're what conditionals and loops branch on.

```
x = True
y = False
print(type(x))
```

```
<class 'bool'>
```

2.1.2.2 Boolean operators

Python's boolean operators, `and`, `or`, and `not`, combine or invert `True`/`False` values, exactly like in everyday logic.

```
x = True
y = False
print("and: ", x and y)
print("or: ", x or y)
print("not: ", not x)
```

```
and:  False
or:  True
not:  False
```

2.1.2.3 Conversion

Booleans are really a special case of integers under the hood: `False` is `0` and `True` is `1`. Conversely, `0` converts to `False` and any other number converts to `True`.

```
# convert bool to int
print(int(False))
print(int(True))
```

```
0
1
```

```
# convert int to bool
print(bool(0))
print(bool(1))
```

```
False
True
```

2.1.3 String Variables

2.1.3.1 Definition

Strings hold text and are indexed like sequences: you can pull out individual characters by position.

```
# define string variable
x = "hello"
print(type(x))
```

```
<class 'str'>
```

```
# get the third character of the string
x = "hello"
y = x[3]
print(y)
```

```
l
```

2.1.3.2 String methods

Strings come with a large set of built-in methods for cleaning and transforming text: useful for anything from normalizing customer names to parsing raw text files, which we'll do later in this chapter.

```
# convert to upper case
"hello".upper()
```

```
'HELLO'
```

```
# convert to lower case
"HELLO".lower()
```

```
'hello'
```

```
# replace substring
"hello".replace("llo", "ro")
```

```
'hero'
```

```
# split string
"hello-world".split("-")
```

```
['hello', 'world']
```

```
# string concatenation
"hello" + " " + "world"
```

```
'hello world'
```

```
# example string methods
x = "hello world "
print("capitalize: ", x.capitalize())
print("index: ", x.index("world"))
print("isalpha: ", x.isalpha())
print("isdigit: ", x.isdigit())
print("islower: ", x.islower())
print("isupper: ", x.isupper())
print("lower: ", x.lower())
print("replace: ", x.replace("world", "students"))
print("split: ", x.split(" "))
print("strip: ", x.strip())
print("upper: ", x.upper())
```

```
capitalize:  Hello world
index:  6
isalpha:  False
isdigit:  False
islower:  True
isupper:  False
lower:  hello world
replace:  hello students
```

```
split:  ['hello', 'world', '']
strip:  hello world
upper:  HELLO WORLD
```

2.1.3.3 Conversion

Just like numbers, strings can be converted to and from other types.

```
# convert int to str
print(str(3))
# convert float to str
print(str(54.3))
# convert bool to str
print(str(True))
```

```
3
54.3
True
```

```
# convert str to int
print(int("75"))
# convert str to float
print(float("45.2"))
# convert str to bool
print(bool("True"))
```

```
75
45.2
True
```

Not every string converts cleanly, though. `int()` only accepts strings that look like whole numbers, so the following raises a `ValueError`: shown here deliberately, since you'll run into this exact error the first time you try to parse a decimal string as an integer.

```
# try to convert str to int
int("4.5")
```

```
ValueError: invalid literal for int() with base 10: '4.5'
---------------------------------------------------------------------------
ValueError                                Traceback (most recent call last)
Cell In[24], line 2
      1 # try to convert str to int
----> 2 int("4.5")
ValueError: invalid literal for int() with base 10: '4.5'
```

2.1.4 None Variables

2.1.4.1 Definition

`None` is Python's way of representing "no value." You'll see it used as a placeholder for missing data or as the default return value of a function that doesn't explicitly return anything.

```
x = None
print(type(x))
```

```
<class 'NoneType'>
```

2.2 Collections

Collections let you group multiple values into a single variable. Python has four built-in collection types, each with different trade-offs around ordering, mutability, and how you look values up.

2.2.1 Lists

2.2.1.1 Definition

A list is an ordered, mutable sequence of values: the values don't have to be the same type.

```
# assign a list with string variables
cities = ["Ghent", "Brussels", "Antwerp"]

# assign a list with numeric variables
scores = [13, 19, 18, 19, 11, 9]

# assign a list with numeric and string variables
names_and_wages = ["tim", 3003, "john", 1600, "sophie", 1900]

print(type(cities))
```

```
<class 'list'>
```

2.2.1.2 Indexing

Lists are indexed starting from `0`, and negative indices count backward from the end. A `start:stop` slice returns a sub-list.

```
# get index of element
cities = ["Ghent", "Brussels", "Antwerp"]
print(cities[1])
```

```
Brussels
```

```
# get index of first element
cities = ["Ghent", "Brussels", "Antwerp"]
print(cities[0])
```

```
Ghent
```

```
# get index of last element
cities = ["Ghent", "Brussels", "Antwerp"]
print(cities[-1])
```

```
Antwerp
```

```
# get index of second last element
cities = ["Ghent", "Brussels", "Antwerp"]
print(cities[-2])
```

```
Brussels
```

```
# get index of first two elements
cities = ["Ghent", "Brussels", "Antwerp"]
print(cities[0:2])
```

```
['Ghent', 'Brussels']
```

Strings support the same indexing and slicing syntax as lists, since a string is really just a sequence of characters.

```
# define a string
string = "hello world"

# get the character at index 6
print(string[6])
```

```
w
```

2.2.1.3 Operations

Because lists are mutable, you can change, add, and remove elements in place rather than creating a new list each time.

```
# change element
cities = ["Ghent", "Brussels", "Antwerp"]
cities[1] = "Louvain"
print(cities)
```

```
['Ghent', 'Louvain', 'Antwerp']
```

```
# number of elements
cities = ["Ghent", "Brussels", "Antwerp"]
print(len(cities))
```

```
3
```

```
# append elements
cities = ["Ghent", "Brussels", "Antwerp"]
cities.append("Louvain")
cities.append("Bruges")
print(cities)
```

```
['Ghent', 'Brussels', 'Antwerp', 'Louvain', 'Bruges']
```

```
# insert elements
cities = ["Ghent", "Brussels", "Antwerp"]
cities.insert(0, "Louvain")
cities.insert(2, "Bruges")
print(cities)
```

```
['Louvain', 'Ghent', 'Bruges', 'Brussels', 'Antwerp']
```

```
# extend list
cities_1 = ["Ghent", "Brussels", "Antwerp"]
cities_2 = ["Leuven", "Bruges"]
cities_1.extend(cities_2)
print(cities_1)
```

```
['Ghent', 'Brussels', 'Antwerp', 'Leuven', 'Bruges']
```

```
# remove element from list
cities = ["Ghent", "Brussels", "Antwerp"]
cities.remove("Ghent")
print(cities)
```

```
['Brussels', 'Antwerp']
```

```
# remove element at index from list
cities = ["Ghent", "Brussels", "Antwerp"]
cities.pop(1)
print(cities)
```

```
['Ghent', 'Antwerp']
```

```
# get index of element
cities = ["Ghent", "Brussels", "Antwerp"]
idx = cities.index("Brussels")
print(idx)
```

```
1
```

2.2.1.4 Nested lists

Lists can contain other lists. You index into them one level at a time.

```
# nested lists
products = [["samsung", "apple", "nokia"],
            ["bmw", "mercedes", "audi", "toyota", "renault"],
            ["apple", "banana"]]

print(products[1][2])
```

```
audi
```

2.2.2 Dictionaries

2.2.2.1 Definition

A dictionary stores `key: value` pairs, where each key maps directly to its value: no scanning through positions like with a list. This is the collection you'll reach for whenever you need to look things up by name or ID, such as a customer's score or a product's price.

```
# assigning a dictionary
students_and_scores = {"tom": 13, "sarah": 20, "john": 9}
print(type(students_and_scores))
```

```
<class 'dict'>
```

2.2.2.2 Indexing

You look up a value by its key in square brackets, not by position, since dictionaries aren't ordered by index the way lists are.

```
# get the score of specific student
students_and_scores = {"tom": 13, "sarah": 20, "john": 9}
print(students_and_scores["sarah"])
```

```
20
```

2.2.2.3 Operations

Dictionaries are mutable: you can change a value, add or remove a key, and inspect all the keys or values at once.

```
# change the value of specific item
students_and_scores = {"tom": 13, "sarah": 20, "john": 9}
```

```
students_and_scores["sarah"] = 4
print(students_and_scores)
```

```
{'tom': 13, 'sarah': 4, 'john': 9}
```

```
# get number of items in dictionary
students_and_scores = {"tom": 13, "sarah": 20, "john": 9}
print(len(students_and_scores))
```

```
3
```

```
# add new item to dictionary
students_and_scores = {"tom": 13, "sarah": 20, "john": 9}
students_and_scores["lisa"] = 15
print(students_and_scores)
```

```
{'tom': 13, 'sarah': 20, 'john': 9, 'lisa': 15}
```

```
# remove item from dictionary
students_and_scores = {"tom": 13, "sarah": 20, "john": 9}
students_and_scores.pop("john")
print(students_and_scores)
```

```
{'tom': 13, 'sarah': 20}
```

```
# get the keys of the dictionary
students_and_scores = {"tom": 13, "sarah": 20, "john": 9}
keys = students_and_scores.keys()
print(keys)
```

```
dict_keys(['tom', 'sarah', 'john'])
```

```
# get the values of the dictionary
students_and_scores = {"tom": 13, "sarah": 20, "john": 9}
values = students_and_scores.values()
print(values)
```

```
dict_values([13, 20, 9])
```

2.2.2.4 Nested dictionaries

Dictionaries can hold other dictionaries as values, which is a natural way to represent structured records: think of each key as a customer ID and each value as that customer's full profile.

```
# nested dictionary
customer_info = {"tom": {"city": "Brussels", "email": "tom@hotmail.com", "age": 24},
                 "sarah": {"city": "Ghent", "email": "sarah@gmail.com", "age": 32},
                 "jack": {"city": "Ghent", "email": "jackjack@gmail.com", "age": 12}}

print(customer_info["tom"]["email"])
```

```
tom@hotmail.com
```

2.2.3 Tuples

2.2.3.1 Definition

A tuple looks like a list but is immutable: once created, you cannot change, add, or remove elements. Tuples are commonly used for values that shouldn't change after creation, like a fixed record or a coordinate pair.

```
# create a tuple with different data types
my_tuple = ("Ghent", 13, 4.20, ["el1", "el2"], {"key_1": 1.0, "key_2": 2.0})

print(type(my_tuple))
```

```
<class 'tuple'>
```

2.2.3.2 Indexing

Indexing works the same as for lists.

```
# retrieve the second element
my_tuple = ("Ghent", 13, 4.20, ["el1", "el2"], {"key_1": 1.0, "key_2": 2.0})
print(my_tuple[1])
```

```
13
```

```
# retrieve the last element
my_tuple = ("Ghent", 13, 4.20, ["el1", "el2"], {"key_1": 1.0, "key_2": 2.0})
print(my_tuple[-1])
```

```
{'key_1': 1.0, 'key_2': 2.0}
```

2.2.3.3 Operations

Because tuples are immutable, most of these operations only read from the tuple rather than changing it: counting elements, finding an index, joining two tuples together, or comparing them.

```
# create a tuple (NOTE: once created, you cannot change any element)
my_tuple = ("Ghent", 13, 13, 4.20, ["el1", "el2"], {"key_1": 1.0, "key_2": 2.0})
```

```
# retrieve number of elements
print(len(my_tuple))
```

```
6
```

```
# retrieve the index of the list in our tuple
print(my_tuple.index(["el1", "el2"]))
```

```
4
```

```
# retrieve the index of the list in our tuple
print(my_tuple.count(13))
```

```
2
```

```
# join two tuples
first_tuple = ("Ghent", "Brussels")
second_tuple = ("Antwerp", "Leuven")
new_tuple = first_tuple + second_tuple # alternative notation: my_tuple += my_tuple2
print(new_tuple)
```

```
('Ghent', 'Brussels', 'Antwerp', 'Leuven')
```

```
# compare two tuples
first_tuple = ("Ghent", "Brussels")
second_tuple = ("Antwerp", "Leuven")
print(first_tuple == second_tuple)
```

```
False
```

```
# compare two tuples
first_tuple = ("Ghent", "Antwerp")
second_tuple = ("Antwerp", "Leuven")
print(first_tuple < second_tuple)
```

```
False
```

```
# other comparisons (NOTE: the comparison is done element by element!)
print((2, 3) == (2, 3))

print((5, 6, 7) < (7, 5, 6))

print((5, 6, 7) > (7, 5, 6))

print((4, 3, 2) <= (4.0, 3.0, 2.0))
```

```
True
True
False
True
```

Focus on the four numeric comparisons in the code cell directly above, not the earlier city-name examples: they compare tuples of numbers using the standard comparison operators. Python runs an **element-by-element comparison**. So, for example:

- In the first expression above, Python compares the 2 in the left tuple and the 2 in the right one. They're equal, and Python continues by comparing 3 and 3 to conclude that both tuples are equal.
- In the second expression, Python compares 5 and 7. They're different. Because 5 is less than 7, this *individual comparison* determines the result of the entire expression, and you get `True` as a result. If we perform the opposite action, using `>` instead of `<`, 5 is not greater than 7, so we get `False` as a result.
- In the last expression, both tuples contain the same values. Because equality is included in the comparison, you get `True` as a result. **Note** that it handles the fact that one tuple uses `integer` and the other tuple uses `float` notation!

2.2.3.4 Nested Tuples

Just like nested lists, you index into a nested tuple one level at a time.

```
# nested tuples
nested_tuple = (("Ghent", 6), ("Brussels", 9))
print(nested_tuple[0][1])
```

```
6
```

2.3 Conditionals

Conditionals let your code take different paths depending on the data it sees, for example, sending a different message to a customer depending on how they prefer to be contacted.

2.3.1 If-Else

2.3.1.1 Definition

An `if/elif/else` block runs exactly one of its branches: whichever one is reached first with a condition that evaluates to `True`.

```
# if-elif-else example
number = 3

if number > 0:

    print("this is a positive number")

elif number == 0:

    print("this number equals zero")

else:
    print("this is a negative number")
```

```
this is a positive number
```

2.3.1.2 Nested if-else

You can nest conditionals inside one another to check several conditions in sequence. Here, it first checks whether a customer exists at all, and only then checks their contact preferences.

```
# nested if-else example
customers = {"Tom": {"mail": True, "phone": False},
             "Julie": {"mail": False, "phone": False},
             "Jasper": {"mail": True, "phone": True}}

contact_customer = "tom"

# check if the customer we want to contact is in our customer list
if contact_customer in customers.keys():

    # check if customer wants to be contacted via mail
    if customers[contact_customer]["mail"] == True:
        print("customer wants to be contacted via mail")
    else:
        print("customer wants not to be contacted via mail")

    # check if customer wants to be contacted via phone
    if customers[contact_customer]["phone"] == True:
        print("customer wants to be contacted via phone")
    else:
        print("customer wants not to be contacted via phone")

else:

    print("customer not in customer list!")
```

```
customer not in customer list!
```

Notice the output here: `contact_customer` is `"tom"` (lowercase) but the dictionary key is `"Tom"` (capitalized), so the membership check fails and we fall into the `else` branch. The take-away is that key lookups in Python are case-sensitive, and mismatched casing is a common source of silent bugs (and frustrations) when matching records across datasets.

2.3.2 For

2.3.2.1 Looping through a list

A `for` loop repeats a block of code once per item in a sequence: a range of numbers, a list, or the keys of a dictionary.

```
# looping through a range of numbers
for i in range(0, 3):
    print(i)
```

```
0
1
2
```

```
# looping through a list
customers = ["tom", "sarah", "julie"]
for cust in customers:
    print(cust)
```

```
tom
sarah
julie
```

2.3.2.2 Looping through a dictionary

Looping over a dictionary by default gives you its keys; `.items()` gives you both the key and its value in one go, which is usually what you want.

```
# looping through keys of a dictionary
product_prices = {"xs45321": 420, "xs5620": 690, "xs0909": 2503}

for key in product_prices.keys():
    print(key, ":", product_prices[key])
```

```
xs45321 : 420
xs5620 : 690
xs0909 : 2503
```

```
# looping through items of a dictionary
product_prices = {"xs45321": 420, "xs5620": 690, "xs0909": 2503}

for key, value in product_prices.items():
    print(key, ":", value)
```

```
xs45321 : 420
xs5620 : 690
xs0909 : 2503
```

2.3.3 While

2.3.3.1 Definition

Unlike a `for` loop, a `while` loop keeps running as long as a condition stays `True`. This is useful when you don't know in advance how many iterations you'll need.

```
# example while loop
counter = 0
```

```
while counter < 4:

    print("counter: ", counter)
    counter = counter + 1
```

```
counter:  0
counter:  1
counter:  2
counter:  3
```

2.3.3.2 Infinite while loop

If the condition never becomes `False`, a `while` loop runs forever. The cell below is commented out on purpose: don't uncomment and run it as-is, since it would never stop!

```
# example infinite while loop: Do not run this!!!
#counter = 0

#while counter >= 0:

    #counter = counter + 1
```

2.3.3.3 Break and continue

`break` exits the loop immediately, while `continue` skips the rest of the current iteration and moves on to the next one.

```
# break
counter = 0

while counter < 6:

    print("counter: ", counter)
    counter = counter + 1

    if counter == 3:
        break
```

```
counter:  0
counter:  1
counter:  2
```

```
# continue
counter = 0

while counter < 6:

    counter = counter + 1

    if counter == 3:
        continue

    print("counter: ", counter)
```

```
counter:  1
counter:  2
counter:  4
```

```
counter:  5
counter:  6
```

2.3.4 Try-Except

2.3.4.1 Definition

A `try/except` block lets your code recover from an error instead of crashing outright. This is especially useful inside a loop over many records, where one bad record shouldn't stop processing of the rest. Below, a customer with zero years of tenure would otherwise cause a division-by-zero error.

```
customer_spendings = {"Tom": {"years_customer": 5, "total_spendings": 304},
                      "Julie": {"years_customer": 0, "total_spendings": 1402},
                      "Jack": {"years_customer": 2, "total_spendings": 459}}

for customer, value in customer_spendings.items():

    try:
        avg_spending_per_year = value["total_spendings"] / value["years_customer"]
        print(customer, ":", avg_spending_per_year)
    except:
        avg_spending_per_year = value["total_spendings"]
        print(customer, ":", avg_spending_per_year, "for less than 1 year a customer")
```

```
Tom : 60.8
Julie : 1402 for less than 1 year a customer
Jack : 229.5
```

2.4 Reading and Writing Files

In every real-world data analytics project, you'll need to get data into and out of Python. Later in the book, you'll see how to read and write Excel and CSV files with `pandas`, but sometimes you need to work with plain-text files directly. Therefore, this section covers the basics of file handling using plain-text files. The examples below use files from this book's `data/raw/` folder.

2.4.1 Reading

2.4.1.1 General procedure

Opening a file gives you a file object; you read its contents, then close it when you're done to free up the underlying resource.

```
# open the corpus.txt file into the file object f
f = open("../data/raw/corpus.txt", "r")
```

```
# read the file
text = f.read()
print(text)
```

```
this is the first sentence
and this is the second sentence
the third sentence has some more words than the previous sentences
this sentence is the last one
```

```
# close the file
f.close()
```

```
# inspect object type of text variable
type(text)
```

```
str
```

2.4.1.2 Read functions

There are a few different ways to pull lines out of a file, depending on whether you want to process them one at a time or all at once.

```
# open the corpus.txt file into the file object f
f = open("../data/raw/corpus.txt", "r")

# initialize empty list
lines_container = []

# loop through lines
for line in f:
    # add line to container
    lines_container.append(line)

# loop through container
for line in lines_container:
    print(line)
```

```
this is the first sentence

and this is the second sentence

the third sentence has some more words than the previous sentences

this sentence is the last one
```

```
# open the corpus.txt file into the file object f
f = open("../data/raw/corpus.txt", "r")

# read lines into container
container = f.readlines()

print(type(container))
print(len(container))
print(container[0])
```

```
<class 'list'>
4
this is the first sentence
```

```
# close the file
f.close()
```

```
# open the corpus.txt file into the file object f
f = open("../data/raw/corpus.txt", "r")

# loop through the lines
for line in f:
    print(line)
```

```
this is the first sentence

and this is the second sentence

the third sentence has some more words than the previous sentences

this sentence is the last one
```

```
# close the file
f.close()
```

A more realistic use case: parsing a semicolon-separated customer file, line by line, into a dictionary keyed by customer ID.

```
# open the customers.csv file
f = open("../data/raw/customers.csv", "r")

# initialize dictionary
customers_dict = dict()


# loop through the lines
for line in f:
    # remove new line
    line = line.replace("\n", "")
    # split line by separator
    data = line.split(";")
    # create dictionary
    cust_dict = {"name": data[1], "country": data[2], "phone": data[3]}
    # store in customers dict with id as key
    customers_dict[data[0]] = cust_dict

# inspect dict
customers_dict
```

```
{'203040': {'name': 'Thomas', 'country': 'Belgium', 'phone': '0456203495'},
 '405061': {'name': 'Steven', 'country': 'Germany', 'phone': '0459602010'},
 '601020': {'name': 'Julie', 'country': 'France', 'phone': '04050103003'},
 '302001': {'name': 'Viktor', 'country': 'Belgium', 'phone': '040501020'},
 '4050102': {'name': 'Sarah', 'country': 'Belgium', 'phone': '0405010034'},
 '3040502': {'name': 'Patrick', 'country': 'France', 'phone': '040502010'}}
```

```
# close the file
f.close()
```

2.4.1.3 With

Manually calling `.close()` every time is easy to forget. For example, if an error happens before you reach the statement, the file stays open. The `with` statement handles closing the file automatically, even if an error occurs inside the block, so it's the pattern you should reach for by default.

```
# use the with statement
with open("../data/raw/corpus.txt", "r") as f:
    text = f.read()
```

2.4.2 Writing Files

2.4.2.1 General procedure

Writing works the same way as reading, but with mode `"w"` instead of `"r"`. Run the cells below and you'll see the corresponding file appear in `data/raw/`.

```
# initialize a string
string = "this is some random string I want to store"
```

```
with open("../data/raw/my_random_string.txt", "w") as f:
    f.write(string)
```

2.4.2.2 Write functions

`.writelines()` writes a list of strings to a file, one after another, with no separator added automatically: if you want line breaks, you need to include `"\n"` yourself.

```
# initialize a list you want to store
customers  = ['Tom', 'Charlotte', 'Lyam', 'Patrick', 'Matts', 'Lesley']
```

```
with open("../data/raw/my_customers.txt", "w") as f:
    f.writelines(customers)
```

```
# initialize a list you want to store
customers  = ['Tom\n', 'Charlotte\n', 'Lyam\n', 'Patrick\n', 'Matts\n', 'Lesley\n']
```

```
with open("../data/raw/my_customers_2.txt", "w") as f:
    f.writelines(customers)
```

You can also build up a file line by line inside a loop, which is handy when each line needs to be assembled from several pieces of data first. Here, the file is created from a list of customer dictionaries.

```
# initialize a dictionary you want to store
customer_info  = [{"name": "Tom", "country": "Belgium", "customer_since": "05/03/1997"},
                  {"name": "Matts", "country": "United States", "customer_since": "14/09/2003"},
                  {"name": "Lesley", "country": "France", "customer_since": "01/11/1993"},
                  {"name": "Catherine", "country": "Belgium", "customer_since": "25/04/2005"}]
```

```
# open the file
with open("../data/raw/my_customers_3.txt", "w") as f:

    # loop through objects
    for customer in customer_info:

        # get the name
        name = customer["name"]
        # get the country
        country = customer["country"]
        # get the start date
        start_date = customer["customer_since"]

        # write data to file and seperate by ;
        f.write(name)
        f.write(";")
        f.write(country)
        f.write(";")
        f.write(start_date)
```

```
        f.write("\n")
```

In the end, you'll find the three newly created customer `.txt` files in `data/raw/`.

2.5 Functions

2.5.1 Definition

A function wraps up a block of reusable code under a specific name. Python also ships with many built-in functions you've already been using without a formal introduction, such as `print()`, `type()`, `float()`, among others.

```
# built in functions
print("hallo")
print(type(3.0))
print(float(2))
print(str(2))
print(bool(0))
print(list())
print(dict())
```

```
hallo
<class 'float'>
2.0
2
False
[]
{}
```

2.5.2 Parameters

Parameters are the inputs a function accepts. You can pass them by referencing their position in the function call or by explicitly naming them.

```
# define a function which adds a constant to a number
def add_constant_to_number(number, constant):

    # calculate result
    result = number + constant

    # return result
    return(result)
```

```
# call the function for a specific number and constant
result = add_constant_to_number(number=15, constant=3)
print(result)
```

```
18
```

```
# define a function that will print the elements of a list
def print_elements_list(elements):
    for el in elements:
        print(el)
```

```
# define a list of elements
e = ["belgium", "france", "germany"]
```

```
# call function on this list
print_elements_list(e)
```

```
belgium
france
germany
```

```
# define a function that divides a numerator by denominator
def division(numerator, denominator):

    return(numerator / denominator)
```

When you call a function with positional arguments, order matters! Swapping the numerator and denominator gives you a different (and wrong) result if you're not careful.

```
# call the function with parameter values at correct place
div = division(4., 2.)
print(div)
```

```
2.0
```

```
# call the function with parameter values not at correct place
div = division(2., 4.)
print(div)
```

```
0.5
```

Using keyword arguments removes any ambiguity, since each value is explicitly matched to its parameter name regardless of the order you write them in.

```
# call the function with parameter names and values specified
div = division(denominator=2., numerator=4.)
print(div)
```

```
2.0
```

Parameters can also have default values, which make them optional at the call site.

```
# define function which prints a customer's name
def print_name(name="unknown"):

    print("the customer's name is: " + name)
```

```
# call function with parameter specified
print_name(name="jack")
```

```
the customer's name is: jack
```

```
# call function with no parameter specified
print_name()
```

```
the customer's name is: unknown
```

2.5.3 Return

A function stops executing as soon as it hits a `return` statement. Any code written after it inside the same branch never runs.

```
# define function with multiple return statements
def divide_and_sum(number, constant_1, constant_2):

```

```
    # divide number by constant_1
    result_1 = number / constant_1
    return(result_1)

    # add constant_2 to result_1
    result_2 = result_1 + constant_2
    return(result_2)
```

```
# call function
result = divide_and_sum(number=10, constant_1=5, constant_2=2)
print(result)
```

```
2.0
```

2.5.4 Flexible Functions

Sometimes you don't know in advance how many arguments a function needs to accept. `*args` collects any number of positional arguments into a tuple, and `**kwargs` collects any number of keyword arguments into a dictionary. You can use either, or both, in the same function.

```
def print_args(*args):
   # loop through non-keyword arguments (stored as a tuple)
   for arg in args:
      print(arg)

print_args(1, 2, 3)
```

```
1
2
3
```

```
def print_kwargs(**kwargs):
   # loop through keyword arguments (stored as a dictionary)
   for key, value in kwargs.items():
      print("{key}: {value}".format(key=key, value=value))

print_kwargs(a=1, b=2, c=3)
```

```
a: 1
b: 2
c: 3
```

```
def print_args_kwargs(*args, **kwargs):
   # loop through non-keyword arguments (stored as a tuple)
   for arg in args:
      print("Non-keyword argument: {arg}".format(arg=arg))

   # loop through keyword arguments (stored as a dictionary)
   for key, value in kwargs.items():
      print("Keyword argument: {key}={value}".format(key=key, value=value))

print_args_kwargs(1, 2, 3, a=1, b=2, c=3)
```

```
Non-keyword argument: 1
Non-keyword argument: 2
Non-keyword argument: 3
```

```
Keyword argument: a=1
Keyword argument: b=2
Keyword argument: c=3
```

2.6 Packages

2.6.1 Usage

Almost everything useful you'll do in Python relies on packages beyond the standard library. Install them from the command line or with `!pip install` inside a Jupyter notebook or `%pip install` inside VScode.

```
# install package from within Jupyter Notebook
# NOTE: replace package_name by the name of the package you want to install!
!pip install package_name
# in VSCode it's better to use: %pip install package_name
```

Once a package (or a built-in module) is installed, you bring it into your script with `import`. You can import an entire module, a single function from it, or give it a shorter alias. All three styles show up throughout this book.

```
# import entire module
import os
```

```
# use the makedirs function from the os module
os.makedirs("./test", exist_ok=True)
```

```
# import the function from the module
from os import listdir
```

```
# use the listdir function from the os module
listdir("./")
```

```
['04-data-exploration.quarto_ipynb',
 'appendix-installation.qmd',
 '07-modeling.quarto_ipynb',
 '05-data-preparation.qmd',
 '06-model-evaluation.qmd',
 '07-modeling.qmd',
 '03-data-understanding.qmd',
 '02-python-basics_files',
 'test',
 '06-model-evaluation.quarto_ipynb',
 '02-python-basics.quarto_ipynb',
 '01-intro.qmd',
 'appendix-installation.quarto_ipynb',
 '05-data-preparation.quarto_ipynb',
 '01-intro.html',
 '04-data-exploration.qmd',
 'references.qmd',
 '02-python-basics.html',
 '03-data-understanding.quarto_ipynb',
 '02-python-basics.qmd',
 '.jupyter_cache']
```

```
# define an alias for the package numpy (if not installed run !pip or %pip install numpy)
import numpy as np
```

> **Module vs Package**
>
> There is some confusion between a module and a package in Python. A **module** is a single file (or script) that is imported under one import and used. For example, you have written a script that contains a set of functions to manually engineer a set of features for a machine learning model. You can save this script as `feature_engineering.py` in your folder and then import it in your notebook with `import feature_engineering`. This will allow you to employ the user-defined functions.
> A **package**, on the other hand, is a collection of modules in directories that give a package hierarchy. For example, `feature-engine` is a package that contains a collection of modules for advanced feature transformations. You can import the entire package with `import feature_engine` or import a specific module with `from feature_engine import variable_transformers`. Hence, there is no need to create your own scripts and modules when you can use the existing packages that are available in Python.

2.7 NumPy

2.7.1 Definition

NumPy is Python's core library for numerical computing. Its central object, the array, stores values of a single type in a grid of one or more dimensions and supports fast, vectorized math operations. This means that NumPy operates on entire arrays at once instead of writing explicit loops. It's also the foundation that libraries like `pandas` and `scikit-learn` are built on, so it's worth being comfortable creating, manipulating, and using NumPy arrays.

```
# import the numpy package
import numpy as np
```

2.7.2 Multi-dimensional array

2.7.2.1 Creation

An array can have any number of dimensions: a 1D array behaves like a list of numbers, a 2D array like a matrix, and so on.

```
# define a 1D array
arr = np.array([3, 4, 1])
```

```
print(arr)
```

```
[3 4 1]
```

```
# define a 2D array
arr = np.array([[1, 3, 0], [3, 2, 5]])
```

```
print(arr)
```

```
[[1 3 0]
 [3 2 5]]
```

NumPy also provides shortcuts for creating arrays filled with a specific set of values, without having to type them out by hand.

```
# create an array full of zeros with shape (2, 3)
zeros = np.zeros(shape=(2, 3))

print(zeros)
```

```
[[0. 0. 0.]
 [0. 0. 0.]]
```

```
# create an array full of ones with shape (1, 3)
ones = np.ones(shape=(1, 3))

print(ones)
```

```
[[1. 1. 1.]]
```

```
# create a matrix full of 5 with the shape (2, 2)
fives = np.full(fill_value=5, shape=(2, 2))

print(fives)
```

```
[[5 5]
 [5 5]]
```

```
# create an array of random numbers between 0 and 1 of shape (2, 2)
random_floats = np.random.random(size=(2, 2))

print(random_floats)
```

```
[[0.73406713 0.81536605]
 [0.70950259 0.72992493]]
```

```
# create an array of random integers between 0 and 10 of shape (2, 2)
random_ints = np.random.randint(low=0, high=10, size=(2, 2))

print(random_ints)
```

```
[[8 0]
 [9 3]]
```

```
# create an array of random normal numbers with mean 0 and stdev 1 of shape (2, 2)
random_normals = np.random.normal(loc=0, scale=1, size=(2, 2))

print(random_normals)
```

```
[[ 0.44364453  0.71790816]
 [-0.85754299  0.18607482]]
```

Every array has a `.shape` attribute describing its size along each dimension. It is worth checking this often, since a huge share of NumPy and `pandas` errors come down to arrays not having the shape you expect.

```
# define another 2D array
arr = np.array([[1, 3, 0], [3, 2, 5], [5, 7, 1], [3, 4, 1]])

print(arr)

# get the shape of the array
arr.shape
```

```
[[1 3 0]
 [3 2 5]
 [5 7 1]
 [3 4 1]]

(4, 3)
```

2.7.2.2 Indexing

Indexing a 2D array takes two coordinates, `[row, column]`, and slicing works along each dimension independently.

```
# define an array
arr = np.array([[1, 2, 3], [4, 5, 6]])

print(arr)
```

```
[[1 2 3]
 [4 5 6]]
```

```
# access the element on the first row and second column
arr[0, 1]
```

```
np.int64(2)
```

```
# access the elements on the second row from the first column until the second column
arr[1, 0:2]
```

```
array([4, 5])
```

2.7.2.3 Mathematical Operations

Arithmetic on arrays is **element-wise** by default: adding two arrays adds each pair of matching positions, rather than doing anything like matrix multiplication.

```
# initialize two arrays
arr_1 = np.array([[1, 3], [-2, 5]])
arr_2 = np.array([[2, 4], [3, -1]])

print(arr_1)
print("\n")
print(arr_2)
```

```
[[ 1  3]
 [-2  5]]


[[ 2  4]
 [ 3 -1]]
```

```
# perform element-wise summation
arr_3 = arr_1 + arr_2

print(arr_3)
```

```
[[3 7]
 [1 4]]
```

```
# perform element-wise subtraction
arr_4 = arr_1 - arr_2

print(arr_4)
```

```
[[-1 -1]
 [-5  6]]
```

```
# perform element-wise multiplication
arr_5 = arr_1 * arr_2

print(arr_5)
```

```
[[ 2 12]
 [-6 -5]]
```

Element-wise operations require compatible shapes. `arr_1` is a 2×2 array, so adding a length-3 vector to it fails. NumPy has no consistent way to line up 2 columns with 3 values. This is the error you'll get whenever your arrays don't broadcast together.

```
arr_5 = arr_1 + np.array([2, 1, 5])
```

```
ValueError: operands could not be broadcast together with shapes (2,2) (3,)
---------------------------------------------------------------------------
ValueError                                Traceback (most recent call last)
Cell In[135], line 1
----> 1 arr_5 = arr_1 + np.array([2, 1, 5])
ValueError: operands could not be broadcast together with shapes (2,2) (3,)
```

Broadcasting is the set of rules NumPy uses to apply an operation between arrays of different (but compatible) shapes, without actually copying data, by conceptually stretching the smaller one to match the bigger one. There are exactly three rules, applied in this order:

1. **Padding:** If the two arrays have a different number of dimensions, the shape of the one with fewer dimensions is padded with 1s on its leading (left) side, until both shapes have the same number of dimensions.
2. **Stretching:** Comparing the two (now equal-length) shapes dimension by dimension: wherever one array has size-1 in a dimension and the other doesn't, the size-1 dimension is stretched (in other words, duplicated) to match the other array's size in that dimension.
3. **Mismatch:** If, in any dimension, the two sizes disagree and neither of them is size-1, broadcasting fails and NumPy raises an error.

Applied to the failing example above: `arr_1` has shape `(2, 2)` and the vector `np.array([2, 1, 5])` has shape `(3,)`. Rule 1 pads the vector's shape to `(1, 3)`. Comparing dimension by dimension, the last dimension is 2 vs. 3 and neither is size-1 so they don't match. In this case, Rule 3 kicks in and NumPy raises the error you saw above.

In the example below, the length-2 vector broadcasts successfully across both rows of `arr_1`. Rule 1 pads its shape from `(2,)` to `(1, 2)`, and then Rule 2 stretches that size-1 first dimension to 2, matching `arr_1`'s shape.

```
# broadcasting
arr_4 = arr_1 + np.array([3, 2])

print(arr_4)
```

```
[[4 5]
 [1 7]]
```

Adding a plain scalar works the same way: a scalar is treated as a 0-dimensional array with shape `()`. Rule 1 pads it to `(1, 1)`, and Rule 2 stretches both dimensions to match `arr_1`'s `(2, 2)`.

```
# add scalar to array
arr_1 + 2
```

```
array([[3, 5],
       [0, 7]])
```

```
# add vector to array
arr_1 + np.array([2, 3])
```

```
array([[3, 6],
       [0, 8]])
```

A higher-dimensional example is shown below to see all three rules working together on a bigger shape mismatch: `a` has shape `(2, 3, 2)` and `b` has shape `(2,)`. Rule 1 pads `b` to `(1, 1, 2)`. Comparing dimension by dimension, the last dimension already matches (`2` vs. `2`), so there's nothing to stretch there; the middle and first dimensions are `1` vs. `3` and `1` vs. `2`, so Rule 2 stretches both of `b`'s size-`1` dimensions to match, giving a broadcast shape of `(2, 3, 2)`.

```
a = np.zeros(shape=(2, 3, 2))
b = np.ones(shape=(2))
a + b
```

```
array([[[1., 1.],
        [1., 1.],
        [1., 1.]],

       [[1., 1.],
        [1., 1.],
        [1., 1.]]])
```

Beyond element-wise arithmetic, NumPy also provides linear-algebra operations like the dot product, along with aggregation functions (e.g., sum, mean, max, min) that can be applied either over the whole array or along a specific axis.

```
# do matrix mulitplication between arr_1 and arr_2
arr = np.dot(arr_1, arr_2)

print(arr)
```

```
[[ 11   1]
 [ 11 -13]]
```

```
# take the sum along the second dimension
np.sum(arr_1, axis=1)
```

```
array([4, 3])
```

```
# take the mean along the first dimension
np.mean(arr_1, axis=0)
```

```
array([-0.5,  4. ])
```

```
# take the maxium of the entire array
np.max(arr_1)
```

```
np.int64(5)
```

```
# take the minimum along the first dimension
np.min(arr_1, axis=0)
```

```
array([-2,  3])
```

2.7.3 Array Manipulation

Beyond creating and combining arrays, you'll often need to reshape or reorganize them into a different layout, for example, to match the input shape a model expects.

> **NumPy version**
>
> `np.reshape()`'s second argument was renamed from `newshape` to `shape` in NumPy 2.1, and the old `newshape` keyword has since been removed entirely. Calls with `np.reshape(arr, newshape=...)` now raise a `TypeError` on current NumPy.

```
# define array
arr_1 = np.array([1, 2, 3, 4])

# reshape array
arr_2 = np.reshape(arr_1, shape=(2, 2))

# check
arr_2
```

```
array([[1, 2],
       [3, 4]])
```

```
# define array
arr_1 = np.array([[1, 2], [3, 4]])

# check
print(arr_1)
```

```
[[1 2]
 [3 4]]
```

```
# transpose array
arr_2 = np.transpose(arr_1)

# check
print(arr_2)
```

```
[[1 3]
 [2 4]]
```

```
# define array
arr_1 = np.zeros(shape=(2, 3))
arr_2 = np.ones(shape=(1, 3))

# concatenate along first dimension
arr_3 = np.concatenate([arr_1, arr_2], axis=0)

# check
arr_3
```

```
array([[0., 0., 0.],
       [0., 0., 0.],
       [1., 1., 1.]])
```

2.8 Pandas

`pandas` is Python's main data manipulation library: it makes it easy to read, explore, wrangle, transform, and combine datasets. Its central object, the `DataFrame`, is a table of rows and columns (similar to a spreadsheet in Excel or a database table) built on top of the NumPy arrays from the previous section. Mastering its core

operations is crucial before moving into the realm of data understanding, exploration, and preparation, which is exactly what the rest of this book does with it.

2.8.1 Definition

Before you can build or load a DataFrame, you need to import the `pandas` package and use the conventional alias as `pd`.

```
# import pandas package
import pandas as pd
```

> **Pandas versions**
>
> pandas 3.0 was released recently, but this book intentionally stays on **pandas 2.x** (`pandas>=2.3,<3`). pandas 3.0 changes default behavior in ways that haven't been fully verified yet against the other packages this book relies on (`feature_engine`, `sklearn_evaluation`, and friends). For example, copy-on-write semantics are now always on, and string columns default to a new `str` dtype instead of `object`.
> Make sure you install the pinned environment from `requirements.txt` in the project root (see also the installation appendix) rather than the default pandas version `pip install pandas` gives you today, otherwise a few examples in this book may behave differently or throw errors.

2.8.2 DataFrame Creation

You can build a `DataFrame` by hand from a dictionary, but far more often you'll load one directly from a file. `pandas` can read most common tabular formats, like CSV and Excel, including files with no header row, or where you want to control each column's type up front.

```
# create DataFrame manually
df = pd.DataFrame(
    {
        "name": ["Sarah", "Tom", "Michael", "Lore"],
        "cust_id": [123, 451, 455, 388],
        "country": ["Germany", "France", "USA", "Belgium"],

    }
)
```

```
# loading csv file
df = pd.read_csv("../data/raw/customers.csv", sep=";")
```

```
# check
df.head()
```

	203040	Thomas	Belgium	0456203495
0	405061	Steven	Germany	459602010
1	601020	Julie	France	4050103003
2	302001	Viktor	Belgium	40501020
3	4050102	Sarah	Belgium	405010034
4	3040502	Patrick	France	40502010

```
# loading xlsx file
#Make sure that you have openpyxl package installed: pip install openpyxl
df = pd.read_excel("../data/raw/customers.xlsx")
```

```
# check
df.head()
```

	Cust_ID	Name	Country	Number
0	203040	Thomas	Belgium	456203495
1	405061	Steven	Germany	459602010
2	601020	Julie	France	4050103003
3	302001	Viktor	Belgium	40501020
4	4050102	Sarah	Belgium	405010034

```
# loading csv file
df = pd.read_csv("../data/raw/customers_no_header.csv",
                 sep=";",
                 header=None,
                 names=["Cust_ID", "Name", "Country", "Number"],
                 dtype={"Cust_ID": int, "Name": str, "Country": str, "Number": float})
```

```
# check
df.head()
```

	Cust_ID	Name	Country	Number
0	203040	Thomas	Belgium	4.562035e+08
1	405061	Steven	Germany	4.596020e+08
2	601020	Julie	France	4.050103e+09
3	302001	Viktor	Belgium	4.050102e+07
4	4050102	Sarah	Belgium	4.050100e+08

2.8.3 DataFrame Exploration

Once your data is loaded, the first step is always to take a first look at it: its size, its column types, and basic summary statistics before you change a single value.

```
# load csv file
df = pd.read_csv("../data/raw/customer_churn.csv")
```

```
# print snapshot of entire dataframe
df
```

	customerID	gender	SeniorCitizen	Partner	Dependents	tenure	PhoneService	MultipleLines	Inte
0	7590-VHVEG	Female	0	Yes	No	1	No	No phone service	DSL
1	5575-GNVDE	Male	0	No	No	34	Yes	No	DSL
2	3668-QPYBK	Male	0	No	No	2	Yes	No	DSL
3	7795-CFOCW	Male	0	No	No	45	No	No phone service	DSL
4	9237-HQITU	Female	0	No	No	2	Yes	No	Fibe
...	...	...	...	...	...	...	...	...	...
7038	6840-RESVB	Male	0	Yes	Yes	24	Yes	Yes	DSL
7039	2234-XADUH	Female	0	Yes	Yes	72	Yes	Yes	Fibe
7040	4801-JZAZL	Female	0	Yes	Yes	11	No	No phone service	DSL
7041	8361-LTMKD	Male	1	Yes	No	4	Yes	Yes	Fibe
7042	3186-AJIEK	Male	0	No	No	66	Yes	No	Fibe

```
# print snapshot of a specific variable
df["gender"]
```

```
0       Female
1         Male
2         Male
3         Male
4       Female
         ...
7038      Male
7039    Female
7040    Female
7041      Male
7042      Male
Name: gender, Length: 7043, dtype: object
```

```
# inspect first 3 rows of dataframe
df.head(3)
```

	customerID	gender	SeniorCitizen	Partner	Dependents	tenure	PhoneService	MultipleLines	Internet
0	7590-VHVEG	Female	0	Yes	No	1	No	No phone service	DSL
1	5575-GNVDE	Male	0	No	No	34	Yes	No	DSL
2	3668-QPYBK	Male	0	No	No	2	Yes	No	DSL

```
# inspect last 3 rows of dataframe
df.tail(3)
```

	customerID	gender	SeniorCitizen	Partner	Dependents	tenure	PhoneService	MultipleLines	Inter
7040	4801-JZAZL	Female	0	Yes	Yes	11	No	No phone service	DSL
7041	8361-LTMKD	Male	1	Yes	No	4	Yes	Yes	Fibe
7042	3186-AJIEK	Male	0	No	No	66	Yes	No	Fibe

```
# inspect the shape of the dataframe
df.shape
```

```
(7043, 21)
```

```
# inspect column names
df.columns
```

```
Index(['customerID', 'gender', 'SeniorCitizen', 'Partner', 'Dependents',
       'tenure', 'PhoneService', 'MultipleLines', 'InternetService',
       'OnlineSecurity', 'OnlineBackup', 'DeviceProtection', 'TechSupport',
       'StreamingTV', 'StreamingMovies', 'Contract', 'PaperlessBilling',
       'PaymentMethod', 'MonthlyCharges', 'TotalCharges', 'Churn'],
      dtype='object')
```

```
# inspect the column names
df.dtypes
```

```
customerID          object
gender              object
SeniorCitizen        int64
Partner             object
```

```
Dependents           object
tenure                int64
PhoneService         object
MultipleLines        object
InternetService      object
OnlineSecurity       object
OnlineBackup         object
DeviceProtection     object
TechSupport          object
StreamingTV          object
StreamingMovies      object
Contract             object
PaperlessBilling     object
PaymentMethod        object
MonthlyCharges      float64
TotalCharges         object
Churn                object
dtype: object
```

`.describe()` behaves differently depending on the column's type: numeric columns get statistics like mean and quartiles, while a string (object) column gets counts instead.

```
# get summary statistics of numeric variable
df["MonthlyCharges"].describe()
```

```
count    7043.000000
mean       64.761692
std        30.090047
min        18.250000
25%        35.500000
50%        70.350000
75%        89.850000
max       118.750000
Name: MonthlyCharges, dtype: float64
```

```
# get summary statistics of string variable
df["gender"].describe()
```

```
count     7043
unique       2
top       Male
freq      3555
Name: gender, dtype: object
```

```
# get summary statistics of all numeric variables
df.describe()
```

	SeniorCitizen	tenure	MonthlyCharges
count	7043.000000	7043.000000	7043.000000
mean	0.162147	32.371149	64.761692
std	0.368612	24.559481	30.090047
min	0.000000	0.000000	18.250000
25%	0.000000	9.000000	35.500000
50%	0.000000	29.000000	70.350000
75%	0.000000	55.000000	89.850000
max	1.000000	72.000000	118.750000

```
# get the mean of the MonthlyCharges variable
df["MonthlyCharges"].mean()
```

```
np.float64(64.76169246059918)
```

```
# get the sum of the MonthlyCharges variable
df["MonthlyCharges"].sum()
```

```
np.float64(456116.6)
```

```
# get the number of categories of the InternetService variable
df["InternetService"].unique()
```

```
array(['DSL', 'Fiber optic', 'No'], dtype=object)
```

```
# check if the InternetService variable only contains unique values
df["InternetService"].is_unique
```

```
False
```

Missing values show up constantly in real data, so knowing how to spot them is essential: `.isnull()` flags them value by value, and chaining `.sum()` onto it counts how many there are.

```
# inspect missing values per row
df["InternetService"].isnull()
```

```
0       False
1       False
2       False
3       False
4       False
        ...
7038    False
7039    False
7040    False
7041    False
7042    False
Name: InternetService, Length: 7043, dtype: bool
```

```
# get number of missing values for column
df["InternetService"].isnull().sum()
```

```
np.int64(0)
```

```
# get number of missing values for each column of DataFrame
df.isnull().sum()
```

```
customerID          0
gender              0
SeniorCitizen       0
Partner             0
Dependents          0
tenure              0
PhoneService        0
MultipleLines       0
InternetService     0
OnlineSecurity      0
OnlineBackup        0
DeviceProtection    0
```

```
TechSupport         0
StreamingTV         0
StreamingMovies     0
Contract            0
PaperlessBilling    0
PaymentMethod       0
MonthlyCharges      0
TotalCharges        0
Churn               0
dtype: int64
```

2.8.4 DataFrame Manipulation

Once a DataFrame is loaded, day-to-day work is mostly about reshaping, cleaning and processing it: converting column types, sorting, subsetting, grouping, joining, dropping unwanted rows or columns, handling missing values, and deriving new columns from existing ones.

2.8.4.1 Type Conversion

Data loaded from a file is often read in as text (or as a generic "object") even when it's really numeric or a date. `pd.to_numeric()` and `pd.to_datetime()` convert it explicitly. `astype()` can convert a column to any type you want, including back to string.

```
# define dataframe
df_1 = pd.DataFrame(
                    {"Postal_Code": ["3410", "4515", "5511", "1390"]}
)
```

```
# show
df_1
```

	Postal_Code
0	3410
1	4515
2	5511
3	1390

```
# check data types
df_1.dtypes
```

```
Postal_Code    object
dtype: object
```

```
# convert string variable to numeric variable
df_1["Postal_Code"] = pd.to_numeric(df_1["Postal_Code"])
```

```
# show
df_1
```

	Postal_Code
0	3410
1	4515
2	5511
3	1390

```
# check data types
df_1.dtypes
```

```
Postal_Code    int64
dtype: object
```

```
# convert numeric variable back to string variable
df_1["Postal_Code"] = df_1["Postal_Code"].astype(str)
```

```
# define dataframe
df_1 = pd.DataFrame(
                    {"dates": ["01/04/1993", "04/11/2003", "05/03/1997", "23/06/2001"]}
)
```

```
df_1
```

	dates
0	01/04/1993
1	04/11/2003
2	05/03/1997
3	23/06/2001

```
# check data types
df_1.dtypes
```

```
dates    object
dtype: object
```

```
# convert string variable to datetime variable
df_1["dates"] = pd.to_datetime(df_1["dates"], format="%d/%m/%Y")
```

```
# check
df_1
```

	dates
0	1993-04-01
1	2003-11-04
2	1997-03-05
3	2001-06-23

```
df_1.dtypes
```

```
dates    datetime64[ns]
dtype: object
```

2.8.4.2 Sorting

Sorting a DataFrame means rearranging its rows in ascending or descending order based on the values of one or more columns.

In `pandas`, the `sort_values()` function sorts a DataFrame by one or more columns. By default, it sorts in ascending order.

```
# define dataframe
df_1 = pd.DataFrame(
                    {"Scores": [3, 18, 15, 6],
                     "Group": [1, 2, 1, 2]}
)
```

```
df_1
```

	Scores	Group
0	3	1
1	18	2
2	15	1
3	6	2

```
# sort in ascending order
df_1.sort_values(by="Scores")
```

	Scores	Group
0	3	1
3	6	2
2	15	1
1	18	2

```
# sort in descending order
df_1.sort_values(by="Scores", ascending=False)
```

	Scores	Group
1	18	2
2	15	1
3	6	2
0	3	1

```
# sort by multiple variables
df_1.sort_values(by=["Group", "Scores"])
```

	Scores	Group
0	3	1
2	15	1
3	6	2
1	18	2

2.8.4.3 Subsetting

Subsetting a DataFrame means selecting a sample of the data, either by column, row, or both.

`pandas` allows you to select columns, positions (`.iloc`), or rows matching a condition. The latter, also called boolean indexing, is what you'll use constantly to filter out observations from a table.

```
# select the gender and tenure columns
df_sub = df[["gender", "tenure"]]
```

```
df_sub.head(3)
```

	gender	tenure
0	Female	1
1	Male	34
2	Male	2

```
# select row 2 and column 3
df_sub = df.iloc[1, 2]
```

```
df_sub
```

```
np.int64(0)
```

```
# select rows 2, 5, 6 and columns 4, 5
df_sub = df.iloc[[1, 4, 5], [3, 4]]
```

```
df_sub
```

	Partner	Dependents
1	No	No
4	No	No
5	No	No

```
# select row 10 until row 30 and columns 3 until 5
df_sub = df.iloc[9:30, 2:5]
```

```
df_sub.head()
```

	SeniorCitizen	Partner	Dependents
9	0	No	Yes
10	0	Yes	Yes
11	0	No	No
12	0	Yes	No
13	0	No	No

```
# select all the rows where gender equals Male
df_sub = df[df["gender"] == "Male"]
```

```
# select all the rows where tenure equals 1, 2 or 3
df_sub = df[df["tenure"].isin([1, 2, 3])]
```

```
# select all the rows where gender equals Male and OnlineSecurity equals yes
df_sub = df[(df["gender"] == "Male") & (df["OnlineSecurity"] == "Yes")]
```

```
# condition on column
df["gender"] == "Male"
```

```
0       False
```

```
1        True
2        True
3        True
4       False
        ...
7038     True
7039    False
7040    False
7041     True
7042     True
Name: gender, Length: 7043, dtype: bool
```

2.8.4.4 Grouping

Grouping summarizes data by one or more categorical variables.

The `groupby()` function splits the DataFrame into groups based on the unique values of the specified column(s), and then applies specific aggregation functions to each group.

> **i** String names vs. function objects
>
> In earlier versions of pandas, when using the `agg()` function in a groupby operation, you had to pass the **actual function**, like `np.mean`, for aggregation. However, in more recent versions of pandas, you can simply pass the **string name of the function** (e.g., `'mean'`, `'sum'`, etc.) instead of the function object itself. This works because pandas internally maps these string names to the corresponding numpy functions.

```
df = pd.read_csv("../data/raw/customer_churn.csv")
```

```
df.head()
```

	customerID	gender	SeniorCitizen	Partner	Dependents	tenure	PhoneService	MultipleLines	Internet
0	7590-VHVEG	Female	0	Yes	No	1	No	No phone service	DSL
1	5575-GNVDE	Male	0	No	No	34	Yes	No	DSL
2	3668-QPYBK	Male	0	No	No	2	Yes	No	DSL
3	7795-CFOCW	Male	0	No	No	45	No	No phone service	DSL
4	9237-HQITU	Female	0	No	No	2	Yes	No	Fiber op

```
df.groupby("gender")["MonthlyCharges"].mean()
```

```
gender
Female    65.204243
Male      64.327482
Name: MonthlyCharges, dtype: float64
```

```
agg_dict = {"tenure": ['mean', 'sum'], "MonthlyCharges": ['min', 'max']}
```

```
df.groupby("gender").agg(agg_dict)
```

	tenure		MonthlyCharges	
	mean	sum	min	max
gender				
Female	32.244553	112469	18.40	118.75

	tenure		MonthlyCharges	
	mean	sum	min	max
gender				
Male	32.495359	115521	18.25	118.35

2.8.4.5 Joining

Joining or merging combines two DataFrames based on a common column or index. `pd.concat()` and `pd.merge()` are the two main functions for this purpose.

`pd.concat()` stacks DataFrames on top of each other by default, matching up columns that share the same name and introducing missing values where they don't.

```
# define dataframe
df_1 = pd.DataFrame(
                    {"Country": ["Belgium", "France", "Germany"],
                     "Name": ["Tom", "Sarah", "Niels"]}
)
```

```
df_1
```

	Country	Name
0	Belgium	Tom
1	France	Sarah
2	Germany	Niels

```
# define dataframe
df_2 = pd.DataFrame(
                    {"Country": ["Belgium", "USA"],
                     "Name": ["Lies", "Jack"]}
)
```

```
df_2
```

	Country	Name
0	Belgium	Lies
1	USA	Jack

```
# stack dataframes
pd.concat([df_1, df_2])
```

	Country	Name
0	Belgium	Tom
1	France	Sarah
2	Germany	Niels
0	Belgium	Lies
1	USA	Jack

```
# define dataframe
df_1 = pd.DataFrame({"Country": ["Belgium", "France", "Germany"],
                     "Name": ["Tom", "Sarah", "Niels"]})
```

```
df_1
```

	Country	Name
0	Belgium	Tom
1	France	Sarah
2	Germany	Niels

```
# define dataframe
df_2 = pd.DataFrame({"Cust_Id": [234, 4531],
                     "Name": ["Lies", "Jack"]})
```

```
df_2
```

	Cust_Id	Name
0	234	Lies
1	4531	Jack

```
# stack dataframes
pd.concat([df_1, df_2])
```

	Country	Name	Cust_Id
0	Belgium	Tom	NaN
1	France	Sarah	NaN
2	Germany	Niels	NaN
0	NaN	Lies	234.0
1	NaN	Jack	4531.0

`pd.merge()`, on the other hand, joins two DataFrames side by side based on a shared key column, exactly like a SQL join with the same four options on how to merge your data: outer, inner, left, and right join.

```
# define dataframe
df_1 = pd.DataFrame({"Country": ["Belgium", "France", "Germany"],
                     "Name": ["Tom", "Sarah", "Niels"],})
```

```
df_1
```

	Country	Name
0	Belgium	Tom
1	France	Sarah
2	Germany	Niels

```
# define 2 dataframes
df_2 = pd.DataFrame({"Country": ["Belgium", "France", "Spain"],
                     "Prefix": ["+32", "+33", "+34"]})
```

```
df_2
```

	Country	Prefix
0	Belgium	+32
1	France	+33
2	Spain	+34

```
# outer join
pd.merge(df_1, df_2, how="outer", on="Country")
```

	Country	Name	Prefix
0	Belgium	Tom	+32
1	France	Sarah	+33
2	Germany	Niels	NaN
3	Spain	NaN	+34

```
# inner join
pd.merge(df_1, df_2, how="inner", on="Country")
```

	Country	Name	Prefix
0	Belgium	Tom	+32
1	France	Sarah	+33

```
# left join
pd.merge(df_1, df_2, how="left", on="Country")
```

	Country	Name	Prefix
0	Belgium	Tom	+32
1	France	Sarah	+33
2	Germany	Niels	NaN

```
# right join
pd.merge(df_1, df_2, how="right", on="Country")
```

	Country	Name	Prefix
0	Belgium	Tom	+32
1	France	Sarah	+33
2	Spain	NaN	+34

2.8.4.6 Dropping

`drop_duplicates()` removes duplicate rows, and `drop()` removes whole columns (or rows) you no longer need.

```
# define dataframe
df_1 = pd.DataFrame({"Name": ["Tom", "Sarah", "Niels", "Tom", "Sarah"],
                     "Country": ["Belgium", "France", "Germany", "USA", "France"]})
```

```
df_1
```

	Name	Country
0	Tom	Belgium
1	Sarah	France
2	Niels	Germany
3	Tom	USA
4	Sarah	France

```
# drop duplicates
df_1.drop_duplicates()
```

	Name	Country
0	Tom	Belgium
1	Sarah	France
2	Niels	Germany
3	Tom	USA

```
# drop duplicates based on column
df_1.drop_duplicates(subset=["Name"])
```

	Name	Country
0	Tom	Belgium
1	Sarah	France
2	Niels	Germany

```
# drop variable
df_1.drop(columns=["Name"])
```

	Country
0	Belgium
1	France
2	Germany
3	USA
4	France

2.8.4.7 Handling Missing Values

`dropna()` removes rows with missing values (with several strategies for exactly which rows to drop), while `fillna()` replaces them with a value you specify, often per column, via a dictionary.

```
import numpy as np
# define dataframe
df_1 = pd.DataFrame({"Name": ["Tom", "Sarah", np.nan, "Tom", np.nan],
                     "Country": ["Belgium", "France", "Germany", "USA", np.nan]})
```

```
df_1
```

	Name	Country
0	Tom	Belgium
1	Sarah	France
2	NaN	Germany
3	Tom	USA
4	NaN	NaN

```
# drop rows with missing values
df_1.dropna()
```

	Name	Country
0	Tom	Belgium
1	Sarah	France
3	Tom	USA

```
# drop rows with missing values for all columns
df_1.dropna(how="all")
```

	Name	Country
0	Tom	Belgium
1	Sarah	France
2	NaN	Germany
3	Tom	USA

```
# keep every row with at least 1 non missing value
df_1.dropna(thresh=1)
```

	Name	Country
0	Tom	Belgium
1	Sarah	France
2	NaN	Germany
3	Tom	USA

```
# define dataframe
df_1 = pd.DataFrame({"Name": ["Tom", "Sarah", np.nan, "Tom", np.nan],
                     "gender": ["male", "female", "male", "male", np.nan],
                     "Spendings": [1023, 500, 120, np.nan, 203]})
```

```
df_1
```

	Name	gender	Spendings
0	Tom	male	1023.0
1	Sarah	female	500.0
2	NaN	male	120.0
3	Tom	male	NaN
4	NaN	NaN	203.0

```
# define imputation dictionary
column_imputations = {"Name": "UNKNOWN_NAME",
                      "Spendings": 0,
                      "gender": "male"}
```

```
# impute missing values
df_1.fillna(value=column_imputations)
```

	Name	gender	Spendings
0	Tom	male	1023.0
1	Sarah	female	500.0
2	UNKNOWN_NAME	male	120.0
3	Tom	male	0.0
4	UNKNOWN_NAME	male	203.0

2.8.4.8 Generating new data

`.apply()` lets you run a custom function over a column (or, with `axis=1`, over each row). This is a fast way to implement a transformation that doesn't already exist as a built-in method.

```
# define dataframe
df_1 = pd.DataFrame(
                    {"Spendings": [120, -5, 293, -1]}
)
```

```
df_1
```

	Spendings
0	120
1	-5
2	293
3	-1

```
# define function
def replace_negative_values(column_obs):
    if column_obs < 0:
        return(0)
    else:
        return(column_obs)
```

```
# apply function on column
df_1["Spendings"].apply(replace_negative_values)
```

```
0    120
1      0
2    293
3      0
Name: Spendings, dtype: int64
```

```
df_1
```

	Spendings
0	120
1	-5
2	293
3	-1

```
# apply same function on column with lambda expression
df_1["Spendings_correct"] = df_1["Spendings"].apply(lambda x: 0 if x < 0 else x)
```

```
# define dataframe
df_1 = pd.DataFrame(
                    {"Spendings": [120, 34, 293, 30],
                     "Country": ["Belgium", "France", "France", "Belgium"]}
)
```

```
df_1
```

	Spendings	Country
0	120	Belgium
1	34	France
2	293	France
3	30	Belgium

```
# define function that will be applied on each column row
def replace_wrong_values(country, spendings):
    if country == "Belgium":
        return(spendings + 100)
    else:
        return(spendings - 50)
```

```
# apply function on column
df_1["Spendings_correct"] = df_1.apply(lambda x: replace_wrong_values(x['Country'], x['Spendings']), ax
```

```
df_1
```

	Spendings	Country	Spendings_correct
0	120	Belgium	220
1	34	France	-16
2	293	France	243
3	30	Belgium	130

Chapter 3

Data Understanding

With the Python and `pandas` fundamentals from the previous chapter in hand, we can start on the first real phase of CRISP-DM: **data understanding**. Before you clean anything, visualize anything, or build a single model, you need to know what you're actually working with: which tables exist, how they relate to each other, what each variable means, where the data came from, and what you're legally allowed to do with it. Skipping this step is a classic rookie mistake: it's tempting to jump straight to modeling, but a model built on data you don't understand is a model you can't trust. And if the data can't be trusted, the model itself can't be trusted either: garbage in, garbage out.

Throughout this chapter, we'll use a single running case to keep things concrete. A newspaper publishing company (NPC) has approached us to help address rising customer **churn** rates, a trend accelerated by the rise of online news, tablets, and news aggregator apps. The NPC has handed over two tables: one describing its customers, and one describing their newspaper subscriptions. Getting to know these two tables is what this chapter is about.

3.1 The three levels of data understanding

It helps to think about data understanding as operating at three levels, from the coarsest to the finest:

- **Table level**: how many tables do you have, and how do they relate to each other?
- **Data level**: what type is each variable, and is the sample you're working with representative of the population you actually care about?
- **Variable level**: what does each individual variable look like, and are there quality issues (missing values, outliers, inconsistencies) you'll need to deal with?

We'll work through each level in turn, grounding every concept in NPC's customers and subscriptions tables. However, the variable level will be covered in the next chapter, data exploration.

3.2 Table level: how your tables relate

3.2.1 Entity-relationship diagrams

An **entity-relationship diagram (ERD)** is an overview of your tables and how they connect. For data understanding purposes, a simplified ERD is enough: you only need the entities (tables) and the relationships between them, not every attribute (or variable) inside each table. Each relationship is annotated with a **cardinality**: how many records on one side correspond to how many records on the other. There are three common cardinalities:

- **One-to-one**: each record on one side matches exactly one record on the other (e.g., one person has one national ID number).

- **One-to-many**: one record on one side can match several on the other (e.g., one customer can have several credit lines, but each credit line belongs to exactly one customer).
- **Many-to-many**: records on both sides can match several on the other (e.g., a subscription can include several products, and a product can appear in several subscriptions).

Every relationship is built on a **key variable**: a column that's unique per record in one table (the *key*) and reappears in the other table as a **foreign key**, pointing back to it.

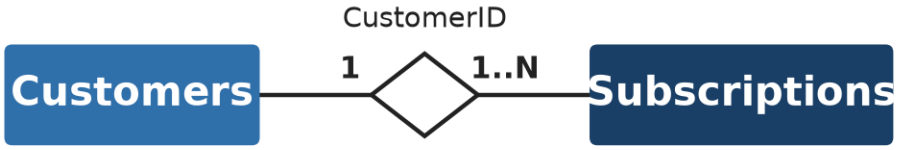


Figure 3.1: Simplified ERD for the NPC case: one customer has one-to-many subscriptions, linked by `CustomerID`.

Figure 3.1 shows the two entities in this simplified ERD: **Customers** and **Subscriptions**. They're connected by a single one-to-many relationship: one customer can have several subscriptions, but each subscription belongs to exactly one customer. The key variable is `CustomerID`: a column that uniquely identifies each customer, and reappears in the subscriptions table as a foreign key pointing back to the customer it belongs to.

3.2.2 Confirming a relationship in code

An ERD describes what you *expect* the relationships between tables to look like, but it's worth actually verifying that against the data itself. Real-world data doesn't always behave the way a diagram says it should. For NPC, we expect a one-to-many relationship between customers and subscriptions: each customer can have several subscriptions over time, but every subscription belongs to exactly one customer. We can check this by joining the two tables on the shared key, `CustomerID`, and counting how many subscriptions each customer has.

```
import pandas as pd

customers = pd.read_csv("../data/raw/customers.txt", sep=";")
subscriptions = pd.read_csv("../data/raw/subscriptions.txt", sep=";")

# left join keeps every customer, even those with zero matching subscriptions
merged = pd.merge(
    customers[["CustomerID"]],
    subscriptions[["CustomerID", "SubscriptionID"]],
    on="CustomerID",
    how="left",
)

# count how many subscriptions each customer has
subs_per_customer = merged.groupby("CustomerID")["SubscriptionID"].count()
subs_per_customer.describe()
```

```
count    1389.000000
mean        5.940965
std         6.657273
min         1.000000
25%         3.000000
50%         4.000000
75%         6.000000
max        50.000000
Name: SubscriptionID, dtype: float64
```

Every one of NPC's 1,389 customers has at least one subscription (the minimum is 1, not 0) and one customer has as many as 50. That confirms the relationship really is one-to-many in both directions we'd expect: no orphaned subscriptions, and no customers with zero subscriptions to explain. If the minimum had been 0, that would have been worth flagging as either a genuine business fact (some customers really have never subscribed) or a data quality problem (a broken join key). This is precisely the kind of information you want to catch here, before it quietly breaks a model three chapters from now.

3.2.3 Table descriptions

A **table description**, also called a **data dictionary**, is a short, plain-language definition of every variable in a table, and it's often the fastest way to get oriented in an unfamiliar dataset. A minimal one for the subscriptions table could look like this:

Variable	Definition
`SubscriptionID`	The subscription identifier
`CustomerID`	The customer identifier
`ProductID`	The product (newspaper) identifier
`StartDate`	The date the subscription started
`TotalPrice`	The total price paid for the subscription

In practice, you'll often build this table description directly from `pandas`. `.info()` gives you a fast overview of every column's name, non-null count, and dtype in one call. This will often be the natural starting point before writing out the fuller, business-meaning version manually.

```
customers.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1389 entries, 0 to 1388
Data columns (total 6 columns):
 #   Column      Non-Null Count  Dtype
---  ------      --------------  -----
 0   CustomerID  1389 non-null   int64
 1   Gender      1166 non-null   object
 2   DOB         1389 non-null   object
 3   District    1389 non-null   int64
 4   ZIP         1389 non-null   int64
 5   StreeID     1389 non-null   int64
dtypes: int64(4), object(2)
memory usage: 65.2+ KB
```

3.3 Data level: getting types and samples right

3.3.1 Data types

Every variable falls into one of a few basic types:

- **Continuous (numerical)**: data takes values on an interval, bounded or not (e.g.,income, sales, number of clicks).
- **Categorical (discrete)**: data takes a limited set of values, and splits further into:
  - **Nominal**: with no meaningful order (e.g., marital status, profession, loan purpose).
  - **Ordinal**: with a meaningful order (e.g., a credit rating, a letter grade (A, B, C, D)).
  - **Binary**: with exactly two values (e.g., gender, employment status, churn (yes/no)).

Getting this classification right matters more than it might seem: if a categorical variable like marital status gets read in as numeric, `pandas` (or any other tool) will happily compute its mean, which is a number with no meaning whatsoever.

When you read a file straight off disk, `pandas` doesn't know any of this by default.

> ⚠ The generic `object` trap
>
> `pandas` defaults almost everything it can't immediately identify as numeric to a generic `object` type, including dates, categories, and IDs. `object` is flexible, but it's rarely what you actually want: it silently hides the fact that a column needs converting, and lets a variable that should be `category` or `datetime` sit around as loosely-typed text until something downstream breaks down. Key take-away: be explicit about the type of each column right when you load the data (or after), rather than discovering the problem three steps later.

```
col_types = {
    "CustomerID": "string",
    "Gender": "category",
    "DOB": "string",
    "District": "category",
    "ZIP": "string",
    "StreeID": "string",
}

for col, dtype in col_types.items():
    customers[col] = customers[col].astype(dtype)

customers.dtypes
```

```
CustomerID    string[python]
Gender              category
DOB           string[python]
District            category
ZIP           string[python]
StreeID       string[python]
dtype: object
```

```
col_types_subs = {
    "SubscriptionID": "string",
    "CustomerID": "string",
    "ProductID": "string",
    "FormulaID": "string",
    "Pattern": "string",
```

```
    "PaymentType": "category",
    "PaymentStatus": "category",
}

for col, dtype in col_types_subs.items():
    subscriptions[col] = subscriptions[col].astype(dtype)

subscriptions.dtypes
```

```
SubscriptionID       string[python]
CustomerID           string[python]
ProductID            string[python]
Pattern              string[python]
StartDate                    object
EndDate                      object
NbrNewspapers                 int64
NbrStart                    float64
RenewalDate                  object
PaymentType                category
PaymentStatus              category
PaymentDate                  object
FormulaID            string[python]
GrossFormulaPrice           float64
NetFormulaPrice             float64
NetNewspaperPrice           float64
ProductDiscount             float64
FormulaDiscount             float64
TotalDiscount               float64
TotalPrice                  float64
TotalCredit                 float64
dtype: object
```

A few of these choices are worth explaining. IDs (`CustomerID`, `ZIP`) are set to `string` rather than a numeric type. Even though they're written as digits, you'll never compute their mean or sum, and treating them as numbers risks losing leading zeros. `Gender` and `District` become `category`, since they're nominal variables with a small, fixed set of values; `pandas` stores categories far more compactly than repeated strings, and several downstream tools (`groupby`, or popular plotting libraries) treat `category` columns specially. Dates like `DOB` are left as `string` for now and converted explicitly in the next section, once we've picked a consistent date format.

The subscriptions table gets the same treatment. Its four ID columns (`SubscriptionID`, `CustomerID`, `ProductID` and `FormulaID`) and `Pattern`, which is a code rather than a quantity, become `string`, and the payment columns `PaymentType` and `PaymentStatus` become `category`. Giving `CustomerID` the same type in both tables matters too: a `string` key cannot be merged with an `int64` key. The date columns stay `string` for now, just like `DOB`.

3.3.2 Sampling

Almost every analytical model is trained on a **sample** of the original data: a subset of past customers, rather than every customer a company will ever have. The key requirement for a good sample is that it stays **representative** of the customers the model will actually be applied to in the future. This creates a real trade-off: more historical data generally means a more robust model, but older data is less representative of today's customers and their behavior. For example, a churn model trained mostly on data from pre-COVID times might not generalize well to the post-COVID world, where their behavior changed drastically. The key takeaway is that your sample should be representative for the population and the business problem you

actually care about, not just the largest sample you can get your hands on (however, the more the better, as long as it's still representative).

> ⚠ Watch out for sampling bias
>
> It's easy to introduce bias without meaning to. If you sample only *currently active* customers to build a churn model, you've quietly excluded everyone who already churned and those are exactly the customers you most need examples of. Focusing on only the active customers is also a clear example of survivorship bias as seen in Chapter 1.

When the outcome you care about is rare, a random sample can end up with too few positive cases to learn from reliably. Churn is a classic example of this, since most customers in any given period *don't* churn. A more extreme case is fraud detection, where the number of fraudulent transactions is typically much smaller than legitimate ones, which is a good thing from a social perspective but poses a challenge for modeling. **Stratified sampling** addresses this by sampling separately within predefined groups (strata): if the full population is 99% non-churners and 1% churners, a stratified sample preserves that exact 99/1 split, rather than leaving it to chance.

3.4 Variable level: a first statistical read

3.4.1 Working with dates

NPC's date columns (when a subscription started, ended, was last paid, or renewed) are read in as plain text, since that's how they're stored in the file. Converting them to an actual `datetime` type is what lets you do date arithmetic (how long has this subscription run?) or extract components (which month did most subscriptions start?) later on. `pd.to_datetime()` handles the conversion; since NPC's dates are written `day/month/year`, we tell it the format explicitly rather than letting it guess.

```
date_cols = ["StartDate", "EndDate", "PaymentDate", "RenewalDate"]

# errors="coerce" turns any value that doesn't match the format into a missing value (NaT),
# instead of raising an error and stopping the whole conversion
subscriptions[date_cols] = subscriptions[date_cols].apply(
    lambda col: pd.to_datetime(col, format="%d/%m/%Y", errors="coerce")
)

customers["DOB"] = pd.to_datetime(customers["DOB"], format="%d/%m/%Y", errors="coerce")
subscriptions[date_cols].dtypes
```

```
StartDate      datetime64[ns]
EndDate        datetime64[ns]
PaymentDate    datetime64[ns]
RenewalDate    datetime64[ns]
dtype: object
```

3.4.2 Describing individual variables

With types and dates sorted out, the last step at the variable level is to actually look at what each column contains: its typical value, its spread, and how it relates to other variables. `pandas` gives you this directly, without needing a single chart yet (charts are exactly where the next chapter picks up).

For a categorical variable, `.value_counts()` shows you every category and how often it occurs:

```
subscriptions["PaymentStatus"].value_counts(dropna=False)
```

```
PaymentStatus
```

```
Paid        7831
Not Paid     421
Name: count, dtype: int64
```

For a numeric variable, `.describe()` gives you count, mean, standard deviation, and quartiles in one call. Calling it on the column of interest, `TotalPrice`, immediately shows something worth flagging: only 8,091 of the 8,252 subscriptions have a non-missing price.

```
subscriptions["TotalPrice"].describe()
```

```
count    8091.000000
mean      125.372010
std        95.119942
min         0.000000
25%        29.600000
50%        79.000000
75%       235.000000
max       635.960000
Name: TotalPrice, dtype: float64
```

Individual statistics are available as their own methods too, which is handy when you only need one number rather than the full summary. For example, the mean, variance, and standard deviation of `TotalPrice`:

```
print("mean:", subscriptions["TotalPrice"].mean(skipna=True))
print("variance:", subscriptions["TotalPrice"].var(skipna=True))
print("std:", subscriptions["TotalPrice"].std(skipna=True))
```

```
mean: 125.3720096403411
variance: 9047.803316850834
std: 95.11994174120815
```

Two variables can also be checked for how strongly they move together, using `.corr()`. Here, the number of newspapers included in a subscription correlates fairly strongly with what the customer ends up paying, which is reassuring, since the alternative (no relationship at all) would suggest a problem with one of the two columns.

```
subscriptions[["TotalPrice", "NbrNewspapers"]].corr().loc["TotalPrice", "NbrNewspapers"]
```

```
np.float64(0.898895317946639)
```

Finally, `.describe(include="all")` extends the summary to every column at once, numeric and categorical alike, and `.select_dtypes()` lets you restrict that to just the numeric columns when that's all you need:

```
subscriptions.select_dtypes(include="number").describe()
```

	NbrNewspapers	NbrStart	GrossFormulaPrice	NetFormulaPrice	NetNewspaperPrice	ProductDiscount
count	8252.000000	8231.000000	8091.000000	8091.000000	8091.000000	8091.000000
mean	159.824285	14.995748	135.877842	133.336547	0.856182	2.459114
std	117.965557	7.290724	92.596151	93.253428	0.201169	24.595209
min	2.000000	0.000000	0.169733	0.000000	0.000000	0.000000
25%	76.000000	10.000000	68.000000	68.000000	0.809211	0.000000
50%	77.000000	10.000000	79.000000	79.000000	0.878289	0.000000
75%	304.000000	25.000000	235.000000	235.000000	0.973684	0.000000
max	354.000000	30.000000	310.914306	276.000000	1.120000	267.000000

Already, from these plain numeric summaries a few things stand out: `TotalPrice` has missing values, and its range (from 0 to over 600) is wide enough to be worth a closer look. Whether those are outliers, a data

entry issue, or a perfectly normal reflection of very different subscription sizes is exactly the kind of question visualizing the data, in the next chapter, will help answer.

3.4.3 Saving your progress

Once a table has been read in with the right types, it's worth saving that cleaned-up version rather than re-deriving it from the raw text file every time you reopen the notebook. **Parquet** is a compressed, columnar file format built for exactly this: it's faster to read and write than CSV, keeps your dtypes intact (a `category` column stays a `category` column), and produces a smaller file on disk.

```
subscriptions.to_parquet("../data/raw/subscriptions.parquet", index=False)
customers.to_parquet("../data/raw/customers.parquet", index=False)
```

3.5 Where does the data come from

Not all data looks like NPC's two neat tables. Broadly, the data an organization works with comes from a handful of sources, each with its own trade-offs:

Source	Description
Transactional data	Structured, detailed records of transactions (purchases, payments, claims), typically stored in a relational database. Easy to aggregate into sums, averages, or trends over time.
Qualitative, expert data	Domain expertise from people with substantial business experience (a credit portfolio manager, a brand manager). Useful for sanity-checking a model's output against what a human expert would expect.
Publicly available data	External data anyone can access: macroeconomic indicators (GDP, inflation, unemployment), or open repositories like the UCI Machine Learning Repository or Kaggle.
Web data	Data gathered from social media, search engines, or websites, either via an API or by web scraping.

Web data deserves a closer look, since the two ways of collecting it, APIs and scraping, differ a lot in practice. An **API** (application programming interface) is a channel a platform deliberately provides for programmatic access to its data; it's usually the quicker, cleaner route, and most programming languages ship packages for talking to the APIs of major platforms. But most platforms require you to apply for access first, and the landscape has gotten stingier in recent years. For example, Meta/Facebook has severely restricted its API, and X/Twitter turned it into a paid service, even Reddit has recently tightened its API access. A few platforms still offer a free API worth knowing about, including decentralized X-alternatives like Mastodon and Bluesky. Be aware though that each platform serves its own specific audience, so data pulled from any single platform carries that audience's biases, and its veracity is worth treating with some skepticism.

Another option is **web scraping**, which involves programmatically loading a page and extracting data from its HTML, without going through an API. This method is more powerful and flexible, since it works on virtually any site. Modern scraping tools actually drive a real browser rather than just parsing raw HTML: they render the page's JavaScript, letting them log in, scroll, and click the way a human user would. That power comes with more legal exposure, and scraping sits in more of a legal gray zone. However, several platforms (X and Facebook among them) actively restrict automated access, and you should always check a site's terms of service before scraping it.

3.6 Data regulation: privacy and intellectual property rights

Being able to access or collect data doesn't automatically mean you're allowed to use it however you like. Two kinds of rights determine what's actually permitted:

- **Privacy rights** govern personal and sensitive information and how it's collected, monitored, and shared. In the EU, this is regulated by the GDPR.
- **Intellectual property rights** protect commercially valuable creations like patents, copyrights, and trade secrets. For data specifically, IP rights typically don't cover the raw data itself, but rather its particular representation: a structured database, or the specific content of a website.

3.6.1 GDPR

The **General Data Protection Regulation** (European Parliament and Council of the European Union 2016) is the EU's regulatory framework for individual privacy rights, and it applies more broadly than most people expect: to any information that can directly *or indirectly* identify someone. That includes not just names, but phone numbers, email addresses, IP addresses, device IDs, and cookies.

> **i** "Anonymous" data can still identify someone
>
> Even data that looks harmless on its own can carry real **re-identification risk**. An address alone might seem anonymous, but "Chapel Street 88, Brussels" identifies a specific person the moment there's only one occupant at that address. Stripping out obvious identifiers like names isn't the same as making data anonymous, and you have to think about what a *combination* of seemingly harmless fields can reveal.

GDPR is built around **seven key principles**. Rather than list them in the abstract, it's more useful to see what actually happens when a company violates each one. By now GDPR enforcement has produced a long list of real fines that make the abstract principles concrete.

- **Lawfulness, fairness, and transparency**: process data legally, without unfair discrimination, and be transparent about how it's used. Spain's La Liga football association was fined €250,000 after its official app used fans' phones to record ambient audio and check location, to detect bars illegally streaming matches. Fans were never clearly told their microphone would be used this way, which is a clear violation of transparency.[1]
- **Purpose limitation**: don't use data for anything beyond the purpose it was originally collected for. In the first-ever fine issued under Belgium's data protection authority, a mayor was fined €2,000 for emailing election campaign material to civilians whose addresses he'd only obtained via a copied-in complaint email. Of course, it was never intended that the complaints would be used for this purpose.[2]
- **Data minimization**: only collect what you actually need for the stated purpose, and be especially careful with sensitive attributes (ethnicity, religion, sexual or political preference). A Greek leasing company was fined €10,000 after a property-sale ad it posted included a photo showing a customer's car with its license plate clearly visible. This allowed his own social circle to identify him as someone whose family property was being sold off, far more personal data than the ad needed to include.[3]
- **Accuracy**: keep data correct and up to date. For example, a loan decision based on stale income data, or repeated contact attempts using an address you know is wrong, can itself be a violation. The starkest real-world example: the Dutch tax authority was fined €750,000 specifically for accuracy violations in an internal fraud-risk blacklist that kept incorrect and outdated fraud indicators on file. In some cases, someone stayed flagged as a suspected fraudster even after an investigation had cleared them, because that clearance was never recorded. Tens of thousands of families were wrongly accused of childcare

[1] "Spain — La Liga fined for using phone microphones in anti-piracy initiative", Linklaters DigiLinks, June 2019. Spanish AEPD decision, €250,000 fine.

[2] "First Belgian GDPR sanction: DPA fines mayor EUR 2,000", Lexology, 2019.

[3] "HDPA (Greece) - 34/2023", GDPRhub, 2023. €10,000 fine for violating the data minimization principle (Article 5(1)(c) GDPR).

benefits fraud as a result, in a scandal that eventually brought down the Dutch government.[4]

- **Storage limitation**: don't keep identifiable data longer than the purpose actually requires. Athens's public transport operator was fined €50,000 for retaining ticketing data including passport numbers for years with no justified retention period,[5] and French retailer Brico Privé was fined €500,000 for, among other issues, keeping inactive customers' data for over five years.[6]
- **Integrity and confidentiality**: keep data secure, and restrict access to only the people who genuinely need it. An Amsterdam hospital, OLVG, was fined €440,000 after an investigation found no effective checks on which staff were accessing patient medical records, some without any work-related need to.[7]
- **Accountability**: organizations must be able to actively demonstrate their compliance, not just claim it. The same Dutch tax authority case is instructive here too: investigators found the agency hadn't properly involved its own data protection officer before rolling out the fraud blacklist. This is exactly the kind of documented internal oversight the accountability principle exists to guarantee.[8]

Two mechanisms are commonly used to reduce privacy risk while still keeping data useful: **anonymized data** has been processed so it can no longer be traced back to an individual at all (hard to achieve in practice without destroying most of its analytical value), while **pseudonymized data** replaces direct identifiers with a code. For example, customer names and emails replaced for an ID like `CUST_001`, with the mapping back to real identities kept separately and securely. Pseudonymization is far more common in practice, since it keeps data usable for analysis while still meaningfully reducing exposure if it leaks.

One trap worth calling out explicitly: GDPR still applies to social media and web data, even though it's publicly visible. A username on a public platform is a weaker anonymizer than it looks. For example, a handle like `@POTUS` unambiguously identifies a specific individual. Moreover, scraping or analyzing public posts still needs a legal basis, such as consent or a demonstrable public interest, even when informing every individual user isn't practically feasible.

3.7 Where we go from here

At this point, we know what tables we have, how they relate, what type each variable is, where the data came from, and what we're legally allowed to do with it. A first look at the plain numbers has already flagged a couple of things worth investigating further (`TotalPrice`'s missing values and wide range chief among them). The next chapter is where we put those numbers into pictures: visualizing distributions, spotting outliers, and digging into the data quality issues this chapter has only hinted at.

[4] "Dutch government fraud scandal leads to record-breaking GDPR fine", Clifford Chance, April 2022. Dutch DPA (Autoriteit Persoonsgegevens) decision of 12 April 2022, €3.7 million total fine made up of six separate fines, including €750,000 for violating the accuracy principle (Article 5(1)(d) GDPR).

[5] "HDPA (Greece) - 30/2023", GDPRhub, 2023. €50,000 fine for violating the storage limitation principle (Article 5(1)(e) GDPR).

[6] "CNIL (France) - SAN-2021-008", GDPRhub, 2021. €500,000 fine covering several violations, including data retention beyond the stated purpose.

[7] "The Netherlands: 440,000 EUR fine for hospital re. unauthorised access to medical records", Lexology, 2021. Dutch DPA decision against OLVG hospital, Amsterdam.

[8] "Dutch government fraud scandal leads to record-breaking GDPR fine", Clifford Chance, April 2022. Dutch DPA (Autoriteit Persoonsgegevens) decision of 12 April 2022, €3.7 million total fine made up of six separate fines, including €750,000 for violating the accuracy principle (Article 5(1)(d) GDPR).

Chapter 4

Data Exploration

The previous chapter discussed the **variable level** of data understanding: once you know which tables you have and what type each column is, the next question is what each variable actually *looks like*, and whether it has quality issues you'll need to deal with before you can trust a model built on it. That's what this chapter covers: visualizing individual variables to spot patterns, and detecting and treating the most common data quality issues: multicollinearity, missing values, and outliers.

We'll continue with NPC's customers and subscriptions tables from the previous chapter.

4.1 Why visualize

Summary statistics alone can be dangerously misleading. **Anscombe's quartet** (Anscombe 1973) is the classic illustration: four small datasets that share the same mean, variance, and correlation for `x` and `y` and yet look completely different when plotted. One is a clean linear relationship, one is a clear curve, another has a straight line that fits poorly due to a single outlier, and one is a vertical line of points plus one outlier. `.describe()` can't tell these apart. A scatter plot can immediately visualize the differences. That's the whole point of visual exploration: it shows patterns, outliers, and relationships that summary statistics alone can hide.

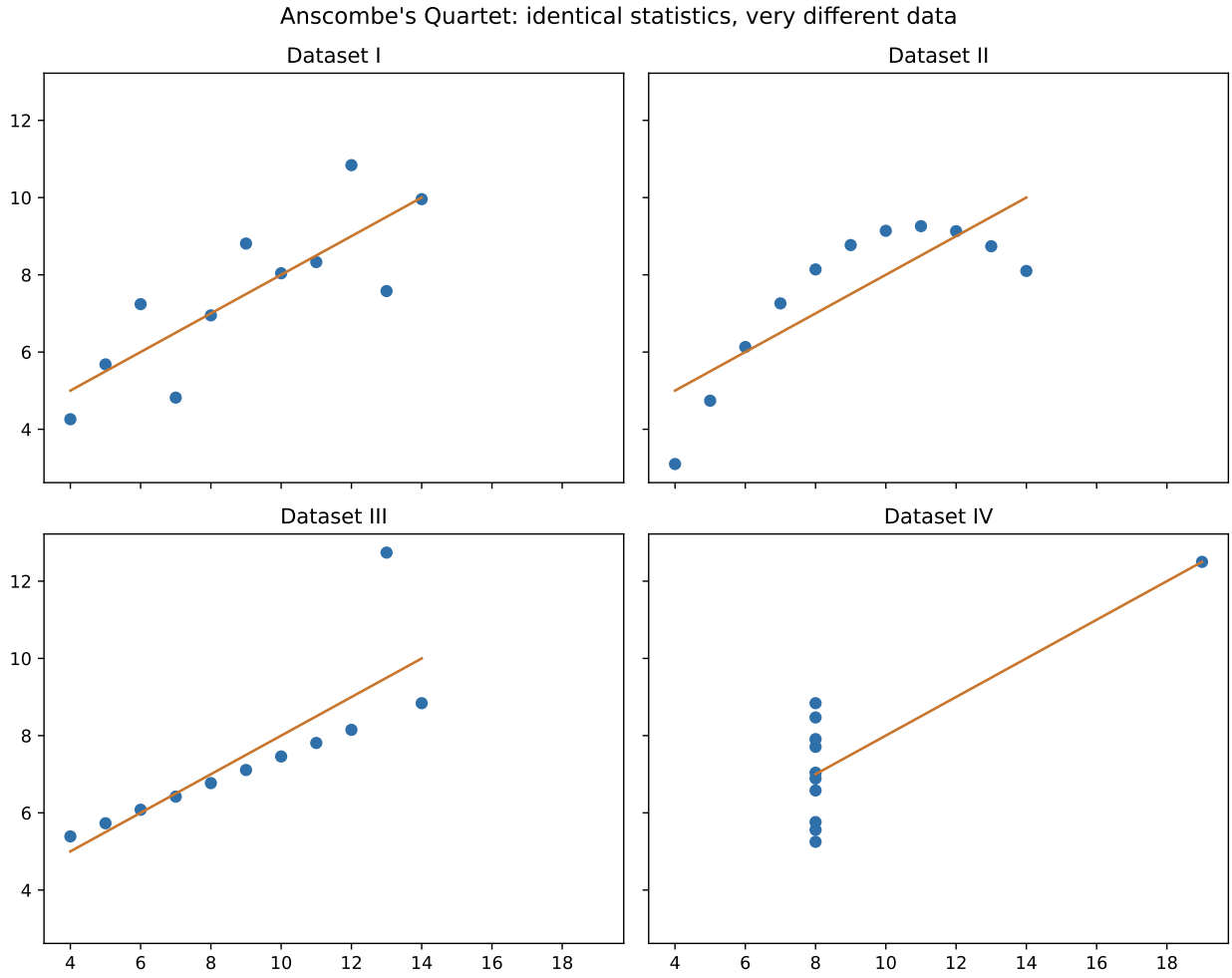
```
import matplotlib.pyplot as plt
import pandas as pd
import numpy as np

anscombe = pd.DataFrame({
    "dataset": ["I"] * 11 + ["II"] * 11 + ["III"] * 11 + ["IV"] * 11,
    "x": [10, 8, 13, 9, 11, 14, 6, 4, 12, 7, 5] * 3
         + [8, 8, 8, 8, 8, 8, 8, 19, 8, 8, 8],
    "y": [8.04, 6.95, 7.58, 8.81, 8.33, 9.96, 7.24, 4.26, 10.84, 4.82, 5.68,
          9.14, 8.14, 8.74, 8.77, 9.26, 8.10, 6.13, 3.10, 9.13, 7.26, 4.74,
          7.46, 6.77, 12.74, 7.11, 7.81, 8.84, 6.08, 5.39, 8.15, 6.42, 5.73,
          6.58, 5.76, 7.71, 8.84, 8.47, 7.04, 5.25, 12.50, 5.56, 7.91, 6.89],
})

# Check whether the four datasets really do share the same summary statistics
summary = anscombe.groupby("dataset").agg(
    mean_x=("x", "mean"), mean_y=("y", "mean"),
    std_x=("x", "std"), std_y=("y", "std"),
    corr=("x", lambda s: s.corr(anscombe.loc[s.index, "y"])),
)
```

```
summary.round(2)
```

dataset	mean_x	mean_y	std_x	std_y	corr
I	9.0	7.5	3.32	2.03	0.82
II	9.0	7.5	3.32	2.03	0.82
III	9.0	7.5	3.32	2.03	0.82
IV	9.0	7.5	3.32	2.03	0.82



It's worth being clear about what kind of visualization this chapter is about. Visualization serves two different purposes:

- **Explore**: you're the analyst, and the goal is discovering insights in the data as fast as possible. Speed and discovery matter more than polish.
- **Explain**: someone else is the audience, and the goal is communicating a specific finding or message clearly. It is also more about creating a compelling narrative and conveying your message in the proper way.

This chapter is entirely about the first one. The charts you'll build here are working tools, not fancy presentation slides.

The charts covered in this chapter are what you need for exploration, but they're a small fraction of what's out there. For example, treemaps, Sankey diagrams, spatial maps, slope charts, and dozens of others each have situations where they're the right option. Once your goal shifts from *exploring* to *explaining*, the chart itself is only half the job; the other half is design choices (how to remove clutter, where the attention should go, what to cut) that turn a chart into a story. That's a large enough topic that it deserves books of its own. Two excellent reading materials, if you want to go deeper: Cole Nussbaumer Knaflic's *Storytelling with Data* and Brent Dykes's *Effective Data Storytelling*.

4.1.1 Matplotlib basics

Python's core plotting library is `matplotlib`. Almost everything in `matplotlib` follows the same three-part structure:

- A **figure** is the overall canvas.
- One or more **axes** are the actual plotting areas inside that pane (confusingly, one "axes" is a single plot, not an axis line).
- An **artist** is everything else you can see (e.g., lines, text, bars, points).

In practice, you'll almost always start a plot the same way: create a figure and axes together with `plt.subplots()`, draw onto the axes using a specific chart type, then label it.

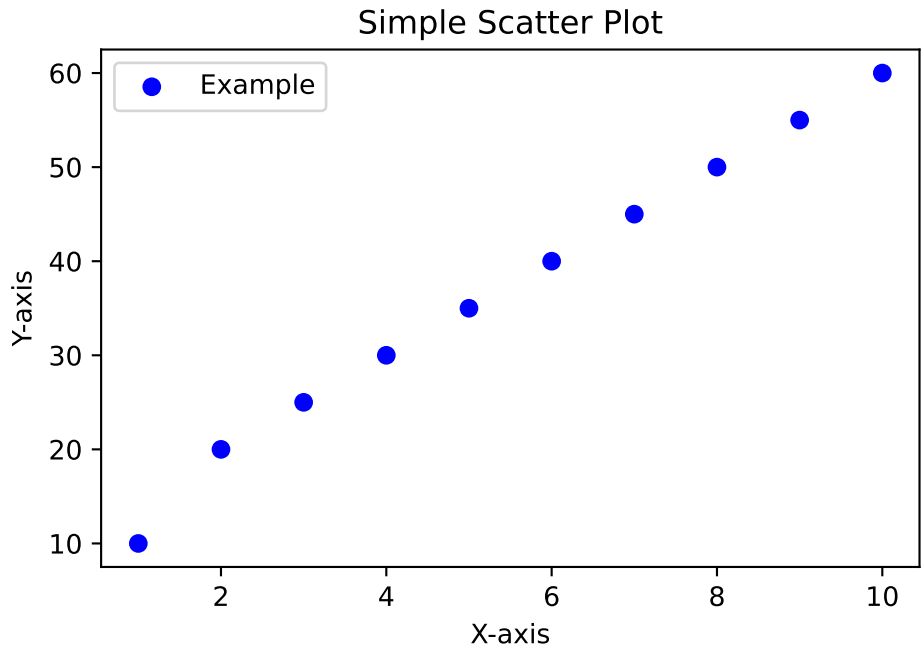
```
import matplotlib.pyplot as plt

x = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
y = [10, 20, 25, 30, 35, 40, 45, 50, 55, 60]

fig, ax = plt.subplots()
ax.scatter(x, y, label="Example", color="blue", marker="o")

ax.set_title("Simple Scatter Plot")
ax.set_xlabel("X-axis")
ax.set_ylabel("Y-axis")
ax.legend()

plt.show()
```



We'll also use `seaborn`, a library built on top of `matplotlib` that trades some of its flexibility for far more concise syntax and better default styling, especially for statistical plots like regression lines, heatmaps, and pair plots.

4.2 The core chart types

There are dozens of chart types, but for data exploration a small toolkit covers almost everything: **bar charts**, **line charts**, **scatter plots**, **histograms**, and **box plots**.

To demonstrate these, we will load the NPC data we saved at the end of the previous chapter.

```
import pandas as pd
import numpy as np
import seaborn as sns

subscriptions = pd.read_parquet("../data/raw/subscriptions.parquet")
customers = pd.read_parquet("../data/raw/customers.parquet")
```

4.2.1 Comparing groups: bar charts

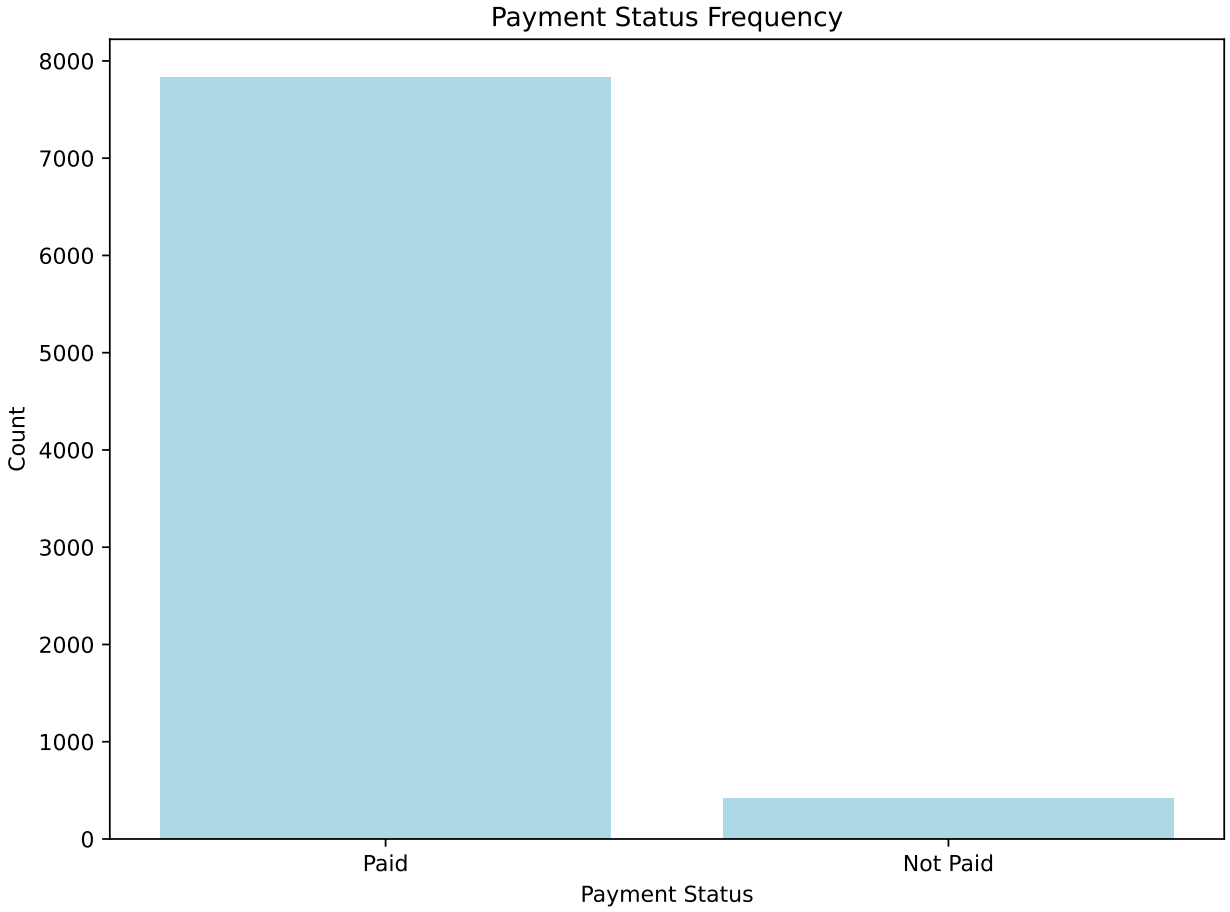
A bar chart represents each category's value or frequency as a vertical or horizontal rectangular bar and is useful whenever you want to compare discrete categories against each other.

```
fig, ax = plt.subplots(figsize=(8, 6))

payment_counts = subscriptions["PaymentStatus"].value_counts()
ax.bar(payment_counts.index, payment_counts.values, color="lightblue")

ax.set_title("Payment Status Frequency")
ax.set_xlabel("Payment Status")
ax.set_ylabel("Count")
```

```
plt.tight_layout()
plt.show()
```



Whereas a vertical bar chart is often the default, a horizontal one, with the counts written directly on the bars, is often easier to read once category labels get long:

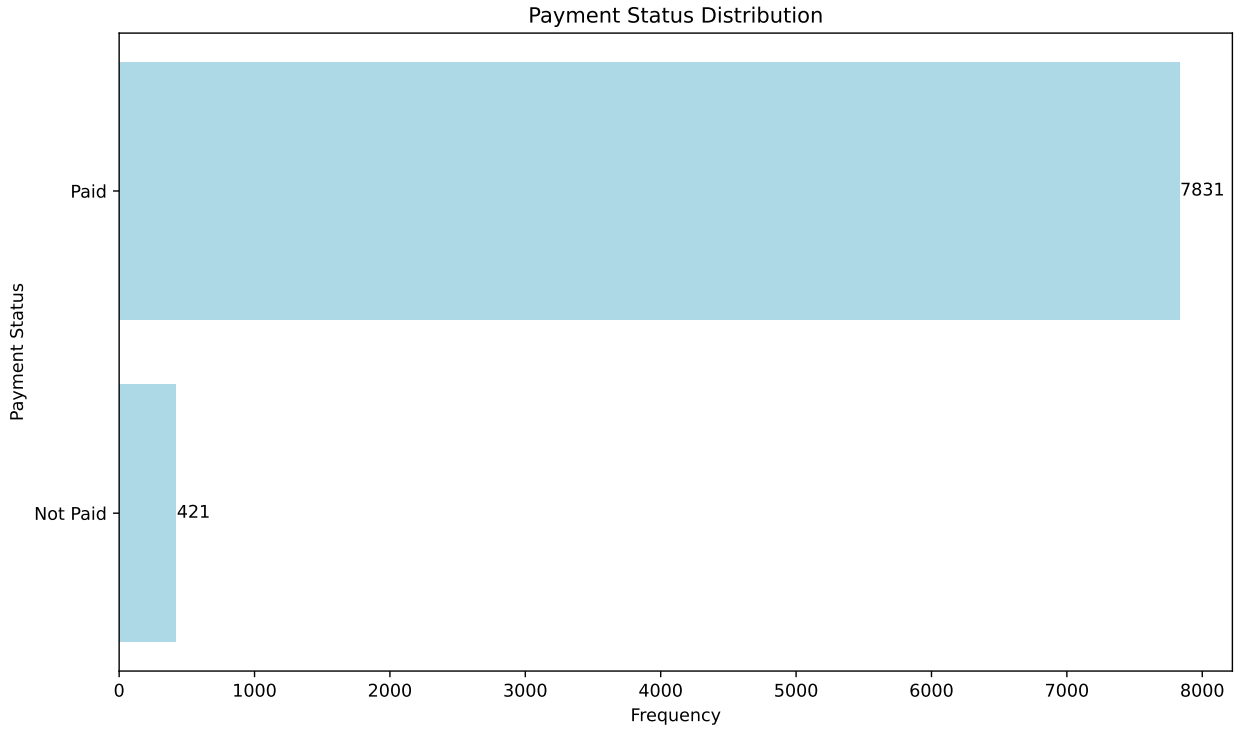
```
fig, ax = plt.subplots(figsize=(10, 6))

payment_counts = subscriptions["PaymentStatus"].value_counts(ascending=True)
bars = ax.barh(payment_counts.index, payment_counts.values, color="lightblue")

for bar, value in zip(bars, payment_counts.values):
    ax.text(bar.get_width() + 5, bar.get_y() + bar.get_height() / 2,
            str(value), ha="left", va="center")

ax.set_title("Payment Status Distribution")
ax.set_xlabel("Frequency")
ax.set_ylabel("Payment Status")

plt.tight_layout()
plt.show()
```



> **Bar chart ground rules**
>
> Always plot bars against a zero baseline. Starting the axis somewhere else exaggerates differences and misleads the reader, even if you didn't mean to. And be cautious with pie and donut charts as an alternative: humans are much better at comparing bar *length* than comparing angles or areas, so a bar chart usually communicates the same information more clearly.

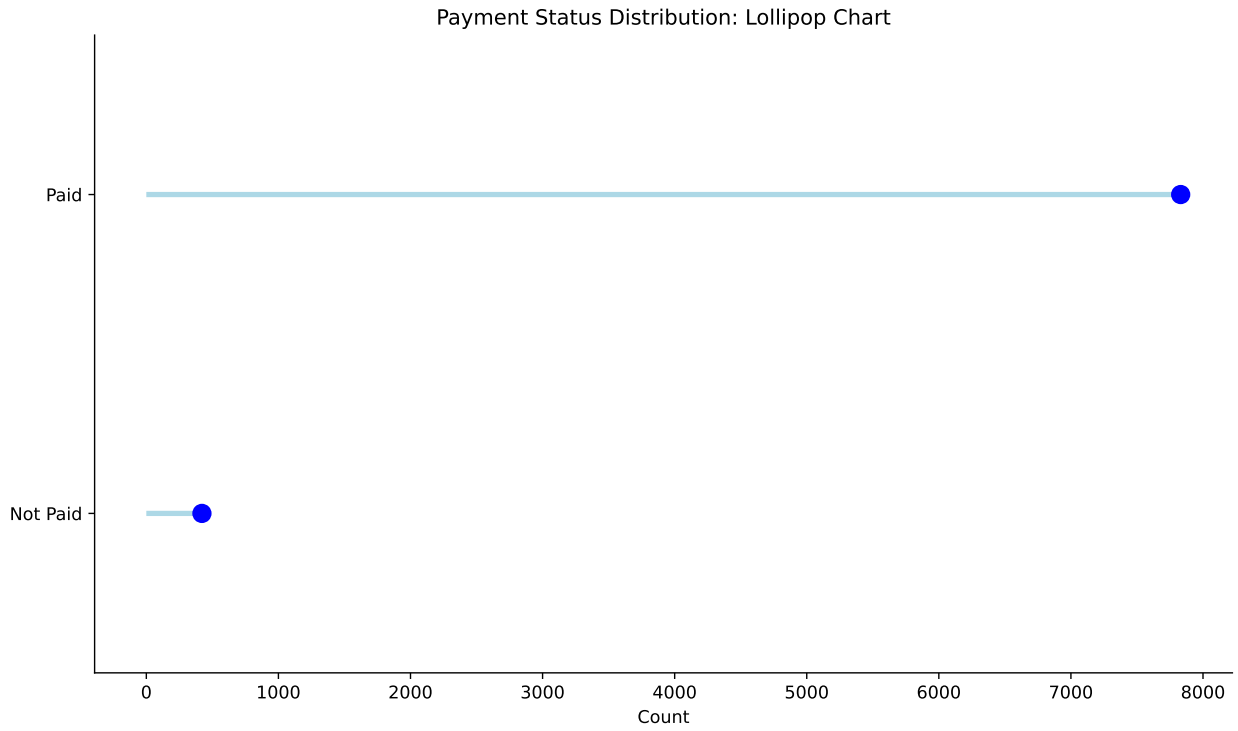
A **lollipop chart**, a thin line plus a single dot instead of a full bar, communicates the same comparison with noticeably less visual noise. With a fraction of the ink of a bar chart, it's often easier for the eye (and brain) to parse (lower noise-to-ink ratio), since there's less shape to take in per category:

```
fig, ax = plt.subplots(figsize=(10, 6))

ax.hlines(y=range(len(payment_counts)), xmin=0, xmax=payment_counts.values,
          color="lightblue", linewidth=3)
ax.scatter(payment_counts.values, range(len(payment_counts)), color="blue", s=100, zorder=3)

ax.set_yticks(range(len(payment_counts)))
ax.set_yticklabels(payment_counts.index)
ax.set_ylim(-0.5, len(payment_counts) - 0.5)
ax.set_title("Payment Status Distribution: Lollipop Chart")
ax.set_xlabel("Count")
ax.spines["top"].set_visible(False)
ax.spines["right"].set_visible(False)

plt.tight_layout()
plt.show()
```



4.2.2 Showing a trend: line charts

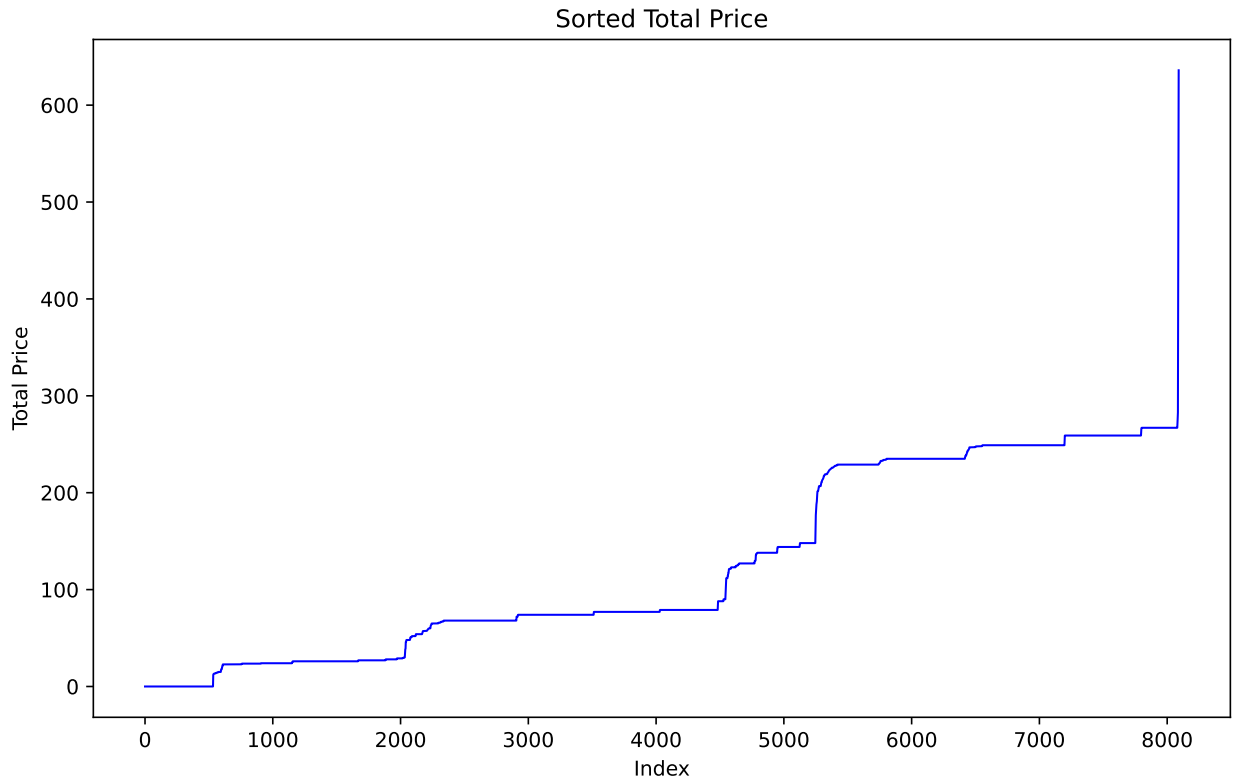
A line chart connects data points to show how a variable changes across an ordered sequence, almost always time but not necessarily.

To show that a line chart can be used to visualize any ordered series, let's look at a simple example. Plotting `TotalPrice`'s sorted values against their rank position isn't a true time series, but it's a quick way to see the overall shape of a distribution and spot outliers at either end:

```
fig, ax = plt.subplots(figsize=(10, 6))

sorted_prices = np.sort(subscriptions["TotalPrice"].dropna())
ax.plot(range(len(sorted_prices)), sorted_prices, color="blue", linewidth=1)

ax.set_title("Sorted Total Price")
ax.set_xlabel("Index")
ax.set_ylabel("Total Price")
plt.show()
```



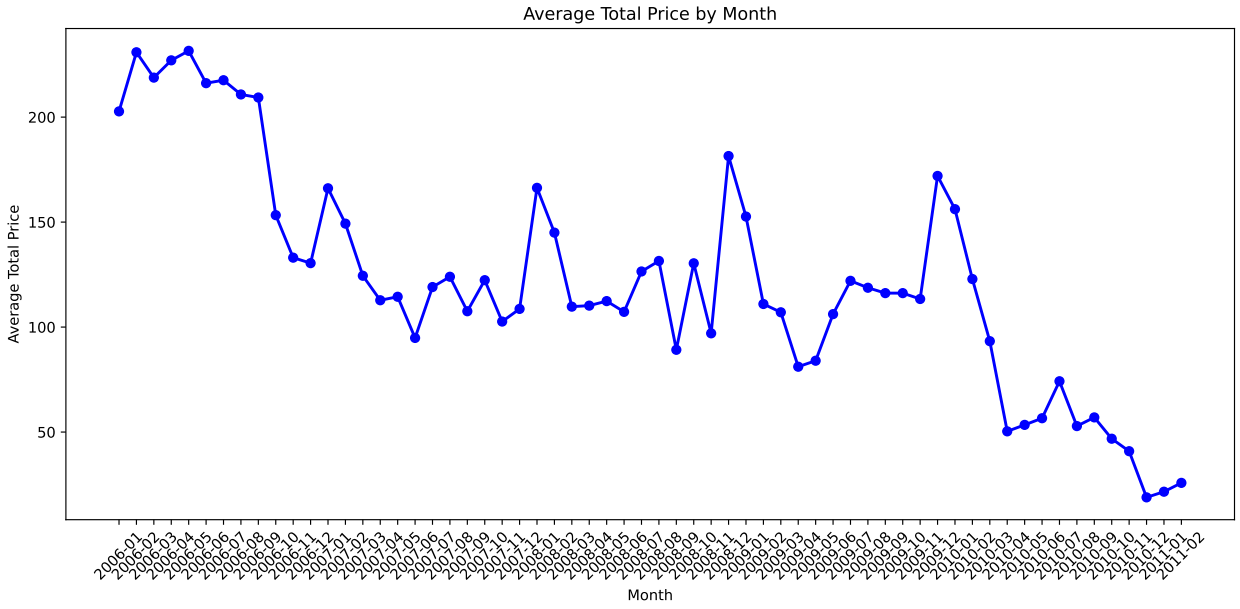
For an actual time series, plot against real dates. Here's the average subscription price by month:

```
subscriptions_monthly = subscriptions.copy()
subscriptions_monthly["StartDate"] = pd.to_datetime(subscriptions_monthly["StartDate"])
subscriptions_monthly["YearMonth"] = subscriptions_monthly["StartDate"].dt.to_period("M")

monthly_avg = subscriptions_monthly.groupby("YearMonth")["TotalPrice"].mean()

fig, ax = plt.subplots(figsize=(12, 6))
ax.plot(monthly_avg.index.astype(str), monthly_avg.values, color="blue", linewidth=2, marker="o")
ax.set_title("Average Total Price by Month")
ax.set_xlabel("Month")
ax.set_ylabel("Average Total Price")

plt.xticks(rotation=45)
plt.tight_layout()
plt.show()
```



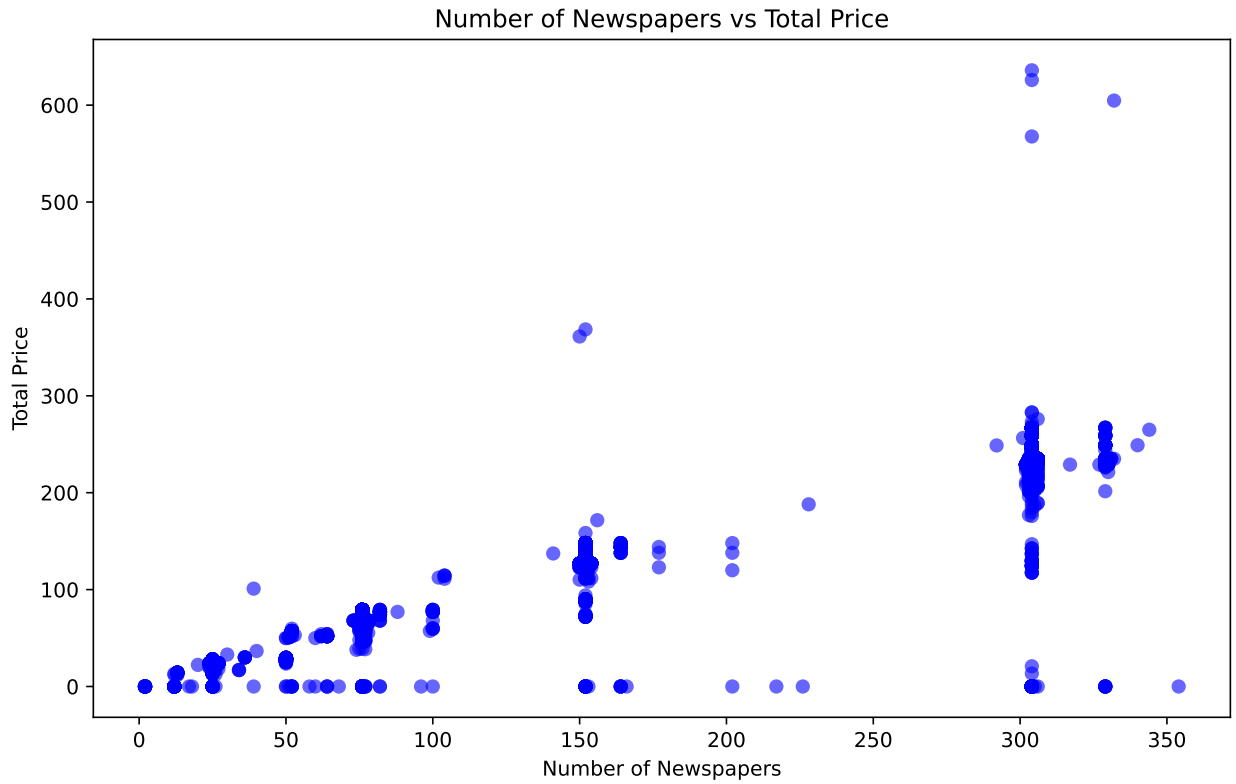
A couple of practical rules of thumb: pick a measurement interval that's neither so long you lose the trend nor so short the chart is dominated by noise. Also, avoid cramming more than about 5–7 lines onto one chart. If you have more than that, split it into several smaller charts instead.

4.2.3 Showing relationships: scatter plots

A scatter plot places one point per observation on two numeric axes, making it easy to spot correlations, clusters, and outliers between two variables at once.

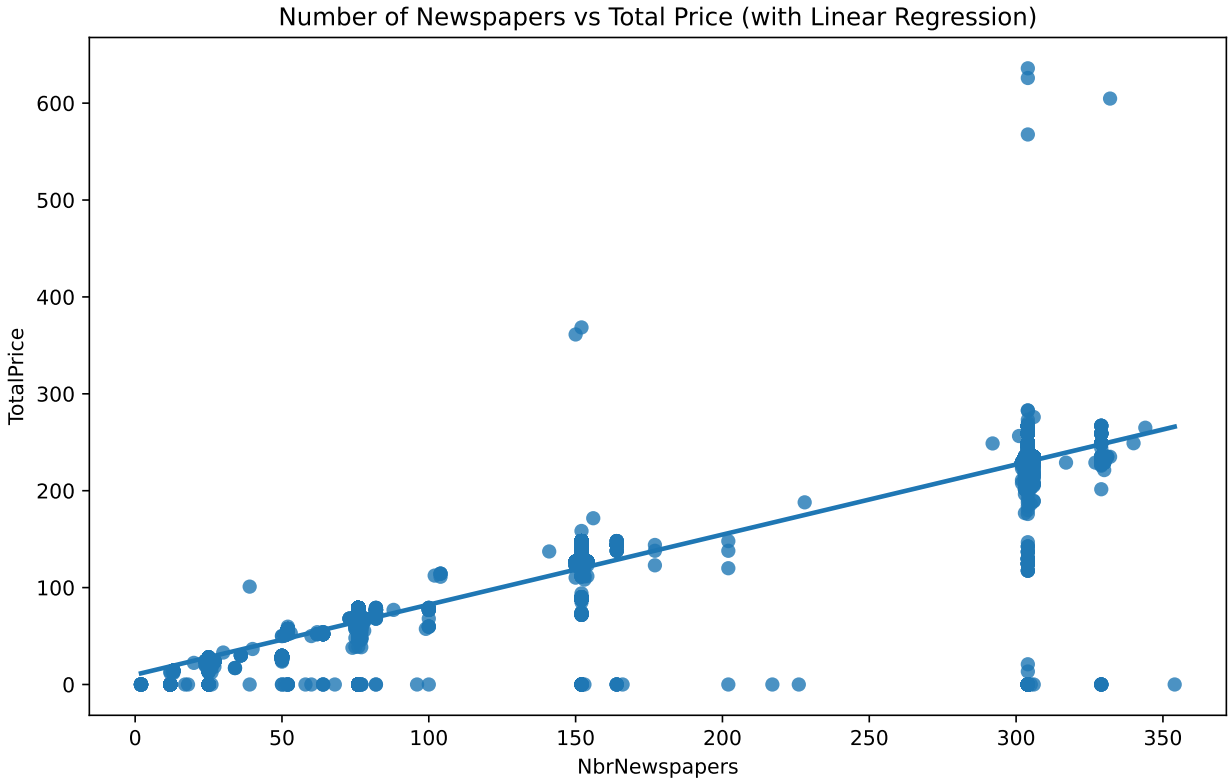
```
fig, ax = plt.subplots(figsize=(10, 6))
ax.scatter(subscriptions["NbrNewspapers"], subscriptions["TotalPrice"], alpha=0.6, color="blue")

ax.set_title("Number of Newspapers vs Total Price")
ax.set_xlabel("Number of Newspapers")
ax.set_ylabel("Total Price")
plt.show()
```



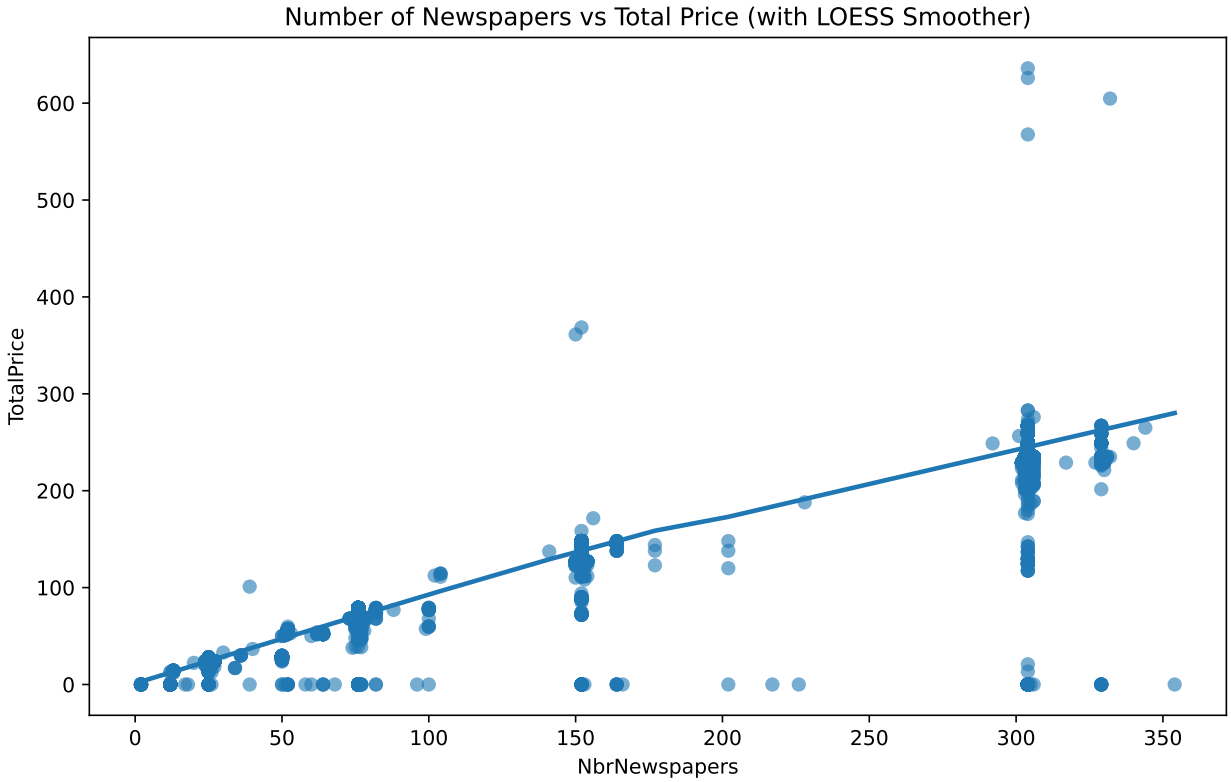
Adding a trend line makes the relationship easier to read at a glance. This is where `seaborn` starts to pay off against plain `matplotlib`. `sns.regplot()` draws the scatter plot and fits a line in one call:

```
fig, ax = plt.subplots(figsize=(10, 6))
sns.regplot(data=subscriptions, x="NbrNewspapers", y="TotalPrice", ax=ax)
ax.set_title("Number of Newspapers vs Total Price (with Linear Regression)")
plt.show()
```



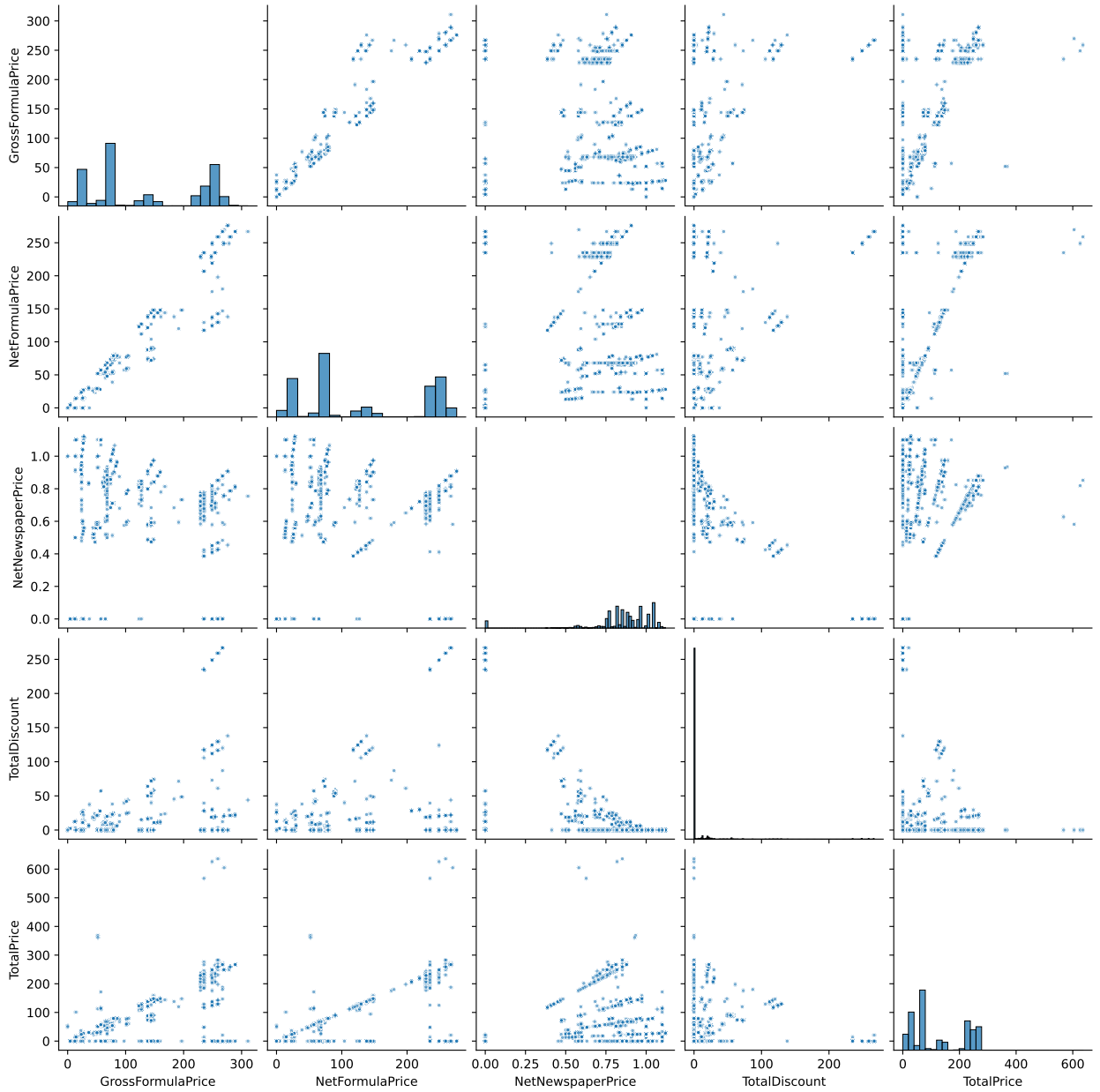
If the relationship isn't linear, a **LOESS** (locally estimated scatterplot smoothing) curve, a nonparametric fit built from many small local regressions, can reveal a nonlinear trend a straight line would miss:

```
fig, ax = plt.subplots(figsize=(10, 6))
sns.regplot(data=subscriptions, x="NbrNewspapers", y="TotalPrice",
            lowess=True, scatter_kws={"alpha": 0.6}, ax=ax)
ax.set_title("Number of Newspapers vs Total Price (with LOESS Smoother)")
plt.show()
```



In practice, you rarely want just one scatter plot in data exploration. You want to see every pair of numeric variables at once to understand their relationships. `seaborn`'s `pairplot()` builds that whole grid in a single call, with histograms of each individual variable down the diagonal:

```
numeric_cols = ["GrossFormulaPrice", "NetFormulaPrice", "NetNewspaperPrice",
                "TotalDiscount", "TotalPrice"]
sns.pairplot(subscriptions[numeric_cols].dropna(), diag_kind="hist",
             plot_kws={"alpha": 0.6, "s": 10})
plt.show()
```



As with any correlation, keep in mind that a strong relationship in a scatter plot doesn't imply causation: it's a pattern worth investigating, not a conclusion on its own.

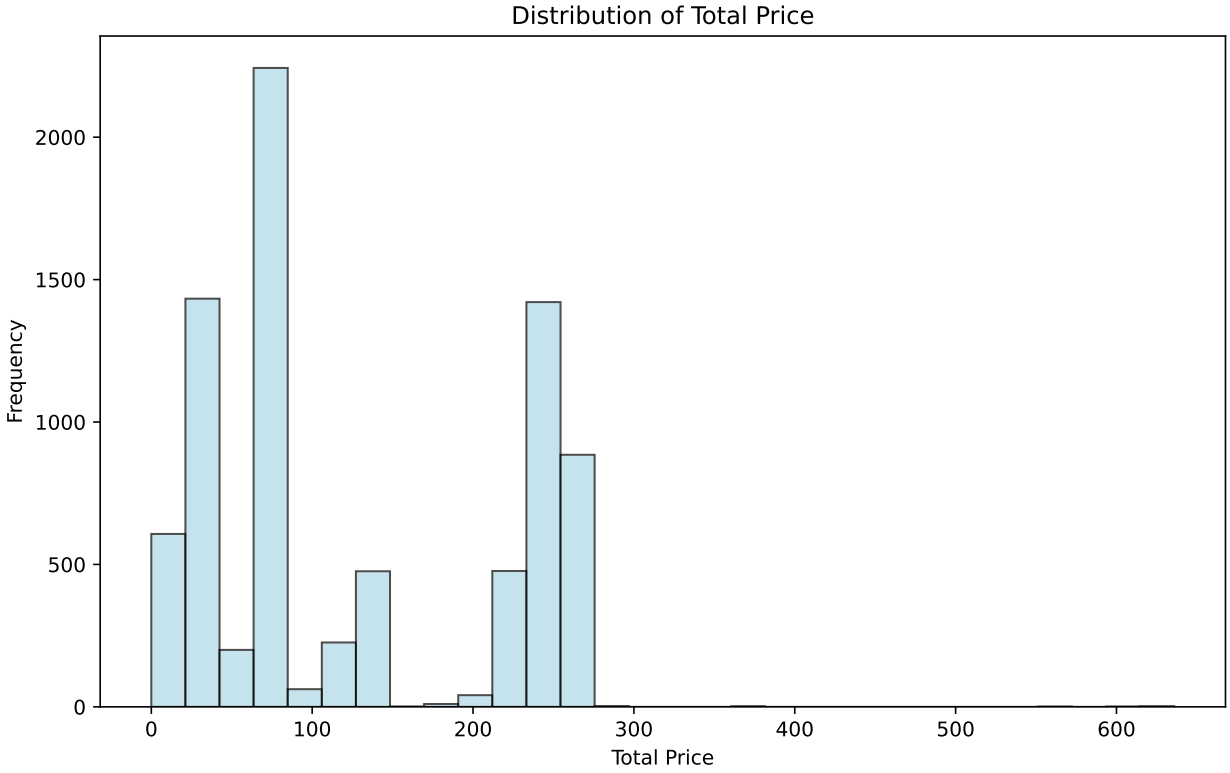
4.2.4 Showing a distribution: histograms and box plots

A histogram groups a variable's values into bins and shows how many observations fall into each one. It is considered the standard way to see a variable's distribution, shape, spread, and skew.

```
fig, ax = plt.subplots(figsize=(10, 6))
ax.hist(subscriptions["TotalPrice"].dropna(), bins=30, color="lightblue",
        edgecolor="black", alpha=0.7)

ax.set_title("Distribution of Total Price")
ax.set_xlabel("Total Price")
ax.set_ylabel("Frequency")
```

```
plt.show()
```

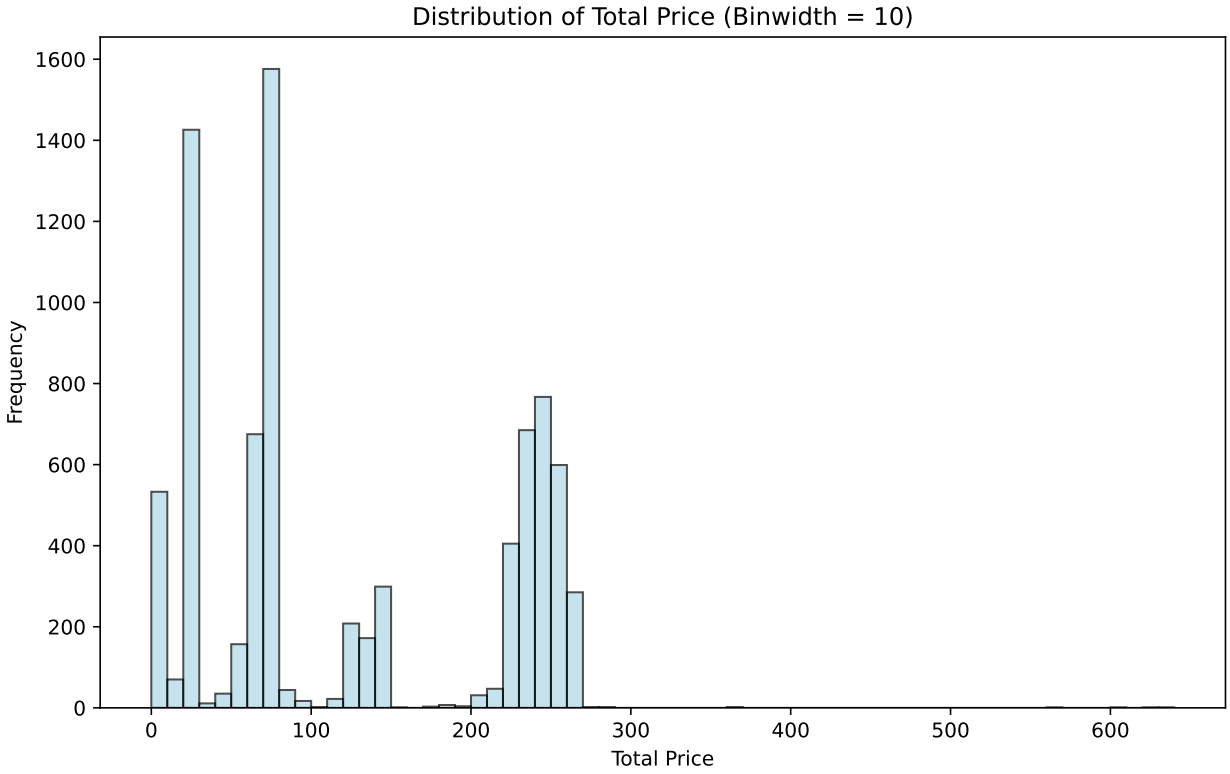


If you don't specify the bins, `matplotlib` picks a bin width automatically (`(max - min) / number of bins`). You can also fix the bin width directly:

```
fig, ax = plt.subplots(figsize=(10, 6))

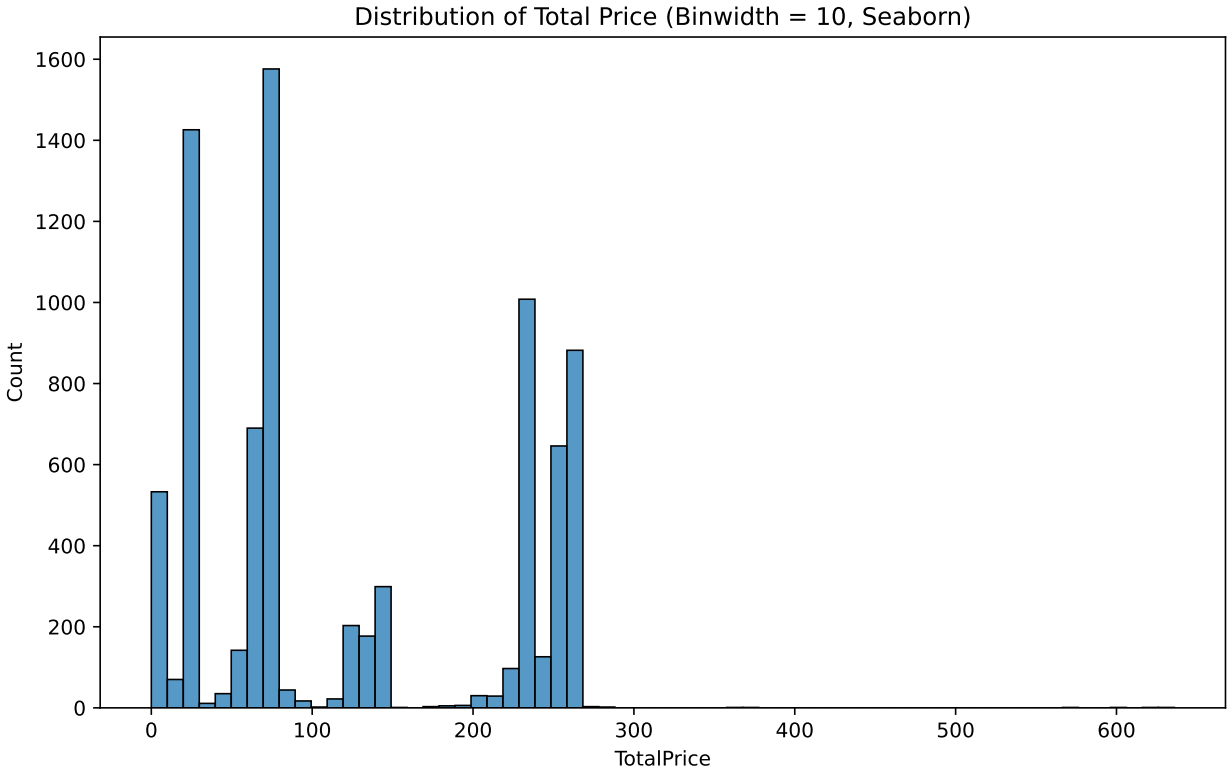
min_price = subscriptions["TotalPrice"].min()
max_price = subscriptions["TotalPrice"].max()
bins = np.arange(min_price, max_price + 10, 10)

ax.hist(subscriptions["TotalPrice"].dropna(), bins=bins, color="lightblue",
        edgecolor="black", alpha=0.7)
ax.set_title("Distribution of Total Price (Binwidth = 10)")
ax.set_xlabel("Total Price")
ax.set_ylabel("Frequency")
plt.show()
```



`seaborn` offers the same thing with a nicer default look, and lets you set `binwidth` directly:

```
fig, ax = plt.subplots(figsize=(10, 6))
sns.histplot(data=subscriptions, x="TotalPrice", binwidth=10, ax=ax)
ax.set_title("Distribution of Total Price (Binwidth = 10, Seaborn)")
plt.show()
```



A **box plot** packs a variable's median, quartiles, and potential outliers into one compact shape, and is especially useful for comparing a distribution and the basic statistics across groups. Figure 4.1 summarizes the key elements of a box plot.

Building this by hand in `matplotlib` needs the data reshaped into one list per group, which gets verbose fast:

```
fig, ax = plt.subplots(figsize=(10, 6))

data_for_boxplot = [group["TotalPrice"].dropna().values
                    for name, group in subscriptions.groupby("PaymentType", observed=True)]
labels = subscriptions["PaymentType"].dropna().unique()

ax.boxplot(data_for_boxplot, tick_labels=labels, patch_artist=True,
           boxprops=dict(facecolor="lightblue"))

ax.set_title("Total Price by Payment Type")
ax.set_xlabel("Payment Type")
ax.set_ylabel("Total Price")
plt.tight_layout()
plt.show()
```

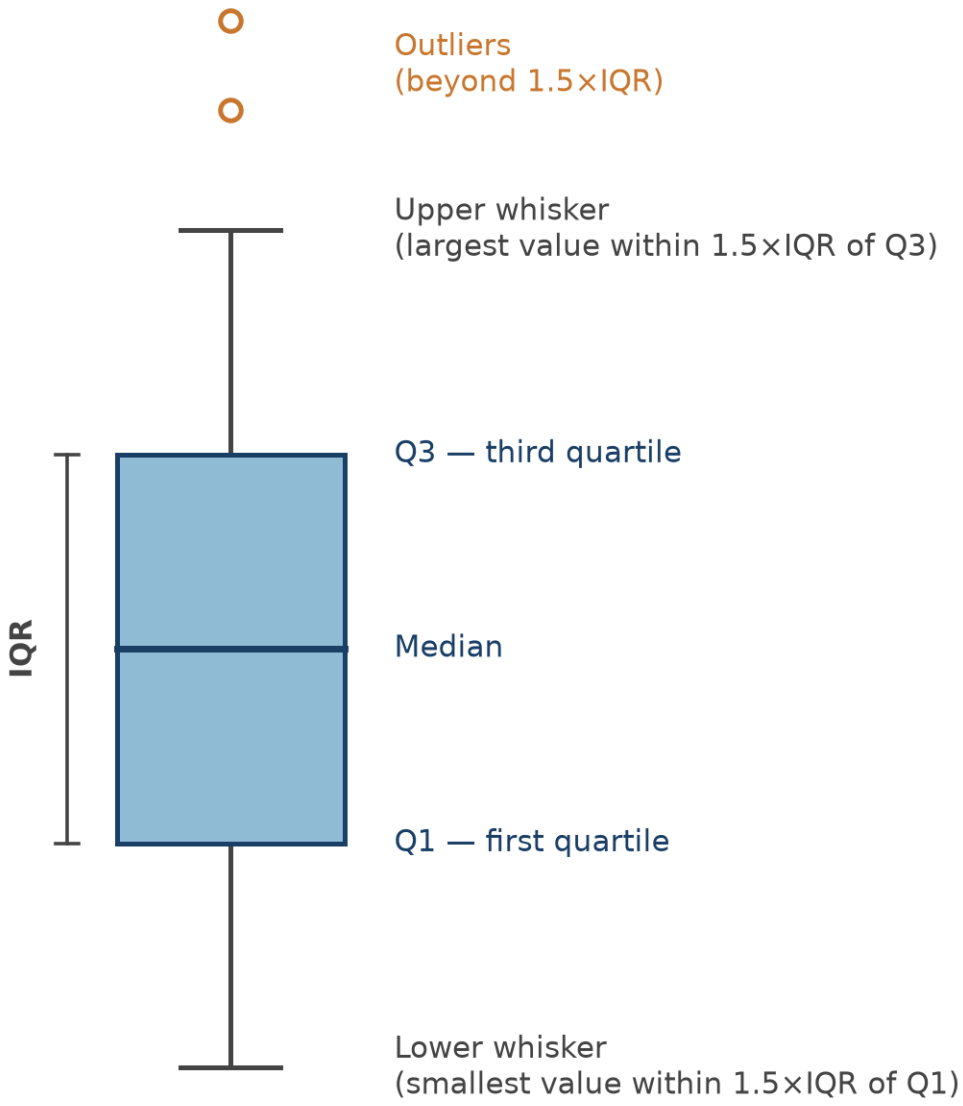
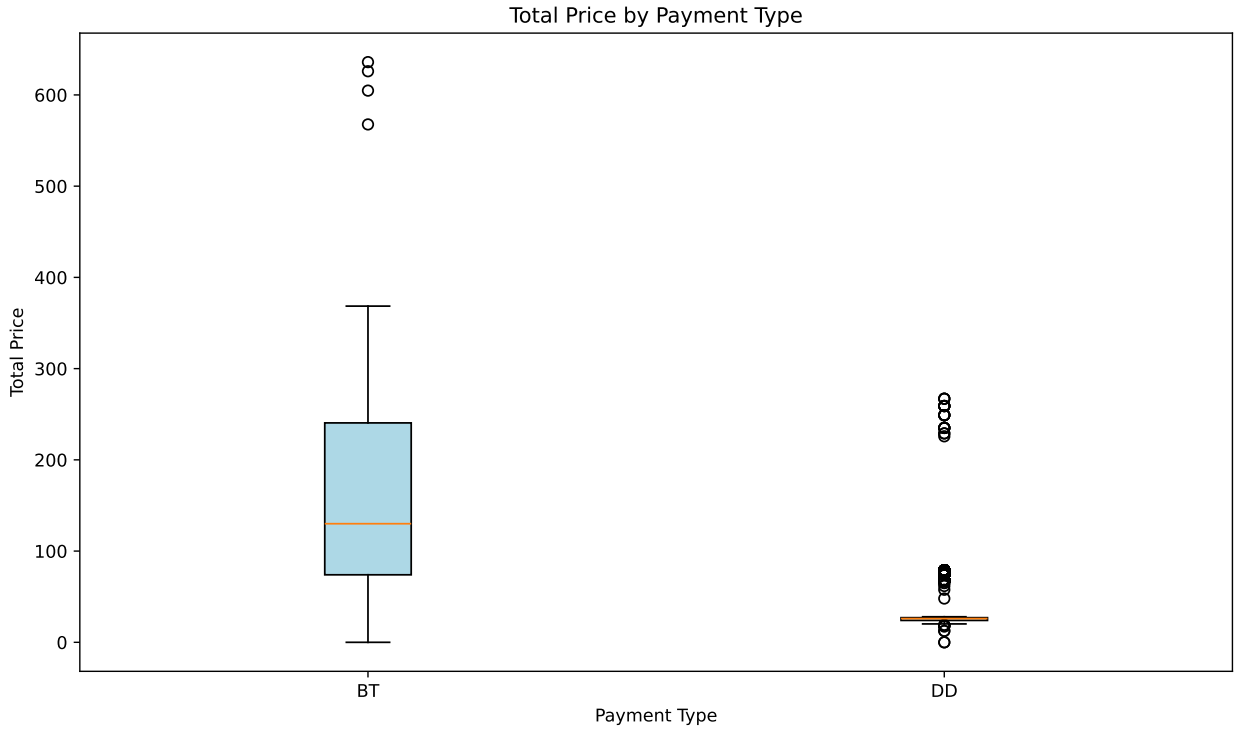
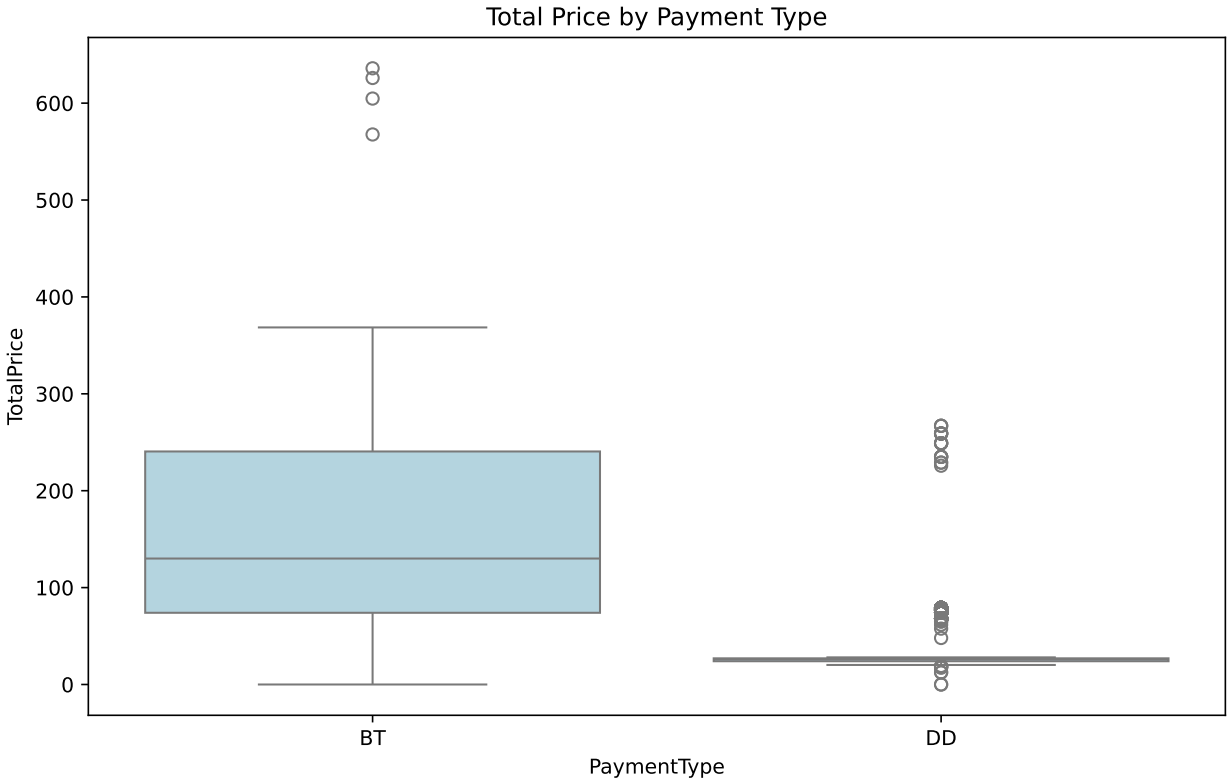


Figure 4.1: Anatomy of a box plot: the box spans the interquartile range (IQR = Q3 − Q1), the line inside marks the median, and the whiskers extend to the most extreme point within 1.5×IQR of the box. Points beyond that are flagged as potential outliers.



`seaborn` does the grouping for you, with no reshaping required:

```
fig, ax = plt.subplots(figsize=(10, 6))
sns.boxplot(data=subscriptions, x="PaymentType", y="TotalPrice", color="lightblue", ax=ax)
ax.set_title("Total Price by Payment Type")
plt.show()
```



4.3 Data quality: multicollinearity

Now that you have a toolkit of charts, it's time to put it to use on the three data quality issues that come up in almost every dataset: multicollinearity, missing values, and outliers.

Multicollinearity occurs when two or more independent variables are highly correlated with each other. It matters because it makes individual coefficients unstable and hard to interpret in a causal model, and it inflates confidence intervals. Even though, as we saw in the introduction, a purely predictive model can often tolerate it without losing accuracy, it can still undermine any post-hoc explanation of that model (e.g., unstable feature importances) and make the model less robust. `pandas` computes a full correlation matrix in one call, here on the numeric columns only. (That is one reason the identifier columns were stored as text in the previous chapter: a correlation between two IDs would mean nothing.)

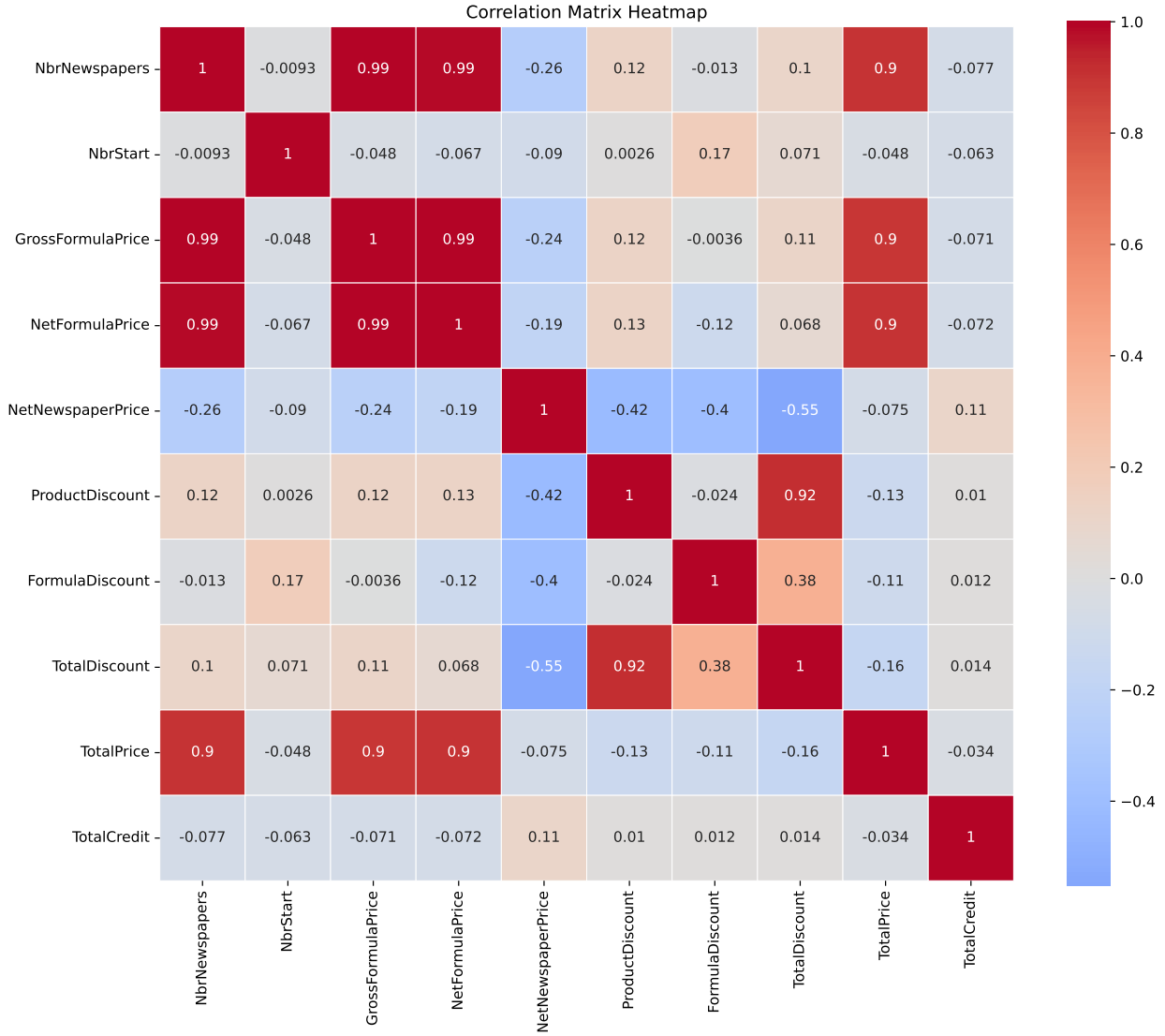
```
numeric_cols = subscriptions.select_dtypes(include=[np.number])
correlation_matrix = numeric_cols.corr()
correlation_matrix.round(2)
```

	NbrNewspapers	NbrStart	GrossFormulaPrice	NetFormulaPrice	NetNewspaperPrice	Produc
NbrNewspapers	1.00	-0.01	0.99	0.99	-0.26	0.12
NbrStart	-0.01	1.00	-0.05	-0.07	-0.09	0.00
GrossFormulaPrice	0.99	-0.05	1.00	0.99	-0.24	0.12
NetFormulaPrice	0.99	-0.07	0.99	1.00	-0.19	0.13
NetNewspaperPrice	-0.26	-0.09	-0.24	-0.19	1.00	-0.42
ProductDiscount	0.12	0.00	0.12	0.13	-0.42	1.00
FormulaDiscount	-0.01	0.17	-0.00	-0.12	-0.40	-0.02
TotalDiscount	0.10	0.07	0.11	0.07	-0.55	0.92
TotalPrice	0.90	-0.05	0.90	0.90	-0.07	-0.13

	NbrNewspapers	NbrStart	GrossFormulaPrice	NetFormulaPrice	NetNewspaperPrice	Produc
TotalCredit	-0.08	-0.06	-0.07	-0.07	0.11	0.01

A heatmap makes the same information much easier to scan than a table of numbers:

```
plt.figure(figsize=(12, 10))
sns.heatmap(correlation_matrix, annot=True, cmap="coolwarm", center=0,
            square=True, linewidths=0.5)
plt.title("Correlation Matrix Heatmap")
plt.tight_layout()
plt.show()
```

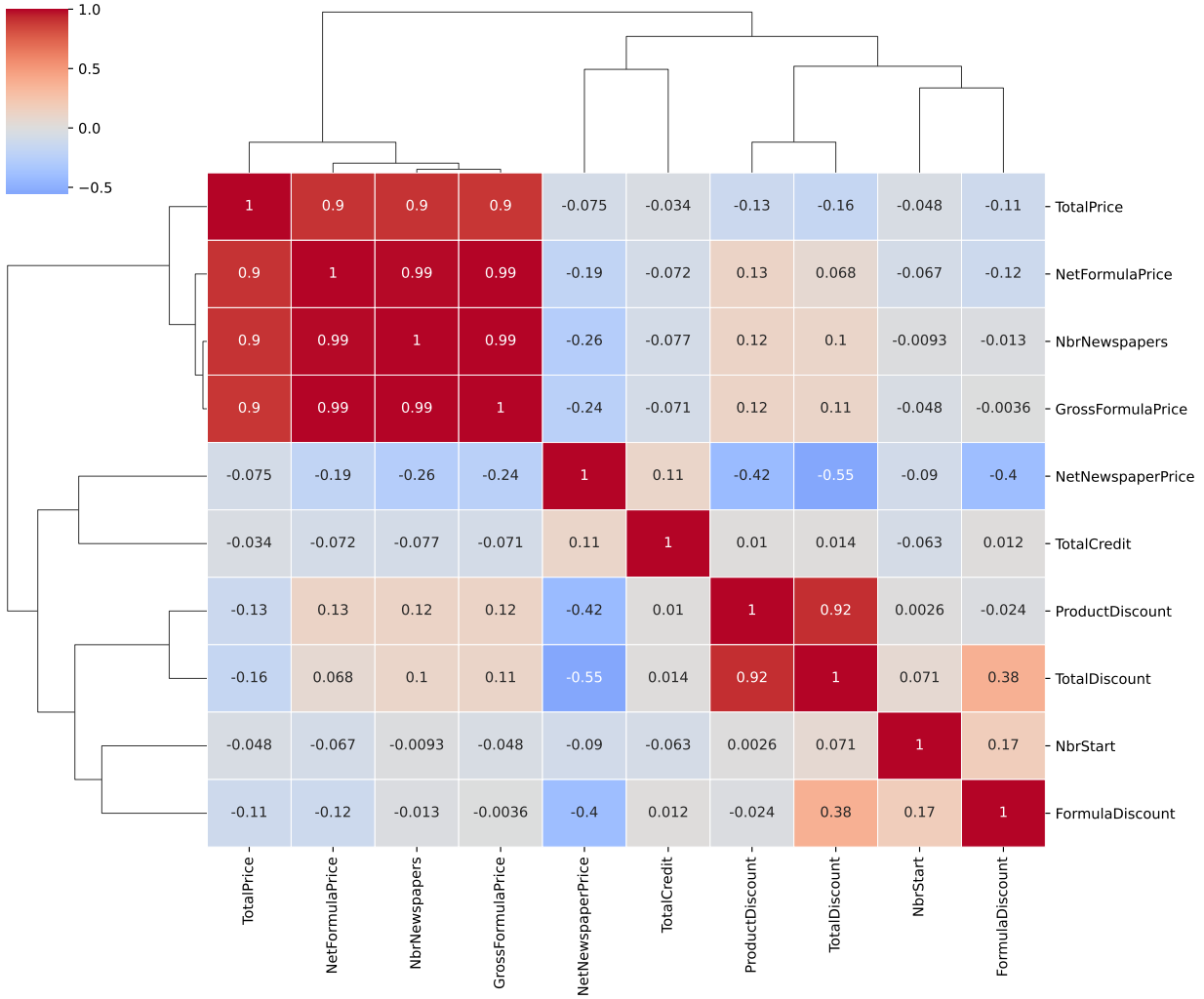


A **cluster map** goes one step further, automatically reordering rows and columns so that similar variables sit next to each other, with a dendrogram showing how they were grouped. Dot heatmaps, where a dot's size or color, rather than a cell's fill color, encodes the correlation, are another common variant.

```
sns.clustermap(correlation_matrix, annot=True, cmap="coolwarm", center=0,
               square=True, linewidths=0.5, figsize=(12, 10))
```

```
plt.show()
```

```
/Users/matthiasbogaert/anaconda3/envs/DA2026/lib/python3.11/site-packages/seaborn/matrix.py:1124: UserWa
  warnings.warn(msg)
```



A common rule of thumb is to consider correlation problematic if it's above **0.75**. Two heuristics are often used to decide which variables to drop:

1. **Highest average correlation**: find the two most correlated variables, drop whichever of the two has the higher *average* correlation with everything else, and repeat until nothing is left above the threshold.
2. **Highest correlation pairs**: scan variables in order of appearance, and for each one, drop any *later* variable that's correlated with it above the threshold. This means that you keep the first occurrence and remove the rest.

The `feature_engine` library implements the second heuristic directly:

```
from feature_engine.selection import DropCorrelatedFeatures

dcf = DropCorrelatedFeatures(variables=None, method="pearson", threshold=0.75)
dcf.fit(numeric_cols.dropna())

print(f"Features to remove: {dcf.features_to_drop_}")
```

```
Features to remove: ['NbrNewspapers', 'NetFormulaPrice', 'TotalPrice', 'TotalDiscount']
```

A plain `pandas` version of the same idea is just as easy to write yourself:

```
def find_correlated_features(corr_matrix, threshold=0.75):
    upper = corr_matrix.where(np.triu(np.ones(corr_matrix.shape), k=1).astype(bool))
    return [column for column in upper.columns if any(upper[column].abs() > threshold)]

find_correlated_features(correlation_matrix, threshold=0.75)
```

```
['GrossFormulaPrice', 'NetFormulaPrice', 'TotalDiscount', 'TotalPrice']
```

For the first heuristic, you can also easily write your own function:

```
def drop_highest_avg_correlation(corr_matrix, threshold=0.75):
    corr_matrix = corr_matrix.abs().copy()
    to_drop = []

    while True:
        upper = corr_matrix.where(np.triu(np.ones(corr_matrix.shape), k=1).astype(bool))
        max_corr = upper.max().max()
        if pd.isna(max_corr) or max_corr <= threshold:
            break

        col1, col2 = upper.stack().idxmax()
        avg_corr = corr_matrix.mean()
        drop_col = col1 if avg_corr[col1] > avg_corr[col2] else col2

        to_drop.append(drop_col)
        corr_matrix = corr_matrix.drop(index=drop_col, columns=drop_col)

    return to_drop

drop_highest_avg_correlation(correlation_matrix, threshold=0.75)
```

```
['NetFormulaPrice', 'GrossFormulaPrice', 'TotalDiscount', 'NbrNewspapers']
```

We won't actually drop these variables at this stage. Better feature-selection tools, like regularization, are covered later in the book. But knowing how to detect multicollinearity, and having these heuristics on hand, is worth having before you get there.

4.4 Data quality: missing values

Missing values show up in almost every real dataset, and the right way to handle them depends a lot on *why* they're missing in the first place:

- **Structurally missing**: the value was never recorded for a structural reason (e.g., a date-of-birth field that simply wasn't collected).
- **Non-applicable**: the field doesn't apply to this record at all (e.g., "time until churn" is meaningless for a customer who hasn't left).
- **Informative missing**: the fact that a value is missing is itself a signal (e.g., a social media user who hides their hometown may be more privacy-conscious than average and this *absence* of the value carries information).

It's also worth distinguishing missing data from **censored data**: with censored data, you don't observe the true value, but you do have partial information about it. It often happens when the true value is not available at the time of observation. For example, a tenancy agreement that's still ongoing when your observation

window ends isn't "missing" its length. You just don't know the actual end date in the future, and you will consider the true length at least as long as what you've observed so far. Hence, in predictive analytics, censored values are usually just used as-is (e.g., treating "still ongoing" as the observed length) rather than treated as missing.

```
print(subscriptions.isnull().sum())
print(f"\nTotal missing values: {subscriptions.isnull().sum().sum()}")
```

```
SubscriptionID          0
CustomerID              0
ProductID               0
Pattern                 0
StartDate               0
EndDate                 0
NbrNewspapers           0
NbrStart               21
RenewalDate          2020
PaymentType             0
PaymentStatus           0
PaymentDate           421
FormulaID               0
GrossFormulaPrice     161
NetFormulaPrice       161
NetNewspaperPrice     161
ProductDiscount       161
FormulaDiscount       161
TotalDiscount         161
TotalPrice            161
TotalCredit           161
dtype: int64

Total missing values: 3750
```

```
print(customers.isnull().sum())
print(f"\nTotal missing values: {customers.isnull().sum().sum()}")
```

```
CustomerID      0
Gender        223
DOB             0
District        0
ZIP             0
StreeID         0
dtype: int64

Total missing values: 223
```

The number of missing values matters as much as the reason: a variable with very few missing values is worth fixing, but one that's mostly empty (like a `RenewalDate` column with over 2,000 missing values here) might be better dropped entirely, or replaced with a simple indicator for "was this ever renewed?", which amounts to the same thing as flagging missingness directly.

4.4.1 A churn model with missing values

To demonstrate missing-value handling on a realistic dataset, we'll switch to a customer-level table, `SubCust`, that's already been built from NPC's raw tables for a churn model. It comes from joining the `subscriptions` and `customers` tables and aggregating to one row per customer, using an **independent period** to compute

predictors from and a **dependent period** to check whether a customer actually churned. Building a table like this is its own topic. For now we'll treat `SubCust` as a given and load it directly.

```
SubCust = pd.read_csv("../data/raw/SubCust.csv")
print(f"Dataset shape: {SubCust.shape}")
SubCust.head()
```

```
Dataset shape: (1178, 40)
```

	CustomerID	ProductID_1_sum	ProductID_2_sum	ProductID_3_sum	ProductID_4_sum	ProductID_5_su
0	100139	0	0	0	0	0
1	100246	0	0	0	0	0
2	1004392	0	0	0	0	0
3	1005451	0	1	0	0	0
4	1005543	0	0	1	0	0

```
missing_counts = SubCust.isnull().sum()
missing_pct = (SubCust.isnull().sum() / len(SubCust)) * 100

missing_summary = pd.DataFrame({"Missing_Count": missing_counts, "Missing_Percentage": missing_pct})
missing_summary[missing_summary["Missing_Count"] > 0].round(2)
```

	Missing_Count	Missing_Percentage
RenewalDate_rec	154	13.07
PaymentDate_rec	4	0.34
Gender	211	17.91

4.4.2 Treating missing values

There are two broad options once you know what's missing and why: **delete** the missing rows or columns, or **impute**. Imputation implies replacing the missing values with a reasonable estimate. Since the missing percentages here are fairly low, imputation is the better choice; deletion mainly makes sense when a column is missing so often it's barely usable, or a row is missing so much it isn't worth keeping.

Imputation itself comes in two flavors:

- **Univariate (simple) imputation**: fill missing values with a single summary statistic, like the median for continuous variables, the mode for categorical ones.
- **Multivariate imputation**: model each variable with missing values as a function of the *other* variables, and use that model's predictions to fill the gaps.

Univariate imputation with `SimpleImputer` is the natural starting point:

```
from sklearn.impute import SimpleImputer

numerical_cols_missing = SubCust.select_dtypes(include=[np.number]).columns[
    SubCust.select_dtypes(include=[np.number]).isnull().any()
].tolist()
categorical_cols_missing = SubCust.select_dtypes(include=["object"]).columns[
    SubCust.select_dtypes(include=["object"]).isnull().any()
].tolist()

print(f"Missing values in {len(numerical_cols_missing)} numerical and "
      f"{len(categorical_cols_missing)} categorical columns")
```

```
numerical_imputer = SimpleImputer(strategy="median")
categorical_imputer = SimpleImputer(strategy="most_frequent")

SubCust_simple = SubCust.copy()
SubCust_simple[numerical_cols_missing] = numerical_imputer.fit_transform(SubCust[numerical_cols_missing]
SubCust_simple[categorical_cols_missing] = categorical_imputer.fit_transform(SubCust[categorical_cols_mi

SubCust_simple[numerical_cols_missing + categorical_cols_missing].isnull().sum()
```

```
Missing values in 2 numerical and 1 categorical columns

RenewalDate_rec    0
PaymentDate_rec    0
Gender             0
dtype: int64
```

Multivariate imputation is more sophisticated: scikit-learn's `IterativeImputer` implements *Multiple Imputation by Chained Equations* (MICE). The idea is to loop over every variable that has missing values, treating it as the target of a small regression model built from the other variables, and repeat until the imputed values stabilize:

1. Fill every missing value with a quick univariate estimate (mean/mode), so every variable has complete data to start from.
2. Pick one variable with missing values as the target; train a model to predict it from the other variables, using only the rows where there were no missing values originally.
3. Use that model to re-predict the missing values for this variable, replacing the placeholder estimate from step 1.
4. Move to the next variable and repeat, now using the freshly updated values as predictors.
5. Cycle through all variables repeatedly (10 times, by default) until the imputed values stop changing much.

```
from sklearn.experimental import enable_iterative_imputer
from sklearn.impute import IterativeImputer

numerical_cols = SubCust.select_dtypes(include=[np.number]).columns
SubCust_numerical = SubCust[numerical_cols].copy()

iterative_imputer = IterativeImputer(random_state=42, max_iter=10, verbose=0)
SubCust_iterative = pd.DataFrame(
    iterative_imputer.fit_transform(SubCust_numerical),
    columns=numerical_cols,
    index=SubCust.index,
)
```

In practice, simple imputation is usually preferred when only a handful of values are missing since it's fast and easy to reason about. Multivariate imputation earns its extra cost when a larger share of the data is missing and there's real structure among the variables for it to exploit.

4.4.3 Doing this properly: fitting only on training data

So far we've imputed using the *entire* dataset. That's fine for a first look, but it hides a real risk once you're building a predictive model: if the imputation model learns anything from observations you'll later use to *test* that model, information has quietly leaked from the test set into training. This is called **data leakage** and will make your test performance look better than the model would actually achieve on genuinely unseen data.

> **A preview of the train/test split**
>
> We'll cover train/test splitting properly in the Model Evaluation chapter, but for now the short version is: before doing anything that *learns* from the data (fitting an imputer, training a model), split your data into a **training set** to learn from and a **test set** you hold back untouched, so you have a fair, unseen sample to check performance on later. Any transformation you *fit*, including an imputer, gets fit on the training set only, then applied or *transformed* to the test set.

Let's redo the imputation the proper way using the `SimpleImputer`. First, split the data, keeping the target variable (`churn`) and `CustomerID` aside:

```
from sklearn.model_selection import train_test_split

y = SubCust["churn"]
X = SubCust.drop(columns=["churn", "CustomerID"])

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.5, random_state=100, stratify=y
)

print(f"Training set: {X_train.shape}")
print(f"Test set: {X_test.shape}")
```

```
Training set: (589, 38)
Test set: (589, 38)
```

Before imputing anything, it's worth keeping a record of *which* values were originally missing, since that's information in its own right, and it can't be recovered once the gaps are filled in:

```
train_missing_flags = pd.DataFrame(
    {f"{col}_missing": X_train[col].isnull() for col in X_train.columns if X_train[col].isnull().any()}
    index=X_train.index,
)
test_missing_flags = pd.DataFrame(
    {f"{col}_missing": X_test[col].isnull() for col in X_test.columns if X_test[col].isnull().any()},
    index=X_test.index,
)

print(f"Created {train_missing_flags.shape[1]} missing value indicators")
```

```
Created 3 missing value indicators
```

Now fit the simple imputers on the training set only, and apply them to both sets:

```
numerical_cols = X_train.select_dtypes(include=[np.number]).columns
categorical_cols = X_train.select_dtypes(include=["object", "category"]).columns

numerical_imputer = SimpleImputer(strategy="median")
categorical_imputer = SimpleImputer(strategy="most_frequent")

numerical_imputer.fit(X_train[numerical_cols])
categorical_imputer.fit(X_train[categorical_cols])

X_train_simple = X_train.copy()
X_test_simple = X_test.copy()
```

```
X_train_simple[numerical_cols] = numerical_imputer.transform(X_train[numerical_cols])
X_test_simple[numerical_cols] = numerical_imputer.transform(X_test[numerical_cols])

X_train_simple[categorical_cols] = categorical_imputer.transform(X_train[categorical_cols])
X_test_simple[categorical_cols] = categorical_imputer.transform(X_test[categorical_cols])

# add the missing-value indicators back in
train_data = pd.concat([X_train_simple, train_missing_flags], axis=1)
test_data = pd.concat([X_test_simple, test_missing_flags], axis=1)

print(f"Final training set with indicators: {train_data.shape}")
print(f"Final test set with indicators: {test_data.shape}")
```

```
Final training set with indicators: (589, 41)
Final test set with indicators: (589, 41)
```

The same fit-on-train, transform-to-both pattern works for `IterativeImputer` too (but only for numerical variables). The only difference is `.fit()` learns the per-variable regression models from the training set, and `.transform()` applies them to new rows without re-learning anything:

```
iterative_imputer = IterativeImputer(random_state=42, max_iter=10, verbose=0)
iterative_imputer.fit(X_train[numerical_cols])

X_train_numerical_imputed = pd.DataFrame(
    iterative_imputer.transform(X_train[numerical_cols]),
    columns=numerical_cols, index=X_train.index,
)
X_test_numerical_imputed = pd.DataFrame(
    iterative_imputer.transform(X_test[numerical_cols]),
    columns=numerical_cols, index=X_test.index,
)
```

From here on, we'll use `train_data` and `test_data`, the simple-imputation version, with missing-value indicators attached, as our working datasets.

4.5 Data quality: outliers

An **outlier** is a data point that sits exceptionally far from the rest of the observations. Not every outlier is a mistake: a genuine CEO salary of €3 million is a *valid* extreme value, while an age of 270 is clearly *invalid*. With a small sample, what looks like an outlier can just be the tail of a skewed distribution, or a handful of points from a different population entirely.

4.5.1 Detecting outliers

Detection is either visual (scatter plots, histograms, box plots) or numerical (checking the min/max, or computing a **z-score** to check how many standard deviations a value sits from the mean). Under a roughly normal distribution, about 95% of values fall within 2 standard deviations of the mean, and 99.7% within 3. So a z-score beyond about ± 3 is a common threshold for flagging a point as a potential outlier.

As with imputation, outlier detection should also be done on the training data only, so that whatever rule you land on is validated against data the model hasn't seen. We'll also restrict this to numeric variables, since the notion of an "outlier" is mostly a continuous-data concept.

```
X_train_numeric = train_data.select_dtypes(include=[np.number])
print(f"Dimensions of the training data: {X_train_numeric.shape}")
```

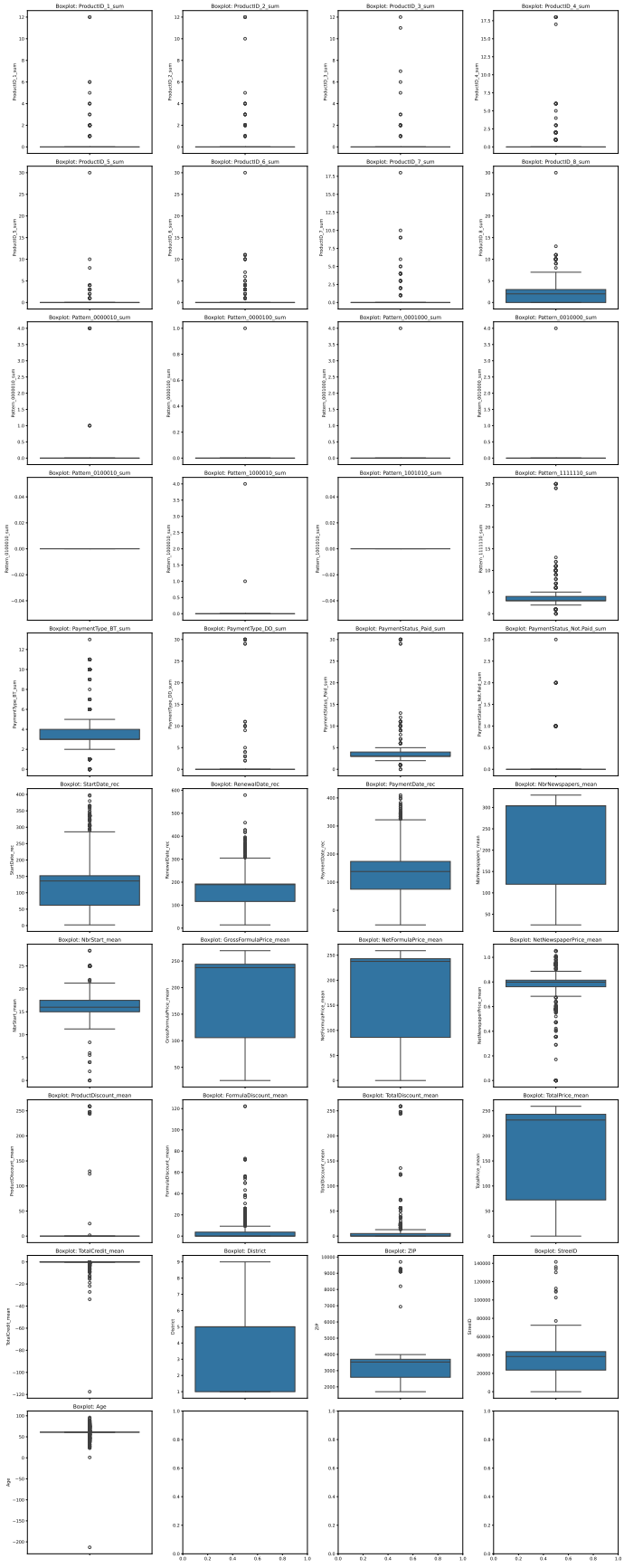
```
Dimensions of the training data: (589, 37)
```

```
n_vars = len(X_train_numeric.columns)
n_cols = 4
n_rows = (n_vars + n_cols - 1) // n_cols

fig, axes = plt.subplots(n_rows, n_cols, figsize=(20, 5 * n_rows))
axes = axes.flatten()

for i, column in enumerate(X_train_numeric.columns):
    sns.boxplot(data=X_train_numeric, y=column, ax=axes[i])
    axes[i].set_title(f"Boxplot: {column}")
    axes[i].set_xlabel("")

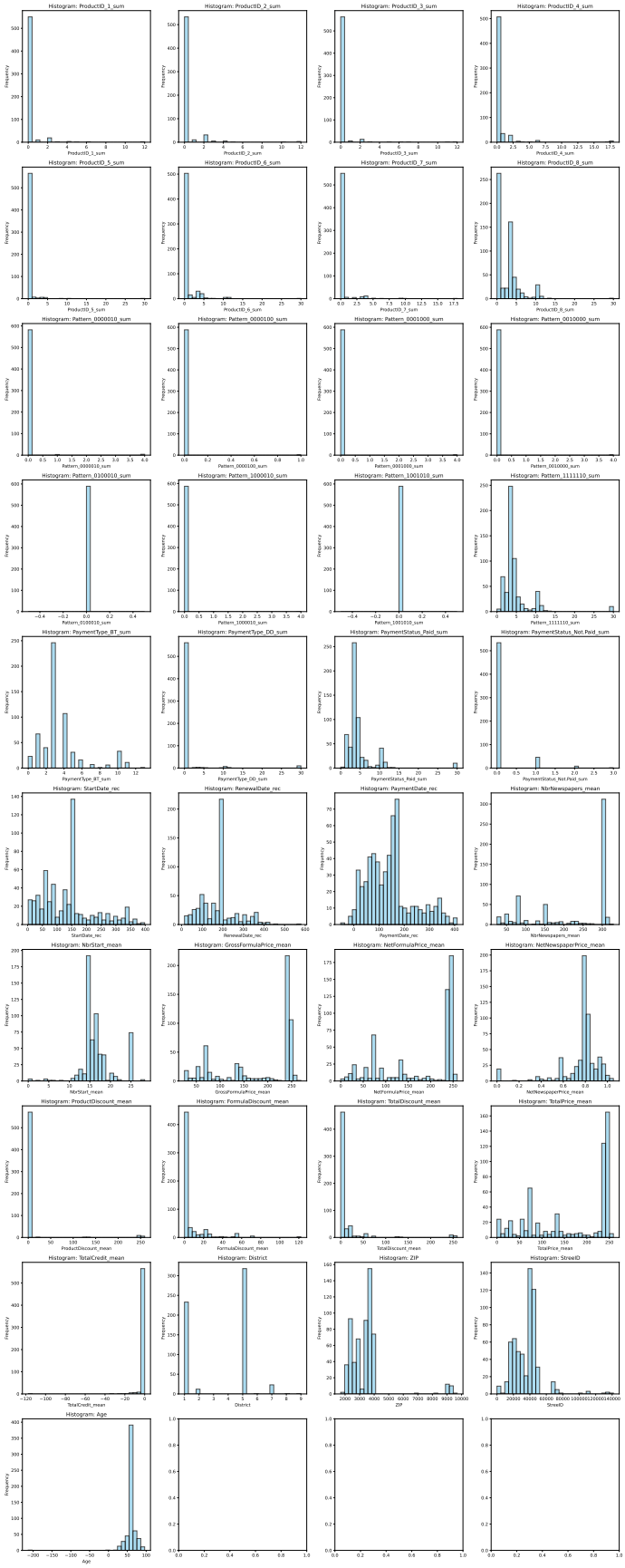
plt.tight_layout()
plt.show()
```


Boxplot: ProductID_1_sum
Boxplot: ProductID_2_sum
Boxplot: ProductID_3_sum
Boxplot: ProductID_4_sum
Boxplot: ProductID_5_sum
Boxplot: ProductID_6_sum
Boxplot: ProductID_7_sum
Boxplot: ProductID_8_sum
Boxplot: Pattern_00000010_sum
Boxplot: Pattern_00000100_sum
Boxplot: Pattern_00010000_sum
Boxplot: Pattern_00100000_sum
Boxplot: Pattern_01000010_sum
Boxplot: Pattern_10000010_sum
Boxplot: Pattern_10010010_sum
Boxplot: Pattern_11111110_sum
Boxplot: PaymentType_BT_sum
Boxplot: PaymentType_DD_sum
Boxplot: PaymentStatus_Paid_sum
Boxplot: PaymentStatus_Not.Paid_sum
Boxplot: StartDate_rec
Boxplot: RenewalDate_rec
Boxplot: PaymentDate_rec
Boxplot: NbrNewspapers_mean
Boxplot: NbrStart_mean
Boxplot: GrossFormulaPrice_mean
Boxplot: NetFormulaPrice_mean
Boxplot: NetNewspaperPrice_mean
Boxplot: ProductDiscount_mean
Boxplot: FormulaDiscount_mean
Boxplot: TotalDiscount_mean
Boxplot: TotalPrice_mean
Boxplot: TotalCredit_mean
Boxplot: District
Boxplot: ZIP
Boxplot: StreetID
Boxplot: Age

```
fig, axes = plt.subplots(n_rows, n_cols, figsize=(20, 5 * n_rows))
axes = axes.flatten()

for i, column in enumerate(X_train_numeric.columns):
    axes[i].hist(X_train_numeric[column].dropna(), bins=30, alpha=0.7,
                 color="skyblue", edgecolor="black")
    axes[i].set_title(f"Histogram: {column}")
    axes[i].set_xlabel(column)
    axes[i].set_ylabel("Frequency")

plt.tight_layout()
plt.show()
```


Histogram: ProductID_1_sum
Histogram: ProductID_2_sum
Histogram: ProductID_3_sum
Histogram: ProductID_4_sum
Histogram: ProductID_5_sum
Histogram: ProductID_6_sum
Histogram: ProductID_7_sum
Histogram: ProductID_8_sum
Histogram: Pattern_0000010_sum
Histogram: Pattern_0000100_sum
Histogram: Pattern_0001000_sum
Histogram: Pattern_0010000_sum
Histogram: Pattern_0100010_sum
Histogram: Pattern_1000010_sum
Histogram: Pattern_1001010_sum
Histogram: Pattern_1111110_sum
Histogram: PaymentType_BT_sum
Histogram: PaymentType_DD_sum
Histogram: PaymentStatus_Paid_sum
Histogram: PaymentStatus_Not.Paid_sum
Histogram: StartDate_rec
Histogram: RenewalDate_rec
Histogram: PaymentDate_rec
Histogram: NbrNewspapers_mean
Histogram: NbrStart_mean
Histogram: GrossFormulaPrice_mean
Histogram: NetFormulaPrice_mean
Histogram: NetNewspaperPrice_mean
Histogram: ProductDiscount_mean
Histogram: FormulaDiscount_mean
Histogram: TotalDiscount_mean
Histogram: TotalPrice_mean
Histogram: TotalCredit_mean
Histogram: District
Histogram: ZIP
Histogram: StreetID
Histogram: Age
Frequency

The plots are a starting point, not the analysis itself and it's worth putting a number on what they show. The box plot rule (anything past 1.5×IQR from the box) gives a quick count of flagged points per variable:

```
Q1 = X_train_numeric.quantile(0.25)
Q3 = X_train_numeric.quantile(0.75)
IQR = Q3 - Q1
lower_bound = Q1 - 1.5 * IQR
upper_bound = Q3 + 1.5 * IQR

iqr_outliers = (X_train_numeric.lt(lower_bound) | X_train_numeric.gt(upper_bound)).sum()
iqr_outlier_pct = (iqr_outliers / len(X_train_numeric) * 100).round(1)

pd.DataFrame({"n_flagged": iqr_outliers, "pct_flagged": iqr_outlier_pct}).query("n_flagged > 0").sort_va
```

	n_flagged	pct_flagged
Age	267	45.3
NetNewspaperPrice_mean	181	30.7
Pattern_1111110_sum	169	28.7
PaymentType_BT_sum	165	28.0
PaymentStatus_Paid_sum	162	27.5
TotalCredit_mean	127	21.6
TotalDiscount_mean	106	18.0
FormulaDiscount_mean	105	17.8
NbrStart_mean	89	15.1
ProductID_6_sum	86	14.6
ProductID_4_sum	82	13.9
RenewalDate_rec	62	10.5
ProductID_2_sum	56	9.5
PaymentStatus_Not.Paid_sum	55	9.3
StartDate_rec	53	9.0
ProductID_8_sum	40	6.8
PaymentDate_rec	39	6.6
ProductID_1_sum	39	6.6
ProductID_7_sum	38	6.5
PaymentType_DD_sum	28	4.8
ProductID_3_sum	26	4.4
ZIP	25	4.2
ProductID_5_sum	24	4.1
ProductDiscount_mean	19	3.2
StreeID	9	1.5
Pattern_0000010_sum	7	1.2
Pattern_1000010_sum	2	0.3
Pattern_0010000_sum	1	0.2
Pattern_0001000_sum	1	0.2
Pattern_0000100_sum	1	0.2

On some variables, this flags well over 10% of all training rows, which is far too many to all be genuine data errors. The **z-score** rule (flag anything with $|z| > 3$) is a useful second opinion, since it reacts differently to skew:

```
z_scores = (X_train_numeric - X_train_numeric.mean()) / X_train_numeric.std()
z_outliers = (z_scores.abs() > 3).sum()
z_outlier_pct = (z_outliers / len(X_train_numeric) * 100).round(1)
```

```
pd.DataFrame({"n_flagged": z_outliers, "pct_flagged": z_outlier_pct}).query("n_flagged > 0").sort_values
```

	n_flagged	pct_flagged
ZIP	24	4.1
FormulaDiscount_mean	23	3.9
NetNewspaperPrice_mean	20	3.4
ProductDiscount_mean	16	2.7
TotalDiscount_mean	15	2.5
PaymentType_BT_sum	12	2.0
ProductID_6_sum	12	2.0
Pattern_1111110_sum	10	1.7
PaymentStatus_Paid_sum	10	1.7
PaymentType_DD_sum	10	1.7
ProductID_2_sum	10	1.7
ProductID_1_sum	10	1.7
PaymentStatus_Not.Paid_sum	9	1.5
StreeID	8	1.4
NbrStart_mean	7	1.2
ProductID_7_sum	7	1.2
ProductID_3_sum	7	1.2
ProductID_4_sum	6	1.0
TotalCredit_mean	5	0.8
Pattern_0000010_sum	4	0.7
ProductID_5_sum	3	0.5
Pattern_1000010_sum	2	0.3
RenewalDate_rec	2	0.3
ProductID_8_sum	2	0.3
Age	2	0.3
Pattern_0010000_sum	1	0.2
Pattern_0001000_sum	1	0.2
Pattern_0000100_sum	1	0.2

The z-score rule flags far fewer points overall, and not always the same variables: `Age` tops the IQR list at over 45% (267 customers), yet the z-score rule flags only 2 of them (0.3%). The explanation is in the data itself. More than half of the customers in the training set (54.7%) have exactly the same age, 61. That looks like a placeholder for an unknown date of birth: in the raw customers table, 715 of the 1,389 customers have a date of birth of 1900-01-01. When more than half of the values are identical, the first quartile, the median and the third quartile are all 61, so the interquartile range is 0 and the box plot rule flags every age that is not exactly 61. The z-score rule reacts differently: it is based on the mean and the standard deviation, and the one clearly invalid age (-213) pushes the standard deviation up from about 11 to about 16, so only the most extreme values get flagged. That gap is the point: neither rule is simply "right" and a mechanical threshold, on its own, doesn't tell you whether a flagged point is a data error or a genuinely extreme (but valid) observation. That judgment call needs the shape of the distribution, which is exactly what the histograms above show, and, in `Age`'s case, some domain knowledge: negative ages aren't valid, and an age shared by more than half of the customers deserves a closer look at where it comes from. (The median we impute with below is that same value, 61.)

> ⚠ Box plots and histograms can tell different stories
>
> As noticed above, box plots flag anything past 1.5×IQR from the box as a potential outlier. On some variables here, that flags a *lot* of points as problematic. But look at the matching histograms: several

> of those distributions are just genuinely skewed (a long right tail of high spenders, say), not full of data errors. The box plot rule is a mechanical threshold, not a verdict. The take-away is: always check the actual shape of the distribution before deciding a flagged point is really a problem.

To be clear, none of this outlier treatment is used to modify `train_data` or `test_data`: it's purely diagnostic, showing what each rule *would* flag.

4.5.2 Treating outliers

How much outliers matter depends on the model you're planning to build. Non-parametric models (decision trees, for instance) are often fairly insensitive to them; parametric models (linear and logistic regression) can be thrown off by a handful of extreme points. Once you've decided a point is worth addressing:

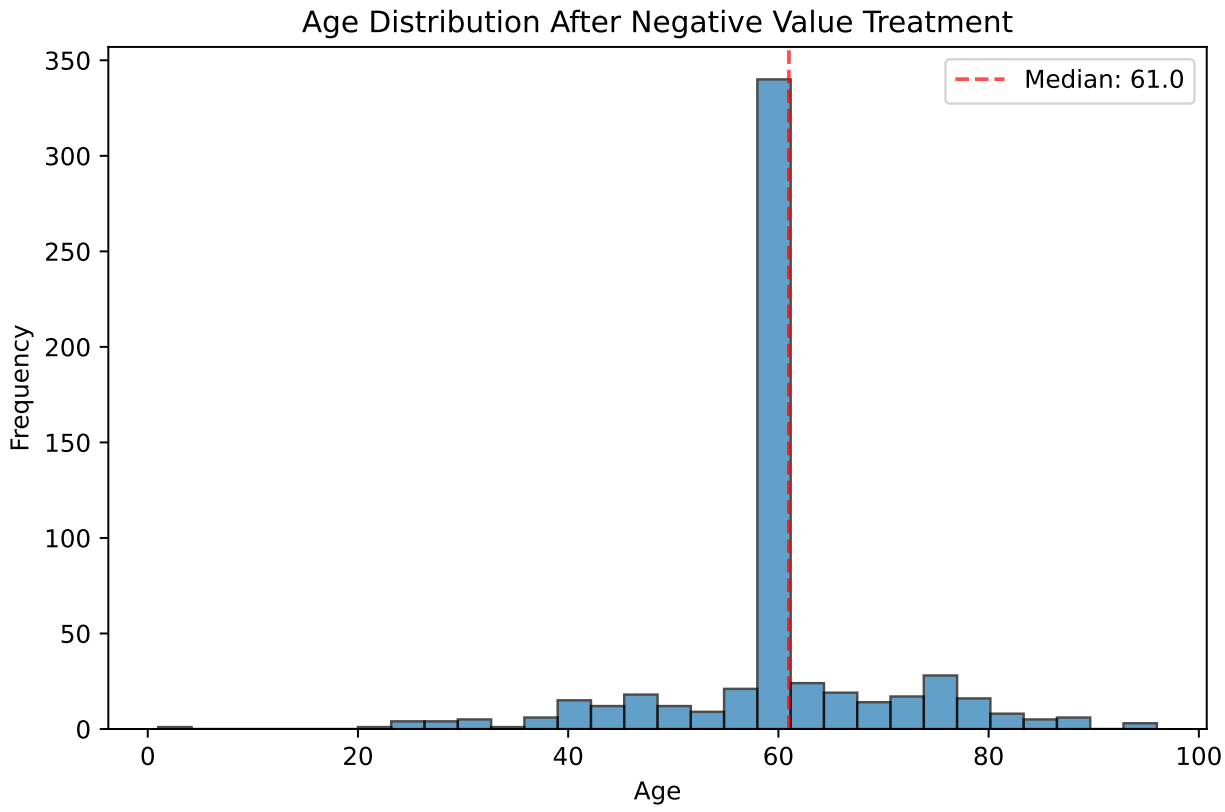
- **Invalid observations** (like the negative ages above): treat them as missing values, and impute them.
- **Valid but extreme observations**: you have three options. Leave them as-is, **cap** them (also called truncating or winsorizing: clamp any value beyond a chosen lower/upper limit back to that limit), or convert the variable into categories, which we'll cover in the next chapter.

In our case, the `Age` variable needs to be treated, since it includes some negative values. These are invalid since nobody has a negative age. That's a case where the fix is simple: treat the invalid values as missing, and impute them like any other missing value. As with every other transformation in this chapter, the median is computed on the training set only and then applied to the test set as well.

```
age_median = train_data["Age"][train_data["Age"] >= 0].median()
train_data.loc[train_data["Age"] < 0, "Age"] = np.nan
train_data["Age"] = train_data["Age"].fillna(age_median)

# apply the same rule to the test set, using the median learned on the training set
test_data.loc[test_data["Age"] < 0, "Age"] = np.nan
test_data["Age"] = test_data["Age"].fillna(age_median)

plt.figure(figsize=(8, 5))
plt.hist(train_data["Age"], bins=30, alpha=0.7, edgecolor="black")
plt.title("Age Distribution After Negative Value Treatment")
plt.xlabel("Age")
plt.ylabel("Frequency")
plt.axvline(age_median, color="red", linestyle="--", alpha=0.7, label=f"Median: {age_median:.1f}")
plt.legend()
plt.show()
```



4.6 Saving your progress

We'll pick this up again in the next chapter, so let's save the cleaned, imputed training and test sets including the target variable as `parquet` files.

```
train_complete = train_data.copy()
train_complete["target"] = y_train

test_complete = test_data.copy()
test_complete["target"] = y_test

train_complete.to_parquet("../data/raw/train_complete.parquet", index=False)
test_complete.to_parquet("../data/raw/test_complete.parquet", index=False)
```

4.7 Where we go from here

We've now covered every level of data understanding: table, data, and variable. The next chapter, **Data Preparation**, picks up right where this one leaves off with transforming, encoding, and engineering the cleaned `train_data` and `test_data` into a model-ready basetable.

Chapter 5

Data Preparation

The previous chapter cleaned up `train_data` and `test_data`: missing values imputed, negative ages fixed, multicollinearity flagged. That's a proper dataset to explore, but most of it still can't go into a predictive model as-is: categorical columns need to become numbers, some continuous variables would benefit from being simplified, and every feature needs to be on a scale a model can work with. That's what **data preparation** covers.

It's easy to underestimate how much this step matters. Some authors argue that it's the data preparation, not the algorithm, that mainly drives predictive performance (Coussement et al. 2017). Variables get transformed or reduced for a few reasons: to be interpretable, to overcome bias, to actually be usable by a given model, and simply to carry more predictive value than the raw column did. It's also the most time-consuming step of the whole process, which is exactly why it's worth doing carefully.

Broadly, this chapter covers four things:

- **Value transformation**: turning continuous variables into simpler categorical ones.
- **Value representation**: turning categorical variables into a numeric form a model can use.
- **Feature engineering**: creating new features and transforming existing ones to help a model fit better.
- **Variable selection**: reducing the number of features to only those that actually carry predictive value.

Let's pick up where the previous chapter left off, and load the cleaned training and test sets:

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

train_complete = pd.read_parquet("../data/raw/train_complete.parquet")
train_data = train_complete.drop(columns=["target"])
train_target = train_complete["target"]

test_complete = pd.read_parquet("../data/raw/test_complete.parquet")
test_data = test_complete.drop(columns=["target"])
test_target = test_complete["target"]

print(f"Training data shape: {train_data.shape}")
train_data["Age"].describe()
```

```
Training data shape: (589, 41)
```

```
count    589.000000
mean      60.779287
```

```
std       10.723909
min        1.000000
25%       61.000000
50%       61.000000
75%       61.000000
max       96.000000
Name: Age, dtype: float64
```

5.1 Value transformations

5.1.1 Continuous categorization (binning)

Binning (also called discretization, coarse-classification, or classing) transforms a continuous variable into a categorical one with a handful of ranges instead of many individual values. There are two main reasons to do this:

- **Fewer parameters to estimate:** A categorical variable with a lot of levels (e.g., in our case the `ZIP` variable has well over a hundred) needs one dummy variable per level (minus one) in an analytical model. Grouping those levels into a handful of bins keeps the model far more compact and robust.
- **Capturing non-monotonic relationships in linear models:** A linear model can only fit a straight line. If the true relationship between a variable and the outcome isn't monotonic, binning lets a linear model approximate it anyway, since each bin gets its own coefficient. For example, in a real credit-scoring model Van Gool et al. (2012) find exactly this for requested loan duration: the riskiest loans turn out to be the medium-term ones, not the longest ones. This is a pattern a single linear term would have missed entirely, and one their binned model picks up directly.

Figure 5.1 illustrates the second point with a hypothetical marketing example: a campaign's response rate against how many days it's been since the customer was last contacted. Response is low right after contact (fatigue since nobody likes being messaged twice in a row), climbs to a "sweet spot" a few weeks out, then drops again as the customer disengages. A straight line can't capture this rise-then-plateau-then-fall pattern. What it actually does is create a misleading representation of the true relationship. Binning recency into a handful of ranges lets even a plain linear model pick up the sweet spot, since each range gets its own estimated response level.

Two very basic binning strategies are **equal-width binning** (bins created with the same range of values) and **equal-frequency binning** (bins containing the same number of observations). Both are simple but don't take the target variable into account at all. For categorical variables, the equivalent ideas are **grouping** (combining several levels into one) and **remapping** (rebuilding categories around their relationship with the target, e.g. via a chi-square analysis).

`sklearn` has `KBinsDiscretizer`, but `feature-engine` offers more convenient binning transformers:

- `EqualFrequencyDiscretiser`: equal-frequency bins.
- `EqualWidthDiscretiser`: equal-width bins.
- `DecisionTreeDiscretiser`: uses a decision tree to find the optimal split points taking into account the target variable (out of scope here).

As with everything so far, binning is *fit* on the training data only, then *applied* or *transformed* to the test set:

```
from feature_engine.discretisation import EqualFrequencyDiscretiser, EqualWidthDiscretiser

freq_binner = EqualFrequencyDiscretiser(variables=["Age"], q=10)
train_freq_binned = freq_binner.fit_transform(train_data)

width_binner = EqualWidthDiscretiser(variables=["Age"], bins=3)
train_width_binned = width_binner.fit_transform(train_data)
```

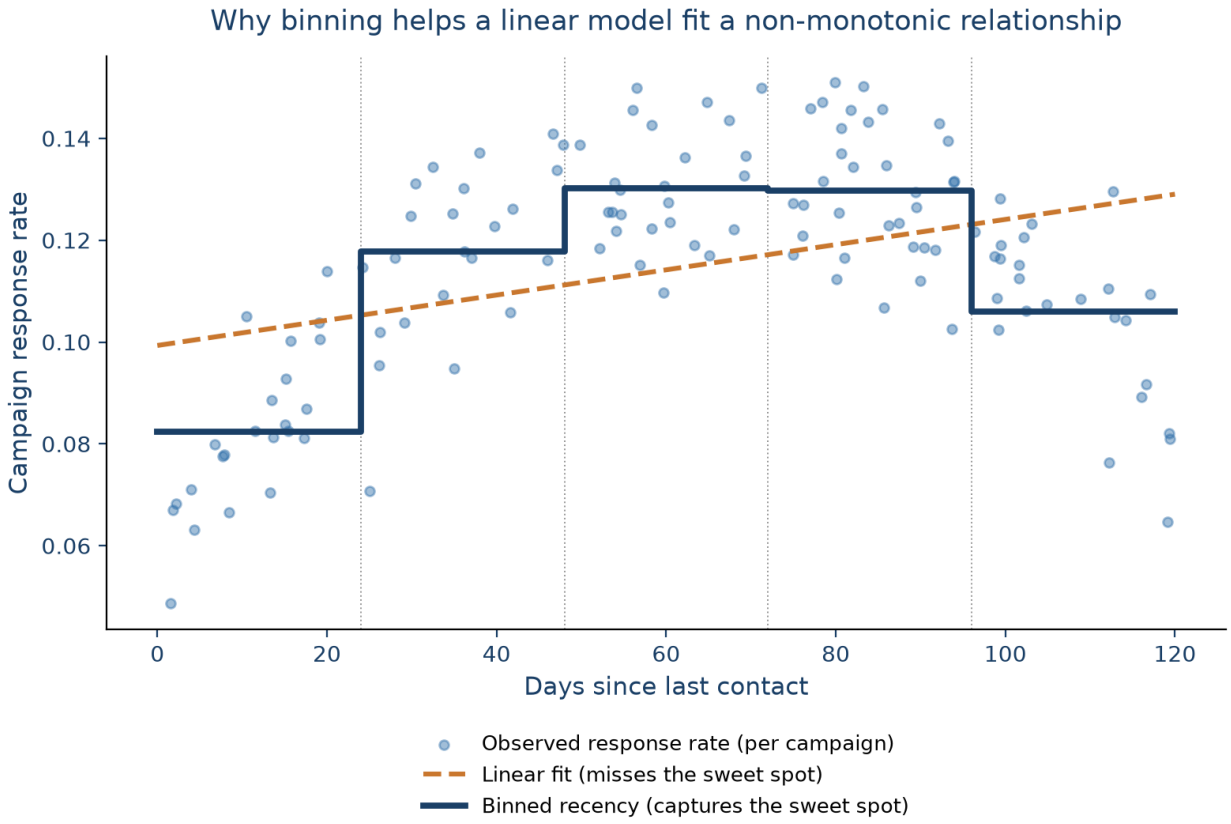


Figure 5.1: Why binning helps a linear model fit a non-monotonic relationship: a straight line (orange) averages away the rise-and-fall pattern, while a binned step function (navy) tracks it closely.

```
print("Equal-frequency bin edges (10 bins):")
print(freq_binner.binner_dict_["Age"])

print("\nEqual-width bin edges (3 bins):")
print(width_binner.binner_dict_["Age"])
```

```
Equal-frequency bin edges (10 bins):
[-inf, np.float64(47.0), np.float64(59.0), np.float64(61.0), np.float64(64.0), np.float64(74.0), inf]

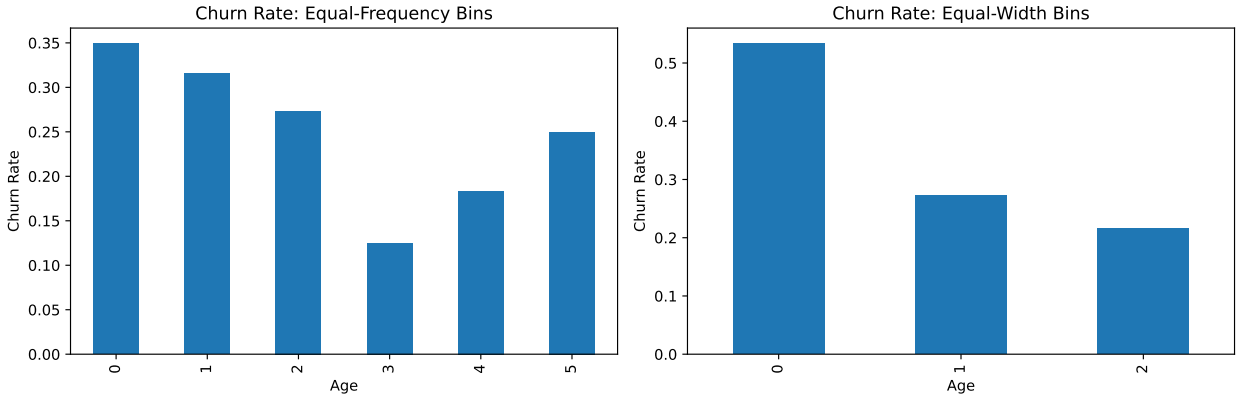
Equal-width bin edges (3 bins):
[-inf, np.float64(32.66666666666667), np.float64(64.33333333333334), inf]
```

Whether a binning is actually *useful* is a separate question from whether it's *possible*. The way to check is to look at how well it separates the outcome you care about. If churn rate differs clearly and fairly monotonically across bins, and each bin still has enough observations to be reliable, the binning is doing real work:

```
train_freq_binned["churn"] = train_target
train_width_binned["churn"] = train_target

freq_churn = train_freq_binned.groupby("Age")["churn"].agg(["mean", "count"])
width_churn = train_width_binned.groupby("Age")["churn"].agg(["mean", "count"])

fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(12, 4))
freq_churn["mean"].plot(kind="bar", ax=ax1, title="Churn Rate: Equal-Frequency Bins")
width_churn["mean"].plot(kind="bar", ax=ax2, title="Churn Rate: Equal-Width Bins")
for ax in [ax1, ax2]:
    ax.set_ylabel("Churn Rate")
plt.tight_layout()
plt.show()
```



The bars are worth reading with actual numbers next to them:

```
print("Equal-frequency bins (target: 10):")
print(freq_churn)
print("\nEqual-width bins (target: 3):")
print(width_churn)
```

```
Equal-frequency bins (target: 10):
         mean  count
Age
0    0.349206     63
1    0.315789     57
```

```
2    0.273556    329
3    0.125000     24
4    0.183333     60
5    0.250000     56

Equal-width bins (target: 3):
         mean  count
Age
0    0.533333     15
1    0.272926    458
2    0.215517    116
```

Two things stand out:

- **Equal-frequency only produced 6 bins, not the requested 10.** `Age` has a lot of repeated values, so several of the 10 requested quantile edges collapsed onto the same value. This is very common in quantile-based binning whenever a variable has many ties.
- **Neither binning is straightforwardly reliable.** Equal-width binning *looks* like the cleaner story (e.g., churn drops monotonically from bin to bin) but its first bin holds only 15 observations, far too few to trust a 53% churn rate from. Equal-frequency binning has properly sized bins throughout, but the churn rate across them doesn't move monotonically at all (35% → 32% → 27% → 13% → 18% → 25%): it dips, then partly recovers.

The take-away: binning `Age` this way doesn't reveal a strong, trustworthy relationship with churn here. It means `Age` on its own is a weaker predictor of churn than the binning exercise might have hoped to show, and any apparent pattern (like the width-binned one) needs its bin sizes checked before it's believed. Therefore, we'll leave `Age` as a continuous variable for now, and move on to the next step: representing categorical variables numerically.

5.2 Value representations

The most common way to represent a discrete variable numerically is **one-hot (dummy) encoding**. In short, each category of a variable becomes its own 0/1 column or *dummy variable*. For example, an `Income category` variable with levels *High*, *Medium*, and *Low* becomes three 0/1 columns, one of which is typically dropped as the **reference category** to avoid multicollinearity between the dummies:

Income category	Income_high	Income_medium
High	1	0
Medium	0	1
Low	0	0

One-hot encoding works well as long as the number of categories stays manageable. When a variable has many categories, it needs a different treatment, covered in its own section below (**Weight of Evidence**), since it's important enough to deserve one.

5.2.1 One-hot encoding

For a categorical variable with a limited number of categories (say, fewer than 10), one-hot encoding is usually the right default since it is:

- **Simple to interpret**: each dummy is just presence/absence of one category.
- **No information loss**: every original category is preserved (unlike binning).
- **Broadly compatible**: works with linear models and tree-based models alike.

One thing to watch for in a predictive setting: training and test data need to end up with the *same* dummy columns. If a category shows up only in the test set, a model trained without it simply can't make use of it. `sklearn`'s `OneHotEncoder`, fit on the training set with `handle_unknown="ignore"`, handles this cleanly. Any category the test set has that training didn't will just come out as all zeros, rather than raising an error.

Let's apply it to `Gender` and `District`:

```
from sklearn.preprocessing import OneHotEncoder

categorical_vars = ["Gender", "District"]

train_cats = {var: set(train_data[var].unique()) for var in categorical_vars}
test_cats = {var: set(test_data[var].unique()) for var in categorical_vars}

inconsistencies = {var: test_cats[var] - train_cats[var] for var in categorical_vars if test_cats[var]
if inconsistencies:
    print(f"Categories in test but not train: {inconsistencies}")
else:
    print("Train and test sets have consistent categories")
```

```
Categories in test but not train: {'District': {np.float64(4.0)}}
```

```
# drop="first" avoids the dummy-variable multicollinearity from the reference category above
# handle_unknown="ignore" sets any unseen test-time category to all zeros
dummy_encoder = OneHotEncoder(drop="first", sparse_output=False, handle_unknown="ignore")
dummy_encoder.fit(train_data[categorical_vars])

feature_names = dummy_encoder.get_feature_names_out(categorical_vars)
print(f"Feature names after dummy encoding: {feature_names}")

train_dummies = pd.DataFrame(
    dummy_encoder.transform(train_data[categorical_vars]),
    columns=feature_names, index=train_data.index,
)
test_dummies = pd.DataFrame(
    dummy_encoder.transform(test_data[categorical_vars]),
    columns=feature_names, index=test_data.index,
)

print(f"Training dummies shape: {train_dummies.shape}")
print(f"Test dummies shape: {test_dummies.shape}")
train_dummies.head()
```

```
Feature names after dummy encoding: ['Gender_M' 'District_2.0' 'District_5.0' 'District_6.0' 'District_
 'District_8.0' 'District_9.0']
Training dummies shape: (589, 7)
Test dummies shape: (589, 7)
```

```
/Users/matthiasbogaert/anaconda3/envs/DA2026/lib/python3.11/site-packages/sklearn/preprocessing/_encode
  warnings.warn(msg, UserWarning)
```

	Gender_M	District_2.0	District_5.0	District_6.0	District_7.0	District_8.0	District_9.0
0	1.0	0.0	1.0	0.0	0.0	0.0	0.0
1	1.0	0.0	0.0	0.0	0.0	0.0	0.0
2	1.0	0.0	1.0	0.0	0.0	0.0	0.0
3	1.0	0.0	0.0	0.0	1.0	0.0	0.0

	Gender_M	District_2.0	District_5.0	District_6.0	District_7.0	District_8.0	District_9.0
4	1.0	0.0	0.0	0.0	1.0	0.0	0.0

Finally, swap the original categorical columns for their dummy versions:

```
train_with_dummies = pd.concat([train_data.drop(columns=categorical_vars), train_dummies], axis=1)
test_with_dummies = pd.concat([test_data.drop(columns=categorical_vars), test_dummies], axis=1)

print(f"Training data with dummies shape: {train_with_dummies.shape}")
print(f"Test data with dummies shape: {test_with_dummies.shape}")
```

```
Training data with dummies shape: (589, 46)
Test data with dummies shape: (589, 46)
```

5.2.2 Weight of Evidence

Two variables in our data, `ZIP` and `StreeID`, are still not usable: they have far too many categories for one-hot encoding to be practical. Turning either one into dummies would blow up the number of predictors and increase computation time enormously. There are two common ways to handle this:

1. **Keep only the top categories** by frequency, grouping everything else into "other". Simple, but throws away information.
2. **Weight of Evidence (WoE)**, which is much better suited to high-cardinality variables.

Let's first check just how high the cardinality actually is:

```
high_cardinality_vars = ["ZIP", "StreeID"]

for var in high_cardinality_vars:
    unique_count = train_with_dummies[var].nunique()
    print(f"{var}: {unique_count} unique categories")
```

```
ZIP: 164 unique categories
StreeID: 562 unique categories
```

Weight of Evidence summarizes a categorical variable into a single numeric column, computed per category as:

$$\text{WOE} = \ln\left(\frac{\%\text{ of positives}}{\%\text{ of negatives}}\right)$$

where "positive" and "negative" refer to the two classes of a binary target (here, churners and non-churners respectively) matching how `churn` is coded (1/0) throughout this book. A positive WoE means a category has relatively more positives (churners) than negatives; a negative WoE means the opposite. It's widely used in practice, particularly in credit scoring (Van Gool et al. 2012).

WoE belongs to a broader family of techniques called **target statistics** (also called target encoding): replace each category with some summary of the target computed within that category. The simplest version computes each category's raw target mean, which we'll use directly later in this chapter for ordinal variables. That simplest version has a real weakness on a variable like `ZIP`, though: many categories have only a handful of observations, and a mean computed from three or four rows is noise dressed up as a number, not a genuine signal. Micci-Barreca (2001) shows this is exactly why a per-category target mean is normally blended with the overall mean (shrinkage) rather than used raw, and Prokhorenkova et al. (2018) describes the same failure mode, there called *target leakage*, as the reason more recent methods such as CatBoost's ordered target statistics exist at all. That's a reliability problem shared by *any* target statistic, WoE included, which

is exactly why grouping rare categories comes first (Step 2 below), before computing anything: it protects whichever statistic you use from being estimated on an unreliably small group.

So why WoE specifically instead of the raw target mean, even after that protection is in place? Two reasons. First, Moeyersoms and Martens (2015) directly compared a raw target-ratio encoding against WoE on a real churn-prediction dataset and found WoE gave the best results across several classifiers, including logistic regression. Second, its scale fits a model like logistic regression more naturally: WoE is a log-odds transform, and logistic regression predicts log-odds, not probability, so a WoE-encoded category sits on the same scale the model is actually estimating, whereas a raw target mean (a bounded probability) doesn't, and tends to behave less smoothly as a category's rate approaches 0% or 100%.

A closely related metric is the **Information Value (IV)**, which sums a weighted version of the WoE across all categories of a variable into a single number summarizing how predictive that whole variable is: $\text{IV} = \sum_i (\%\text{positive}_i - \%\text{negative}_i) \times \text{WOE}_i$.

An illustrative example below shows how to compute WoE and IV for a hypothetical `PaymentType` variable against churn (positive = churner, negative = non-churner):

Payment-Type	Count	Distr. Count	Positive	Distr. Positive	Negative	Distr. Negative	WOE	IV
Direct debit	800	40.00%	50	13.51%	750	46.01%	−1.2252	0.3982
Bank transfer	500	25.00%	80	21.62%	420	25.77%	−0.1754	0.0073
Credit card	450	22.50%	120	32.43%	330	20.25%	+0.4712	0.0574
Cash	250	12.50%	120	32.43%	130	7.98%	+1.4028	0.3431
Total	**2000**		**370**		**1630**			**0.8060**

Cash customers churn far more than the base rate (a strongly positive WoE), direct debit customers churn far less (a strongly negative WoE), and the variable's total IV of 0.81 would count as a **strong** predictor by the usual rule of thumb:

IV	Predictive power
< 0.02	Unpredictive
0.02 to 0.1	Weak
0.1 to 0.3	Medium
> 0.3	Strong

Two things make WoE attractive for high-cardinality variables: it **reduces dimensionality** (many categories become one numeric column), and it creates a **monotonic relationship** with the target, which linear models can exploit directly.

Implementing WoE takes a few steps: check data quality, group rare categories, fit WoE, then apply it to build the final dataset.

Step 1: check data quality. WoE breaks down for a category that's 100% or 0% one class (the log ratio is undefined), so it's worth checking how common that is per variable first. A rough heuristic: if more than about 80% of a variable's categories are problematic like this, it's probably not a good candidate for WoE at all.

```
train_demo = train_with_dummies.copy()
train_demo["churn"] = train_target
```

```
for var in high_cardinality_vars:
    churn_analysis = train_demo.groupby(var).agg(
        Total_Count=("churn", "count"), Churn_Count=("churn", "sum"), Churn_Rate=("churn", "mean"),
    ).round(3)

    problematic = churn_analysis[(churn_analysis["Churn_Rate"] == 0) | (churn_analysis["Churn_Rate"] ==

    print(f"{var}: {len(churn_analysis)} categories, {len(problematic)} problematic "
          f"({len(problematic) / len(churn_analysis):.0%})")
```

```
ZIP: 164 categories, 93 problematic (57%)
StreeID: 562 categories, 555 problematic (99%)
```

`StreeID` turns out to be almost entirely problematic categories, so we'll exclude it from WoE and just drop it later. `ZIP` looks usable.

Step 2: group rare categories. Before fitting WoE, grouping infrequent categories into a single "Rare" bucket with `RareLabelEncoder` makes the resulting WoE values more stable:

```
from feature_engine.encoding import RareLabelEncoder

suitable_vars = ["ZIP"]

rare_encoder = RareLabelEncoder(tol=0.02, n_categories=5, variables=suitable_vars, ignore_format=True)

train_rare = rare_encoder.fit_transform(train_demo[suitable_vars])
train_rare["churn"] = train_demo["churn"].values

test_demo = test_with_dummies.copy()
test_rare = rare_encoder.transform(test_demo[suitable_vars])

print(f"ZIP: {train_rare['ZIP'].nunique()} categories after grouping (was {train_demo['ZIP'].nunique()}
```

```
ZIP: 7 categories after grouping (was 164)
```

Step 3: fit WoE on the training set, apply to both:

```
from feature_engine.encoding import WoEEncoder

woe_encoder = WoEEncoder(variables=suitable_vars, ignore_format=True)
woe_encoder.fit(train_rare[suitable_vars], train_rare["churn"])

train_woe = woe_encoder.transform(train_rare[suitable_vars])
test_woe = woe_encoder.transform(test_rare[suitable_vars])

woe_summary = pd.DataFrame({"Category": train_rare["ZIP"], "WoE": train_woe["ZIP"]}).drop_duplicates()
woe_summary = woe_summary.merge(train_rare.groupby("ZIP")["churn"].agg(["count", "mean"]), left_on="Cat
woe_summary.columns = ["Category", "WoE", "Count", "Churn_Rate"]
woe_summary.sort_values("WoE", ascending=False).round(4)
```

	Category	WoE	Count	Churn_Rate
9	3740.0	0.5335	13	0.3846
0	3550.0	0.4927	16	0.3750
2	3500.0	0.1562	20	0.3000
18	3530.0	0.0872	14	0.2857
1	Rare	-0.0261	494	0.2632

	Category	WoE	Count	Churn_Rate
47	3630.0	-0.2005	13	0.2308
12	3600.0	-0.3182	19	0.2105

Step 4: assemble the final dataset, replacing `ZIP` with its WoE column and dropping `StreeID` entirely:

```
train_final = train_with_dummies.copy()
test_final = test_with_dummies.copy()

train_final["ZIP_woe"] = train_woe["ZIP"].values
test_final["ZIP_woe"] = test_woe["ZIP"].values

train_final = train_final.drop(columns=["ZIP", "StreeID"])
test_final = test_final.drop(columns=["ZIP", "StreeID"])

print(f"Training set: {train_final.shape}")
print(f"Test set: {test_final.shape}")
```

```
Training set: (589, 45)
Test set: (589, 45)
```

Dummy encoding vs. WoE: use both, for different variables

They try to solve the same problem, namely turning a categorical variable into numbers, but they're not really competitors, since they sit at opposite ends of the cardinality spectrum. Dummy encoding preserves the most detail per category but doesn't scale well after 10 categories; WoE scales to hundreds of categories and guarantees a monotonic relationship with the target, at the cost of collapsing each category down to a single number. In practice, a single dataset often needs both at once, depending on the needs of different columns, which is exactly as we just did.

5.3 Feature engineering

Feature engineering is the process of creating new features, or transforming existing ones, to help a model perform better in terms of predictive performance, interpretability, or both. It pays to keep the target model's biases in mind: a linear model needs features that separate the classes roughly linearly, so a raw date-of-birth column is far less useful to it than an age computed from it.

Recency, Frequency, and Monetary value (RFM) features are a classic example of this kind of engineering (Baesens et al. 2021): instead of feeding a model per-transaction data, summarize each customer's behavior along three simple dimensions:

- **Recency**: how long since the customer's last transaction, payment, or contact.
- **Frequency**: how often the customer transacts, e.g. per month or per year.
- **Monetary**: how much value those transactions carry, e.g. average, minimum, or maximum spend.

A fourth dimension, **length of relationship** (time since the customer's *first* transaction), is often added alongside the classic three. In fact, it's closely related to the customer-tenure and duration modeling used in CRM attrition research (Van den Poel and Larivière 2004). Each dimension can also be operationalized in several ways: frequency over different windows (last month vs. last year), monetary value summarized differently (average vs. highest vs. lowest), or any of the above computed separately per product, time period, or channel. RFM-style features are heavily used in CRM (Buckinx and Van den Poel 2005), fraud analytics (Baesens et al. 2021), and credit scoring (Hsieh 2004). Several of the aggregated columns already in our dataset (purchase counts, average discount, days since last payment) are exactly this kind of feature. We

won't build RFM features from scratch here since they're already present, but it's worth knowing the term and the reasoning behind it.

Let's take stock of where the dataset stands after value representation:

```
print(f"Shape: {train_final.shape}")
train_final.head()
```

```
Shape: (589, 45)
```

	ProductID_1_sum	ProductID_2_sum	ProductID_3_sum	ProductID_4_sum	ProductID_5_sum	ProductI
0	0.0	0.0	0.0	0.0	0.0	0.0
1	0.0	2.0	0.0	1.0	0.0	0.0
2	0.0	0.0	0.0	0.0	0.0	0.0
3	0.0	0.0	0.0	0.0	4.0	0.0
4	0.0	0.0	0.0	0.0	1.0	0.0

5.3.1 Non-linear relationships

5.3.1.1 Visual inspection for non-linear relationships

Before reaching for a transformation, it's worth checking whether one is actually needed. A convenient way to do that for a binary target is to fit a logistic regression on everything, then plot each feature against the model's **logit** (the log-odds it predicts). If a LOESS curve through that relationship tracks a straight line closely, the feature is already behaving roughly linearly with respect to the target; if it diverges, a transformation might help.

```
from sklearn.linear_model import LogisticRegression

numerical_features = ["TotalDiscount_mean", "PaymentDate_rec"]
X_demo = train_final[numerical_features].copy()
y_demo = train_target.copy()

model = LogisticRegression(random_state=42, max_iter=1000)
model.fit(train_final, train_target)

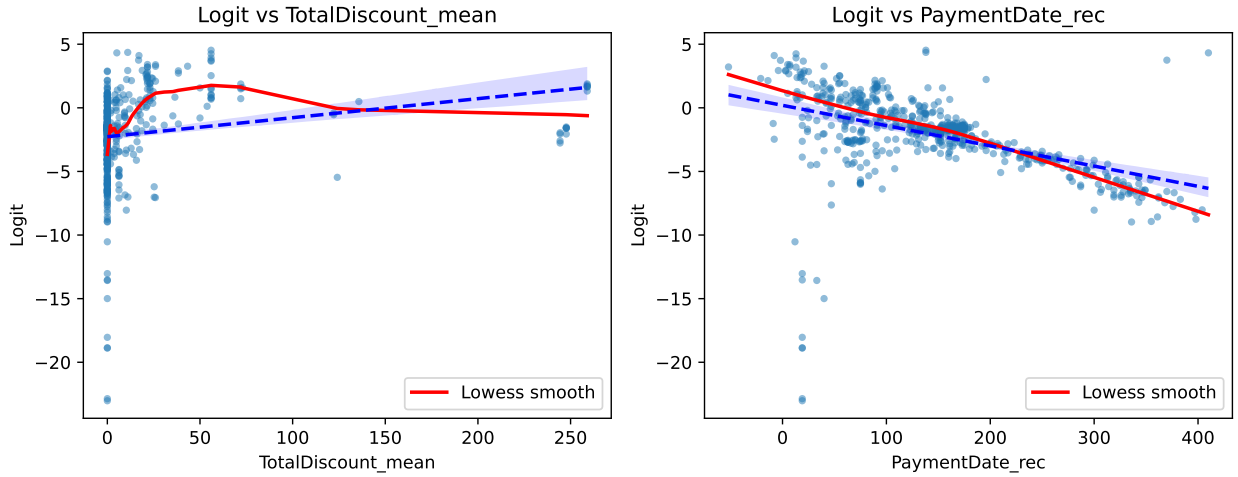
probabilities = np.clip(model.predict_proba(train_final)[:, 1], 1e-10, 1 - 1e-10)
logits = np.log(probabilities / (1 - probabilities))

fig, axes = plt.subplots(1, len(numerical_features), figsize=(5 * len(numerical_features), 4))
for i, feature in enumerate(numerical_features):
    sns.regplot(x=X_demo[feature], y=logits, ax=axes[i], scatter_kws={"alpha": 0.5, "s": 10},
                lowess=True, line_kws={"color": "red", "linewidth": 2, "label": "Lowess smooth"})
    sns.regplot(x=X_demo[feature], y=logits, ax=axes[i], scatter=False,
                line_kws={"color": "blue", "linewidth": 2, "linestyle": "--", "label": "Linear fit"})
    axes[i].set_xlabel(feature)
    axes[i].set_ylabel("Logit")
    axes[i].set_title(f"Logit vs {feature}")
    axes[i].legend()

plt.tight_layout()
plt.show()
```

```
/Users/matthiasbogaert/anaconda3/envs/DA2026/lib/python3.11/site-packages/sklearn/linear_model/_logisti
STOP: TOTAL NO. OF ITERATIONS REACHED LIMIT
```

```
Increase the number of iterations to improve the convergence (max_iter=1000).
You might also want to scale the data as shown in:
    https://scikit-learn.org/stable/modules/preprocessing.html
Please also refer to the documentation for alternative solver options:
    https://scikit-learn.org/stable/modules/linear_model.html#logistic-regression
  n_iter_i = _check_optimize_result(
/Users/matthiasbogaert/anaconda3/envs/DA2026/lib/python3.11/site-packages/statsmodels/nonparametric/smo
  res, _ = _lowess(y, x, x, np.ones_like(x),
```



The two lines are close for both features here, which suggests a fairly linear relationship already. `TotalDiscount_mean` diverges slightly more than `PaymentDate_rec`, so it's the more plausible candidate for a transformation. Let's try a couple anyway, to see what they'd actually buy us.

5.3.1.2 Polynomial features

Polynomial features add higher-order and interaction terms, letting a linear model capture curvature it otherwise couldn't. Let's create a degree-2 polynomial expansion of `TotalDiscount_mean`:

```
from sklearn.preprocessing import PolynomialFeatures

target_feature = numerical_features[0]

poly_transformer = PolynomialFeatures(degree=2, include_bias=False)
target_poly = poly_transformer.fit_transform(train_final[[target_feature]])
poly_names = poly_transformer.get_feature_names_out([target_feature])

target_poly_df = pd.DataFrame(target_poly, columns=poly_names, index=train_final.index)
print(f"New polynomial features: {list(poly_names)}")
```

```
New polynomial features: ['TotalDiscount_mean', 'TotalDiscount_mean^2']
```

Adding it to the dataset and comparing training accuracy against the original model shows whether it actually helped:

```
train_final_with_poly = pd.concat([train_final, target_poly_df.iloc[:, 1:]], axis=1)  # skip the origina

model_poly = LogisticRegression(random_state=42, max_iter=1000)
model_poly.fit(train_final_with_poly, train_target)
```

```
original_score = model.score(train_final, train_target)
poly_score = model_poly.score(train_final_with_poly, train_target)

print(f"Original accuracy: {original_score:.4f}")
print(f"With polynomial feature: {poly_score:.4f}  (Δ = {poly_score - original_score:+.4f})")
```

```
Original accuracy: 0.8727
With polynomial feature: 0.8778  (Δ = +0.0051)

/Users/matthiasbogaert/anaconda3/envs/DA2026/lib/python3.11/site-packages/sklearn/linear_model/_logisti
STOP: TOTAL NO. OF ITERATIONS REACHED LIMIT

Increase the number of iterations to improve the convergence (max_iter=1000).
You might also want to scale the data as shown in:
    https://scikit-learn.org/stable/modules/preprocessing.html
Please also refer to the documentation for alternative solver options:
    https://scikit-learn.org/stable/modules/linear_model.html#logistic-regression
  n_iter_i = _check_optimize_result(
```

5.3.1.3 Power transformations

Power transformations reshape a variable's distribution, often to make it more Gaussian-like and to stabilize its variance, which can help both linear models and any method that assumes a normal distribution of errors. A simple **logarithmic transformation** (`x -> log(x)`, or `x -> log(1 + x)` when zeros are present) is the most common case, typically applied to strictly-positive, right-skewed "size" variables like loan amounts or revenue. More general power transforms (`x -> x^ `) trade off skew in either direction depending on , at the cost of needing to fit that .

Box-Cox and **Yeo-Johnson** are the two standard ways to fit automatically rather than guessing it. Box-Cox only works for strictly positive values; Yeo-Johnson extends it to handle zero and negative values too (and reduces to Box-Cox exactly when all values are positive). In `sklearn`, this works the same way as everything else fit in this book: `PowerTransformer`'s `.fit()` estimates directly from the training data (i.e., the value that makes the transformed distribution closest to Gaussian) rather than you having to search for it by hand. Let's try Yeo-Johnson on the same feature:

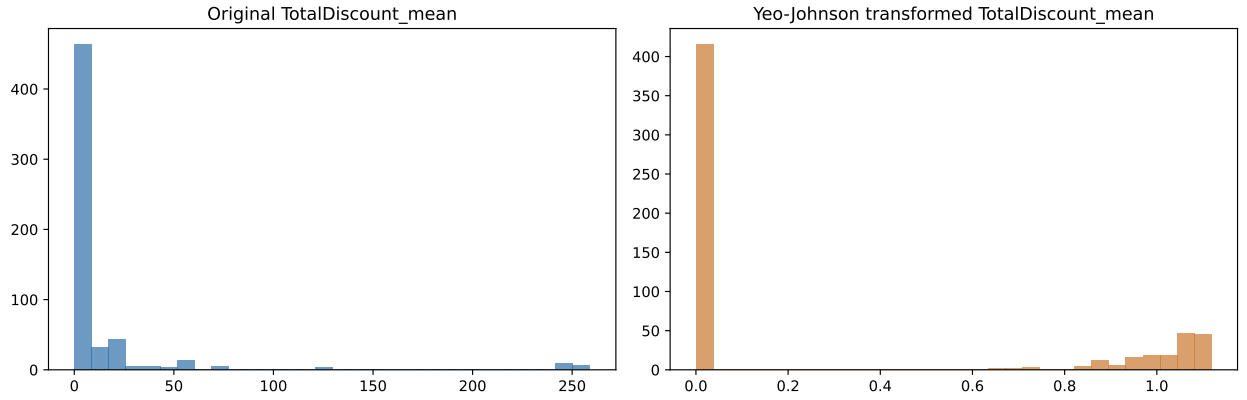
```
from sklearn.preprocessing import PowerTransformer

power_transformer = PowerTransformer(method="yeo-johnson", standardize=False)
target_transformed = power_transformer.fit_transform(train_final[[target_feature]])

print(f"Optimal lambda: {power_transformer.lambdas_[0]:.4f}")

fig, axes = plt.subplots(1, 2, figsize=(12, 4))
axes[0].hist(train_final[target_feature], bins=30, alpha=0.7, color="#2f6faa")
axes[0].set_title(f"Original {target_feature}")
axes[1].hist(target_transformed[:, 0], bins=30, alpha=0.7, color="#c9772e")
axes[1].set_title(f"Yeo-Johnson transformed {target_feature}")
plt.tight_layout()
plt.show()
```

```
Optimal lambda: -0.8871
```




```
train_final_with_yj = train_final.copy()
train_final_with_yj[f"{target_feature}_yj"] = target_transformed.flatten()

model_yj = LogisticRegression(random_state=42, max_iter=1000)
model_yj.fit(train_final_with_yj, train_target)

yj_score = model_yj.score(train_final_with_yj, train_target)
print(f"Original accuracy: {original_score:.4f}")
print(f"With Yeo-Johnson feature: {yj_score:.4f}  (Δ = {yj_score - original_score:+.4f})")
```

```
Original accuracy: 0.8727
With Yeo-Johnson feature: 0.8795  (Δ = +0.0068)

/Users/matthiasbogaert/anaconda3/envs/DA2026/lib/python3.11/site-packages/sklearn/linear_model/_logisti
STOP: TOTAL NO. OF ITERATIONS REACHED LIMIT

Increase the number of iterations to improve the convergence (max_iter=1000).
You might also want to scale the data as shown in:
    https://scikit-learn.org/stable/modules/preprocessing.html
Please also refer to the documentation for alternative solver options:
    https://scikit-learn.org/stable/modules/linear_model.html#logistic-regression
  n_iter_i = _check_optimize_result(
```

In this case, neither the polynomial term nor the Yeo-Johnson transform moved training accuracy much. This is consistent with the logit plots above, which already showed a fairly linear relationship. That's a useful negative result: it means we can discard both here rather than carrying extra, unhelpful complexity forward.

That's not always the case, though. Sometimes a transformation like this is the difference between a model that's good enough to detect a real, actionable relationship and one that isn't. Wang et al. (2019) study how a seller's mix of marketing versus non-marketing social media posts relates to how popular those posts become, and specifically whether there's an *optimal* level of marketing aggressiveness. The effect of aggressiveness level is an inverted U-shape curve, quantified with a quadratic term in a plain linear regression too. But applying a Yeo-Johnson transform to one of their control variables (a seller's number of followers) nearly triples the model's R^2 compared to the untransformed version. The take-away: the transformation itself didn't reveal the U-shape; it made the model reliable enough to trust the U-shape it found.

Interaction effects

Sometimes a variable has no linear effect on its own: $y \not\leftarrow f(x_1)$ and $y \not\leftarrow f(x_2)$ individually, but the *combination* $y \leftarrow f(x_1, x_2)$ does, often represented simply as the product $x_1 \times x_2$. A classic example: `Gender` on its own might show no relationship with an outcome, but `Gender × Income` might, if the

effect of income differs by gender. This is sometimes called moderation, with the interaction term acting as the moderator.

5.4 Standardization and normalization

Different algorithms need scaling for different reasons: distance-based methods like k-nearest neighbors and linear models with regularization need it directly, neural networks need it to avoid numerical overflow, and tree-based models don't need it at all. The two most common approaches are:

- **Standardization**: rescale to mean 0, standard deviation 1 (the z-score). Good for distance-based algorithms and for comparing coefficients across features.
- **Min-max normalization**: rescale to a fixed range, typically [0, 1]. Common for neural networks and any method that expects bounded input.

As with the transformations above, `sklearn` estimates the numbers each scaler needs directly from the data: `StandardScaler.fit()` computes each feature's mean and standard deviation, `MinMaxScaler.fit()` computes its min and max, and `.transform()` applies them.

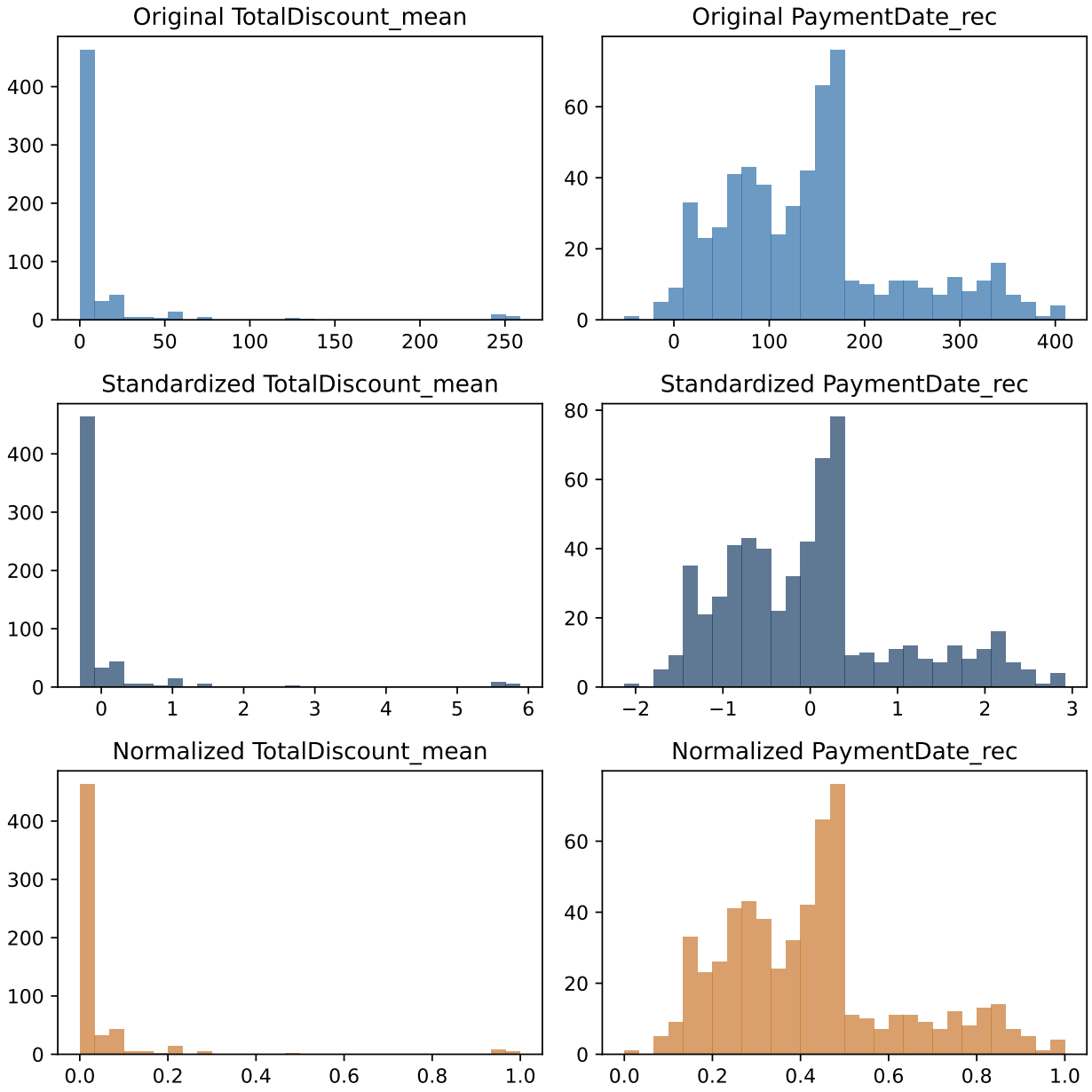
```
from sklearn.preprocessing import StandardScaler, MinMaxScaler

scaler_standard = StandardScaler()
X_demo_standardized = pd.DataFrame(scaler_standard.fit_transform(X_demo), columns=X_demo.columns)

scaler_minmax = MinMaxScaler()
X_demo_normalized = pd.DataFrame(scaler_minmax.fit_transform(X_demo), columns=X_demo.columns)

fig, axes = plt.subplots(3, len(X_demo.columns), figsize=(4 * len(X_demo.columns), 8))
for i, feature in enumerate(X_demo.columns):
    axes[0, i].hist(X_demo[feature], bins=30, alpha=0.7, color="#2f6faa")
    axes[0, i].set_title(f"Original {feature}")
    axes[1, i].hist(X_demo_standardized[feature], bins=30, alpha=0.7, color="#1a3f66")
    axes[1, i].set_title(f"Standardized {feature}")
    axes[2, i].hist(X_demo_normalized[feature], bins=30, alpha=0.7, color="#c9772e")
    axes[2, i].set_title(f"Normalized {feature}")

plt.tight_layout()
plt.show()
```



⚠ Fit on training, transform on test and never scale dummy variables!

The fit-on-train, transform-on-both rule from the previous chapter applies here too: fit the scaler on training data only, then use it to transform both sets. There's a second, easy-to-miss mistake specific to scaling: **don't scale dummy/binary columns**. A one-hot column only ever takes the values 0 and 1, and standardizing it doesn't make it more useful. More importantly, it destroys the direct 0/1 interpretation. Hence, only continuous features should go through the scaler.

```
binary_features, continuous_features = [], []
for col in train_final.columns:
    unique_values = set(train_final[col].unique())
    if unique_values == {0, 1}:
        binary_features.append(col)
```

```
    else:
        continuous_features.append(col)

print(f"Binary features (not scaled): {len(binary_features)}")
print(f"Continuous features (scaled): {len(continuous_features)}")

train_binary, train_continuous = train_final[binary_features], train_final[continuous_features]
test_binary, test_continuous = test_final[binary_features], test_final[continuous_features]
```

```
Binary features (not scaled): 11
Continuous features (scaled): 34
```

```
scaler = StandardScaler()
scaler.fit(train_continuous)

train_continuous_scaled = pd.DataFrame(
    scaler.transform(train_continuous), columns=continuous_features, index=train_final.index,
)
test_continuous_scaled = pd.DataFrame(
    scaler.transform(test_continuous), columns=continuous_features, index=test_final.index,
)

# recombine, keeping the original column order
train_final_scaled_df = pd.concat([train_continuous_scaled, train_binary], axis=1)[train_final.columns]
test_final_scaled_df = pd.concat([test_continuous_scaled, test_binary], axis=1)[test_final.columns]

print(f"Final scaled training shape: {train_final_scaled_df.shape}")
print(f"Final scaled test shape: {test_final_scaled_df.shape}")
```

```
Final scaled training shape: (589, 45)
Final scaled test shape: (589, 45)
```

Transformation	When to use	Example algorithms
Standardization	Distance-based methods, comparing coefficients	SVM, KNN, linear/logistic regression
Min-max normalization	Bounded features needed	Neural networks, some clustering methods
Polynomial features	Non-linear relationships in a linear model	Linear/logistic regression
Power transforms	Skewed data, improving normality	Linear models, parametric tests
No scaling	Split-based models	Random forests, gradient boosting, decision trees

5.5 Variable selection

Real datasets usually come with far more variables than actually needed to achieve good predictive performance. Searching every possible subset is computationally infeasible (even in today's world). So **variable selection** methods use a heuristic instead: keep predictors that are strongly related to the response and weakly related to each other, and drop the rest. In other words, delete both the ones that are **irrelevant** (unrelated to the target) and the ones that are **redundant** (correlated with another predictor that already carries the same information). This isn't purely a matter of convenience: more compact models are also faster to estimate, and can genuinely perform *better* (thanks to the curse of dimensionality), since more variables for a fixed number of observations makes the prediction task harder, not easier (Verbeke et al. 2012).

Three broad families of methods exist: **filter** methods (fast, univariate, model-independent), **wrapper** methods (recursive feature elimination, and stepwise selection, considered more expensive since they retrain a model repeatedly), and **embedded** methods (regularization such as L1/L2/elastic net, built directly into the model-fitting process). Filter methods are covered here, and the other two families are introduced in the next chapters.

5.5.1 Filter methods

Filter methods score each predictor's univariate relationship with the target, independent of any model, which makes them a fast first screening pass. Which score to use depends on the variable type:

	Continuous target	Categorical target
Continuous variable	Pearson correlation	Fisher score / ANOVA
Categorical variable	Fisher score / ANOVA	Information Value, Cramér's V

For our continuous features against the (categorical) churn target, `sklearn`'s `f_classif` plays the role of the Fisher score, paired with `SelectKBest` to keep only the top-scoring ones:

```
from sklearn.feature_selection import SelectKBest, f_classif

k_best_numerical = min(20, len(continuous_features))
selector_numerical = SelectKBest(score_func=f_classif, k=k_best_numerical)
selector_numerical.fit(train_continuous_scaled, train_target)

selected_numerical_features = np.array(continuous_features)[selector_numerical.get_support()].tolist()

numerical_results = pd.DataFrame({
    "feature": continuous_features, "f_score": selector_numerical.scores_,
}).sort_values("f_score", ascending=False)

numerical_results.head(10)
```

```
/Users/matthiasbogaert/anaconda3/envs/DA2026/lib/python3.11/site-packages/sklearn/feature_selection/_uni
  warnings.warn("Features %s are constant." % constant_features_idx, UserWarning)
/Users/matthiasbogaert/anaconda3/envs/DA2026/lib/python3.11/site-packages/sklearn/feature_selection/_uni
  f = msb / msw
```

	feature	f_score
19	StartDate_rec	105.595305
21	PaymentDate_rec	93.995147
28	FormulaDiscount_mean	71.133723
26	NetNewspaperPrice_mean	52.653478
23	NbrStart_mean	34.391696
30	TotalPrice_mean	33.057998
17	PaymentStatus_Paid_sum	31.056359
14	Pattern_1111110_sum	28.830724
15	PaymentType_BT_sum	24.807797
25	NetFormulaPrice_mean	22.855152

For the binary (dummy) features, chi-squared and mutual information are the equivalent tools:

```
from functools import partial
from sklearn.feature_selection import mutual_info_classif, chi2

k_best_categorical = min(5, len(binary_features))

selector_chi2 = SelectKBest(score_func=chi2, k=k_best_categorical)
selector_chi2.fit(train_binary, train_target)
selected_chi2 = [f for f, keep in zip(binary_features, selector_chi2.get_support()) if keep]

# mutual_info_classif adds a little random noise internally: fix the seed so the result is reproducible
selector_mi = SelectKBest(score_func=partial(mutual_info_classif, random_state=42), k=k_best_categorical
selector_mi.fit(train_binary, train_target)
selected_mi = [f for f, keep in zip(binary_features, selector_mi.get_support()) if keep]

print(f"Top by chi-squared: {selected_chi2}")
print(f"Top by mutual information: {selected_mi}")
```

```
Top by chi-squared: ['RenewalDate_rec_missing', 'PaymentDate_rec_missing', 'Gender_missing', 'District_
Top by mutual information: ['RenewalDate_rec_missing', 'Gender_M', 'District_5.0', 'District_7.0', 'Dis
```

Combining the two: take every continuous feature the numerical filter selected, plus only the binary features *both* categorical filters agreed on:

```
selected_binary = list(set(selected_chi2) & set(selected_mi))
all_selected_features = selected_numerical_features + selected_binary

train_final_selected = train_final_scaled_df[all_selected_features]
test_final_selected = test_final_scaled_df[all_selected_features]

print(f"Selected {len(all_selected_features)} features "
      f"({len(selected_numerical_features)} continuous, {len(selected_binary)} binary), "
      f"down from {train_final_scaled_df.shape[1]}")
```

```
Selected 23 features (20 continuous, 3 binary), down from 45
```

Filter methods are fast and a good first pass, but they're univariate. They don't account for correlations *between* features, so a certain amount of redundancy can slip through. That's exactly why they're normally used as a quick first cut, often followed by a wrapper or embedded method for the final selection. Determining the actual optimal number of features is left for later, once cross-validation is introduced.

5.6 Ordinal encoding

Ordinal variables have a natural ranking in their categories (e.g., credit rating, education level, or a customer loyalty tier) unlike a plain nominal categorical variable like `Gender` or `District`. Our dataset doesn't happen to have one, so to demonstrate the encoding options, we'll build a synthetic **customer value tier** from two existing features: total product purchases and payment recency.

> **i** This section is a synthetic, self-contained demo
>
> The customer tiers and the churn labels used below are artificially generated purely to illustrate the three encoding methods. They are *not* the real `churn` target from earlier in the chapter, even though the code reuses familiar variable names for convenience.

```
np.random.seed(42)
```

```
product_cols = [col for col in train_final.columns if "ProductID" in col and "_sum" in col]
total_products = train_final[product_cols].sum(axis=1)
recency_factor = 1 / (1 + train_final["PaymentDate_rec"])
value_score = total_products * 0.7 + recency_factor * 0.3

q1, q2, q3 = np.percentile(value_score, [25, 50, 75])

def assign_tier(score):
    if score <= q1:
        return "Bronze"
    elif score <= q2:
        return "Silver"
    elif score <= q3:
        return "Gold"
    return "Platinum"

customer_tiers = [assign_tier(score) for score in value_score]
print(pd.Series(customer_tiers).value_counts())
```

```
Silver      153
Bronze      148
Platinum    147
Gold        141
Name: count, dtype: int64
```

```
# Synthetic churn, deliberately lower for higher tiers, so the encodings below have something to pick u
tier_churn_probs = {"Bronze": 0.6, "Silver": 0.4, "Gold": 0.25, "Platinum": 0.15}
synthetic_churn = [np.random.binomial(1, tier_churn_probs[tier]) for tier in customer_tiers]

X_train_ordinal = pd.DataFrame({"customer_tier": customer_tiers})
y_train_ordinal = pd.Series(synthetic_churn, name="churn")

print(pd.DataFrame({"customer_tier": customer_tiers, "churn": synthetic_churn})
      .groupby("customer_tier")["churn"].agg(["mean", "count"]).round(3))
```

```
                mean  count
customer_tier
Bronze         0.554    148
Gold           0.291    141
Platinum       0.129    147
Silver         0.399    153
```

Manual ordinal encoding uses domain knowledge to specify the order directly (Bronze < Silver < Gold < Platinum):

```
from sklearn.preprocessing import OrdinalEncoder

tier_order = [["Bronze", "Silver", "Gold", "Platinum"]]
sklearn_encoder = OrdinalEncoder(categories=tier_order)
sklearn_encoder.fit(X_train_ordinal)

print("Bronze → 0, Silver → 1, Gold → 2, Platinum → 3")
```

```
Bronze → 0, Silver → 1, Gold → 2, Platinum → 3
```

Automated ordinal encoding orders categories by their relationship with the target instead of by domain

knowledge. `feature-engine`'s version always orders ascending by target mean:

```
from feature_engine.encoding import OrdinalEncoder as FeatureEngineOrdinalEncoder

fe_encoder = FeatureEngineOrdinalEncoder(variables=["customer_tier"])
fe_encoder.fit(X_train_ordinal, y_train_ordinal)

print(f"Categories ordered by churn rate (ascending): {fe_encoder.encoder_dict_['customer_tier']}")
```

```
Categories ordered by churn rate (ascending): {'Platinum': 0, 'Gold': 1, 'Silver': 2, 'Bronze': 3}
```

Target (mean) encoding goes a step further and replaces each category with the actual mean of the target for that category, rather than just a rank. This is the plain target statistic flagged back in the Weight of Evidence section. Here, it's reliable enough since our synthetic tiers are each backed by hundreds of observations. But remember, this is exactly the kind of raw, unshrunk per-category mean that gets noisy fast on a variable with many small categories:

```
from feature_engine.encoding import MeanEncoder

target_encoder = MeanEncoder(variables=["customer_tier"])
target_encoder.fit(X_train_ordinal, y_train_ordinal)

print(f"Encoding: {target_encoder.encoder_dict_['customer_tier']}")
```

```
Encoding: {'Bronze': 0.5540540540540541, 'Gold': 0.2907801418439716, 'Platinum': 0.1292517006802721, 'S
```

```
comparison = pd.DataFrame({
    "customer_tier": X_train_ordinal["customer_tier"].unique(),
})
comparison["sklearn_ordinal"] = [sklearn_encoder.categories_[0].tolist().index(t) for t in comparison["
comparison["feature_engine_ordinal"] = comparison["customer_tier"].map(fe_encoder.encoder_dict_["custom
comparison["target_encoding"] = comparison["customer_tier"].map(target_encoder.encoder_dict_["customer_
comparison.sort_values("target_encoding")
```

	customer_tier	sklearn_ordinal	feature_engine_ordinal	target_encoding
3	Platinum	3	0	0.129252
1	Gold	2	1	0.290780
2	Silver	1	2	0.398693
0	Bronze	0	3	0.554054

Which to use depends on what you have and what you need:

- **Manual encoding**: use when you have a clear, defensible business hierarchy. It stays interpretable, but isn't necessarily optimal for prediction.
- **Automated (feature-engine) encoding**: use when you'd rather let the data reveal the order. Useful for finding surprising patterns, at some cost to interpretability (e.g., it might not be obvious *why* Platinum ends up coded as 0 in a given fit).
- **Target encoding**: use when predictive power matters most. It creates the strongest statistical relationship with the target, but is also the most prone to overfitting, especially with small categories, and needs careful cross-validation (or an approach like WoE) to use safely.

5.7 Putting it all together

The exact order can vary by project, but a reasonable default sequence for everything covered in this chapter (and the previous one) is:

1. Impute missing values.
2. Resolve outliers and other data quality issues.
3. Engineer individual features (transformations, computed columns).
4. Discretize, where useful.
5. Encode categorical variables.
6. Create interaction effects.
7. Normalize or standardize.
8. Apply variable selection or dimensionality reduction.

> **i** The order isn't arbitrary
>
> Each step assumes the previous ones are already done: a transformation expects a variable's original distribution rather than a rescaled one, an interaction term expects numeric inputs rather than raw category labels, a scaler expects genuinely continuous features. Swapping steps around doesn't just change the output: it can make a step mean something different from what it's meant to.

5.8 Where we go from here

Between this chapter and the last one, `train_data` and `test_data` have gone from a raw, messy table to a fully numeric, model-ready basetable: imputed, encoded, transformed, scaled, and filtered down to a manageable set of predictors. The next chapter, **Model Evaluation**, is where that basetable finally meets a model and where the train/test split gets formalized properly, alongside the metrics needed to evaluate whether a model is actually any good.

Chapter 6

Model Evaluation

The previous two chapters turned a raw table into a clean, model-ready basetable. The obvious next step is to fit a model to it. Before that's worth doing, though, we need to answer a more basic question: *how would we even properly set up an experiment and how would we know if the model is any good?*

In CRISP-DM terms this is the **evaluation** phase, and it's where most predictive modelling projects actually go wrong. The mistakes rarely happen in the modelling itself; they happen in the **experimental setup**, the way the data is split into parts for learning and for testing. Get that wrong and every performance number you report afterwards is fictional and unreliable.

This chapter covers the two things you need before you can trust a model:

- **An experimental setup**: a sampling strategy that produces a fair, unseen sample to measure performance on (holdout, cross-validation, nested cross-validation, the bootstrap), plus the data-leakage traps that quietly break all of them.
- **Performance metrics**: how to actually score a model once you have an honest test set, for both regression and classification targets.

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

RANDOM_STATE = 123
```

6.1 The experimental setup

The single most important principle in predictive modelling is that a model must be evaluated on **unseen** data. Scoring a model on the same rows it was trained on is a self-fulfilling prophecy: it will look accurate because you're checking patterns against the exact data those patterns were extracted from. If every elderly customer in the training data happened to churn, the model will "learn" that elderly customers churn, and score perfectly on that training data, whether or not the relationship is real. Only data the model has never touched can tell you whether a learned pattern generalizes well or not.

The fix is to split the available data into separate parts: one for *learning* the patterns, another for *evaluating* them. Several strategies exist for doing that, and the rest of this section works through them.

We'll use a compact **churn** basetable with just four predictors: a monetary score, a transaction count, a recency score, and the number of days the customer has been active. This is an RFM plus length-of-relationship feature set, exactly the kind discussed in the previous chapter.

```
basetable = pd.read_csv("../data/processed/basetable.csv")

print(f"Dataset shape: {basetable.shape}")
print(f"Churn rate: {basetable['churn'].mean():.3f}")
basetable.head()
```

```
Dataset shape: (41739, 5)
Churn rate: 0.038
```

	Monetary_score	Total_quantity	recency_score	days_customer	churn
0	12519.0	715	16	15503	0
1	7315.2	309	36	16964	0
2	3267.8	176	15	22442	0
3	1289.0	74	21	16598	0
4	6789.0	390	1	15137	0

```
X = basetable.drop("churn", axis=1)
y = basetable["churn"]
```

6.1.1 The holdout set: train and test

The simplest strategy is the **holdout set**: shuffle the data and split it once into a **training set** to learn from and a **test set** to evaluate on (Figure 6.1).

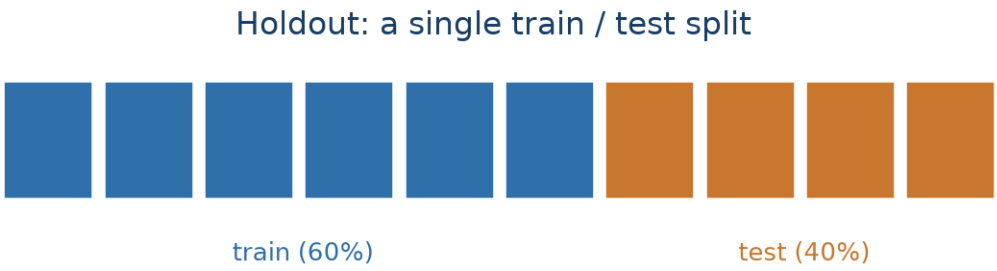


Figure 6.1: A single holdout split: most of the data trains the model, the rest is held back to test it.

Two practical points. We use a **60/40 split**: slightly more data in training (more training data generally means a better model) while still leaving a substantial test set. And we set a **random state** (seed) so the "random" sample is reproducible. That is essential while learning and debugging, so everyone gets the same split. `stratify=y` keeps the churn rate the same in both sets, which matters when the positive class is rare.

```
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.4, random_state=RANDOM_STATE, stratify=y
)

print(f"Training set: {X_train.shape[0]} rows ({X_train.shape[0] / len(X):.0%})")
print(f"Test set:     {X_test.shape[0]} rows ({X_test.shape[0] / len(X):.0%})")
print(f"Churn rate, train: {y_train.mean():.3f}, test: {y_test.mean():.3f}")
```

```
Training set: 25043 rows (60%)
```

```
Test set:     16696 rows (40%)
Churn rate, train: 0.038, test: 0.038
```

> **i** A note on the split ratio
>
> There is no universal rule for how much data to hold out. This book uses a 60/40 split, but 70/30 and 80/20 are also common, and scikit-learn's own `train_test_split` defaults to a 75/25 split when `test_size` and `train_size` are both left unspecified. The choice is a direct trade-off, not a fixed convention: a bigger training set gives the model more to learn from and lowers its bias, while a bigger test set gives a less variable, more trustworthy estimate of how well it actually performs (James et al. 2013). With a small or moderately sized dataset, protecting the test-set estimate a bit more, as with a 60/40 split, is a reasonable choice.

The holdout set is simple and works well on large samples (roughly 10,000+ rows). Its weakness is that the test-set score depends on *which* rows happened to land in the test set: a different random split can give a meaningfully different number.

6.1.2 The holdout set: train, validation, and test

As soon as you compare several models or fine-tune hyperparameter settings, a plain train/test split isn't enough. If you evaluate many candidate models on the test set and keep the best, you've effectively tuned to that specific test set (also called *test-set torturing*) and its score is no longer an honest estimate of future performance.

The fix is a **three-way split** (Figure 6.2): a **training set** to fit models, a **validation set** to compare them and pick the winner, and a **test set** that stays untouched until the very end for the best, final, unbiased measurement. We'll use **60/20/20**. In `sklearn` this takes two calls to `train_test_split`: first create the 20% test set, then split the remaining 80% into train (75% of 80% = 60%) and validation (25% of 80% = 20%).

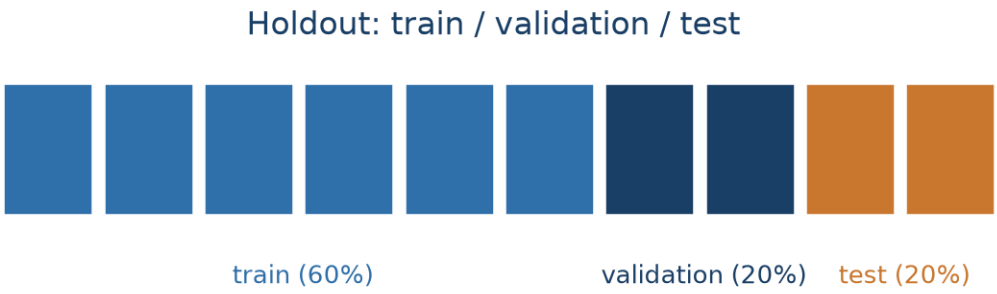


Figure 6.2: A three-way split: the validation set is used to compare models and pick hyperparameters, and only the test set is kept for the final measurement.

```python
X_temp, X_test, y_temp, y_test = train_test_split(
    X, y, test_size=0.2, random_state=RANDOM_STATE, stratify=y
)
X_train, X_val, y_train, y_val = train_test_split(
    X_temp, y_temp, test_size=0.25, random_state=RANDOM_STATE, stratify=y_temp
)

print(f"Train: {len(X_train)}  |  Validation: {len(X_val)}  |  Test: {len(X_test)}")
```

```
Train: 25043  |  Validation: 8348  |  Test: 8348
```

> ⚠ Two things people get wrong with the three-way split
>
> **All data preparation belongs on the training set.** Bin width, imputation values, scaling statistics, encodings: everything gets *fitted* from training data only, then applied or *transformed* unchanged to validation and test. Fitting a scaler or an imputer on the full dataset leaks information about the test rows into the model and makes the model overly optimistic.
> **Never merge the validation and test sets.** It's fine (and recommended) to combine *train + validation* to refit the final model once the hyperparameter is chosen (albeit not needed if the training set is sufficiently big). The test set stays separate.

To make this concrete, let's tune one hyperparameter of a **K-nearest-neighbours (KNN)** classifier. KNN classifies an observation by finding the K most similar observations in the training data (by Euclidean distance) and taking a majority vote. K is a **hyperparameter**: it controls the model's behaviour but isn't estimated from the data the way a regression coefficient is, so we have to choose it ourselves by trying several values.

Because KNN uses distances, all features must be on the same scale, so we wrap a `StandardScaler` and the classifier in a **pipeline**, which guarantees the scaler is fit on training data only and the same transformation is reapplied to every other set. We score each K by AUC on the validation set (AUC is covered later in this chapter; for now, read it as "higher is better, 0.5 is random").

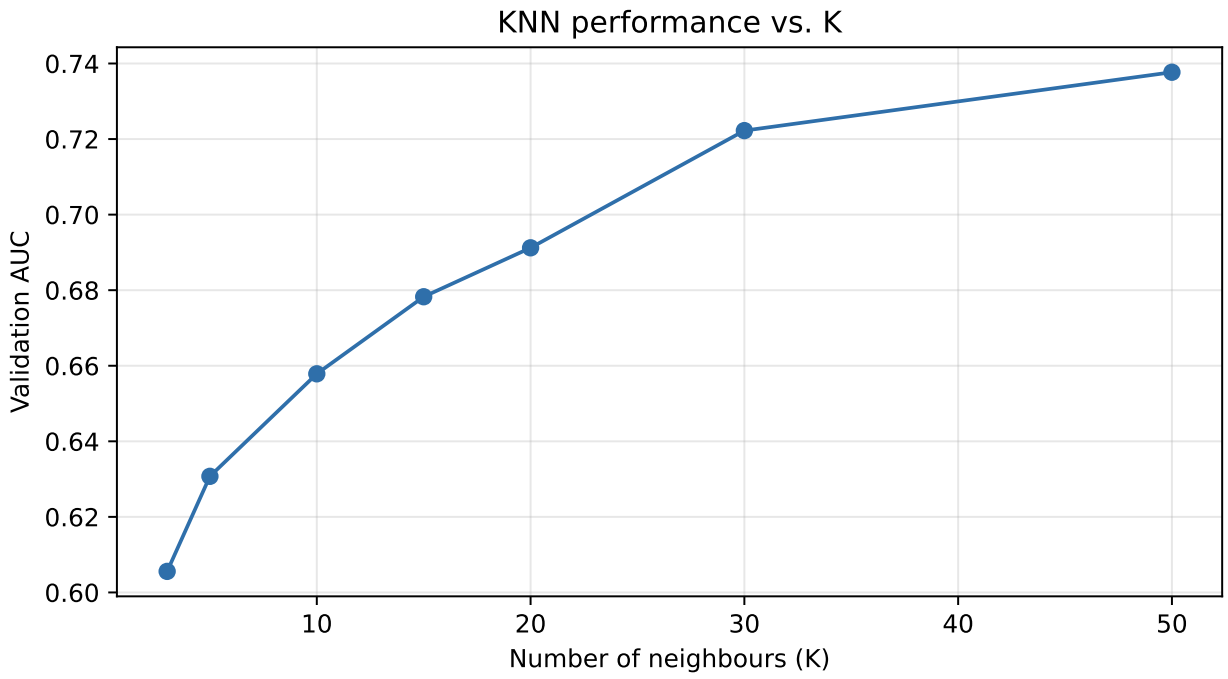
```
from sklearn.neighbors import KNeighborsClassifier
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import Pipeline
from sklearn.metrics import roc_auc_score

k_values = [3, 5, 10, 15, 20, 30, 50]
validation_scores = []

for k in k_values:
    pipeline = Pipeline([
        ("scaler", StandardScaler()),
        ("classifier", KNeighborsClassifier(n_neighbors=k)),
    ])
    pipeline.fit(X_train, y_train)
    val_probs = pipeline.predict_proba(X_val)[:, 1]
    validation_scores.append(roc_auc_score(y_val, val_probs))
    print(f"K={k:2d}: validation AUC = {validation_scores[-1]:.4f}")

plt.figure(figsize=(8, 4))
plt.plot(k_values, validation_scores, "o-", color="#2f6faa")
plt.xlabel("Number of neighbours (K)")
plt.ylabel("Validation AUC")
plt.title("KNN performance vs. K")
plt.grid(alpha=0.3)
plt.show()
```

```
K= 3: validation AUC = 0.6056
K= 5: validation AUC = 0.6307
K=10: validation AUC = 0.6579
K=15: validation AUC = 0.6783
K=20: validation AUC = 0.6912
K=30: validation AUC = 0.7222
K=50: validation AUC = 0.7377
```



Once the best K is identified on the validation set, we combine train + validation, refit the final model on all of it, and touch the test set exactly once:

```
best_k = k_values[int(np.argmax(validation_scores))]
best_val_score = max(validation_scores)

X_train_full = pd.concat([X_train, X_val])
y_train_full = pd.concat([y_train, y_val])

final_pipeline = Pipeline([
    ("scaler", StandardScaler()),
    ("classifier", KNeighborsClassifier(n_neighbors=best_k)),
])
final_pipeline.fit(X_train_full, y_train_full)

test_probs = final_pipeline.predict_proba(X_test)[:, 1]
test_auc = roc_auc_score(y_test, test_probs)

print(f"Best K = {best_k}  (validation AUC {best_val_score:.4f})")
print(f"Final test AUC: {test_auc:.4f}")
```

```
Best K = 50  (validation AUC 0.7377)
Final test AUC: 0.7439
```

The small gap between validation and test AUC is normal. It's the natural variation between two different random samples of the same data.

6.1.3 Cross-validation

A single train/validation split has the same weakness as a single holdout: the score depends on which rows landed where. **Cross-validation (CV)** averages that variation away. The data is cut into K equally sized **folds**; each fold serves as the test set exactly once while the other $K-1$ folds train the model, giving K scores that get averaged (Figure 6.3). **10-fold CV** is the most common choice in practice; but $K = 5$ is also a good

compromise between reliable estimates and computation time.

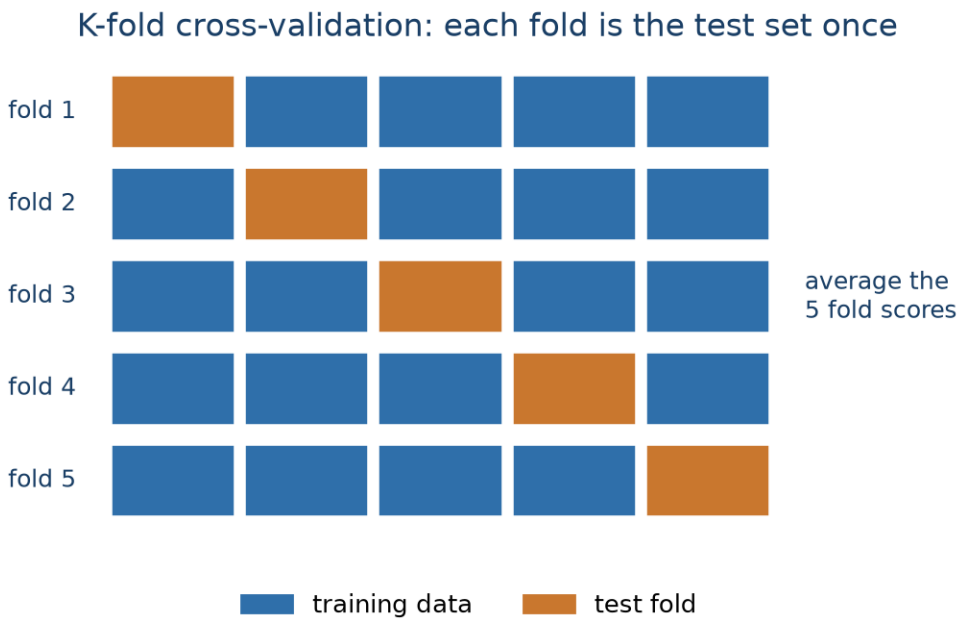


Figure 6.3: K-fold cross-validation: the data is cut into K folds, and each one takes a turn as the test set while the rest train the model.

`sklearn`'s `cross_validate` handles all the splitting, fitting, and scoring internally. Here we run it with a fixed KNN ($K = 50$) just to see how much a single split can vary. This is sometimes called the **big outer loop**: cross-validation used only to estimate final performance, with no hyperparameter tuning. We'll reuse this exact K-fold strategy twice more below: first as the tuning mechanism *inside* a search, then wrapped *around* that search. The "Nested cross-validation" section further down names these three roles precisely.

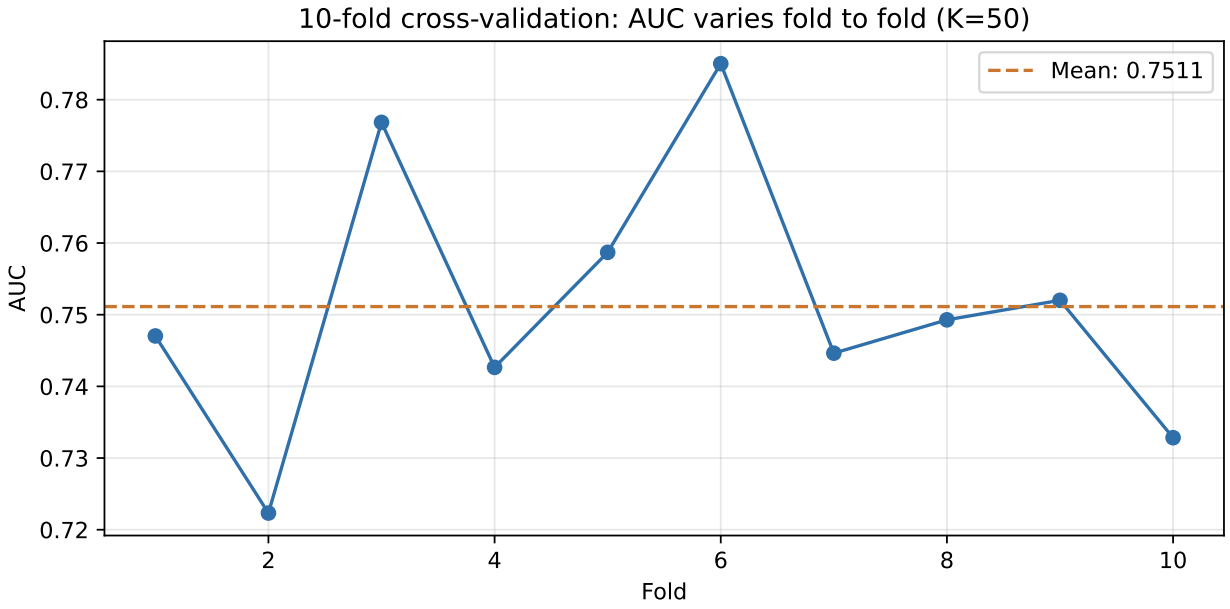
```
from sklearn.model_selection import cross_validate, KFold

pipeline_cv = Pipeline([
    ("scaler", StandardScaler()),
    ("classifier", KNeighborsClassifier(n_neighbors=50)),
])
kfold = KFold(n_splits=10, shuffle=True, random_state=RANDOM_STATE)

cv_results = cross_validate(pipeline_cv, X, y, cv=kfold, scoring="roc_auc")
cv_scores = cv_results["test_score"]
test_auc_cv = cv_scores.mean()

plt.figure(figsize=(9, 4))
plt.plot(range(1, 11), cv_scores, "o-", color="#2f6faa")
plt.axhline(test_auc_cv, color="#c9772e", ls="--", label=f"Mean: {test_auc_cv:.4f}")
plt.xlabel("Fold")
plt.ylabel("AUC")
plt.title("10-fold cross-validation: AUC varies fold to fold (K=50)")
plt.legend()
plt.grid(alpha=0.3)
plt.show()
```

```
print(f"AUC ranges from {cv_scores.min():.4f} to {cv_scores.max():.4f}; mean {test_auc_cv:.4f}")
```




```
AUC ranges from 0.7223 to 0.7850; mean 0.7511
```

6.1.4 Hyperparameter tuning with K-fold CV

In `sklearn`, `GridSearchCV` automates the inner train/validation loop: you give it a **pipeline**, a grid of hyperparameter values, and a CV strategy, and it fits every combination, scores it by cross-validation *within the training set*, keeps the best, and refits on the full training set, all without ever touching the test set.

The workflow becomes: split off a test set manually (the **outer loop**), let `GridSearchCV` do the validation splitting (the **inner loop**), and evaluate once on the test set at the end. This is less code, harder to get wrong, and needs no manual refitting.

A single holdout split for the outer loop, combined with full K-fold CV for the inner loop, is what's called the **big inner loop**: cheaper than also repeating the outer split, and the usual choice in engineering and research settings, where getting the hyperparameter right matters more than polishing the final performance number and the test set is already large enough to be reliable.

Below we tune K for KNN, but this time with `GridSearchCV`. The scoring metric is AUC, and the inner CV loop is 10-fold. The final test set in the outer loop is already present with the existing 20% holdout, and only touched once at the very end.

```
from sklearn.model_selection import GridSearchCV

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=RANDOM_STATE, stratify=y
)

pipeline = Pipeline([
    ("scaler", StandardScaler()),
    ("classifier", KNeighborsClassifier()),
])
param_grid = {"classifier__n_neighbors": [3, 5, 10, 15, 20, 30, 50]}
```

```
grid_search = GridSearchCV(
    pipeline, param_grid, cv=kfold, scoring="roc_auc", n_jobs=-1
)
grid_search.fit(X_train, y_train)

test_probs_grid = grid_search.predict_proba(X_test)[:, 1]
test_auc_grid = roc_auc_score(y_test, test_probs_grid)

print(f"Best K = {grid_search.best_params_['classifier__n_neighbors']}")
print(f"Best CV AUC (mean of 10 folds): {grid_search.best_score_:.4f}")
print(f"Final test AUC: {test_auc_grid:.4f}")
```

```
Best K = 50
Best CV AUC (mean of 10 folds): 0.7471
Final test AUC: 0.7439
```

`n_jobs=-1` tells `GridSearchCV` to use all CPU cores. The folds and hyperparameter settings are independent, so they run in parallel and the search finishes much faster on larger grids.

6.1.5 Nested cross-validation

Above, `GridSearchCV` gives a robust *hyperparameter* estimate, but the *final performance* number still relies on one random train/test split. If that split happened to be favourable, the reported AUC is not representative of the model's true performance.

Every example above is built from the same two roles, combined differently:

- The **outer loop** splits the data into training and test folds. It is used to estimate generalization performance: each outer test fold acts as a final, untouched holdout for scoring.
- The **inner loop** runs *inside* each outer training fold. It is used for hyperparameter tuning and model selection, choosing the best configuration before that configuration is ever scored on the outer test fold.

Cross-validation can play either role, or both at once, which gives three potential setups:

- **Big outer loop**: K-fold CV in the outer loop, a simple holdout split for tuning in the inner loop. The plain 10-fold example above is the most simple case of this, where there's no hyperparameter to tune at all, so only the outer loop is left. Common in business and applied settings, where compute is limited and the model is often already fixed.
- **Big inner loop**: a single holdout split in the outer loop, full K-fold CV in the inner loop. This is exactly the `GridSearchCV` example above. Common in engineering and research settings, where the hyperparameter search matters more than a polished final estimate.
- **Fully nested CV**: K-fold CV in *both* loops. The most rigorous of the three, and the most expensive.

Hence, **fully nested cross-validation** removes the last dependence on a single lucky split by running CV at two levels:

- an **outer loop** (e.g., 10-fold CV) that produces 10 independent train/test splits for performance estimation, and
- an **inner loop** (e.g., `GridSearchCV` with 10-fold CV) that tunes the hyperparameter separately *inside each outer training fold.*

Figure 6.4 draws this fully nested setup with 5-fold CV at both levels instead of 10. This makes it easier to visualize since each fold is larger. However, the logic remains identical for any K.

The code below uses 10-fold at both levels, matching the rest of the chapter. The result is 10 performance estimates instead of one, which gives a mean, a standard deviation, and a confidence interval, and no single lucky split can distort it. The cost is computation: $10 \times 10 \times 7 = 700$ model fits instead of 70. As a rule of

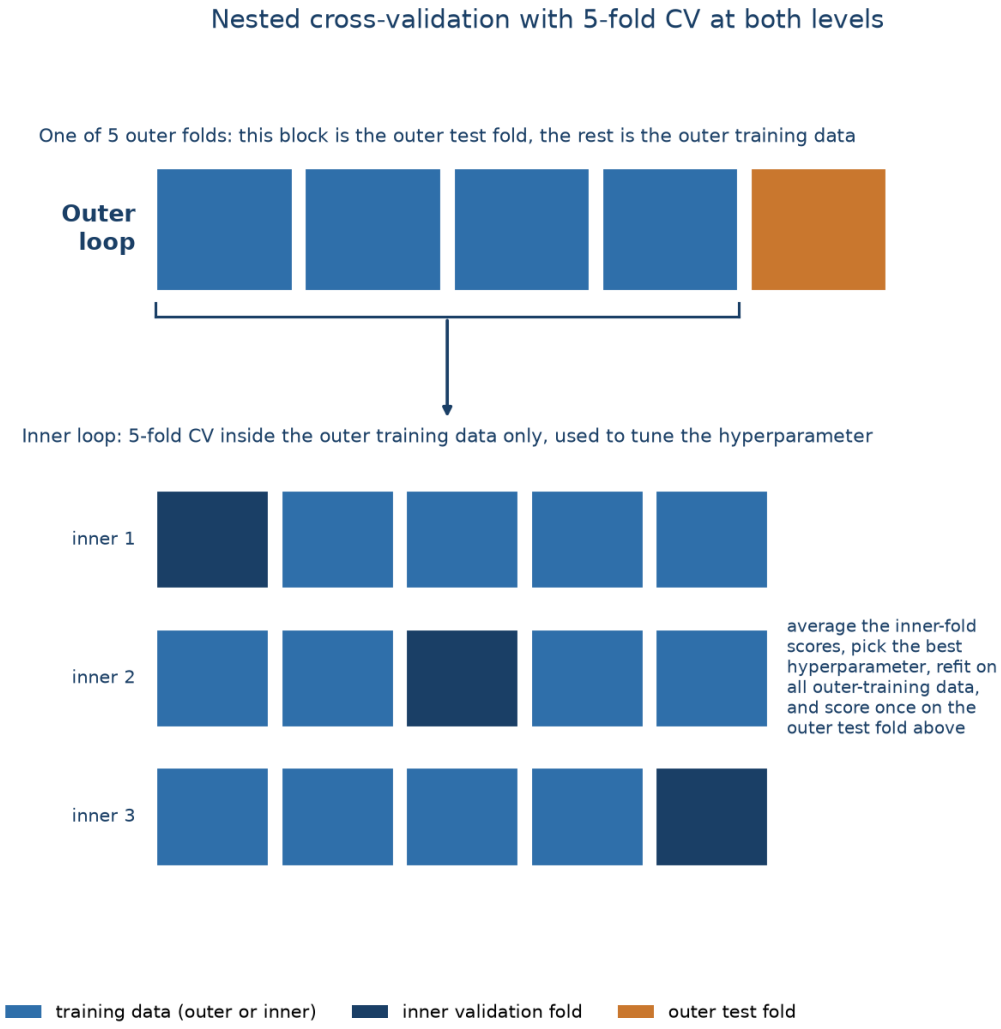


Figure 6.4: Nested cross-validation with 5-fold CV at both levels: one outer fold is shown on top, blue training data and an orange test fold; the arrow zooms into that training data below, where a second, inner 5-fold CV tunes the hyperparameter before the model is scored once on the outer test fold. The same logic applies for any choice of K.

thumb, academic work favours fully nested CV for maximum rigour; industry usually settles for the big inner loop (`GridSearchCV`, inner CV only) for speed.

```
nested_results = cross_validate(grid_search, X, y, cv=kfold, scoring="roc_auc", n_jobs=1)
nested_scores = nested_results["test_score"]
test_auc_nested = nested_scores.mean()

print(f"Mean AUC across 10 outer folds: {test_auc_nested:.4f}")
print(f"Standard deviation: {nested_scores.std():.4f}")
lo, hi = test_auc_nested - 1.96 * nested_scores.std(), test_auc_nested + 1.96 * nested_scores.std()
print(f"95% confidence interval: [{lo:.4f}, {hi:.4f}]")
```

```
Mean AUC across 10 outer folds: 0.7511
Standard deviation: 0.0178
95% confidence interval: [0.7163, 0.7860]
```

Here the nested-CV mean lands almost exactly on the plain outer-loop mean from before, because $K = 50$ really is the best setting for this data, so tuning inside each fold keeps picking it. On a problem where the best hyperparameter is genuinely unknown (which is often the case with more complex models), the two would diverge, and the nested estimate would be the trustworthy one.

6.1.6 Leave-one-out cross-validation

Leave-one-out cross-validation (LOOCV) is K-fold CV taken to its extreme: K equals the number of rows, so each model trains on all data but one row and is tested on that single left-out row (Figure 6.5). It uses almost all the data for training every time, but it needs n model fits, which is impractical for anything but tiny datasets, and the per-fold test score (right or wrong on one row) is extremely noisy.

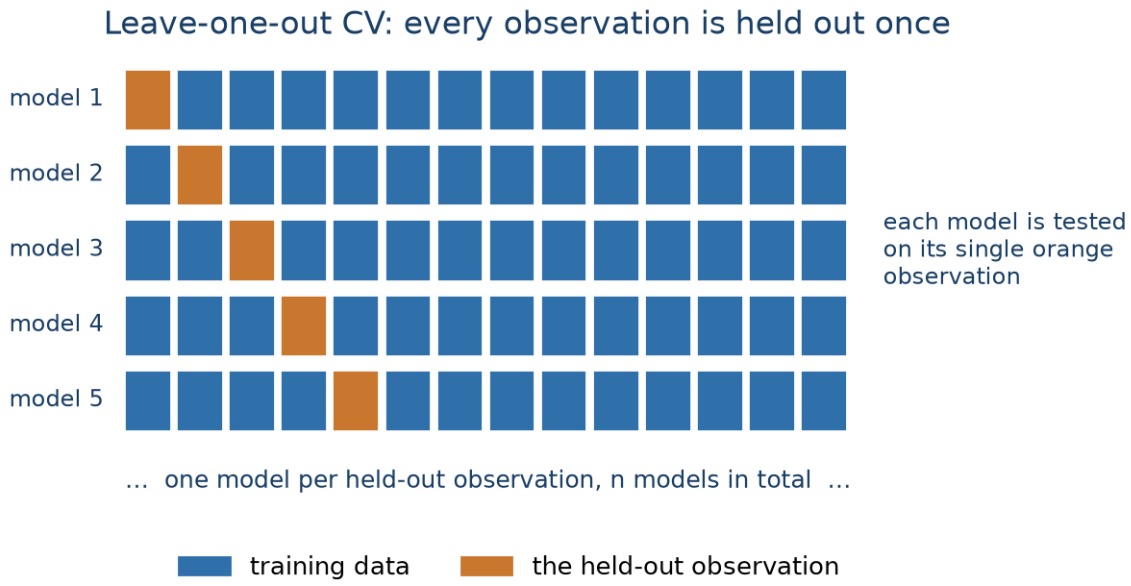


Figure 6.5: Leave-one-out cross-validation: each model trains on every row but one and is tested on that single held-out observation, for n models in total.

We demonstrate it on a 500-row stratified subsample to keep the runtime reasonable:

```
from sklearn.model_selection import LeaveOneOut

X_small, _, y_small, _ = train_test_split(
```

```
    X, y, train_size=500, random_state=RANDOM_STATE, stratify=y
)

loo = LeaveOneOut()
loo_preds, loo_true = [], []
for train_idx, test_idx in loo.split(X_small):
    pipeline_cv.fit(X_small.iloc[train_idx], y_small.iloc[train_idx])
    loo_preds.append(pipeline_cv.predict_proba(X_small.iloc[test_idx])[0, 1])
    loo_true.append(y_small.iloc[test_idx].iloc[0])

loocv_auc = roc_auc_score(loo_true, loo_preds)
print(f"LOOCV AUC ({len(y_small)} fits): {loocv_auc:.4f}")
```

```
LOOCV AUC (500 fits): 0.6287
```

The lower AUC here isn't LOOCV being "wrong": it's the much smaller training set (499 rows, only a handful of churners per fit) doing worse than the models trained on tens of thousands of rows. LOOCV is really only worth it for final evaluation on very small datasets.

6.1.7 The bootstrap

The **bootstrap** builds a training set by sampling n rows **with replacement** from the data, so some rows appear multiple times and, on average, about 36.8% never get drawn at all. Those unused rows form the **out-of-bag (OOB)** test set. Repeating this many times gives a distribution of scores, much like CV, but with training sets that stay the original size. Note that the bootstrap was mainly designed as a statistical resampling technique, not a true model evaluation method, so it is less common in practice than CV. `sklearn` has no built-in bootstrap splitter, so we use `resample`:

```
from sklearn.utils import resample

boot_scores = []
for i in range(50):
    X_boot, y_boot = resample(X, y, n_samples=len(X), random_state=RANDOM_STATE + i, stratify=y)
    oob_idx = X.index.difference(X_boot.index)
    pipeline_cv.fit(X_boot, y_boot)
    oob_probs = pipeline_cv.predict_proba(X.loc[oob_idx])[:, 1]
    boot_scores.append(roc_auc_score(y.loc[oob_idx], oob_probs))

boot_scores = np.array(boot_scores)
print(f"Bootstrap AUC: {boot_scores.mean():.4f} ± {boot_scores.std():.4f}  (50 iterations)")
```

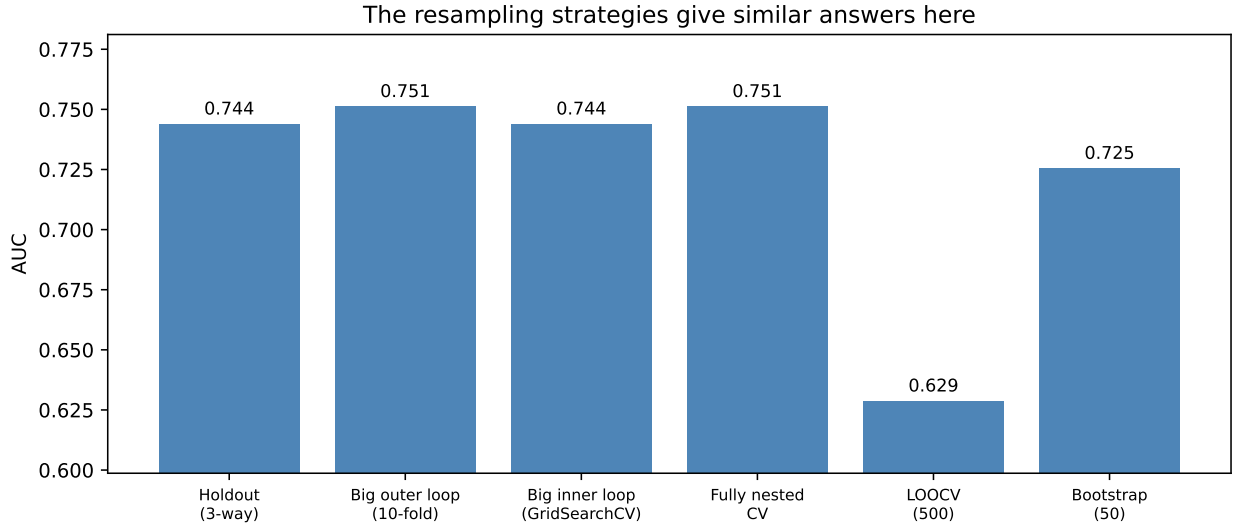
```
Bootstrap AUC: 0.7254 ± 0.0114  (50 iterations)
```

6.1.8 Comparing the resampling strategies

```
methods = ["Holdout\n(3-way)", "Big outer loop\n(10-fold)", "Big inner loop\n(GridSearchCV)", "Fully nes
scores = [test_auc, test_auc_cv, test_auc_grid, test_auc_nested, loocv_auc, boot_scores.mean()]

plt.figure(figsize=(9, 4))
bars = plt.bar(methods, scores, color="#2f6faa", alpha=0.85)
for bar, s in zip(bars, scores):
    plt.text(bar.get_x() + bar.get_width() / 2, bar.get_height() + 0.004, f"{s:.3f}",
             ha="center", fontsize=9)
plt.ylabel("AUC")
plt.title("The resampling strategies give similar answers here")
```

```
plt.ylim(min(scores) - 0.03, max(scores) + 0.03)
plt.xticks(fontsize=8)
plt.tight_layout()
plt.show()
```



For this dataset every strategy lands around AUC 0.74 to 0.75, which is itself reassuring: it means the model is stable and not sensitive to how the data is split. In general:

- **Fully nested CV** is the most robust approach: an unbiased estimate with a confidence interval.
- **The big outer loop** (outer CV only) is a good compromise between speed and reliability, and is the usual choice in business research and applied settings.
- **The big inner loop** (`GridSearchCV`, inner CV only) gives reliable *hyperparameter* choices at a fraction of the cost, and is the usual industry choice. Works well when the test set is large enough to be trustworthy.
- **Single holdout splits** are fastest but least reliable: fine for quick exploration, risky for a final number.
- **LOOCV** is computationally impractical except on very small data.
- **The bootstrap** is an alternative to CV that keeps training sets at full size. However, it is less common in practice and has no built-in support in `sklearn`.

6.2 Data leakage

Almost every broken experimental setup comes down to the same root cause: **data leakage**. This is information from outside the training set (especially information about the target, about the future or the test data) sneaking into the model. The model then "learns" things it could not possibly know at prediction time, its measured performance is inflated, and its performances collapses on genuinely new data (Kaufman et al. 2012).

Leakage is fundamentally about **legitimacy**: which inputs would actually be available, and accurate, at the moment you'd want a prediction. Three rules capture most of it:

- **The target is never an input.** Neither the target itself nor a proxy that's really a restatement of it.
- **No time machine.** Every feature must come from a time period strictly *before* the target is determined. A feature that's only recorded *after* the outcome is leakage even if it looks innocuous.
- **No peeking at the test set.** Every preprocessing step (scaling, imputation, encoding, feature selection) must be fit on training data only and applied unchanged to validation and test.

Some classic examples:

- Predicting whether a prospect will *open a bank account*, with **account number** as a feature. You only get an account number once the account exists.
- Predicting *churn*, with an **interviewer ID** as a feature. Interviewers are assigned only *after* a customer has already been flagged as a likely leaver.
- Identifying prospects from **website keywords** that were actually scraped after those people became customers, so the keywords describe products they've already bought.
- Building a **WoE encoding** for payment type in a *churn* model (Chapter 5) using each payment type's churn proportion computed across the entire dataset, then splitting into train and test. Every test row's encoded value already reflects its own category's churn outcome, information the model isn't supposed to have before it makes a prediction.

> **i** A famous mistake: feature selection outside the CV loop
>
> You have 5,000 rows and 500 variables. To fight the curse of dimensionality, you run variable selection **on the full dataset** to get down to 100 variables, then cross-validate a classifier on those 100.
> This is a classical example of data leakage. The variable selection step already looked or peeked at every row's label, including the rows that later act as CV test folds, and chose features *because* they correlated with those labels. Cross-validating afterwards can't undo that; on the contrary, each fold is now contaminated with information from the test set and the folds were never truly unseen. The key lesson here is that feature selection is part of *training* and must happen **inside** each CV fold, on the training portion only.

How do you detect it? Performance that looks too good-to-be-true is also often not true. For example, a churn model with AUC 0.99 should raise suspicion before celebration. A large drop in performance once the model meets a genuinely new batch of data, for example next month's customers, should also raise suspicion. That drop is leakage showing itself: the model looked great on data it had secretly already seen, and it collapses once it can no longer cheat.

How do you prevent it?

- Decide on the time separation up front: know exactly which point in time separates "known" from "future" before touching the data and stick to this in your preprocessing and splitting.
- Run *every* preprocessing step (scaling, imputation, encoding, feature selection) *inside* the cross-validation loop, never on the full dataset beforehand. In `sklearn`, a `Pipeline` passed to `cross_validate` or `GridSearchCV` does exactly this automatically.
- Keep a final holdout test set that is touched exactly once, at the very end, and never used for any other decisions along the way.

6.3 The bias–variance trade-off

Choosing a model's **complexity** (e.g., how many features, what polynomial degree, how many parameters) is a balancing act between two kinds of error (Figure 6.6):

- **Bias** is error from a model too simple to capture the real relationship. High bias means **underfitting**: consistently wrong, in the same way, on training *and* test data. Your model is too simple.
- **Variance** is how much the fitted model would change if you retrained it on a different sample. High variance means **overfitting**: the model has learned the training data's noise, so it does great on that data and poorly on anything new. This leads to stellar training performance but disappointing test performance. Your model is overly complex.

Expected test error is roughly bias^2 + variance + an **irreducible** noise floor you can never beat. Lowering one of the first two tends to raise the other, so there's an optimal complexity where the total error is smallest. Increasing complexity beyond this (more variables, complex transformations, trees) makes things *worse*, not better (James et al. 2013).

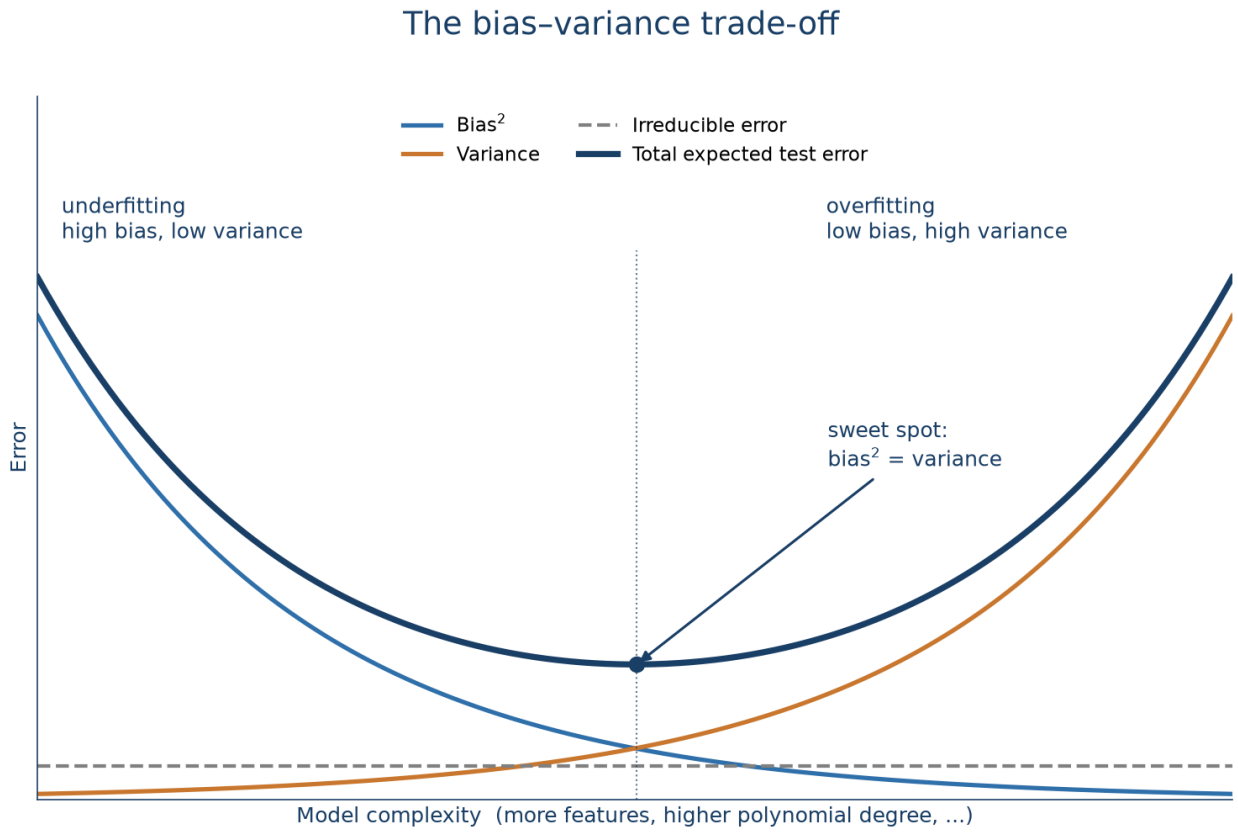


Figure 6.6: The bias–variance trade-off: as model complexity increases, bias² falls and variance rises. Total expected test error is their sum plus an irreducible noise floor, and its minimum falls exactly where the bias² and variance curves cross.

The synthetic example below shows things clearer. We generate data from a known quadratic relationship plus noise, then fit three models of increasing complexity:

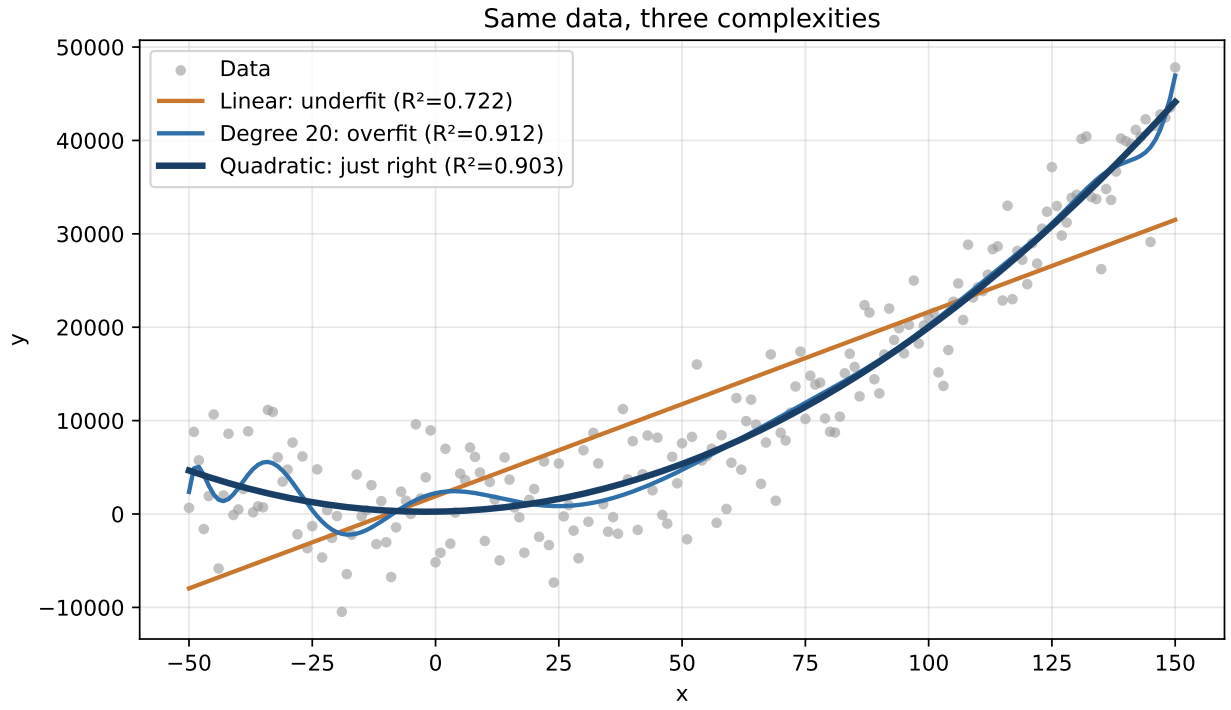
```
from sklearn.linear_model import LinearRegression
from sklearn.preprocessing import PolynomialFeatures
from sklearn.metrics import r2_score

np.random.seed(RANDOM_STATE)
x = np.arange(-50, 151)
y_true = 3 + 2 * x**2
y_noisy = y_true + np.random.normal(0, 4000, len(x))

def fit_poly(degree):
    # standardize x first: raw powers up to x**20 are so large that the fit becomes numerically unstable
    x_std = (x - x.mean()) / x.std()
    Xp = PolynomialFeatures(degree=degree, include_bias=False).fit_transform(x_std.reshape(-1, 1))
    model = LinearRegression().fit(Xp, y_noisy)
    return model.predict(Xp)

pred_linear = fit_poly(1)
pred_quad = fit_poly(2)
pred_poly20 = fit_poly(20)

fig, ax = plt.subplots(figsize=(9, 5))
ax.scatter(x, y_noisy, s=15, color="#999999", alpha=0.6, label="Data")
ax.plot(x, pred_linear, color="#c9772e", lw=2, label=f"Linear: underfit (R²={r2_score(y_noisy, pred_lin
ax.plot(x, pred_poly20, color="#2f6faa", lw=2, label=f"Degree 20: overfit (R²={r2_score(y_noisy, pred_p
ax.plot(x, pred_quad, color="#1a3f66", lw=3, label=f"Quadratic: just right (R²={r2_score(y_noisy, pred_
ax.set_xlabel("x")
ax.set_ylabel("y")
ax.set_title("Same data, three complexities")
ax.legend()
ax.grid(alpha=0.3)
plt.show()
```



The linear model can't bend to the curve at all (underfitting, high bias). The degree-20 polynomial wiggles to chase individual noisy points (overfitting, high variance). Note that its R^2 is *higher* than the quadratic's on this data, which is exactly why R^2 on the training data you fit is a poor guide to a model's real quality. The quadratic matches the true relationship and will generalise best to new data, even if its R^2 is lower on this particular training sample.

6.4 Performance metrics

The demonstration above was easy because we *knew* the true relationship. In practice you don't, which is why you need evaluation metrics, computed on a proper held-out set, to compare models and pick one.

6.4.1 Regression metrics

Remember that a regression model doesn't output a class label, it outputs a **point estimate**: a single predicted number $\hat{y}$ for each unknown value y. Evaluating the model means comparing these predictions to the actual values on a held-out set: the closer $\hat{y}$ sits to y across all test rows, the better the model. There is no single "right" way to summarise that closeness, different metrics have their own strengths and weaknesses (e.g., how to weigh large errors, outliers, and scale differently) so it's common to look at several at once.

Four metrics cover most needs:

- **R^2**: the proportion of variance in y explained by $\hat{y}$ (higher is better, max 1). It is primarily a goodness-of-fit measure. On a test set it can even go **negative** if the model does worse than just predicting the mean. Note that R^2 is not the same as the squared correlation between $\hat{y}$ and y: the two coincide on the data a linear model was fitted to, but on held-out data a model can have a high correlation and still a poor (even negative) R^2, for example when its predictions are systematically shifted or scaled.
- **RMSE**: the root mean squared difference between $\hat{y}$ and y, in the same units as the target. It squares the errors, so large errors are penalised heavily (sensitive to outliers).
- **MAE**: the mean absolute difference between $\hat{y}$ and y, also in target units, but penalising all errors proportionally.

- **MAPE**: mean absolute *percentage* error. Scale-free and easy to communicate, but undefined when actual values are near zero. So be careful with this metric if you have small or zero values in your dependent variable.

Only R^2 is maximised; the rest are minimised.

To make this concrete, let's consider a synthetic time series example: `x` is a time index, `y` is profit in that period, and we want to predict future profit. We split it **chronologically** rather than randomly (for a time series, the validation and test set must be the *later* periods): 60% train, 20% validation, 20% test. Then we compare a linear, a quadratic, and a high-order polynomial fit.

```
reg = pd.DataFrame({"x": x, "x2": x**2, "x4": x**4, "x5": x**5, "y": y_noisy})
train, val, test = reg.iloc[:121], reg.iloc[121:161], reg.iloc[161:]

linear_reg = LinearRegression().fit(train[["x"]], train["y"])
quad_reg = LinearRegression().fit(train[["x", "x2"]], train["y"])
poly_reg = LinearRegression().fit(train[["x4", "x5"]], train["y"])

val_pred = {
    "Linear": linear_reg.predict(val[["x"]]),
    "Quadratic": quad_reg.predict(val[["x", "x2"]]),
    "High-order poly": poly_reg.predict(val[["x4", "x5"]]),
}
```

Scoring the three candidate models on the validation set:

```
from sklearn.metrics import mean_squared_error, mean_absolute_error

def mape(y_true, y_pred):
    return np.mean(np.abs((y_true - y_pred) / y_true))

rows = []
for name, pred in val_pred.items():
    rows.append({
        "Model": name,
        "R²": r2_score(val["y"], pred),
        "RMSE": np.sqrt(mean_squared_error(val["y"], pred)),
        "MAE": mean_absolute_error(val["y"], pred),
        "MAPE": mape(val["y"], pred),
    })
pd.DataFrame(rows).round(3)
```

	Model	R^2	RMSE	MAE	MAPE
0	Linear	-4.864	12320.046	11360.358	0.635
1	Quadratic	0.368	4045.516	3354.931	0.193
2	High-order poly	-3.158	10374.303	8295.971	0.492

The quadratic model wins on every metric. The linear model underfits: it systematically underestimates the high values. The high-order polynomial overfits, producing poor predictions off the training range. Refitting the quadratic on train + validation and scoring once on the test set gives the final number:

```
final_model = LinearRegression().fit(
    pd.concat([train, val])[["x", "x2"]], pd.concat([train, val])["y"]
)
test_pred = final_model.predict(test[["x", "x2"]])
```

```
print("Final quadratic model, test set:")
print(f"  R²   = {r2_score(test['y'], test_pred):.3f}")
print(f"  RMSE = {np.sqrt(mean_squared_error(test['y'], test_pred)):,.0f}")
print(f"  MAE  = {mean_absolute_error(test['y'], test_pred):,.0f}")
print(f"  MAPE = {mape(test['y'], test_pred):.3f}")
```

```
Final quadratic model, test set:
  R²   = 0.683
  RMSE = 3,666
  MAE  = 2,493
  MAPE = 0.080
```

6.4.2 Classification metrics

Classification needs different evaluation metrics: you can't average an "error" when every prediction is either exactly right (1–1 or 0–0) or exactly wrong.

We switch back to the churn basetable, take a fresh **50/50 split** (so the test set is large enough for stable metrics), and fit a **random forest** model. Random forests build hundreds of decision trees and average them; they're one of the best off-the-shelf classifiers even without tuning. You don't have to know the details of this model, but just see it as a good performing default model. Here the random forest is just a means to generate predictions to evaluate.

```
from sklearn.ensemble import RandomForestClassifier

X_clf = basetable.drop("churn", axis=1)
y_clf = basetable["churn"]
X_tr, X_te, y_tr, y_te = train_test_split(
    X_clf, y_clf, test_size=0.5, random_state=RANDOM_STATE, stratify=y_clf
)

rf = RandomForestClassifier(n_estimators=100, random_state=RANDOM_STATE)
rf.fit(X_tr, y_tr)

y_proba = rf.predict_proba(X_te)[:, 1]
```

6.4.2.1 Scoring vs. deterministic classifiers

Before looking at any metric, it helps to be clear about what a classifier actually returns, because that's what the rest of this section relies on (Hernández-Orallo et al. 2012). There are two kinds:

- A **deterministic classifier** returns a class label directly, 0 or 1. There's nothing else to decide.
- A **scoring classifier** returns a continuous score in [0, 1] instead, or a score that shows confidence that the event (e.g., churn) will happen. That score only becomes a label once you pick a **cut-off** (or threshold): above it, predict 1; below it, predict 0.

Most models used in practice, including random forests, logistic regression, and gradient boosting, are scoring classifiers. Random forest above is no exception: `rf.predict_proba` gives its raw scores, `y_proba`, and `rf.predict` is simply what you get after applying `sklearn`'s default cut-off of 0.5 to those same scores (a probability of exactly 0.5 is the one edge case that rounds down to 0).

> **i** A note on the word "probability"
>
> This book, like most of `sklearn`'s documentation, calls a scoring classifier's output a "probability." In practice, the raw score a model outputs is often not a genuine probability: many popular classifiers are known to be poorly calibrated out of the box (Niculescu-Mizil and Caruana 2005). A classifier is **well-calibrated** if, for example, 80% of the instances to which it assigns a 80% probability are actually members of that class. So a score of 0.8 across a group of customers should mean that roughly 80% of them actually churn, which for most raw model outputs isn't true. Turning a raw score into something that behaves like a real probability takes an extra step, **calibration**, or more rigorously, **conformal prediction** (Manokhin 2023), which is a topic on its own and beyond the scope of this book. But it is worth keeping in mind that the "probabilities" you see here are not guaranteed to be probabilities in the strict sense and a purist would call them "scores" instead.

Both kinds of classifier can be evaluated with the same starting point: the **confusion matrix**, a cross-tabulation of actual versus predicted labels. For a deterministic classifier that's the whole story; for a scoring classifier, the matrix (and therefore everything built on it) still depends on which cut-off you chose.

That split runs through everything that follows. **Threshold-dependent metrics**, like accuracy, precision, recall, and the rest, are computed from the confusion matrix at one specific cut-off. **Threshold-independent metrics**, ROC and AUC, precision-recall curves, and lift, look instead at how a scoring classifier performs across *every possible* cut-off at once. We cover the threshold-dependent metrics first, then move on to the threshold-independent ones.

6.4.2.2 The confusion matrix and threshold-dependent metrics

The confusion matrix at a chosen threshold has four cells: true positives (tp) and true negatives (tn), **false positives** (fp, predicted churn, when the customer did not: a type I error) and **false negatives** (fn, missed a churner: a type II error).

Almost every threshold-dependent metric is just a different ratio of these four counts:

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN} \qquad \text{Precision} = \frac{TP}{TP + FP}$$

$$\text{Recall (Sensitivity, TPR)} = \frac{TP}{TP + FN} \qquad \text{Specificity (TNR)} = \frac{TN}{TN + FP}$$

$$\text{FNR (miss rate)} = \frac{FN}{TP + FN} = 1 - \text{Recall} \qquad F_1 = 2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}$$

Recall and specificity each look at one actual class: of actual churners, how many did we catch (recall), and of actual non-churners, how many did we correctly clear (specificity, also called the **true negative rate**, TNR). Their complements are exactly the two error rates marked on the confusion matrix above: 1 minus recall is the **FNR**, the type II error rate, and 1 minus specificity is the **false positive rate (FPR)**, the type I error rate. Precision instead looks at the predictions themselves: of everyone flagged as a churner, how many actually were one.

Two more metrics are built specifically for imbalanced data, where the positive class is rare:

$$\text{Balanced accuracy} = \frac{\text{Recall} + \text{Specificity}}{2} \qquad F_\beta = (1 + \beta^2) \cdot \frac{\text{Precision} \cdot \text{Recall}}{(\beta^2 \cdot \text{Precision}) + \text{Recall}}$$

Balanced accuracy averages the per-class accuracies instead of counting rows: a model that just predicts the majority class gets 50% here, not the inflated number plain accuracy would give it, because a large majority class can no longer hide a near-zero recall on the minority class. F_β generalises F_1 (the special case

$\beta = 1$) to let you weight the two error types by how much they actually cost: $\beta > 1$ weights recall more heavily, appropriate when missing a churner is worse than a false alarm; $\beta < 1$ weights precision more, when a false alarm is the more expensive mistake.

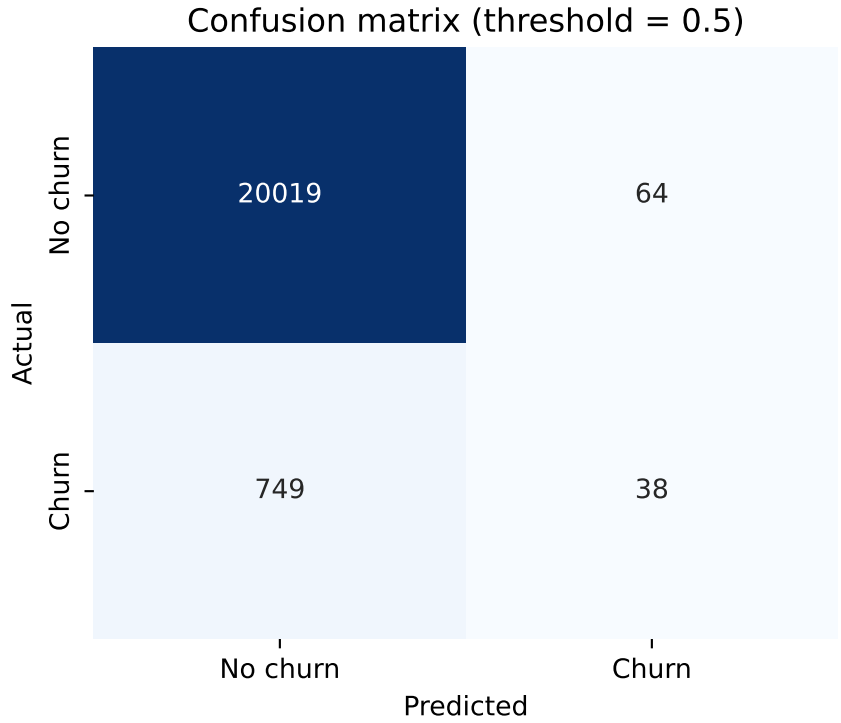
Let's see these metrics in action on the random forest model above, at the default 0.5 threshold:

```
from sklearn.metrics import (confusion_matrix, accuracy_score, precision_score,
                             recall_score, f1_score, balanced_accuracy_score, fbeta_score)

y_pred = rf.predict(X_te)
cm = confusion_matrix(y_te, y_pred)

fig, ax = plt.subplots(figsize=(5, 4))
sns.heatmap(cm, annot=True, fmt="d", cmap="Blues", cbar=False,
            xticklabels=["No churn", "Churn"], yticklabels=["No churn", "Churn"], ax=ax)
ax.set_xlabel("Predicted")
ax.set_ylabel("Actual")
ax.set_title("Confusion matrix (threshold = 0.5)")
plt.show()

tn, fp, fn, tp = cm.ravel()
print(f"Accuracy:          {accuracy_score(y_te, y_pred):.4f}")
print(f"Balanced accuracy: {balanced_accuracy_score(y_te, y_pred):.4f}")
print(f"Precision:         {precision_score(y_te, y_pred):.4f}   (of predicted churners, how many churne
print(f"Recall:            {recall_score(y_te, y_pred):.4f}   (of actual churners, how many we caught)")
print(f"Specificity:       {tn / (tn + fp):.4f}   (of actual non-churners, how many we cleared)")
print(f"FNR:               {fn / (fn + tp):.4f}   (of actual churners, how many we missed)")
print(f"F1-score:          {f1_score(y_te, y_pred):.4f}   (harmonic mean of precision and recall)")
print(f"F2-score:          {fbeta_score(y_te, y_pred, beta=2):.4f}   (weights recall twice as much as pr
```




```
Accuracy:          0.9610
```

```
Balanced accuracy: 0.5225
Precision:         0.3725   (of predicted churners, how many churned)
Recall:            0.0483   (of actual churners, how many we caught)
Specificity:       0.9968   (of actual non-churners, how many we cleared)
FNR:               0.9517   (of actual churners, how many we missed)
F1-score:          0.0855   (harmonic mean of precision and recall)
F2-score:          0.0585   (weights recall twice as much as precision)
```

The numbers show the classic failure mode on imbalanced data. Accuracy looks great at 96%, but that is almost entirely the 96% of customers who don't churn: specificity is 99.7%, the model clears almost every non-churner. Balanced accuracy tells the true story: 52.3%, barely above the 50% coin flip, because it no longer lets the huge non-churn class hide the model's near-zero recall. Recall itself confirms it: only 4.8% of actual churners are caught, an FNR above 95%. At a 0.5 threshold the model almost never predicts churn, because churn probabilities rarely climb above 0.5 when only ~4% of customers churn in the first place. Weighting recall more heavily makes this even more visible: F2 drops to 0.058, below F1's already weak 0.086, because F2 punishes the missed churners harder than F1 does. Every one of these numbers depends on the threshold, which is exactly the weakness the next section addresses.

Let's drop the threshold to 0.05 and the picture changes completely:

```
for t in [0.5, 0.05]:
    yp = (y_proba >= t).astype(int)
    tn, fp, fn, tp = confusion_matrix(y_te, yp).ravel()
    print(f"threshold {t:>4}:  recall {recall_score(y_te, yp):.3f}   "
          f"precision {precision_score(y_te, yp):.3f}   "
          f"specificity {tn / (tn + fp):.3f}   FPR {fp / (fp + tn):.3f}   "
          f"FNR {fn / (fn + tp):.3f}   accuracy {accuracy_score(y_te, yp):.3f}")
```

```
threshold  0.5:  recall 0.053   precision 0.382   specificity 0.997   FPR 0.003   FNR 0.947   accuracy
threshold 0.05:  recall 0.615   precision 0.099   specificity 0.781   FPR 0.219   FNR 0.385   accuracy
```

Recall jumps from 5% to 61%, and the FNR falls from 95% to 38%: three times as many churners caught. That comes at a real cost. The false-alarm rate is read directly off specificity: it drops from 99.7% to 78%, which is the same as saying the **false positive rate** (1 minus specificity) rises from 0.3% to 22%, so far more non-churners now get incorrectly flagged. Precision falls too, from 38% to 10%, but for a related, distinct reason: it's not a rate over non-churners, it's the fraction of *flagged* customers who actually churn, and that fraction drops because the flagged group now contains many more of those false alarms relative to the true churners it also catches. Which trade-off is right depends entirely on the business context: a retention campaign that's cheap per contact can afford more false positives to catch more real churners; an expensive one-on-one intervention cannot.

6.4.2.3 Choosing a threshold deliberately

A fixed cut-off like `sklearn`'s default of 0.5 quietly assumes two things that often aren't true: that the classifier's scores behave like well-calibrated probabilities (see the callout above), and that 0.5 happens to match what the business actually needs. Neither assumption is often the case. Comparing two classifiers at the same fixed cut-off can be misleading if their score distributions differ: one model's recall can look like 0 and another's clearly positive purely because their scores are scaled differently, not because one is genuinely worse. And even for a single well-behaved classifier, there is no reason 0.5 should be the cut-off a business actually wants to act on.

Three ways to pick a better one:

1. **Calibrate the scores first**, so a fixed cut-off means the same thing for every classifier. Out of scope for this book (see the callout above).
2. **Work with percentile ranks instead of raw scores.** Pick a cut-off so that a *fixed proportion* of the data is flagged, for example the top 10%, rather than trusting the raw score scale. A common heuristic

is to set that proportion equal to the positive rate seen in training or validation.

3. **Post-tune the threshold after training**, searching for the threshold that maximises a chosen performance metric via cross-validation, so the choice isn't overfit to the training data.

Solution 2 is simple enough to do by hand:

```
pos_rate = y_tr.mean()
cutoff_score = np.quantile(y_proba, 1 - pos_rate)
y_pred_pct = (y_proba >= cutoff_score).astype(int)

print(f"Training positive rate: {pos_rate:.3f}")
print(f"Score cut-off at that percentile: {cutoff_score:.3f}")
print(f"  recall    {recall_score(y_te, y_pred_pct):.3f}")
print(f"  precision {precision_score(y_te, y_pred_pct):.3f}")
print(f"  F1        {f1_score(y_te, y_pred_pct):.3f}")
```

```
Training positive rate: 0.038
Score cut-off at that percentile: 0.230
  recall    0.241
  precision 0.233
  F1        0.237
```

Solution 3 does the same job more rigorously. `TunedThresholdClassifierCV` searches for the threshold that maximises a chosen metric, using internal cross-validation. For churn, F1 (balancing precision and recall) is a reasonable target. Note the use of `StratifiedKFold`: with imbalanced data you always want folds that preserve the class ratio.

```
from sklearn.model_selection import TunedThresholdClassifierCV, StratifiedKFold

tuned = TunedThresholdClassifierCV(
    estimator=RandomForestClassifier(n_estimators=100, random_state=RANDOM_STATE),
    scoring="f1",
    cv=StratifiedKFold(n_splits=5, shuffle=True, random_state=RANDOM_STATE),
    n_jobs=-1,
)
tuned.fit(X_tr, y_tr)
y_pred_tuned = tuned.predict(X_te)

print(f"F1-optimal threshold: {tuned.best_threshold_:.3f}")
print(f"  recall    {recall_score(y_te, y_pred_tuned):.3f}")
print(f"  precision {precision_score(y_te, y_pred_tuned):.3f}")
print(f"  F1        {f1_score(y_te, y_pred_tuned):.3f}  (was {f1_score(y_te, y_pred):.3f} at 0.5)")
```

```
F1-optimal threshold: 0.207
  recall    0.267
  precision 0.221
  F1        0.242  (was 0.085 at 0.5)
```

The two solutions land in a similar place here: F1 around 0.24 either way, a real improvement over the 0.085 the default 0.5 cut-off gave, but still modest in absolute terms. Threshold tuning merely repositions the operating point; it doesn't make a weak classifier strong. Genuinely fixing imbalanced-class performance needs resampling, cost-sensitive learning, a stronger model or a combination of these, which are all beyond the scope of this book.

6.4.2.4 Threshold-independent metrics: ROC and AUC

Often you don't know the operating conditions at evaluation time. For a retention campaign, the marketing budget, and so the fraction of customers you can target, may not be decided yet. **Threshold-independent** metrics sidestep this by aggregating over *all* thresholds at once.

The **ROC curve** plots the true positive rate (recall) against the false positive rate (1 − specificity) as the threshold sweeps from 1 down to 0 (Fawcett 2006). Two corners anchor it: a perfect model reaches the top-left corner (100% TPR, 0% FPR), a model that gets everything backwards sits in the bottom-right (0% TPR, 100% FPR), and the diagonal between them is what pure random guessing looks like. The closer a curve hugs the top-left corner, the better the trade-off it offers across every threshold at once, which also makes ROC curves useful for comparing two classifiers directly: whichever curve dominates (sits above the other almost everywhere) is the better model, threshold for threshold.

The **area under the curve (AUC)** collapses that whole curve into one number:

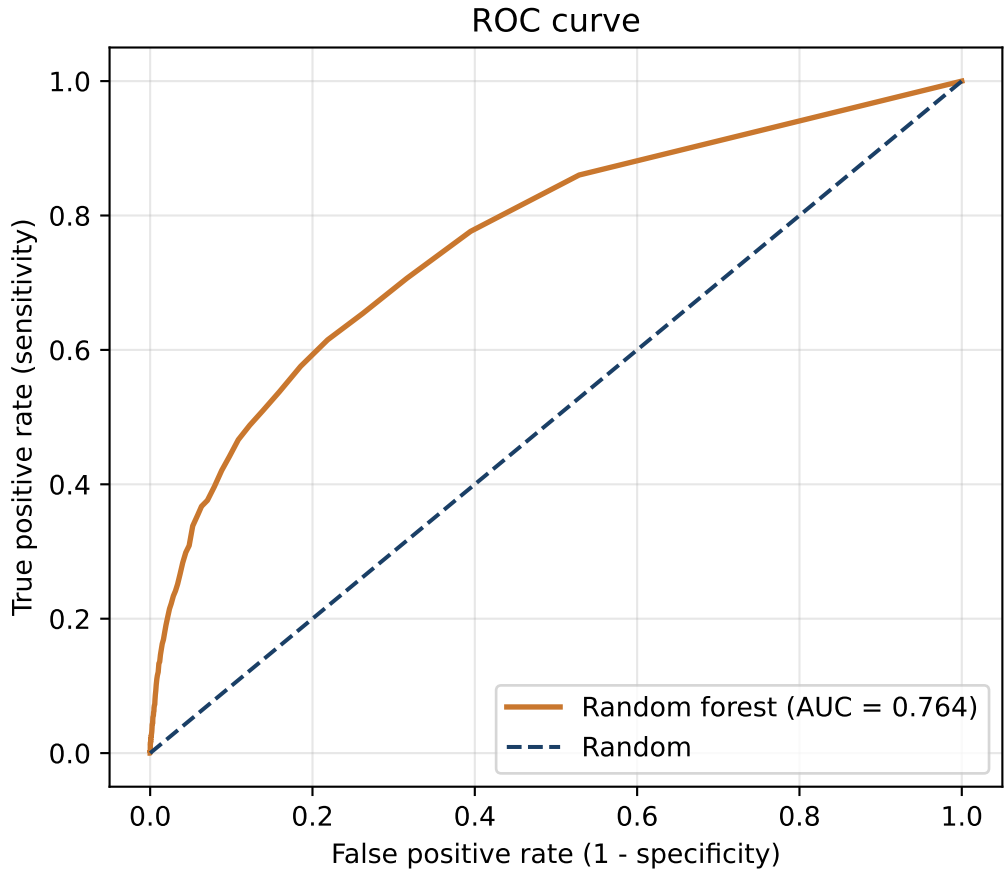
$$\text{AUC} = \int_0^1 TPR(FPR)\, d(FPR)$$

The integral has a cleaner, ranking-based interpretation in practice: the probability that a randomly chosen positive instance (e.g., churn) is scored higher than a randomly chosen negative instance (e.g., non-churn). A random classifier scores 0.5; a perfect one scores 1.

```
from sklearn.metrics import roc_curve, roc_auc_score

fpr, tpr, _ = roc_curve(y_te, y_proba)
auc = roc_auc_score(y_te, y_proba)

plt.figure(figsize=(6, 5))
plt.plot(fpr, tpr, color="#c9772e", lw=2, label=f"Random forest (AUC = {auc:.3f})")
plt.plot([0, 1], [0, 1], color="#1a3f66", lw=1.5, ls="--", label="Random")
plt.xlabel("False positive rate (1 - specificity)")
plt.ylabel("True positive rate (sensitivity)")
plt.title("ROC curve")
plt.legend(loc="lower right")
plt.grid(alpha=0.3)
plt.show()
```



The random forest model reaches an AUC of 0.764: pick a random churner and a random non-churner from the test set, and about 76% of the time the churner gets the higher score, comfortably better than the 0.5 a random classifier would get, but far from the 1.0 of a perfect one. Reading a single point off the curve makes the trade-off concrete: to catch 80% of churners (TPR = 0.8) this model needs a false positive rate of roughly 53%, more than half of all non-churners flagged along the way. That is exactly what AUC hides by summarising the whole curve: it judges the overall ranking ability, but any one operating point on that curve can still be a poor deal in business terms. AUC's single-number convenience has a deeper, well-documented catch too: it implicitly weighs misclassification costs differently for different classifiers, which can make it favour the wrong model when two classifiers are compared, and can lead to suboptimal model selection in terms of actual profit (Verbraken et al. 2013). That critique is beyond the scope of this book, but worth knowing the number isn't as theoretically clean as it looks.

6.4.2.5 Precision–recall curves and average precision

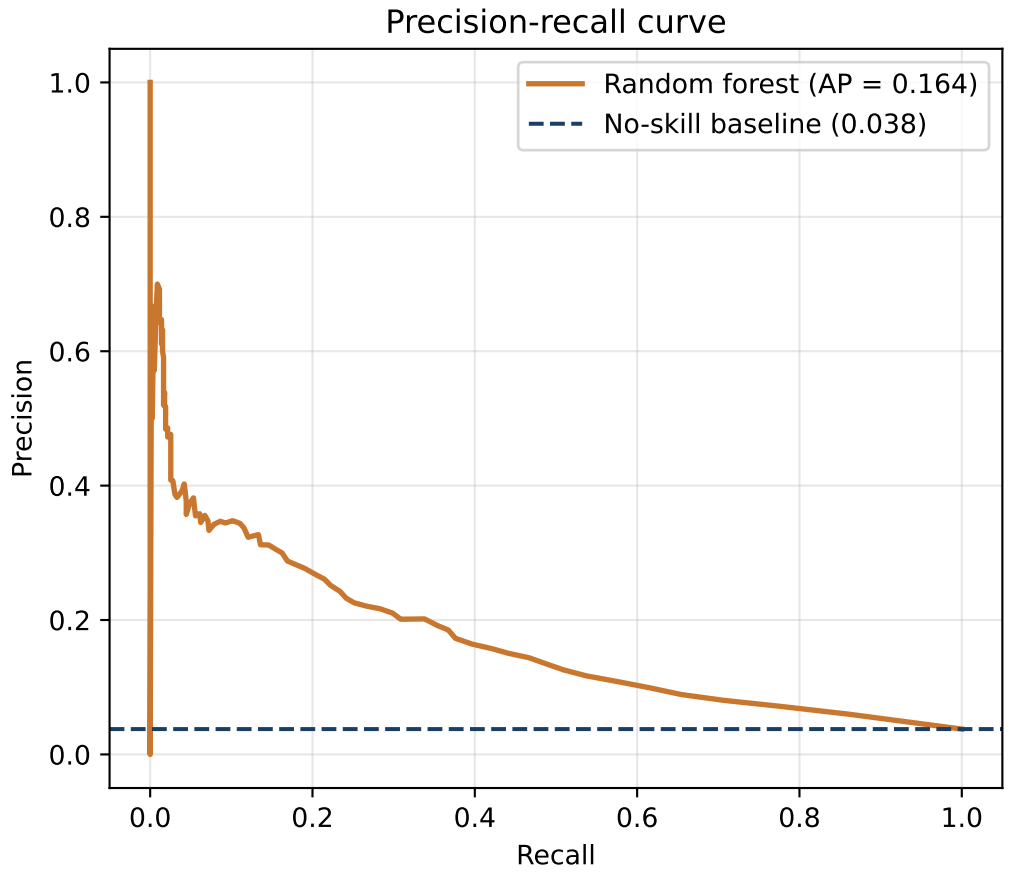
On strongly imbalanced data, ROC/AUC can be misleading too, even producing overly optimistic results. The false positive rate has a huge true-negative denominator, so it stays low even when the model is producing many false alarms in absolute terms (Van Belle et al. 2022). The **precision–recall (PR) curve** ignores true negatives entirely and focuses on the minority class. Its summary number, **average precision (AP)** or **area under the PR curve (AUC-PR)**, is the PR curve's alternative to AUC. Note that a random classifier's AP equals the positive rate in the test set (its "no-skill" baseline).

```
from sklearn.metrics import precision_recall_curve, average_precision_score

precision, recall, _ = precision_recall_curve(y_te, y_proba)
ap = average_precision_score(y_te, y_proba)
no_skill = y_te.mean()
```

```
plt.figure(figsize=(6, 5))
plt.plot(recall, precision, color="#c9772e", lw=2, label=f"Random forest (AP = {ap:.3f})")
plt.axhline(no_skill, color="#1a3f66", lw=1.5, ls="--", label=f"No-skill baseline ({no_skill:.3f})")
plt.xlabel("Recall")
plt.ylabel("Precision")
plt.title("Precision-recall curve")
plt.legend(loc="upper right")
plt.grid(alpha=0.3)
plt.show()

print(f"AUC = {auc:.3f}  vs  AP = {ap:.3f}  (no-skill AP = {no_skill:.3f})")
```




```
AUC = 0.764  vs  AP = 0.164  (no-skill AP = 0.038)
```

The gap tells the story: an AUC of ~0.76 sounds healthy, but the AP of ~0.16, against a no-skill baseline of ~0.04, is a more realistic read of how the model does at actually pinpointing churners.

6.4.2.6 ROC vs. PR: a worked comparison

A workable decision rule for imbalanced problems is to look at **both** curves. If the ROC curves don't cross, pick the dominant one on AUC. If they cross, or the AUC gap is small (under ~0.02), fall back to **AP** and the **PR curves** instead, since that ambiguous case is exactly where ROC/AUC's optimism on imbalanced data can hide a real difference between models.

Here it is in practice, comparing random forest against logistic regression:

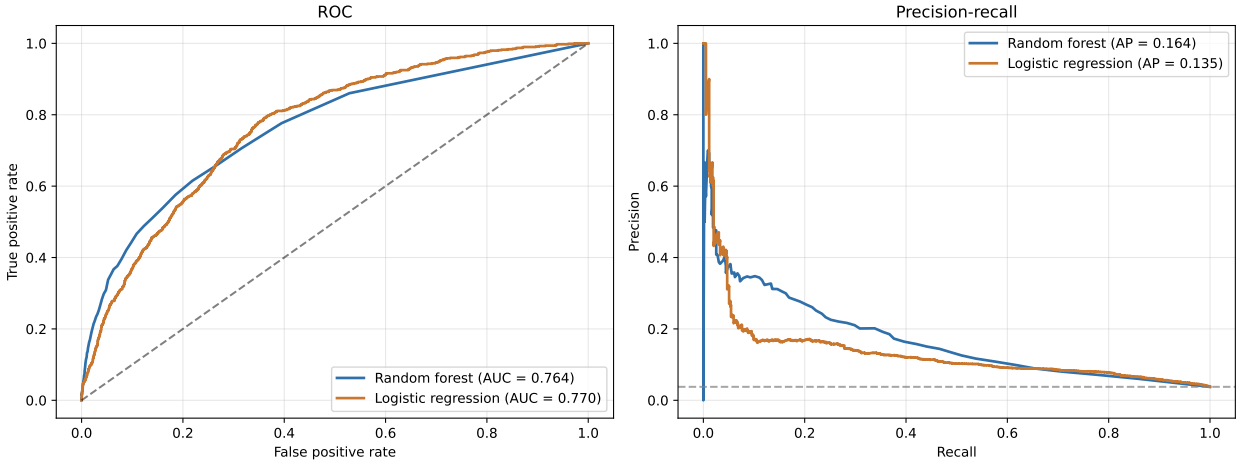
```
from sklearn.linear_model import LogisticRegression

lr = Pipeline([
    ("scaler", StandardScaler()),
    ("classifier", LogisticRegression(random_state=RANDOM_STATE, max_iter=1000)),
]).fit(X_tr, y_tr)

rf_proba = y_proba
lr_proba = lr.predict_proba(X_te)[:, 1]

fig, axes = plt.subplots(1, 2, figsize=(13, 5))
for name, p, color in [("Random forest", rf_proba, "#2f6faa"), ("Logistic regression", lr_proba, "#c977
    f, t, _ = roc_curve(y_te, p)
    axes[0].plot(f, t, color=color, lw=2, label=f"{name} (AUC = {roc_auc_score(y_te, p):.3f})")
    pr_p, pr_r, _ = precision_recall_curve(y_te, p)
    axes[1].plot(pr_r, pr_p, color=color, lw=2, label=f"{name} (AP = {average_precision_score(y_te, p):

axes[0].plot([0, 1], [0, 1], "k--", alpha=0.5)
axes[0].set(xlabel="False positive rate", ylabel="True positive rate", title="ROC")
axes[0].legend(loc="lower right"); axes[0].grid(alpha=0.3)
axes[1].axhline(no_skill, color="gray", ls="--", alpha=0.7)
axes[1].set(xlabel="Recall", ylabel="Precision", title="Precision-recall")
axes[1].legend(loc="upper right"); axes[1].grid(alpha=0.3)
plt.tight_layout()
plt.show()
```



The AUCs are close to each other and the ROC curves cross, exactly the ambiguous case the rule above is for. Falling back to AP settles the discussion: random forest's AP is clearly higher, so it's the pick.

The final call is still a business decision. A **high-precision** strategy (few, high-confidence targets) suits a small retention budget; a **high-recall** strategy (catch as many churners as possible, tolerate false alarms) suits a case where losing a customer is very costly; a **balanced** strategy optimises F1 or ideally creates an explicit cost matrix tailored to the specific business context (Verbeke et al. 2012; Verbraken et al. 2013).

6.4.2.7 Lift and top-decile lift

Retention campaigns rarely contact everyone: budgets are limited, so in practice a campaign targets a fixed slice of the customer base, sorted by score, from the most likely churners down. **Lift** measures how much better than random that targeting is and is especially useful for evaluating the effectiveness of a model in

identifying high-risk customers or for guiding retention campaigns. Linking back to the confusion matrix, it is precision divided by the overall churn rate (De Caigny et al. 2018):

$$\text{Lift} = \frac{TP/(TP+FP)}{(TP+FN)/(TP+TN+FP+FN)}$$

The numerator is precision at that cut-off: of the customers targeted, how many actually churn. The denominator is the base rate: how many would churn if targeted at random. A lift of 1 means the targeted slice is no more concentrated in churners than the population as a whole, the model adds nothing; a lift of 3 means it is three times as concentrated.

The **top-decile lift (TDL)**, the lift at the cut-off that flags exactly the top 10% of customers by score, is the most common single-number version in practice. Ten percent isn't mathematically special: Neslin et al. (2006) show that a retention campaign's expected profit is a direct, increasing function of lift, which is why (top-decile) lift became one of the most widely used performance criteria in churn prediction in the first place (Verbeke et al. 2012; De Caigny et al. 2018).

```
order = np.argsort(y_proba)[::-1]
top10 = order[: int(0.10 * len(y_te))]
top_decile_lift = y_te.iloc[top10].mean() / y_te.mean()
print(f"Top-decile lift: {top_decile_lift:.2f}x  (baseline churn rate {y_te.mean():.1%})")
```

```
Top-decile lift: 4.17x  (baseline churn rate 3.8%)
```

The interpretation is direct: at the top 10%, the model detects 4.17 times more churners than a random 10% of customers would, exactly the efficiency gain a limited retention budget is trying to get.

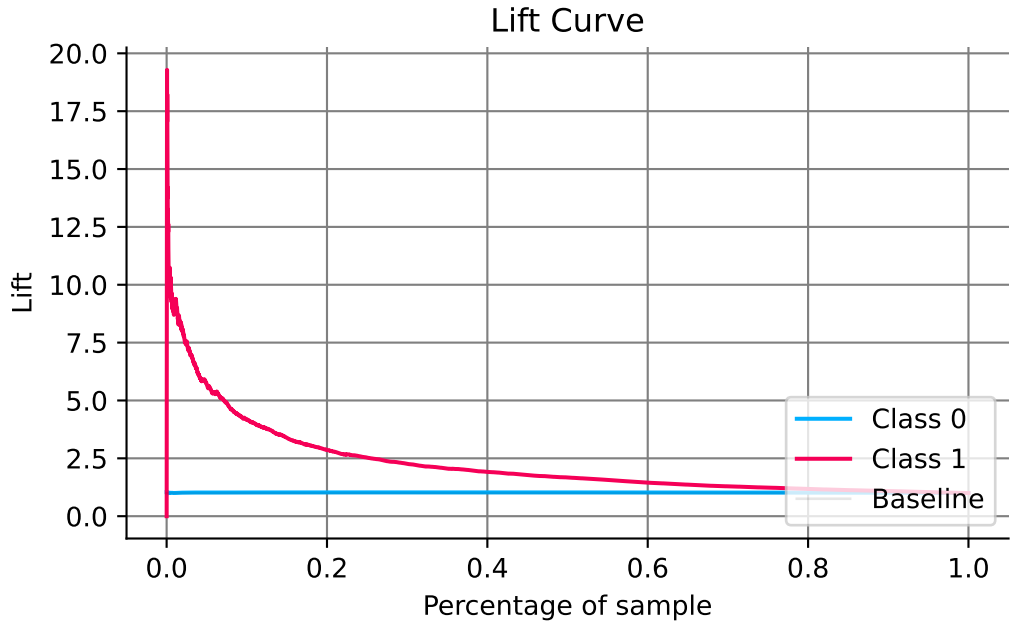
> **i** Why a fixed 10% cut-off can pick the wrong model
>
> Fixing the cut-off at 10% by convention is still only a stand-in for whatever the real campaign size will be, and that choice matters more than it looks. In practice, lift curves should be compared directly: Verbeke et al. show two churn models whose lift curves *cross*, one has the better lift at the top 5%, the other at the top 10% (Verbeke et al. 2012). Picking a model on top-decile lift can therefore mean walking away with the wrong one if the real campaign only reaches the top 5%, or vice versa. Since expected profit is a direct function of lift at whatever fraction is actually used, the safe rule is to read lift at the fraction you genuinely intend to target, not at a fixed 10% by habit. Fully profit-driven metrics remove that guesswork by searching over every fraction directly (Verbraken et al. 2013), but that is beyond the scope of this book.

The **cumulative lift curve** extends the top decile lift to every possible campaign size, from the top 1% to the whole population: at each point it's the cumulative churn rate among the highest-scored customers divided by the overall rate. It starts highest at the very smallest, most concentrated slice (lift of about 9 at the top 1% here), decays as more, less certain customers are added, and settles at 1 once the whole population is targeted, since targeting everyone is the same as targeting at random.

```
from sklearn_evaluation.plot import lift_curve

y_proba_2d = np.column_stack([1 - y_proba, y_proba])
plt.figure(figsize=(7, 4))
lift_curve(y_te, y_proba_2d)
plt.tight_layout()
plt.show()
```

```
<Figure size 2100x1200 with 0 Axes>
```



6.4.2.8 Cumulative gains and the KS statistic

The **cumulative gains chart** is the lift curve's close cousin, and the two are related by a simple identity. Both share the same x-axis: the fraction of customers targeted, sorted from the highest score down to the lowest, growing from the smallest slice to the whole population. Where they differ is what the y-axis counts. Lift is a *ratio*, how concentrated is this slice compared to random, so it starts high and decays toward 1 as more, less certain customers are added. Gains is a *running total*, how many churners have been captured so far, so it starts at 0% and climbs to 100% once everyone is included. The two are tied together exactly:

$$\text{Gains}(x) = x \cdot \text{Lift}(x)$$

Working that out in terms of the confusion matrix at the cut-off that targets fraction x gives an even cleaner formula:

$$\text{Gains}(x) = \frac{TP}{TP + FN}$$

That's exactly **recall**. The cumulative gains chart is nothing more than recall traced across every possible cut-off instead of read at one fixed threshold: of all actual churners, what share are in the group targeted so far. Targeting a larger fraction x always trades a lower lift for a larger absolute share of churners caught. Targeting the top 10% here captures roughly 40% of all churners ($0.10 \times 4.17 \approx 0.42$, the same top-decile lift computed above, just read as a running total instead of a ratio).

Let's see that in action, and read the cumulative gains at 10% directly off the chart:

```
from sklearn_evaluation.plot import cumulative_gain

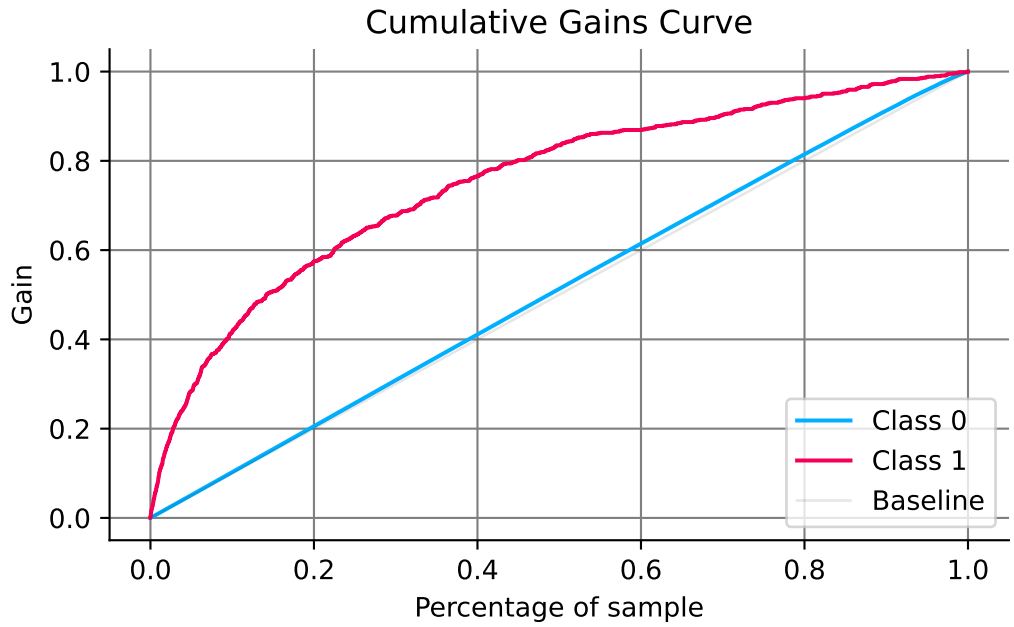
gains_at_10 = y_te.iloc[top10].sum() / y_te.sum()
print(f"Cumulative gains at 10%: {gains_at_10:.1%} of all churners captured")

plt.figure(figsize=(7, 4))
cumulative_gain(y_te, y_proba_2d)
```

```
plt.tight_layout()
plt.show()
```

```
Cumulative gains at 10%: 41.7% of all churners captured
```

```
<Figure size 2100x1200 with 0 Axes>
```



The **Kolmogorov–Smirnov (KS) statistic** measures how well the scores *separate* the two classes. It comes from a classic two-sample test: plot the cumulative distribution of scores separately for churners and for non-churners, and KS is the largest vertical gap between those two curves. Since TPR and FPR are themselves just those two cumulative distributions read from the top down, that same gap shows up on the ROC curve as `max(TPR - FPR)`. Hence, it is the widest vertical distance between the ROC curve and the diagonal.

Let's compute KS both ways and see that they agree:

```
from scipy.stats import ks_2samp

ks_scipy = ks_2samp(y_proba[y_te == 1], y_proba[y_te == 0]).statistic

fpr, tpr, thresholds = roc_curve(y_te, y_proba)
ks_idx = np.argmax(tpr - fpr)
ks_roc = (tpr - fpr)[ks_idx]

print(f"KS statistic (two score distributions): {ks_scipy:.4f}")
print(f"KS statistic (max TPR - FPR on the ROC curve): {ks_roc:.4f}")
print(f"Reached at threshold {thresholds[ks_idx]:.3f}")

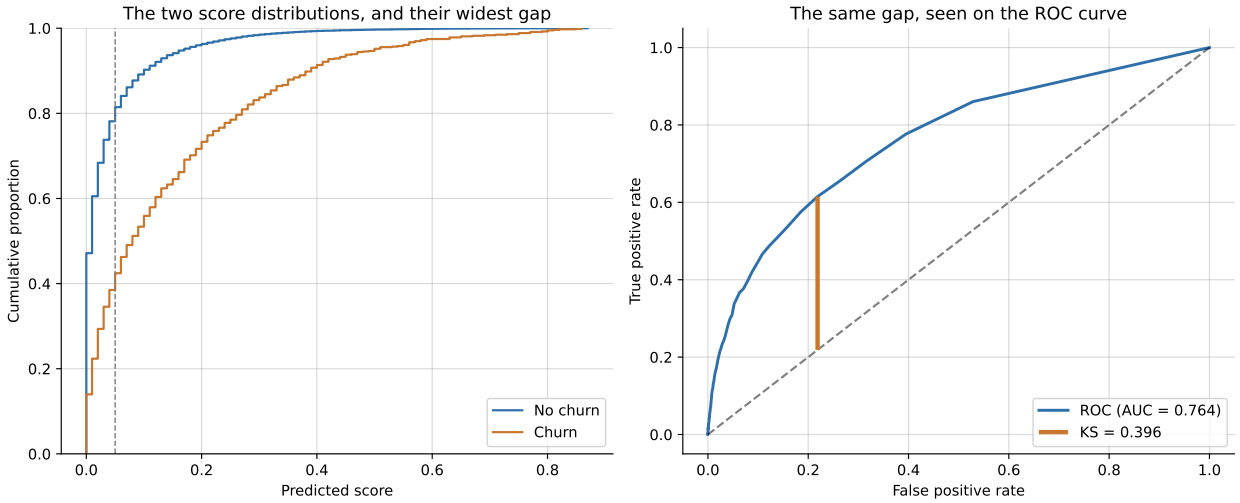
fig, axes = plt.subplots(1, 2, figsize=(12, 5))

sns.ecdfplot(y_proba[y_te == 0], ax=axes[0], color="#2f6faa", label="No churn")
sns.ecdfplot(y_proba[y_te == 1], ax=axes[0], color="#c9772e", label="Churn")
axes[0].axvline(thresholds[ks_idx], color="gray", ls="--", lw=1)
axes[0].set(xlabel="Predicted score", ylabel="Cumulative proportion",
```

```
            title="The two score distributions, and their widest gap")
axes[0].legend(loc="lower right")
axes[0].grid(alpha=0.3)

axes[1].plot(fpr, tpr, color="#2f6faa", lw=2, label=f"ROC (AUC = {auc:.3f})")
axes[1].plot([0, 1], [0, 1], "k--", alpha=0.5)
axes[1].vlines(fpr[ks_idx], fpr[ks_idx], tpr[ks_idx], color="#c9772e", lw=3,
               label=f"KS = {ks_roc:.3f}")
axes[1].set(xlabel="False positive rate", ylabel="True positive rate",
            title="The same gap, seen on the ROC curve")
axes[1].legend(loc="lower right")
axes[1].grid(alpha=0.3)
plt.tight_layout()
plt.show()
```

```
KS statistic (two score distributions): 0.3963
KS statistic (max TPR - FPR on the ROC curve): 0.3963
Reached at threshold 0.050
```



Both routes agree: KS = 0.396, reached at a score threshold of 0.05, the cut-off where the two classes are most cleanly separated. That's a different threshold from the F1-optimal one found earlier (0.207): maximising separation and maximising F1 don't have to agree. At this KS-optimal threshold the model would flag about 23% of customers as likely churners, noticeably more than the 10% a typical campaign budget allows, a reminder that "the threshold with the best statistical property" and "the threshold the business can actually act on" are two different questions.

As a rule of thumb: KS below 0.2 is poor separation, 0.2 to 0.4 acceptable, 0.4 to 0.75 good, and above 0.75 is either excellent or a sign of leakage worth investigating. At 0.396, this model sits right at the top of the acceptable band, just short of good.

6.5 Where we go from here

We now have both sides of a trustworthy evaluation: an experimental setup that produces an honest test set, and metrics to score against it for regression and classification targets. The models used throughout this chapter, KNN and random forests, were deliberately treated as black boxes: useful for producing predictions to evaluate, not yet explained. The next chapter, **Modeling**, opens up the two workhorses instead, **linear regression** and **logistic regression**, including how regularization keeps them from overfitting, so you

understand not just how to score a model but how the most widely used one actually works.

Chapter 7

Modeling

The previous chapter treated the models producing predictions, a random forest and a KNN classifier, as black boxes: useful for generating scores to evaluate, not yet fully explained. This chapter introduces the two workhorses of predictive modeling instead: **linear regression** for a quantitative response, and **logistic regression** for a qualitative (typically binary) one. Both are **parametric** techniques: they assume a specific functional form for the relationship between predictors and response up front, which is what makes them fast to fit and, unlike a random forest, directly interpretable. That interpretability is also why, despite decades of newer alternatives, they remain the workhorse models in many business settings such as credit scoring and churn management, where a model's predictions often have to be explained to a regulator, a manager, or a customer, not just be accurate (Lessmann et al. 2015; Crook et al. 2007).

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

RANDOM_STATE = 123
```

7.1 Linear Regression

Linear regression predicts a **quantitative** response, a number rather than a category: next month's sales, a customer's lifetime value, the price of a house, stock prices, or, in the professor's own published research, the number of friends a person has on a social network, predicted from their activity on the platform (Ballings et al. 2016). It is a **parametric** technique: it assumes the response Y is a straight-line (linear) function of the predictors X, plus some irreducible noise ε:

$$Y = \beta_0 + \beta_1 X_1 + \beta_2 X_2 + \cdots + \beta_p X_p + \varepsilon$$

With one predictor ($p = 1$) this is **simple linear regression**, a straight line through a scatterplot. With more than one, it's **multiple linear regression**, and the same equation now describes a plane (two predictors) or a hyperplane (more than two), but the intuition doesn't change. Two things are being estimated, and both have a direct, plain-language reading:

- **The intercept,** β_0, is the model's prediction when every predictor equals zero: the baseline the rest of the model adjusts up or down from. It's not always meaningful on its own, a customer with zero transactions and zero recency may not exist in practice, but it's still needed to place the line (or plane) correctly.
- **Each slope,** β_j, is the change in Y for a one-unit increase in X_j, holding every other predictor fixed. This "holding everything else fixed" part matters: it is the coefficient's own, isolated effect, not the raw correlation you'd see from that predictor alone.

7.1.1 When the linear assumption fails

Because linear regression commits to a straight line, it struggles whenever the true relationship bends. The example below is synthetic on purpose, so we know the true relationship: it grows steeply up to a point, then flattens out, a common shape for e.g. diminishing returns on an advertising budget.

```
rng = np.random.default_rng(RANDOM_STATE)

x = np.linspace(0, 60, 200)
y_true = np.where(x < 30, 2 * x, 60 + 0.2 * (x - 30))
y = y_true + rng.normal(0, 8, size=len(x))
```

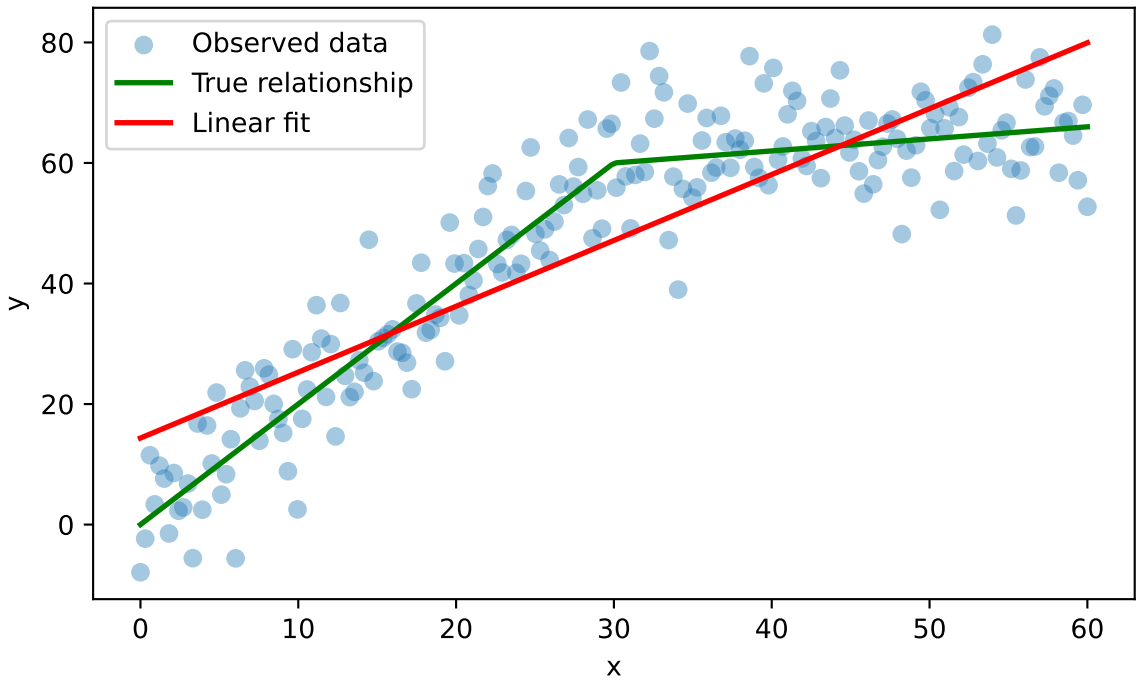
```
from sklearn.linear_model import LinearRegression

lin_reg = LinearRegression().fit(x.reshape(-1, 1), y)
y_pred = lin_reg.predict(x.reshape(-1, 1))

print(f"Intercept: {lin_reg.intercept_:.2f}")
print(f"Slope:     {lin_reg.coef_[0]:.2f}")
```

```
Intercept: 14.36
Slope:     1.09
```

```
plt.figure(figsize=(7, 4))
plt.scatter(x, y, alpha=0.4, label="Observed data")
plt.plot(x, y_true, color="green", linewidth=2, label="True relationship")
plt.plot(x, y_pred, color="red", linewidth=2, label="Linear fit")
plt.xlabel("x")
plt.ylabel("y")
plt.legend()
plt.show()
```



The fitted line, with an intercept around 14 and a slope around 1.1, is a compromise: a single slope has to

average the true, steeper early growth rate (2 per unit of x) against the flatter later one (0.2 per unit), so it ends up too steep for the flat part and too flat for the steep part. On average the error is not catastrophic, but the model systematically misses both ends of the range. Even so, linear regression remains a reasonable first model to try in practice: it is cheap to fit, easy to explain to a non-technical audience, and, as this example shows, "wrong" does not always mean "useless". As covered in the previous chapter, interaction terms and polynomial features let a linear model absorb some of this curvature without abandoning the linear framework entirely.

7.1.2 Fitting the line: sklearn versus statsmodels

Fitting a line means choosing the intercept and slope that make the line pass as close as possible to the data points, in the sense of minimizing the **residual sum of squares** (RSS), the sum of the squared vertical distances between each observed point and the line:

$$\text{RSS} = \sum_{i=1}^{n} (y_i - \hat{y}_i)^2$$

`sklearn`'s `LinearRegression` and the traditional statistical technique **Ordinary Least Squares** (OLS) both minimize exactly this quantity and will always produce identical coefficients. Where they differ is in what they report back:

```
import statsmodels.api as sm

X_const = sm.add_constant(x)
ols_model = sm.OLS(y, X_const).fit()

print(f"sklearn:     intercept = {lin_reg.intercept_:.4f}, slope = {lin_reg.coef_[0]:.4f}")
print(f"statsmodels: intercept = {ols_model.params[0]:.4f}, slope = {ols_model.params[1]:.4f}")
```

```
sklearn:     intercept = 14.3557, slope = 1.0932
statsmodels: intercept = 14.3557, slope = 1.0932
```

The coefficients match exactly. `statsmodels`, however, also gives you the full statistical picture: standard errors, t-statistics, and p-values for every coefficient, along with R^2 for the whole model.

```
print(ols_model.summary())
```

```
                            OLS Regression Results
==============================================================================
Dep. Variable:                      y   R-squared:                       0.758
Model:                            OLS   Adj. R-squared:                  0.757
Method:                 Least Squares   F-statistic:                     619.7
Date:                Tue, 22 Sep 2026   Prob (F-statistic):           6.82e-63
Time:                        11:59:20   Log-Likelihood:                -758.88
No. Observations:                 200   AIC:                             1522.
Df Residuals:                     198   BIC:                             1528.
Df Model:                           1
Covariance Type:            nonrobust
==============================================================================
                 coef    std err          t      P>|t|      [0.025      0.975]
------------------------------------------------------------------------------
const         14.3557      1.523      9.425      0.000      11.352      17.359
x1             1.0932      0.044     24.894      0.000       1.007       1.180
==============================================================================
Omnibus:                        0.007   Durbin-Watson:                   0.904
Prob(Omnibus):                  0.997   Jarque-Bera (JB):                0.069
```

```
Skew:                          0.012   Prob(JB):                     0.966
Kurtosis:                      2.912   Cond. No.                      69.2
==============================================================================

Notes:
[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.
```

Two things in this particular table are worth reading concretely, not just knowing where to find. The slope's p-value is essentially 0, so the relationship between x and y is real, not noise, which is unsurprising since we built the data ourselves. More interesting is $R^2 = 0.758$: the line explains about 76% of the variance in y, which on its own would read as a strong fit. It isn't one, since we already know this exact model systematically misfits both ends of the range. A high R^2 only tells you the predictors and the response move together strongly; it says nothing about whether a straight line is the *right shape* for that relationship. It is entirely possible to have a good R^2 and a wrong functional form at the same time, exactly as shown above.

`sklearn` deliberately leaves all of this out. It is built for prediction, where the coefficients themselves are a means to an end, so it optimizes for speed and for working the same way across dozens of very different model types. `statsmodels` is built for inference, where the coefficients *are* the end, so it computes the extra statistics that let you ask "is this effect real, or could it just be noise?" Use `sklearn` when the goal is a prediction pipeline; reach for `statsmodels` whenever you need to defend a specific coefficient's value.

7.1.3 A worked example: predicting Airbnb demand

Let's apply this to a real dataset: predicting how many days per month (`demand`) an Airbnb listing gets booked, based on properties of the listing and its host.

```
airbnb = pd.read_csv("../data/processed/basetable_regression.csv")
airbnb = airbnb.drop(columns=["Unnamed: 0"])

print(f"Shape: {airbnb.shape}")
airbnb.isnull().sum()
```

```
Shape: (6862, 10)
```

```
host_id                   0
accommodates              0
review_scores_rating    1292
distance                  0
demand                    0
entire_home               0
washer_drier              0
superhost                 0
response                869
price_pp                  0
dtype: int64
```

Two columns have missing values, `review_scores_rating` and `response`, so the pipeline built below needs an imputation step. One predictor, `host_id`, is not a real feature at all, it is just an identifier, and is deliberately kept in the basetable as a check: a well-behaved variable selection method should be able to recognize it carries no signal and downweight or drop it.

```
from sklearn.model_selection import train_test_split

X = airbnb.drop(columns=["demand"])
y = airbnb["demand"]

X_train, X_test, y_train, y_test = train_test_split(
```

```
    X, y, test_size=0.3, random_state=RANDOM_STATE
)

print(f"Training set: {X_train.shape[0]} rows, test set: {X_test.shape[0]} rows")
```

```
Training set: 4803 rows, test set: 2059 rows
```

As with any model, missing values need to be handled before fitting. Rather than imputing by hand as in earlier chapters, we wrap the imputer and the model together in a single `Pipeline`. That guarantees the exact same imputation logic (fit only on the training data) is applied consistently to both the training and the test set, and it lets the whole thing be treated, fit, and reused as one object.

```
from sklearn.pipeline import Pipeline
from sklearn.impute import SimpleImputer
from sklearn.metrics import mean_squared_error, r2_score

lr_pipeline = Pipeline([
    ("imputer", SimpleImputer(strategy="median")),
    ("regressor", LinearRegression()),
])
lr_pipeline.fit(X_train, y_train)

y_pred = lr_pipeline.predict(X_test)
test_rmse = np.sqrt(mean_squared_error(y_test, y_pred))
test_r2 = r2_score(y_test, y_pred)

baseline_rmse = np.sqrt(mean_squared_error(y_test, np.full_like(y_test, y_train.mean(), dtype=float)))

print(f"Test RMSE:     {test_rmse:.3f} days")
print(f"Baseline RMSE: {baseline_rmse:.3f} days (always predicting the training mean)")
print(f"Test R²:       {test_r2:.3f}")
```

```
Test RMSE:     5.482 days
Baseline RMSE: 5.659 days (always predicting the training mean)
Test R²:       0.061
```

RMSE on its own is hard to judge: is 5 days good or bad? It depends entirely on the scale of the target. The **baseline** above answers that: predicting nothing but the training set's average `demand` for every listing is the simplest model imaginable, so any model worth using has to beat it. The gap between the two numbers is the actual value the model adds, and here that gap is narrow: the model only edges out the baseline by a small margin.

The R^2 tells the same story from a different angle. At around 0.06, the model explains only about 6% of the variance in `demand`, which sounds discouraging next to the OLS demo's R^2 of 0.758 above. That comparison isn't quite fair, though: the synthetic demo used a single predictor built to have a strong, direct relationship with y by construction, while booking demand for a real listing depends heavily on things this basetable simply doesn't capture, seasonality, location beyond a single distance measure, competing listings nearby, price relative to the market, and plain randomness in traveller behaviour. A low R^2 here isn't a sign the model was fit badly; it's a sign that host and listing attributes alone are a genuinely weak predictor of demand, which is itself a useful finding to report back to a business stakeholder who might have expected more.

```
feature_names = X_train.columns
lr_model = lr_pipeline.named_steps["regressor"]

coef_df = pd.concat([
    pd.DataFrame({"Feature": ["Intercept"], "Coefficient": [lr_model.intercept_]}),
```

```
    pd.DataFrame({"Feature": feature_names, "Coefficient": lr_model.coef_}),
])
coef_df
```

	Feature	Coefficient
0	Intercept	11.068960
0	host_id	0.244975
1	accommodates	-0.352595
2	review_scores_rating	-0.079644
3	distance	0.210747
4	entire_home	1.113333
5	washer_drier	-0.684775
6	superhost	2.336448
7	response	0.006102
8	price_pp	-0.002349

The **intercept** is the model's predicted `demand` when every feature equals zero, the baseline that every slope below then adjusts. Since a real listing with zero `accommodates` and zero `distance` doesn't exist, that exact baseline is a mathematical anchor for the line rather than a prediction you'd ever actually use on its own, but it is still required to place the fitted plane correctly.

The **slopes** are read one at a time, holding everything else in the model fixed. For example, a positive coefficient on `superhost` means that, for two otherwise identical listings, being a superhost is associated with more booked days per month, on top of whatever the intercept already accounts for. This is what makes linear regression attractive to a business audience: every number in the table above is a direct, additive statement about the outcome, with no translation required.

Not every coefficient in that table is trustworthy, though. Some of these features may just be adding noise rather than signal, exactly what `host_id` was included to test for. To find out which, we need the p-values `sklearn` doesn't give us:

```
X_train_imputed = lr_pipeline.named_steps["imputer"].transform(X_train)
X_train_with_const = sm.add_constant(X_train_imputed)

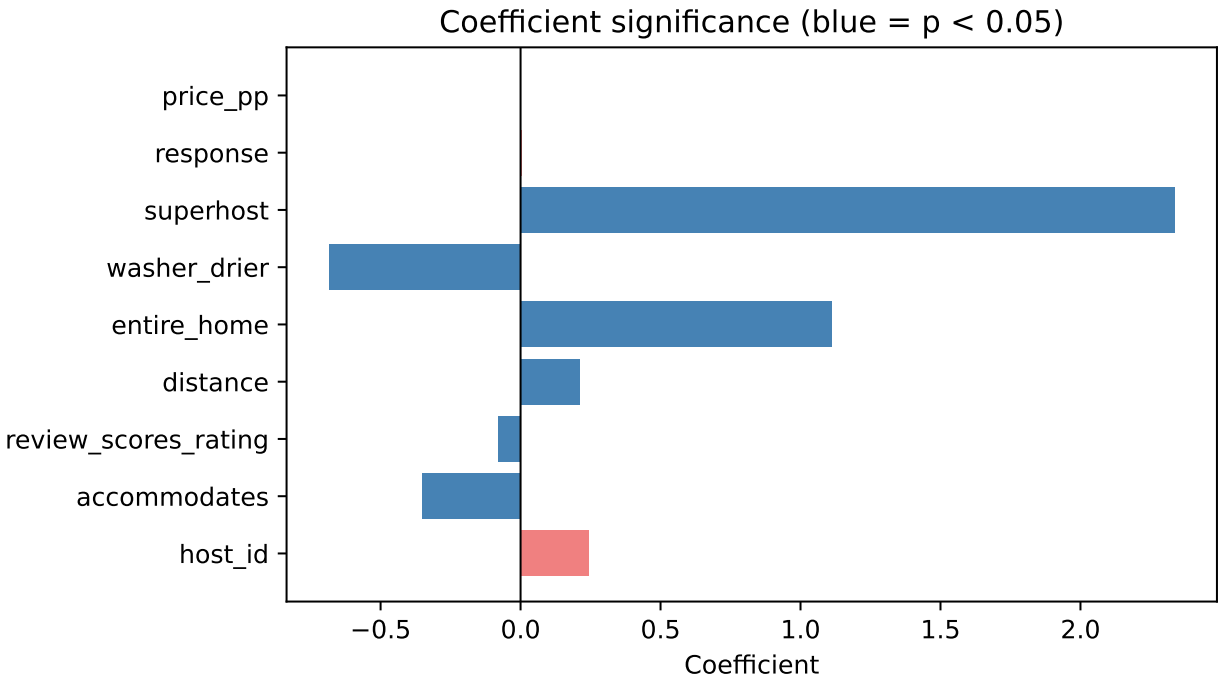
ols_airbnb = sm.OLS(y_train.values, X_train_with_const).fit()

coef_stats = pd.DataFrame({
    "Feature": feature_names,
    "Coefficient": ols_airbnb.params[1:],
    "P_value": ols_airbnb.pvalues[1:],
})
coef_stats["Significant"] = coef_stats["P_value"] < 0.05
coef_stats.sort_values("P_value")
```

	Feature	Coefficient	P_value	Significant
6	superhost	2.336448	1.580566e-42	True
1	accommodates	-0.352595	6.628124e-15	True
3	distance	0.210747	3.795619e-12	True
4	entire_home	1.113333	8.311393e-10	True
2	review_scores_rating	-0.079644	1.030395e-09	True
5	washer_drier	-0.684775	1.139220e-04	True
8	price_pp	-0.002349	1.643469e-03	True
0	host_id	0.244975	1.284363e-01	False

	Feature	Coefficient	P_value	Significant
7	response	0.006102	4.401556e-01	False

```
plt.figure(figsize=(7, 4))
colors = coef_stats["Significant"].map({True: "steelblue", False: "lightcoral"})
plt.barh(coef_stats["Feature"], coef_stats["Coefficient"], color=colors)
plt.axvline(0, color="black", linewidth=0.8)
plt.xlabel("Coefficient")
plt.title("Coefficient significance (blue = p < 0.05)")
plt.tight_layout()
plt.show()
```



`host_id` and `response` come back not significant, `host_id` exactly as expected, since it's an identifier with no real relationship to demand, and `response` more usefully: a host's response rate looks like it should matter, but it doesn't move the needle once the other predictors are already in the model. Both are still sitting in the model above, quietly adding complexity without adding predictive power. `washer_drier`, on the other hand, *is* significant despite looking like a minor amenity, a reminder that statistical significance depends on the actual data, not on how important a feature sounds. The next section covers how to find and remove the genuinely uninformative ones, rather than by eye or an arbitrary p-value threshold.

i Model comparison beyond a single test set

Comparing candidate linear models does not always require a held-out test set. Statistics offers two broad routes (James et al. 2013): **directly estimating** the test error via cross-validation (covered in the previous chapter, and the more broadly correct approach), or **approximating** it by adjusting the training error for the number of predictors used, via measures like adjusted R^2, AIC, or BIC. These metrics add a penalty for the number of predictors used, helping to prevent overfitting and approximate the true test error. The second route is popular in classical statistics, mainly because it doesn't require setting aside data you might not have much of, but cross-validation is generally preferred whenever

enough data is available for it.

7.1.4 Where linear regression breaks down

Linear regression rests on a handful of assumptions about the data that real datasets routinely violate to some degree:

- **Nonlinearity**: the true relationship isn't actually a straight line, as in the first example of this chapter.
- **Correlated errors**: residuals aren't independent of each other, common in time series data, where an unusually large error today makes an unusually large error tomorrow more likely.
- **Heteroscedasticity**: the spread of the residuals changes across the range of the predictors, instead of staying roughly constant.
- **Multicollinearity**: two or more predictors are highly correlated with each other, which makes their individual coefficients unstable, sometimes even flipping sign between two very similar samples of the same data.

Diagnosing and correcting these properly is an econometrics topic, well beyond this book's scope. The one thing worth carrying over into a predictive-modelling context is where the damage actually lands: these violations usually hurt a model's raw predictive accuracy far less than you would expect, but they hurt the reliability of its *coefficients* much more. A coefficient's size, and even its sign, can become unstable well before the model's predictions do. This maps directly onto the sklearn-versus-statsmodels distinction from earlier: if the model is only ever used to produce a score, these assumption violations are a secondary concern, but the moment you tell a manager "increasing X by one unit changes Y by β", it is worth checking whether that particular assumption actually holds.

7.2 Selecting Variables and Taming Complexity

The previous chapter's variable selection section introduced three families of methods: **filter** methods, which score each predictor against the target independently of any model, **wrapper** methods, which retrain a model on different feature subsets and keep the best-performing one, and **embedded** methods, which build variable selection directly into how the model is fit. Filter methods were covered there; both of the remaining families belong here, since they only make sense once you have an actual model to wrap around or embed into.

7.2.1 Wrapper methods: sequential feature selection

The classical wrapper approach is **stepwise selection**: start from nothing and add the single best predictor at each step (forward selection), start from everything and remove the least useful one at each step (backward selection), or some combination of both. The trouble is what "best" and "least useful" get judged by: a p-value or an AIC score, re-tested again and again on the exact same training data, once per candidate variable at every single step.

That repeated testing against one fixed dataset is exactly what **data dredging** (also called **p-hacking**) means: the more hypotheses you test on the same data, the more likely you are to find one that exactly matches your preconceptions, not because it reflects a real pattern. A p-value of 0.05 already means a 1-in-20 false-positive rate on a *single* test; run dozens of these tests along the way, one per candidate predictor at every step, and finding a handful of "significant" variables that are really just noise stops being unlikely and starts being expected. This is why stepwise selection is unstable: run it again on a slightly different sample of the same population, and you will often get a different final set of variables, exactly what you'd expect if some of the original picks were random noise rather than genuine signal in the first place.

`sklearn` sidesteps this by never testing a candidate feature against the data it was chosen from. `SequentialFeatureSelector` is a modern equivalent that judges each candidate by cross-validated performance instead of a p-value: every decision is checked against held-out folds the feature had no part in, so a feature can only look good by actually improving out-of-sample predictions, not by getting lucky on the training data.

```
from sklearn.feature_selection import SequentialFeatureSelector

sfs_pipeline = Pipeline([
    ("imputer", SimpleImputer(strategy="median")),
    ("selector", SequentialFeatureSelector(
        LinearRegression(), n_features_to_select=5, direction="forward",
        scoring="neg_root_mean_squared_error", cv=5,
    )),
    ("regressor", LinearRegression()),
])
sfs_pipeline.fit(X_train, y_train)

selected_sfs = feature_names[sfs_pipeline.named_steps["selector"].get_support()]
rmse_sfs = np.sqrt(mean_squared_error(y_test, sfs_pipeline.predict(X_test)))

print(f"Selected features: {list(selected_sfs)}")
print(f"Test RMSE: {rmse_sfs:.3f} days (full model: {test_rmse:.3f})")
```

```
Selected features: ['accommodates', 'review_scores_rating', 'distance', 'entire_home', 'superhost']
Test RMSE: 5.497 days (full model: 5.482)
```

Forward selection here means specifying up front how many features to keep, `5` above, which is both its main advantage (aggressive, deliberate feature reduction) and its main limitation (finding the *right* number still needs its own cross-validation loop, comparing several choices of `n_features_to_select` against each other). The RMSE with only 5 of the 9 features is essentially tied with the full model, so nothing of real value was lost by cutting the feature count almost in half.

Notice that this cross-validated selection doesn't fully match the p-value picture from before: it correctly drops `host_id` and `response`, the two genuinely non-significant features, but it also drops `washer_drier` and `price_pp`, which *were* statistically significant. That isn't a contradiction, a coefficient can be significant on its own and still be redundant once combined with the other four features that already explain most of what it was picking up, which is exactly the kind of correlation-driven redundancy a p-value alone can't detect.

7.2.2 Embedded methods: regularization

Regularization takes a different approach entirely: instead of removing predictors from the model, keep all of them, but add a penalty to the fitting procedure that discourages large coefficients. The model is *constrained* rather than *reduced*. This "shrinks" coefficients toward zero, which trades a little bias for a real reduction in variance, making the model more robust to noise and less sensitive to any one training sample (James et al. 2013; Kuhn and Johnson 2013). Two types of penalty are most common: **Ridge** (L2) regularization, which penalizes the sum of squared coefficients, and **Lasso** (L1) regularization, which penalizes the sum of absolute coefficients:

$$\text{Ridge (L2):}\ \min_{\beta}\ \text{RSS} + \lambda \sum_{j=1}^{p} \beta_j^2 \qquad \text{Lasso (L1):}\ \min_{\beta}\ \text{RSS} + \lambda \sum_{j=1}^{p} |\beta_j|$$

The parameter λ controls how aggressively coefficients are penalized, and it is a direct lever on the bias-variance trade-off from the previous chapter: $\lambda = 0$ recovers ordinary least squares (low bias, potentially high variance), while a very large λ shrinks every coefficient toward zero and eventually toward a model that predicts only the mean (high bias, low variance). The right value in between is found the same way any other hyperparameter is, via cross-validation.

The practical difference between the two penalties matters for interpretation. **Ridge** shrinks every coefficient but essentially never sets one to exactly zero, so all predictors stay in the model, just with smaller weights.

Lasso can shrink a coefficient all the way to zero, which makes it a built-in variable selection method: whichever predictors survive are the ones the model considers worth keeping.

There is a simple geometric reason for that difference. Minimizing RSS plus a penalty is mathematically equivalent to minimizing RSS alone subject to a hard budget on the coefficients: a total absolute size for Lasso (a **diamond**-shaped region of allowed coefficient values) or a total squared size for Ridge (a **circular** region). The unconstrained least-squares solution $\hat{\beta}$ usually sits outside that budget, so the regularized solution is whichever point is closest to it.

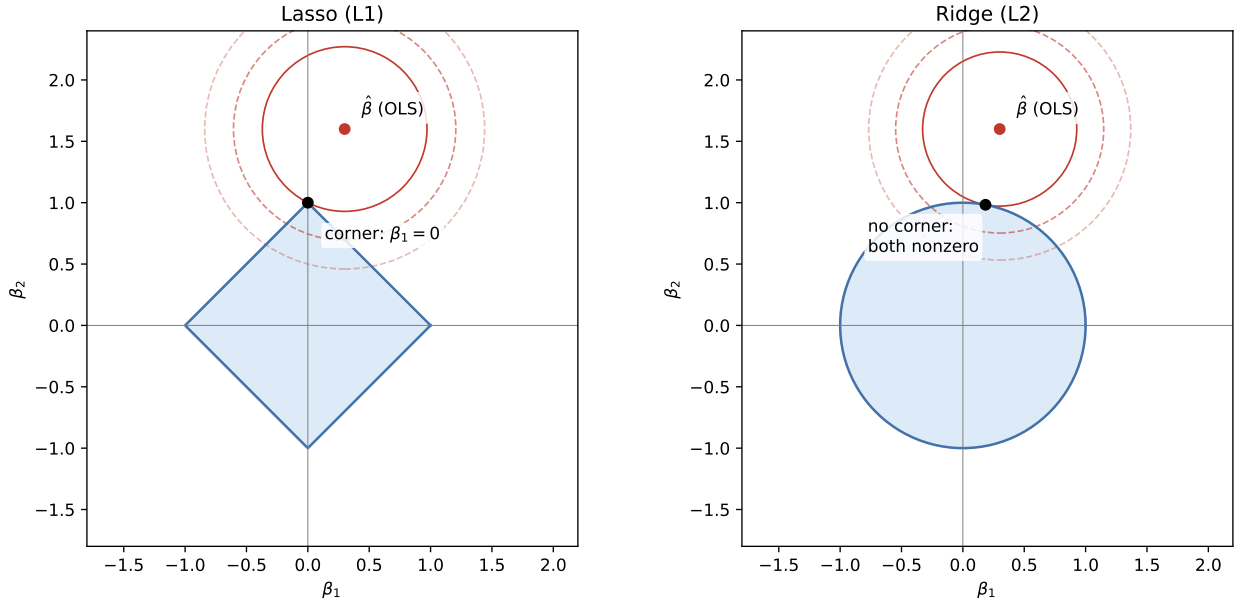


Figure 7.1: Why Lasso zeroes coefficients and Ridge doesn't: the diamond has corners on the axes, the circle has none.

The red circles are contours of equal RSS, growing outward from $\hat{\beta}$ as the fit gets worse. The regularized solution is wherever the smallest such contour first touches the shaded budget region. Because the diamond has corners sitting exactly on the axes, that first touch often lands on a corner, where one coefficient is exactly zero. The circle has no corners: its closest point is just some point on its boundary, essentially never exactly on an axis. That is the entire geometric reason Lasso performs variable selection and Ridge does not.

Always scale before you regularize

Both penalties act directly on the coefficients' *magnitudes*, and a coefficient's magnitude depends on the scale of its predictor: a variable measured in the thousands only needs a tiny coefficient to have a meaningful effect on the outcome, while one measured in single digits needs a much larger coefficient for that same effect, regardless of how important either actually is. Left uncorrected, the penalty would fall almost entirely on the small-scale predictor: its naturally larger coefficient gets shrunk hard, while the large-scale predictor's already-tiny coefficient barely feels the penalty at all, even if the two are equally important to the outcome.
The fix is exactly the **standardization** step from the previous chapter: scale every predictor before fitting a regularized model, fit the scaler on the training data only inside a pipeline, and apply it to the test data.

Below, Ridge and Lasso both get fit the exact same way: impute, scale, then search a grid of λ values (`alpha` in `sklearn`) with cross-validation to find the one that minimizes RMSE, and evaluate the winner on the held-out test set. Only the estimator and its hyperparameter grid change between the two, so rather than

repeat that recipe twice, the cell below writes it once as a reusable helper function, `fit_regularized`. Right after defining it, we call it once for Ridge on the Airbnb data.

```
from sklearn.linear_model import Ridge, Lasso, ElasticNet
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import GridSearchCV

def fit_regularized(estimator, param_grid, X_train, y_train, X_test, y_test):
    pipeline = Pipeline([
        ("imputer", SimpleImputer(strategy="median")),
        ("scaler", StandardScaler()),
        ("regressor", estimator),
    ])
    grid = GridSearchCV(pipeline, param_grid, cv=5, scoring="neg_root_mean_squared_error", n_jobs=-1)
    grid.fit(X_train, y_train)

    rmse = np.sqrt(mean_squared_error(y_test, grid.predict(X_test)))
    coefs = grid.best_estimator_.named_steps["regressor"].coef_
    return grid, rmse, coefs

alpha_grid = {"regressor__alpha": np.logspace(-2, 2, 10)}

ridge_grid, ridge_rmse, ridge_coefs = fit_regularized(
    Ridge(), alpha_grid, X_train, y_train, X_test, y_test
)
print(f"Ridge:  best alpha = {ridge_grid.best_params_['regressor__alpha']:.3f}, test RMSE = {ridge_rmse
print(f"Features with a non-zero coefficient: {(ridge_coefs != 0).sum()} / {len(feature_names)}")
```

```
Ridge:  best alpha = 100.000, test RMSE = 5.482
Features with a non-zero coefficient: 9 / 9
```

Cross-validation picked a fairly large alpha here, near the top of the range searched, yet Ridge's test RMSE barely moves from the unregularized model's. That is consistent with the earlier finding that this basetable isn't carrying much harmful noise: there's little overfitting for regularization to fix, so shrinking the coefficients costs essentially nothing, but it doesn't buy much either. Ridge keeps every feature at a non-zero weight regardless.

```
lasso_grid, lasso_rmse, lasso_coefs = fit_regularized(
    Lasso(max_iter=5000), alpha_grid, X_train, y_train, X_test, y_test
)
print(f"Lasso:  best alpha = {lasso_grid.best_params_['regressor__alpha']:.3f}, test RMSE = {lasso_rmse
print(f"Features with a non-zero coefficient: {(lasso_coefs != 0).sum()} / {len(feature_names)}")
print(f"Dropped: {list(feature_names[lasso_coefs == 0])}")
```

```
Lasso:  best alpha = 0.010, test RMSE = 5.482
Features with a non-zero coefficient: 9 / 9
Dropped: []
```

The cross-validated alpha for Lasso lands at the *smallest* value in the search grid, and the "Dropped" list above comes back empty: on this dataset, even Lasso's built-in variable selection concludes that every feature is worth at least a small, non-zero weight. That is a legitimate result, not a failed demo. It simply means none of the remaining noise here is harmful enough for cross-validated RMSE to reward removing it outright, which is a different, milder conclusion than the p-value test found (`host_id` and `response` weren't statistically significant) or the wrapper method found (it happily dropped four features for a similar RMSE). All three answers are consistent with each other: this basetable has some redundant, low-value features, but no features that actively hurt a plain, unregularized fit.

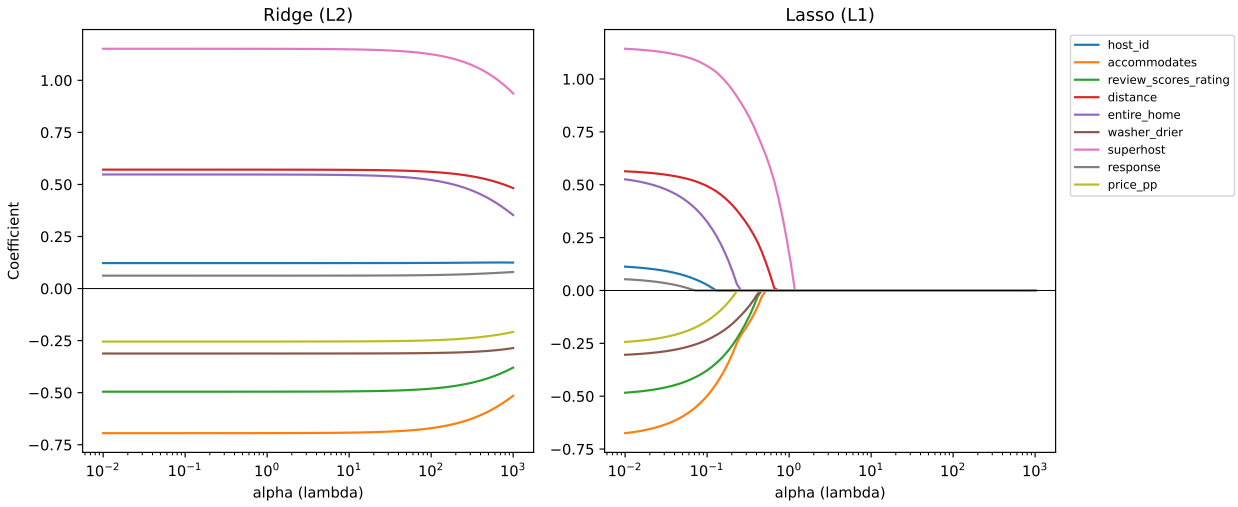
Watching the coefficients shrink as λ (called `alpha` in `sklearn`) increases makes the Ridge-versus-Lasso difference visible directly:

```
alphas_path = np.logspace(-2, 3, 100)
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(
    SimpleImputer(strategy="median").fit_transform(X_train)
)

ridge_paths = np.array([Ridge(alpha=a).fit(X_train_scaled, y_train).coef_ for a in alphas_path])
lasso_paths = np.array([Lasso(alpha=a, max_iter=5000).fit(X_train_scaled, y_train).coef_ for a in alphas

fig, axes = plt.subplots(1, 2, figsize=(12, 5))
for i, name in enumerate(feature_names):
    axes[0].plot(alphas_path, ridge_paths[:, i], label=name)
    axes[1].plot(alphas_path, lasso_paths[:, i], label=name)

for ax, title in zip(axes, ["Ridge (L2)", "Lasso (L1)"]):
    ax.set_xscale("log")
    ax.set_xlabel("alpha (lambda)")
    ax.axhline(0, color="black", linewidth=0.6)
    ax.set_title(title)
axes[0].set_ylabel("Coefficient")
axes[1].legend(loc="upper left", bbox_to_anchor=(1.02, 1), fontsize=8)
plt.tight_layout()
plt.show()
```



Every Ridge line bends toward zero but never quite touches it. Several Lasso lines flatten out at exactly zero well before the largest alpha values, and stay there: those are the features Lasso has fully excluded. In general, Lasso tends to work well when there is high variance in the true effect sizes: a small number of predictors have large coefficients while the rest are close to zero (a small "true" model hiding among many candidates). Ridge tends to work well when the effect sizes have low variance instead, i.e. most predictors carry a similar, modest amount of signal. Lasso's sparser, more selective solutions also make it the more commonly preferred default of the two, especially when interpretability matters as much as raw predictive accuracy.

Ridge and Lasso aren't the only two options. **Elastic Net** combines both penalties, controlled by a mixing parameter `l1_ratio` between 0 (pure Ridge) and 1 (pure Lasso), which is useful when predictors are highly

correlated with each other: pure Lasso tends to arbitrarily keep just one variable from a correlated group and drop the rest, while Elastic Net's Ridge component encourages correlated predictors to be kept or reduced together. Fitting it needs nothing new: it's the same `fit_regularized` helper from above, just called a third time with `ElasticNet` and its own hyperparameter grid.

```
elastic_grid_params = {
    "regressor__alpha": np.logspace(-2, 2, 6),
    "regressor__l1_ratio": [0.1, 0.5, 0.9],
}
elastic_grid, elastic_rmse, elastic_coefs = fit_regularized(
    ElasticNet(max_iter=5000), elastic_grid_params, X_train, y_train, X_test, y_test
)
print(f"Elastic Net: best alpha = {elastic_grid.best_params_['regressor__alpha']:.3f}, "
      f"l1_ratio = {elastic_grid.best_params_['regressor__l1_ratio']}, test RMSE = {elastic_rmse:.3f}")
```

```
Elastic Net: best alpha = 0.010, l1_ratio = 0.1, test RMSE = 5.482
```

Cross-validation settles on a low `l1_ratio`, meaning Elastic Net's own search concluded this problem is closer to a Ridge-style one than a Lasso-style one, and, unsurprisingly given the last two results, its RMSE lands in the same narrow band as both of them. Three different penalty shapes converging on the same performance is itself informative: it says the *choice* of penalty matters far less here than the decision to regularize at all, which for this dataset barely moved the needle either.

7.2.3 Importance-based selection: SelectFromModel

A quicker, less exhaustive alternative to wrapping a model in cross-validation is `SelectFromModel`: fit an estimator once, then keep whichever features have a coefficient (or, for tree-based models, a feature importance) above some threshold. Passing `threshold="median"` keeps roughly the stronger half of the features by coefficient magnitude. It is commonly applied right after a Lasso fit, to turn "many small coefficients" into an explicit, smaller feature list.

```
from sklearn.feature_selection import SelectFromModel

sfm_pipeline = Pipeline([
    ("imputer", SimpleImputer(strategy="median")),
    ("scaler", StandardScaler()),
    ("selector", SelectFromModel(Lasso(alpha=lasso_grid.best_params_["regressor__alpha"], max_iter=5000)
                                 threshold="median")),
    ("regressor", LinearRegression()),
])
sfm_pipeline.fit(X_train, y_train)

selected_sfm = feature_names[sfm_pipeline.named_steps["selector"].get_support()]
rmse_sfm = np.sqrt(mean_squared_error(y_test, sfm_pipeline.predict(X_test)))

print(f"Selected features: {list(selected_sfm)}")
print(f"Test RMSE: {rmse_sfm:.3f}")
```

```
Selected features: ['accommodates', 'review_scores_rating', 'distance', 'entire_home', 'superhost']
Test RMSE: 5.497
```

`SelectFromModel` fits the Lasso passed into it, takes the absolute value of its coefficients as each feature's importance, and drops every feature below the threshold. `threshold="median"` sets that cutoff to the median of those 9 importances, so roughly the weaker half gets dropped and the stronger half survives.

The `threshold="median"` above is not optional decoration, though. Leave it unset, and for an L1-penalized estimator like Lasso, `SelectFromModel` quietly swaps in a tiny fixed threshold instead of the median, one

small enough that it drops nothing once most coefficients are already nonzero. That's exactly the situation the cross-validated Lasso above left us in: at its best alpha it zeroed out none of the 9 features, so an unset threshold would let all 9 straight through. Setting `threshold="median"` explicitly is what actually gets a smaller model out of this step.

With it, the method lands on the same five features Sequential Feature Selection picked out earlier, from a completely different selection process (coefficient magnitude here, cross-validated performance there), at essentially the same test RMSE. Two unrelated methods agreeing is a good sign that these five really are the ones carrying the signal in this basetable.

7.2.4 Comparing the approaches

```
comparison = pd.DataFrame({
    "Method": ["Full model", "Sequential selection", "Ridge", "Lasso", "Elastic Net", "SelectFromModel"]
    "Features used": [
        len(feature_names), len(selected_sfs),
        (ridge_coefs != 0).sum(), (lasso_coefs != 0).sum(),
        (elastic_coefs != 0).sum(), len(selected_sfm),
    ],
    "Test RMSE": [test_rmse, rmse_sfs, ridge_rmse, lasso_rmse, elastic_rmse, rmse_sfm],
}).round(3)
comparison
```

	Method	Features used	Test RMSE
0	Full model	9	5.482
1	Sequential selection	5	5.497
2	Ridge	9	5.482
3	Lasso	9	5.482
4	Elastic Net	9	5.482
5	SelectFromModel	5	5.497

No method here beats the full model by much: this particular dataset just doesn't have enough truly harmful noise variables for aggressive selection to pay off. That is a realistic outcome, not a failed experiment. What every method *does* agree on is that a meaningfully smaller model is possible at little to no cost in performance, and a smaller, cleaner model is worth having on its own merits: easier to explain, faster to maintain, and less exposed to the "noise" predictors identified earlier.

7.3 Logistic Regression

The Airbnb example predicted a number, namely the demand for listings. Many business problems instead ask for a yes/no answer: will this customer churn, will this transaction turn out to be fraud, will this loan default? `churn`, the target used throughout the last two chapters, is exactly this kind of **qualitative** (binary, 0/1) response, and **logistic regression** is its natural, linear-regression-style counterpart.

7.3.1 Why not just use linear regression?

It's tempting to just plug a 0/1 target straight into `LinearRegression` and see what happens.

```
basetable = pd.read_csv("../data/processed/basetable.csv")

X_naive = basetable.drop(columns=["churn"])
y_naive = basetable["churn"]
```

```
naive_fit = LinearRegression().fit(X_naive, y_naive)
naive_preds = naive_fit.predict(X_naive)

print(f"Predicted values range from {naive_preds.min():.3f} to {naive_preds.max():.3f}")
```

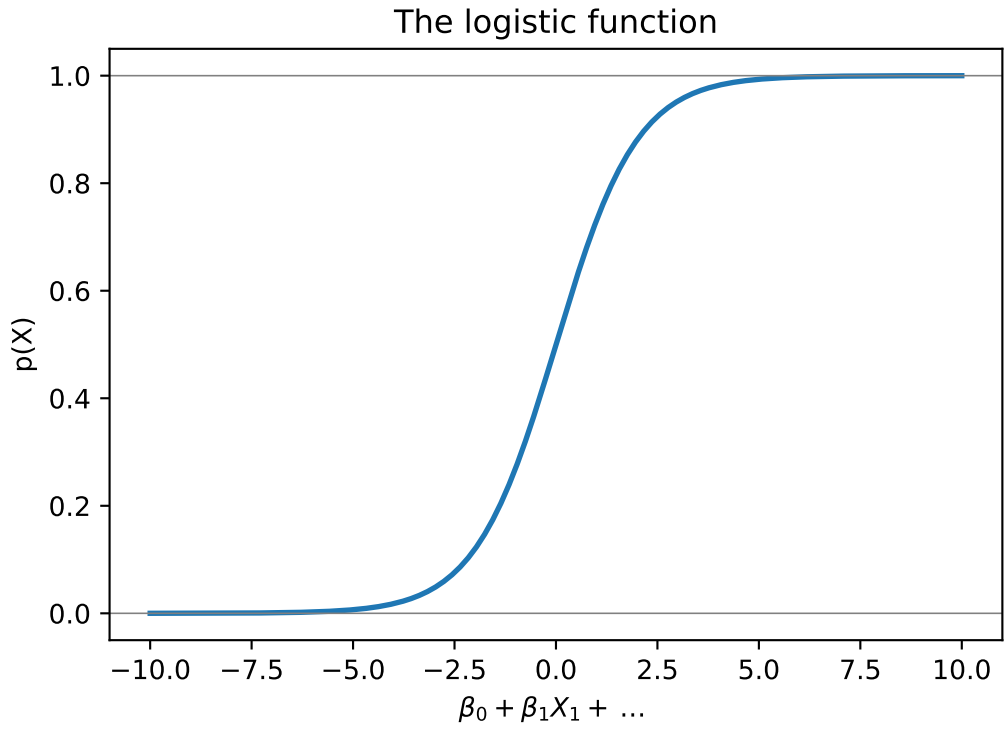
```
Predicted values range from -0.153 to 0.898
```

Some of the "predicted probabilities" fall below 0 or above 1, values that make no sense as a probability of churn. Linear regression has no built-in notion that the response should stay between 0 and 1; it just fits the best straight line it can through a cloud of 0s and 1s. **Logistic regression** fixes exactly this by passing the linear combination of predictors through the **logistic/sigmoid** function, which squashes any real number into the $(0, 1)$ range:

$$p(X) = \frac{e^{\beta_0+\beta_1 X_1+\cdots+\beta_p X_p}}{1 + e^{\beta_0+\beta_1 X_1+\cdots+\beta_p X_p}}$$

```
z = np.linspace(-10, 10, 200)
p = np.exp(z) / (1 + np.exp(z))

plt.figure(figsize=(6, 4))
plt.plot(z, p, linewidth=2)
plt.axhline(0, color="gray", linewidth=0.6)
plt.axhline(1, color="gray", linewidth=0.6)
plt.xlabel(r"$\beta_0 + \beta_1 X_1 + \dots$")
plt.ylabel("p(X)")
plt.title("The logistic function")
plt.show()
```



Regardless of how extreme the linear combination of predictors gets, $p(X)$ stays safely inside $(0, 1)$, which is exactly the S-shaped curve above. This probability comes directly out of a proper statistical estimation

procedure, which also tends to make logistic regression's predicted probabilities somewhat better calibrated out of the box than many tree ensembles (Niculescu-Mizil and Caruana 2005), though calibration is never guaranteed and is always worth checking, as noted in the previous chapter.

7.3.2 Interpreting coefficients: log-odds and odds ratios

The main advantage of logistic regression is that it is essentially a linear regression model, but with a link function that maps the linear combination of predictors to the $(0, 1)$ range. Its coefficients can also easily be converted into odds ratios, which are considerably more interpretable than the raw log-odds coefficients themselves.

Rearranging the logistic function gives the **log-odds**, or **logit**, which is linear in the predictors, exactly like ordinary linear regression:

$$\log\left(\frac{p(X)}{1-p(X)}\right) = \beta_0 + \beta_1 X_1 + \cdots + \beta_p X_p$$

This is what logistic regression is actually fitting a straight line to. A one-unit increase in X_j, holding everything else fixed, adds β_j to the log-odds, or equivalently, multiplies the **odds** themselves, $p(X)/(1-p(X))$, by e^{β_j}: the **odds ratio**. So:

- $\beta_j > 0$ (so $e^{\beta_j} > 1$): the odds, and therefore the probability, of the event increase with X_j.
- $\beta_j < 0$ (so $e^{\beta_j} < 1$): the odds, and therefore the probability, decrease with X_j.

Odds ratios are a genuinely useful way to talk to a business audience: "each extra point of `recency_score` multiplies the odds of churn by 1.4" is a concrete, actionable statement in a way that a raw logit coefficient is not. We'll compute actual odds ratios for the churn model below.

> **i** Estimating the coefficients: maximum likelihood
>
> Unlike linear regression's RSS, logistic regression's coefficients are fit by **maximum likelihood**: choose the β's that make the observed outcomes (e.g., who actually churned and who didn't) as probable as possible under the model. In practice this means finding the β's that push $p(X)$ close to 1 for every customer who did churn, and close to 0 for every customer who didn't. `sklearn` and `statsmodels` both solve this optimization for you internally, so writing the likelihood function out by hand is not something you need to do in practice, but it explains why a logistic regression fit can occasionally fail to converge on separable or near-separable data, where a very extreme coefficient would make the likelihood arbitrarily large.

7.3.3 A worked example: predicting churn

We reuse the same churn basetable and train/test split convention as the previous chapter with a stratified split, so the churn rate stays the same in both sets.

```
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import roc_auc_score, accuracy_score, roc_curve

X = basetable.drop(columns=["churn"])
y = basetable["churn"]

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=RANDOM_STATE, stratify=y
)
```

```
print(f"Churn rate: {y_test.mean():.3%}")
print(f"Accuracy of always predicting \"no churn\": {1 - y_test.mean():.3%}")
```

```
Churn rate: 3.769%
Accuracy of always predicting "no churn": 96.231%
```

That second line matters for what comes next: only about 4% of customers actually churn, so a classifier that never bothers to learn anything and just predicts "no churn" every time would already be right roughly 96% of the time. Keep that number in mind, since accuracy on its own will be almost useless for judging the models below, exactly the class-imbalance problem covered in the previous chapter.

Ridge, Lasso, and Elastic Net logistic regression follow the same one-recipe-three-calls pattern as their linear counterparts above, so the cell below again defines a single reusable helper, `fit_regularized_logistic`, and each of the three techniques just calls it with a different `l1_ratio`.

Two things are worth mentioning. First, just like the regularized linear models earlier, regularized logistic regression is scale-sensitive, so the pipeline again includes a `StandardScaler`. Second, `sklearn` parameterizes the penalty strength as `C`, the *inverse* of λ: a **smaller** `C` means **stronger** regularization, the opposite of how λ/`alpha` worked above, so it is easy to get backwards. The penalty type itself is set through `l1_ratio`: `0` for a pure L2 (Ridge) penalty, `1` for a pure L1 (Lasso) penalty, and anything in between for Elastic Net, all handled by the same `"saga"` solver.

```
def fit_regularized_logistic(l1_ratio_grid, C_grid=None):
    pipeline = Pipeline([
        ("imputer", SimpleImputer(strategy="median")),
        ("scaler", StandardScaler()),
        ("classifier", LogisticRegression(solver="saga", random_state=RANDOM_STATE, max_iter=5000)),
    ])
    param_grid = {"classifier__l1_ratio": l1_ratio_grid}
    if C_grid is not None:
        param_grid["classifier__C"] = C_grid

    grid = GridSearchCV(pipeline, param_grid, cv=5, scoring="roc_auc", n_jobs=-1)
    grid.fit(X_train, y_train)

    proba = grid.predict_proba(X_test)[:, 1]
    auc = roc_auc_score(y_test, proba)
    acc = accuracy_score(y_test, grid.predict(X_test))
    coefs = grid.best_estimator_.named_steps["classifier"].coef_[0]
    return grid, auc, acc, coefs, proba

C_grid = np.logspace(-3, 3, 10)

ridge_log, ridge_auc, ridge_acc, ridge_log_coefs, ridge_proba = fit_regularized_logistic([0.0], C_grid)
print(f"Ridge logistic: best C = {ridge_log.best_params_['classifier__C']:.4f}")
print(f"Test AUC = {ridge_auc:.3f}, test accuracy = {ridge_acc:.3f}")
```

```
Ridge logistic: best C = 1000.0000
Test AUC = 0.774, test accuracy = 0.963
```

Look at those two numbers side by side, and the trap from the previous chapter shows up immediately: the test accuracy is barely above the "always predict no churn" baseline computed above, roughly a fraction of a percentage point better. Read on its own, that accuracy looks like the model learned almost nothing. The AUC tells a completely different story: at around 0.77, the model is substantially better than random (0.5) at ranking customers by churn risk, it just doesn't change many predicted *labels* at the default 0.5 cut-off, because churn is rare enough that most predicted probabilities stay under 0.5 regardless. This is precisely

why the previous chapter insisted on threshold-independent metrics for imbalanced targets: accuracy alone would have you conclude this model is worthless, when it's actually doing real, usable work at separating higher-risk customers from lower-risk ones.

With the model fit, the odds-ratio interpretation from earlier becomes concrete:

```
odds_ratios = pd.DataFrame({
    "Feature": X_train.columns,
    "Coefficient": ridge_log_coefs,
    "Odds ratio": np.exp(ridge_log_coefs),
}).sort_values("Odds ratio", ascending=False)
odds_ratios
```

	Feature	Coefficient	Odds ratio
2	recency_score	0.549717	1.732763
3	days_customer	-0.420364	0.656808
1	Total_quantity	-1.168432	0.310854
0	Monetary_score	-4.169311	0.015463

An odds ratio above 1 increases the odds of churning, below 1 lowers them. Note that because every predictor was standardized first, each odds ratio is "per one standard deviation increase", not "per one raw unit", which is the price of the scaling step from earlier: convenient for a fair comparison across coefficients, less convenient for a direct real-world statement like "per extra euro spent."

`recency_score` is the only predictor above 1: a standard-deviation increase multiplies the odds of churn by roughly 1.7, so customers who haven't purchased in a while are noticeably more likely to leave, which matches the intuition behind including recency in the basetable in the first place. `Monetary_score` sits at the other extreme, an odds ratio near 0.015, an unusually large protective effect: customers who have spent more are less likely to churn. `days_customer` and `Total_quantity` also pull the odds down, more moderately. All four line up with a simple business story: recent, frequent, high-spending customers stay, and it took nothing more than a logistic regression to read that off the model directly.

```
lasso_log, lasso_auc, lasso_acc, lasso_log_coefs, lasso_proba = fit_regularized_logistic([1.0], C_grid)
print(f"Lasso logistic: best C = {lasso_log.best_params_['classifier__C']:.4f}")
print(f"Test AUC = {lasso_auc:.3f}, test accuracy = {lasso_acc:.3f}")
print(f"Non-zero coefficients: {(lasso_log_coefs != 0).sum()} / {len(X_train.columns)}")
```

```
Lasso logistic: best C = 1000.0000
Test AUC = 0.774, test accuracy = 0.963
Non-zero coefficients: 4 / 4
```

With only four predictors to begin with, and all four already carrying predictive value, Lasso keeps every one of them, and AUC and accuracy both come out essentially identical to Ridge's. There just isn't a weak predictor here for Lasso to prune.

```
elastic_log, elastic_auc, elastic_acc, elastic_log_coefs, elastic_proba = fit_regularized_logistic(
    [0.1, 0.5, 0.9], np.logspace(-2, 2, 6)
)
print(f"Elastic Net logistic: best C = {elastic_log.best_params_['classifier__C']:.4f}, "
      f"l1_ratio = {elastic_log.best_params_['classifier__l1_ratio']}")
print(f"Test AUC = {elastic_auc:.3f}, test accuracy = {elastic_acc:.3f}")
```

```
Elastic Net logistic: best C = 100.0000, l1_ratio = 0.9
Test AUC = 0.774, test accuracy = 0.963
```

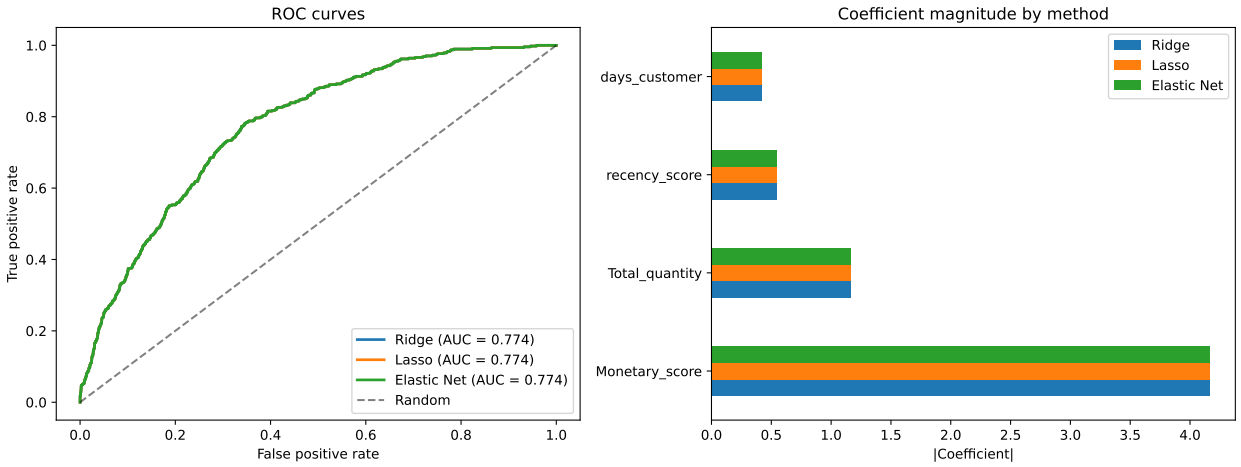
Cross-validation leans Elastic Net heavily toward the Lasso end (`l1_ratio` close to 1) but, again, lands on essentially the same AUC as the other two.

```
fig, axes = plt.subplots(1, 2, figsize=(13, 5))

for proba, name in [(ridge_proba, "Ridge"), (lasso_proba, "Lasso"), (elastic_proba, "Elastic Net")]:
    fpr, tpr, _ = roc_curve(y_test, proba)
    auc = roc_auc_score(y_test, proba)
    axes[0].plot(fpr, tpr, linewidth=2, label=f"{name} (AUC = {auc:.3f})")
axes[0].plot([0, 1], [0, 1], "k--", alpha=0.5, label="Random")
axes[0].set_xlabel("False positive rate")
axes[0].set_ylabel("True positive rate")
axes[0].set_title("ROC curves")
axes[0].legend()

coef_comparison = pd.DataFrame({
    "Ridge": np.abs(ridge_log_coefs),
    "Lasso": np.abs(lasso_log_coefs),
    "Elastic Net": np.abs(elastic_log_coefs),
}, index=X_train.columns)
coef_comparison.plot(kind="barh", ax=axes[1])
axes[1].set_xlabel("|Coefficient|")
axes[1].set_title("Coefficient magnitude by method")

plt.tight_layout()
plt.show()
```



With only four predictors to begin with, all three regularization methods land on very similar AUC, and Lasso doesn't have much room to drop any of them entirely. That is a genuinely common outcome once the feature set is already compact and each predictor carries some real signal: the choice between Ridge, Lasso, and Elastic Net then comes down more to interpretability and how you want coefficients to behave than to a meaningful difference in predictive accuracy.

7.3.4 Beyond two classes

Logistic regression, as defined above, assumes a binary response. Extending it to $K > 2$ classes takes one of two standard strategies:

- **One-vs-rest** (also one-vs-all): fit K binary classifiers, each one separating a single class from all the others combined, and assign each observation to whichever classifier is most confident. This is the most common default and scales linearly in the number of classes. It becomes impractical for very large K because each classifier sees the full dataset, which can be slow for some algorithms.

- **One-vs-one**: fit a binary classifier for every *pair* of classes ($K(K-1)/2$ of them in total) and let them vote. Each individual classifier only ever trains on the rows belonging to its two classes, rather than the full dataset, which helps for algorithms that get disproportionately slow as the training set grows. Logistic regression isn't one of them, so for it, one-vs-rest's smaller number of classifiers is simply the better default.

7.3.5 Where logistic regression breaks down

Like linear regression, logistic regression assumes a specific functional form, here a linear relationship between the predictors and the log-odds, so genuinely nonlinear relationships are just as much of a blind spot as they were before. It is also prone to the same overfitting pattern as linear regression when the predictor set grows large relative to the amount of data: low training error, poor generalization, low bias, high variance. Everything covered under regularization above applies identically here; it is not a coincidence that `sklearn`'s `LogisticRegression` accepts the same `l1_ratio` and `C` arguments a regularized linear model does.

7.4 Where we go from here

Across the last four chapters, a raw, messy table has gone through the full analytics pipeline: understood, cleaned and prepared into a model-ready basetable, evaluated with a proper experimental setup and the right metrics, and finally modeled with two of the most widely used techniques in business analytics. That is the complete loop this book set out to cover.

Linear and logistic regression are not the end of the modeling story, only its foundation. Decision trees, random forests, and gradient boosting (of which the random forest from the previous chapter was one, deliberately unexplained, example) relax the linearity assumption entirely and often predict better on complex, real-world data at the cost of some interpretability (Prokhorenkova et al. 2018; De Caigny et al. 2018). Learning to build them well is a natural next step once the fundamentals covered in this book are second nature. However, with proper feature engineering combined with proper regularization and variable selection, linear and logistic regression can be powerful tools and competitive baselines for many real-world problems, and they are often the first models deployed because of their speed, simplicity, and interpretability.

Appendix A

Installation Guide

This appendix walks through setting up Python and an IDE (VS Code is recommended) on your own laptop for this course, so that every chapter in this book runs exactly as shown. The idea is to create a dedicated, self-contained **virtual environment** with a known-good set of package versions, rather than relying on whatever happens to already be installed on your machine.

A.1 Getting the course files

Before installing anything, get a local copy of the book's repository. The notebooks refer to `requirements.txt` and the `data/` folder by a relative path, so those only resolve correctly once you have the whole project folder, not just one downloaded notebook. Downloading a single notebook by itself, for example via right-click "Save link as" on one file, will not work: it won't come with `requirements.txt` or `data/` alongside it, and every `pip install` or `pd.read_csv` call in this book assumes they're there.

The simplest way, no `git` required:

1. Go to the book's GitHub repository.
2. Click the green **Code** button, then **Download ZIP**.
3. Unzip it somewhere on your machine. This folder is your project root for the rest of this appendix and for every notebook in the course.

If you're already comfortable with `git`, cloning works the same way and makes it easier to pull future updates to the course material:

```
git clone https://github.com/MatthBogaert/IDAFBE-book.git
```

A.2 Create a virtual environment

A virtual environment is a self-contained directory that holds a Python installation for a particular version of Python, plus a specific set of additional packages. Without one, packages installed for different projects can silently conflict with each other: a version of `pandas` that one project needs might break another. Learning to use virtual environments now is a worthwhile habit: it's exactly how you'll be expected to manage Python projects later, e.g. in a company.

In this book `conda` is used to create the environment (you should already have `conda` available if you installed Anaconda). This book pins everything to **Python 3.11**: see `requirements.txt` in the project root for the exact package versions.

A.2.1 VS Code

If you'd like a visual walkthrough of creating a conda environment from within VS Code, this video covers the general steps. Note that the video uses its own example environment name and Python version: when it gets to `conda create --name mynewenv`, use the command from the next section instead, with **this book's** environment name and Python version.

A.2.2 Command line

Open a terminal (not a notebook cell) and run:

```
conda create --name DA2026 python=3.11
```

Then activate it:

```
conda activate DA2026
```

You may need to restart VS Code (or your notebook) afterward and explicitly select this new environment as your kernel: see Selecting the right kernel below.

A.3 Installing required packages

With the `DA2026` environment activated, install the exact package versions this book was written and tested against:

```
pip install -r requirements.txt
```

Make sure `requirements.txt` is the one in the root of this book's project folder: running this from the wrong directory, or skipping it entirely and just `pip install`-ing packages one at a time, is the single most common source of "it works in the book but not on my machine."

> **Installing the right versions**
>
> You'll sometimes be tempted to just install the newest version of a package. We deliberately don't: for example, `pandas` 3.0 was released recently, but this book stays on `pandas>=2.3,<3` because several packages it depends on (`feature_engine`, `sklearn_evaluation`) haven't been fully verified against pandas 3.0's changed default behavior yet. See the note in Python for Data Analytics for details. The pinned versions in `requirements.txt` are the ones every example in this book was actually run against.

If you're running this from inside a Jupyter notebook rather than a terminal, you can install the same requirements with:

```
%pip install -r requirements.txt
```

A.4 Selecting the right kernel

Installing the packages into the `DA2026` environment isn't enough on its own: VS Code (or Jupyter) also needs to actually *use* that environment when it runs your code. This trips people up more often than the installation itself, so it's worth being deliberate about it.

For `.ipynb` notebooks: open the notebook, click the kernel name in the top-right corner of the notebook editor, choose **"Select Another Kernel..."**, and pick **"Python (DA2026)"** from the list.

For your overall Python interpreter (affects linting, IntelliSense, and some extensions): `Cmd+Shift+P` (or `Ctrl+Shift+P` on Windows/Linux) → **"Python: Select Interpreter"** → choose the interpreter inside your `DA2026` environment.

For `quarto preview`/`quarto render` on `.qmd` files: this project's `_quarto.yml` already pins `jupyter: da2026` at the top, so Quarto picks the right kernel automatically regardless of whatever interpreter VS Code itself is set to. If a preview still seems to be using the wrong environment, close and reopen the preview tab: it can cache the kernel it started with from before you had `DA2026` set up.

> ⚠ The conda environment trap
>
> If you have multiple conda environments with the *same* Python version, they can look identical in VS Code's picker unless you check the environment name specifically, not just the version number. Picking the wrong one is easy to do, and the symptom is usually confusing: code that runs fine in one environment throws an error like `AttributeError: _ARRAY_API not found` or `A module that was compiled using NumPy 1.x cannot be run in NumPy 2.x` in another. That specific error means the active kernel has a `numpy` version that doesn't match the `pandas`/`pyarrow` build it's trying to import: you're not actually in `DA2026`. Re-check the kernel/interpreter selection above.

A.5 Running Python

Once your kernel is set correctly, you're ready to run code. As a first check, try the classic **"Hello, world!"** example: Python's `print()` function displays text or results.

```
print("Hello, world!")
```

```
Hello, world!
```

If that runs without error, your environment is set up correctly and you're ready for the rest of this book.

References

Anscombe, Francis J. 1973. "Graphs in Statistical Analysis." *The American Statistician* 27 (1): 17–21.

Baesens, Bart. 2014. *Analytics in a Big Data World: The Essential Guide to Data Science and Its Applications*. Wiley.

Baesens, Bart, Sébastien Höppner, and Tim Verdonck. 2021. "Data Engineering for Fraud Detection." *Decision Support Systems* 150: 113492.

Ballings, Michel, Dirk Van den Poel, and Matthias Bogaert. 2016. "Social Media Optimization: Identifying an Optimal Strategy for Increasing Network Size on Facebook." *Omega* 59: 15–25.

Bhattacharya, C. B. 1998. "When Customers Are Members: Customer Retention in Paid Membership Contexts." *Journal of the Academy of Marketing Science 1998 26:1* 26: 31–44. https://link.springer.com/article/10.1177/0092070398261004.

Buckinx, Wouter, and Dirk Van den Poel. 2005. "Customer Base Analysis: Partial Defection of Behaviourally Loyal Clients in a Non-Contractual FMCG Retail Setting." *European Journal of Operational Research* 164 (1): 252–68.

Chapman, Pete, Julian Clinton, Randy Kerber, et al. 2000. *CRISP-DM 1.0: Step-by-Step Data Mining Guide*. SPSS Inc.

Coussement, Kristof, Stefan Lessmann, and Geert Verstraeten. 2017. "A Comparative Analysis of Data Preparation Algorithms for Customer Churn Prediction: A Case Study in the Telecommunication Industry." *Decision Support Systems* 95: 27–36.

Crook, Jonathan N., David B. Edelman, and Lyn C. Thomas. 2007. "Recent Developments in Consumer Credit Risk Assessment." *European Journal of Operational Research* 183: 1447–65.

De Bock, Koen W., Kristof Coussement, Arno De Caigny, et al. 2024. "Explainable AI for Operational Research: A Defining Framework, Methods, Applications, and a Research Agenda." *European Journal of Operational Research* 317 (September): 249–72. https://www.sciencedirect.com/science/article/pii/S0377221723007294.

De Caigny, Arno, Kristof Coussement, and Koen W. De Bock. 2018. "A New Hybrid Classification Algorithm for Customer Churn Prediction Based on Logistic Regression and Decision Trees." *European Journal of Operational Research* 269 (2): 760–72.

Delen, Dursun, and Sudha Ram. 2018. "Research Challenges and Opportunities in Business Analytics." *Journal of Business Analytics* 1 (1): 2–12.

European Parliament and Council of the European Union. 2016. *Regulation (EU) 2016/679 of the European*

Parliament and of the Council of 27 April 2016 on the Protection of Natural Persons with Regard to the Processing of Personal Data and on the Free Movement of Such Data, and Repealing Directive 95/46/EC (General Data Protection Regulation) (Text with EEA Relevance). No. 119.

European Parliament and Council of the European Union. 2024. *Regulation (EU) 2024/1689 of the European Parliament and of the Council of 13 June 2024 Laying down Harmonised Rules on Artificial Intelligence and Amending Regulations (EC) No 300/2008, (EU) No 167/2013, (EU) No 168/2013, (EU) 2018/858, (EU) 2018/1139 and (EU) 2019/2144 and Directives 2014/90/EU, (EU) 2016/797 and (EU) 2020/1828 (Artificial Intelligence Act) (Text with EEA Relevance)*. No. 1689.

Fawcett, Tom. 2006. "An Introduction to ROC Analysis." *Pattern Recognition Letters* 27 (8): 861–74.

Fayyad, Usama, Gregory Piatetsky-Shapiro, and Padhraic Smyth. 1996. "From Data Mining to Knowledge Discovery in Databases." *AI Magazine* 17 (3): 37–54.

Geisser, Seymour. 1993. *Predictive Inference: An Introduction*. Chapman; Hall/CRC.

Hernández-Orallo, José, Peter Flach, and Cèsar Ferri. 2012. "A Unified View of Performance Metrics: Translating Threshold Choice into Expected Classification Loss." *Journal of Machine Learning Research* 13 (91): 2813–69.

Hindle, Giles, Martin Kunc, Michael Mortensen, Asil Oztekin, and Richard Vidgen. 2020. "Business Analytics: Defining the Field and Identifying a Research Agenda." In *European Journal of Operational Research*, No. 3, vol. 281. Elsevier.

Hsieh, Nan-Chen. 2004. "An Integrated Data Mining and Behavioral Scoring Model for Analyzing Bank Customers." *Expert Systems with Applications* 27 (4): 623–33.

James, Gareth, Daniela Witten, Trevor Hastie, and Robert Tibshirani. 2013. *An Introduction to Statistical Learning: With Applications in r*. Springer.

Kaufman, Shachar, Saharon Rosset, Claudia Perlich, and Ori Stitelman. 2012. "Leakage in Data Mining: Formulation, Detection, and Avoidance." *ACM Transactions on Knowledge Discovery from Data* 6 (4): 1–21.

Kuhn, Max, and Kjell Johnson. 2013. *Applied Predictive Modeling*. Springer.

Lagioia, Francesca, Riccardo Rovatti, and Giovanni Sartor. 2022. "Algorithmic Fairness Through Group Parities? The Case of COMPAS-SAPMOC." *AI & SOCIETY 2022 38:2* 38 (April): 459–78. https://link.springer.com/article/10.1007/s00146-022-01441-y.

Lessmann, Stefan, Bart Baesens, Hsin Vonn Seow, and Lyn C. Thomas. 2015. "Benchmarking State-of-the-Art Classification Algorithms for Credit Scoring: An Update of Research." *European Journal of Operational Research* 247 (November): 124–36. https://www.sciencedirect.com/science/article/abs/pii/S0377221715004208.

Manokhin, Valeriy. 2023. *Practical Guide to Applied Conformal Prediction in Python: Learn and Apply the Best Uncertainty Frameworks to Your Industry Applications*. Packt Publishing.

Martens, David, Jan Vanthienen, Wouter Verbeke, and Bart Baesens. 2011. "Performance of Classification Models from a User Perspective." *Decision Support Systems* 51 (4): 782–93.

Micci-Barreca, Daniele. 2001. "A Preprocessing Scheme for High-Cardinality Categorical Attributes in Classification and Prediction Problems." *ACM SIGKDD Explorations Newsletter* 3 (1): 27–32.

Moeyersoms, Julie, and David Martens. 2015. "Including High-Cardinality Attributes in Predictive Models: A Case Study in Churn Prediction in the Energy Sector." *Decision Support Systems* 72: 72–81.

Mortenson, Michael J, Neil F Doherty, and Stewart Robinson. 2015. "Operational Research from Taylorism to Terabytes: A Research Agenda for the Analytics Age." *European Journal of Operational Research* 241 (3): 583–95.

Neslin, Scott A., Sunil Gupta, Wagner Kamakura, Junxiang Lu, and Charlotte H. Mason. 2006. "Defection Detection: Measuring and Understanding the Predictive Accuracy of Customer Churn Models." *Journal of Marketing Research* 43 (2): 204–11.

Niculescu-Mizil, Alexandru, and Rich Caruana. 2005. "Predicting Good Probabilities with Supervised Learning." *Proceedings of the 22nd International Conference on Machine Learning*, 625–32.

Prokhorenkova, Liudmila, Gleb Gusev, Aleksandr Vorobev, Anna Veronika Dorogush, and Andrey Gulin. 2018. "CatBoost: Unbiased Boosting with Categorical Features." *Advances in Neural Information Processing Systems* 31.

Van Belle, Rafaël, Charles Van Damme, Hendrik Tytgat, and Jochen De Weerdt. 2022. "Inductive Graph Representation Learning for Fraud Detection." *Expert Systems with Applications* 193: 116463.

Van den Poel, Dirk, and Bart Larivière. 2004. "Customer Attrition Analysis for Financial Services Using Proportional Hazard Models." *European Journal of Operational Research* 157 (August): 196–217. https://www.sciencedirect.com/science/article/abs/pii/S0377221703000699.

Van Gool, Joris, Wouter Verbeke, Piet Sercu, and Bart Baesens. 2012. "Credit Scoring for Microfinance: Is It Worth It?" *International Journal of Finance & Economics* 17 (2): 103–23.

Verbeke, Wouter, Karel Dejaeger, David Martens, Joon Hur, and Bart Baesens. 2012. "New Insights into Churn Prediction in the Telecommunication Sector: A Profit Driven Data Mining Approach." *European Journal of Operational Research* 218 (1): 211–29.

Verbraken, Thomas, Wouter Verbeke, and Bart Baesens. 2013. "A Novel Profit Maximizing Metric for Measuring Classification Performance of Customer Churn Prediction Models." *IEEE Transactions on Knowledge and Data Engineering* 25 (5): 961–73.

Wang, Xu, Bart Baesens, and Zhen Zhu. 2019. "On the Optimal Marketing Aggressiveness Level of C2C Sellers in Social Media: Evidence from China." *Omega* 85: 83–93.

Wickham, Hadley. 2014. "Tidy Data." *Journal of Statistical Software* 59 (10): 1–23.